软件定义网络

Software Defined Network

SDN

https://www.scholat.com/Phomepage.html

SDN

软件定义网络

Software Defined Network

第一章概述

课程考核及成绩评定方法

本门课程加大过程性考核力度，落实过程性评价理念，学业总成绩由过程性评价成绩和期末考试成绩构成，过程性评价成绩占30%，期末考试成绩占70%。其中，过程性评价采取考勤、雨课堂测试以及激励（反转课堂、研究报告、实验复现）相结合的形式进行考核，考勤在平时成绩基础上实行减分制(缺勤6次以上无平时成绩)。

课程教材及主要学习参考资源

1. 黄辉、施晓秋、彭达卫编著《软件定义网络技术》，高等教育出版社，2020年10月。
2. https://www.icourse163.org/learn/WZU-1205809832?tid=1471225500#/learn/announce慕课,温州大学软件定义网络技术,黄辉,施晓秋,潘舒笑.

作业

1. 软件定义物联网、软件定义低空网络、软件定义园区网、软件定义广域网、软件定义6G通信、软件定义接入网、软件定义云计算、软件定义网络安全等
2. 开发实践活动
3. 科研训练研究报告

问题引入

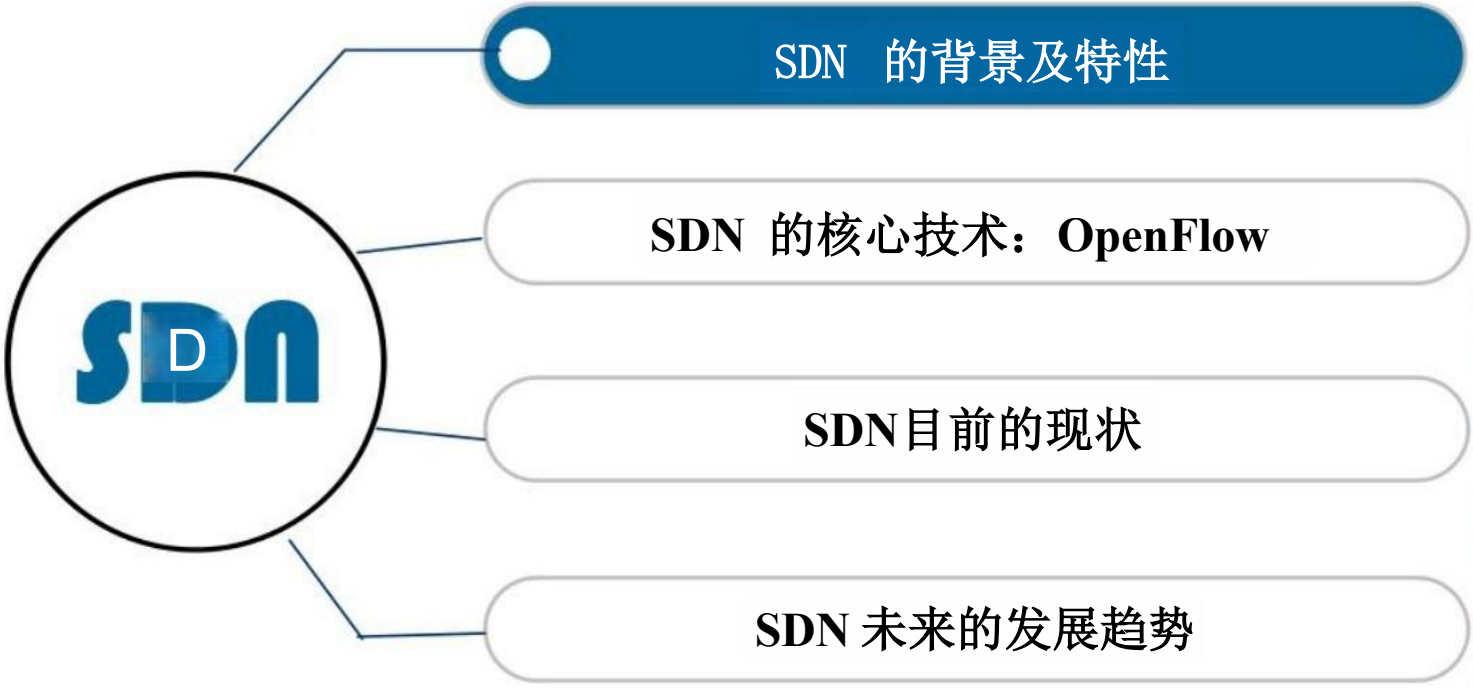
. 什么是软件定义网络？

. Software Defined Networking

https://www.bilibili.com/video/BV1Kh411273v/?spm_id_from=333.337.search-card.all.click&vd_source=363a1a6da8510506c2a4be69351ff671

https://www.bilibili.com/video/BV1nN411y7ur/?spm_id_from=333.337.search-card.all.click&vd_source=363a1a6da8510506c2a4be69351ff671

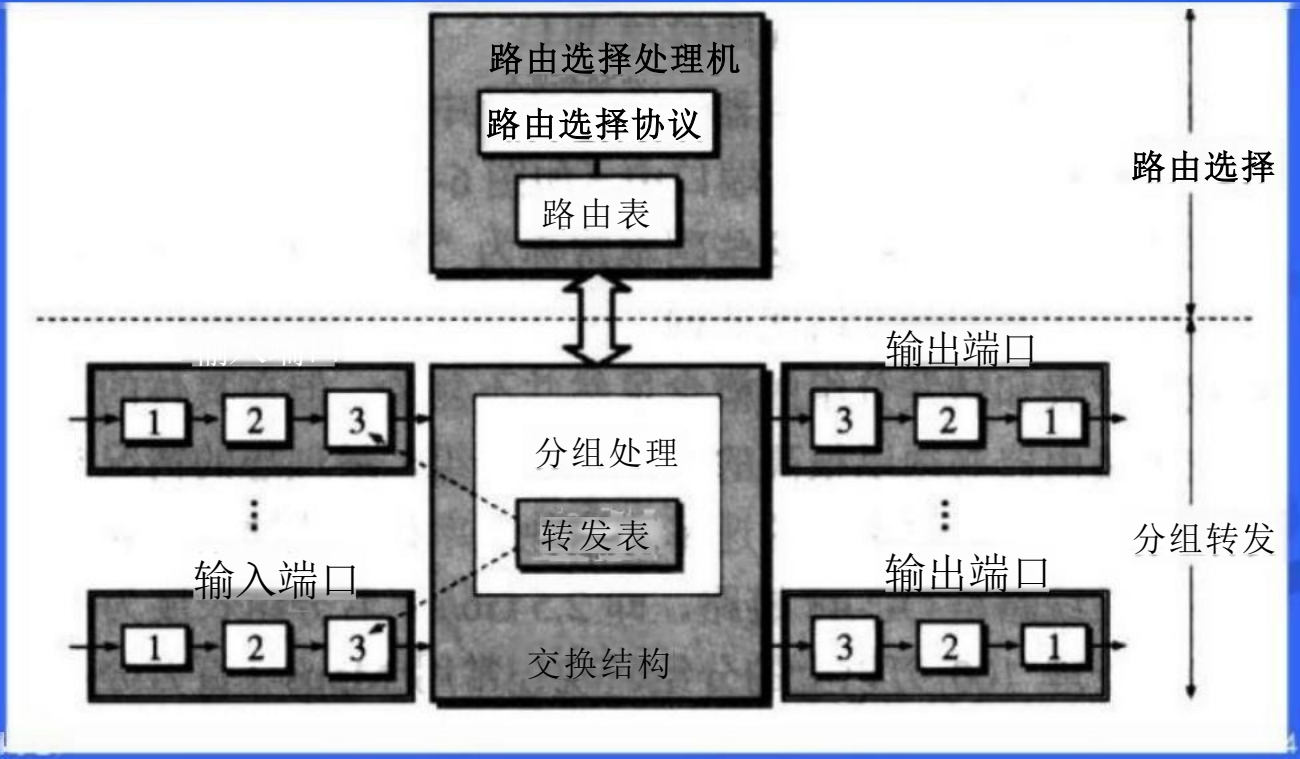
目录



背景

传统的网络设备(交换机、路由器)的固件是由设备制造商锁定和控制，因为交换机和路由器在制造时，已经将分组的转发功能和控制功能结合到一起，在这些传统的网络中，有关对网络的管理控制和分组的转发都是由交换机、路由器完成的。

典型的路由器的结构
（注意：图中的数字1、2、3表示相应层次的构件，路由器的缓存在图外）



背 景

这样带来的问题是交换机、路由器对网络的管理控制功能过强，网络的管理分散，网络的调整困难。

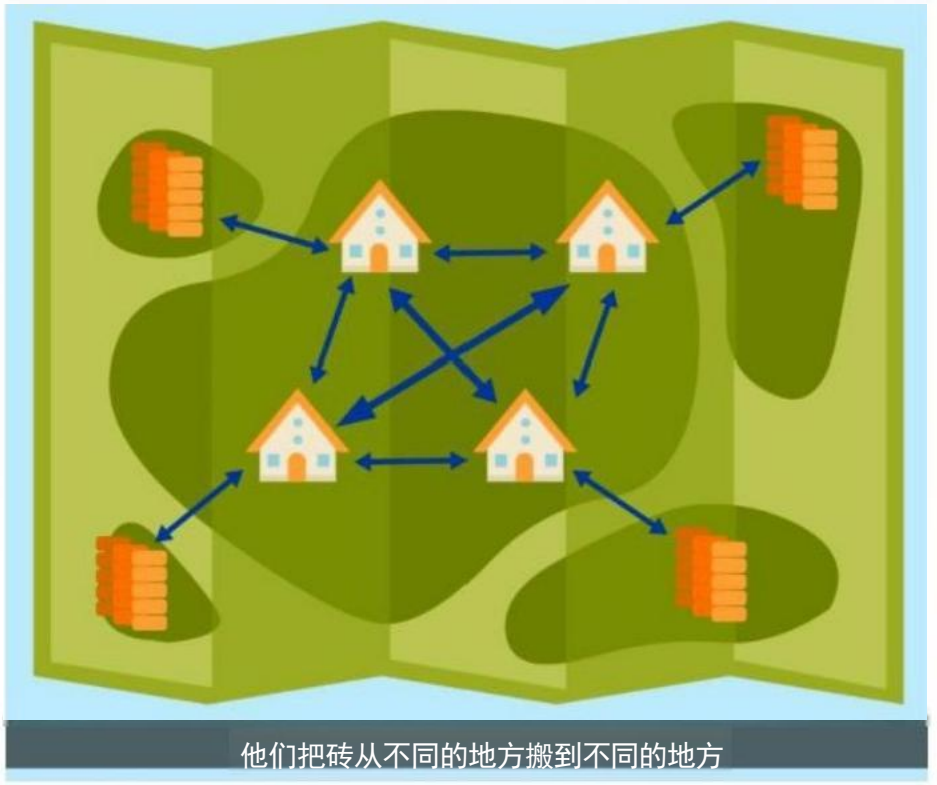
希望将网络控制与物理网络拓扑分离，从而摆脱硬件对网络架构的限制。这样企业便可以像升级、安装软件一样对网络架构进行修改，满足企业对整个网站架构进行调整、扩容或升级。而底层的交换机、路由器等硬件则无需替换，节省大量的成本的同时，网络架构迭代周期将大大缩短。

关于SDN出现的背景可举例来形象化说明

A A
—故事和资料来源于互联网—
在很久很久以前有个小村子

男人
女人
每户人家都是以搬砖为生

每户人家都是以搬砖为生

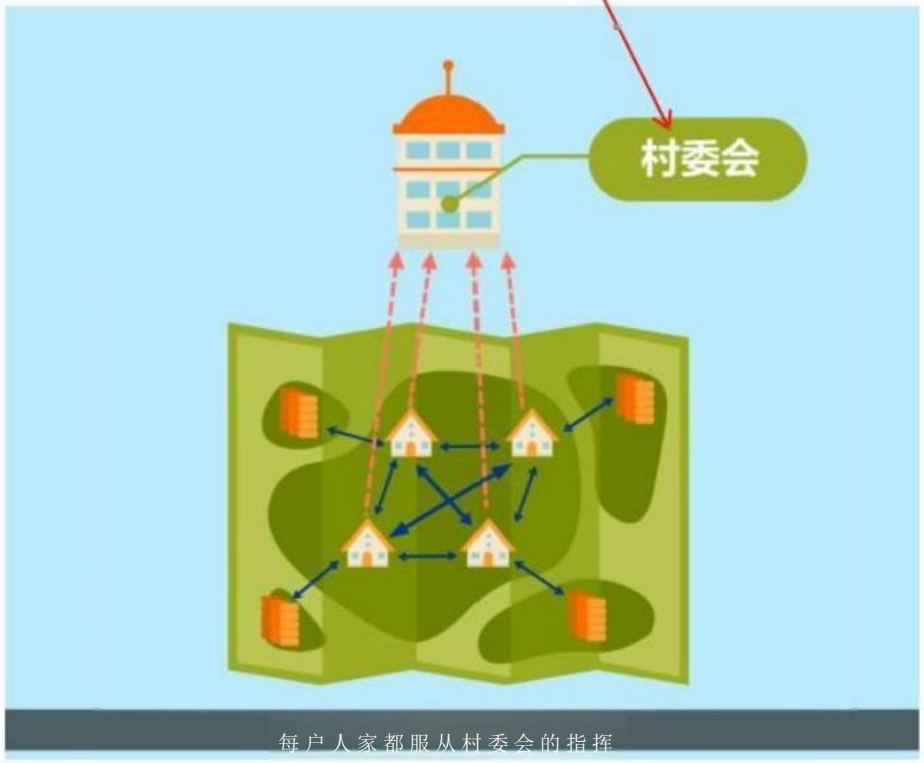


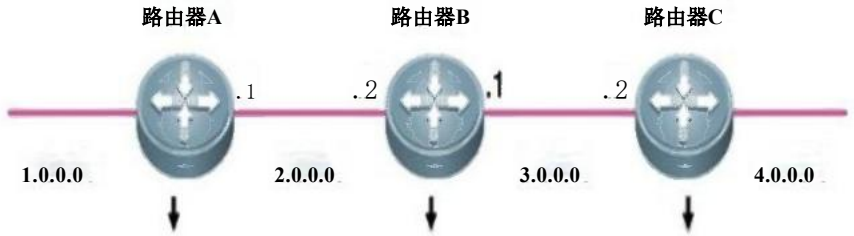
他们把砖从不同的地方搬到不同的地方

男人们负责搬砖

而女人们负责告诉男人往哪里搬

这里村委会
代表什么？





Routing Table

	NET	Nexthop	Metric
C	1.0.00		0
C	2.0.00		0
R	3.0.0.0	2.0.0.2	
B	4.0.0.0	2.0.0.2	2

Routing Table

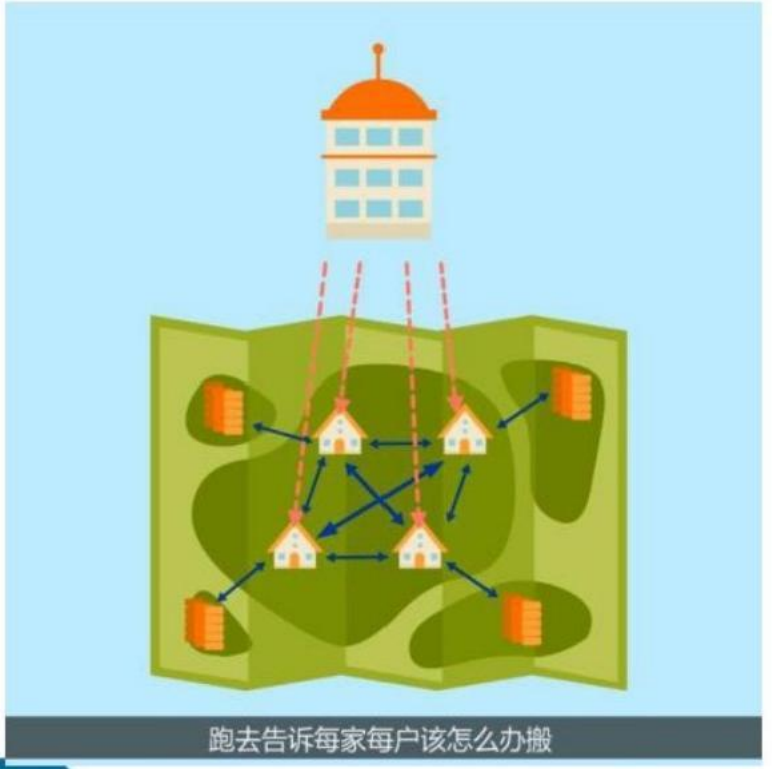
	NET	Next hop	Metic
	2.0.0.0		0
C	3.0.0.0		0
R	1.0.0.0	2.0.0.1	1
R	4.0.0.0	3.0.0.2	1

Routing Table

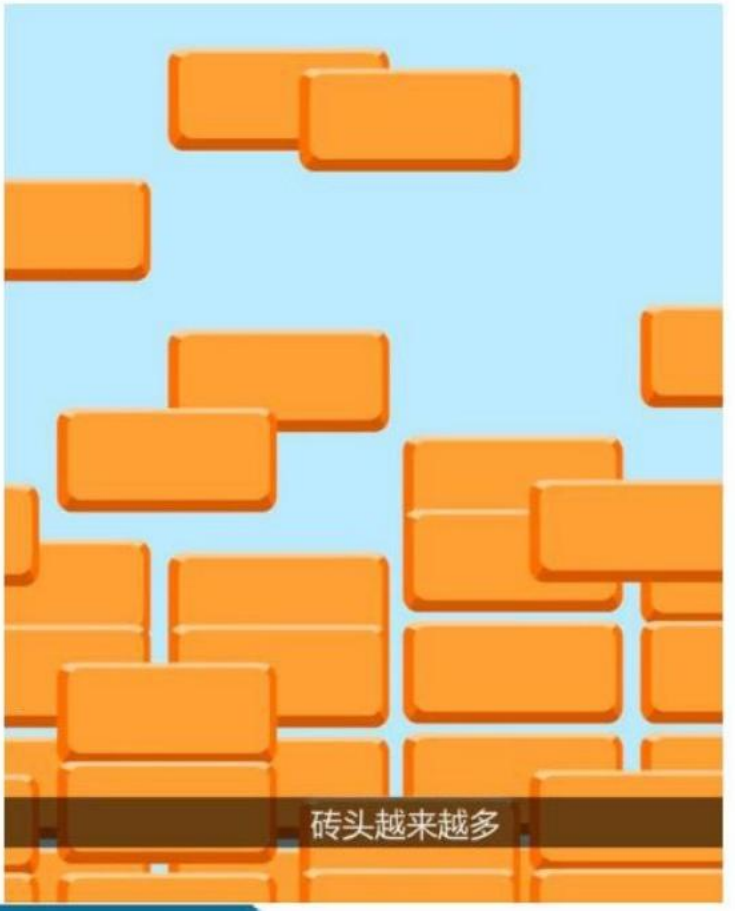
	NET	Next hop	Metric
C	3.0.0.0		0
C	4.0.0.0		0
R	1.0.0.0	3.0.0.1	2
R	2.0.0.0	3.0.0.1	1

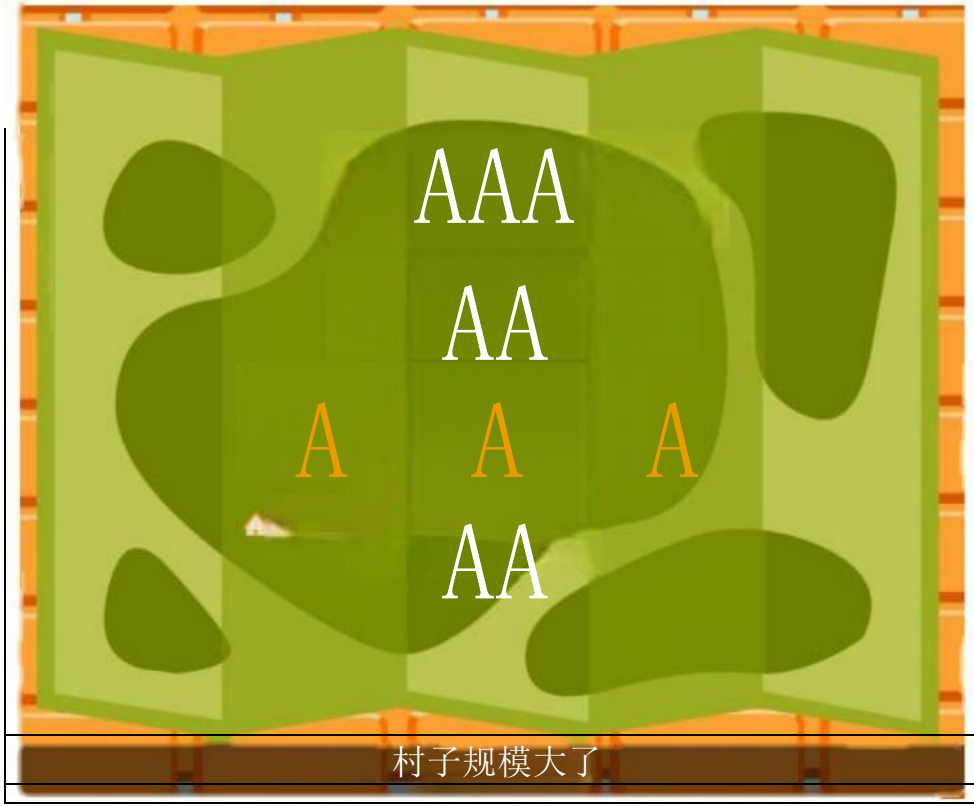
每个路由器的路由表告诉路由器数据如何转发，每户人家代表路由器或交换机，村委会代表什么？

村委会
BOSS
村委会根据砖厂老板的需求


跑去告诉每家每户该怎么办搬

一切都很顺利


砖头越来越多



村子规模大了

村户也多了

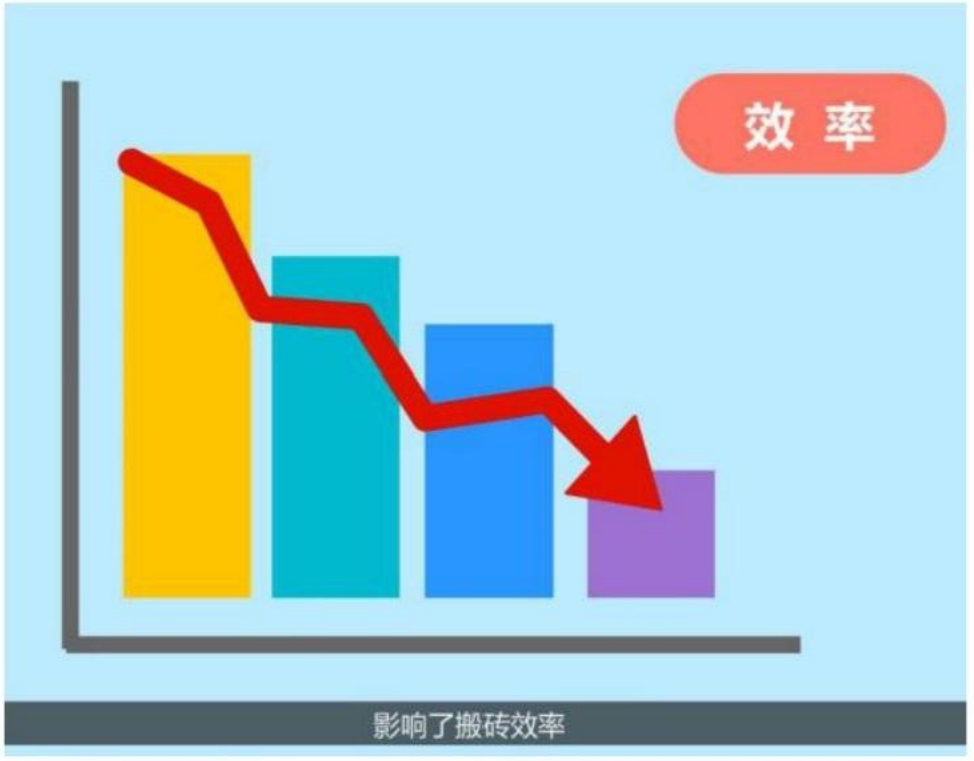
计划变更
每次搬砖计划有变动

村委会
村委会都要挨家挨户通知

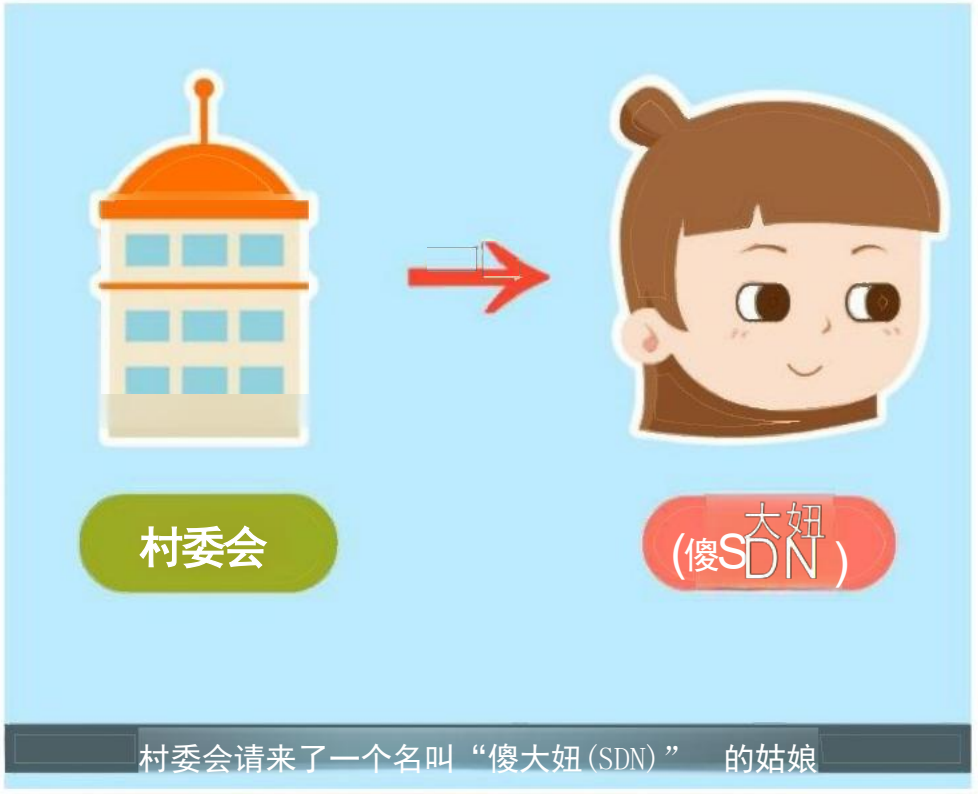
而且

村委会不能及时通知到位

思考：为什么不能及时通知到位？


效率
影响了搬砖效率

砖厂老板也很不开心

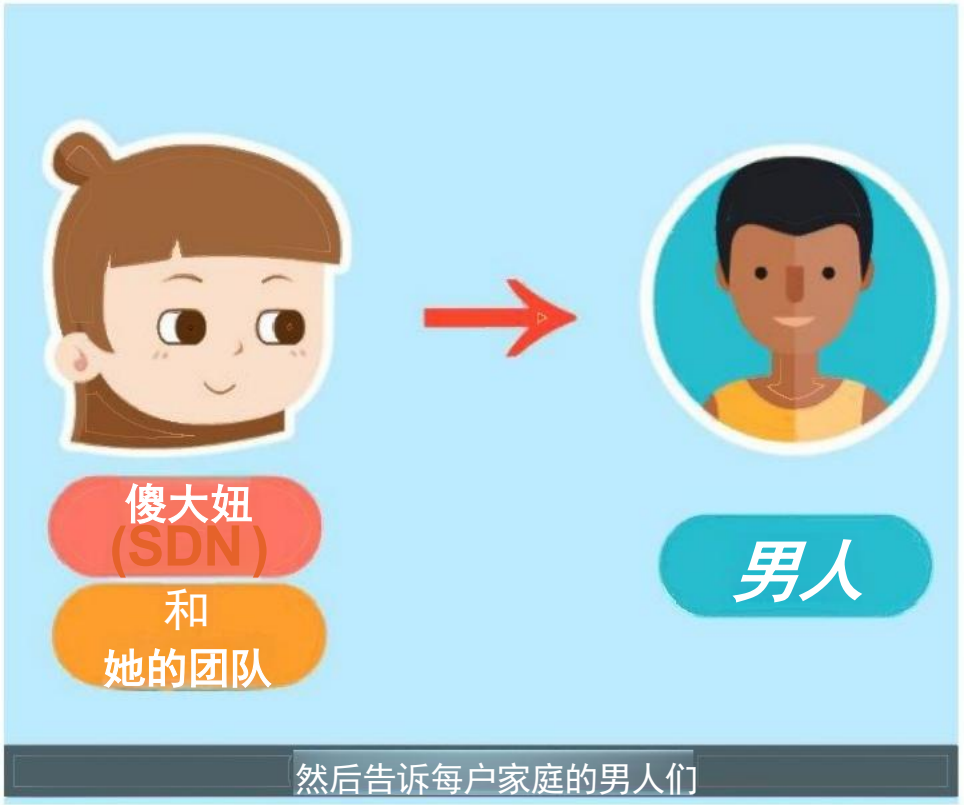

村委会
(傻SDN)
大妞
村委会请来了一个名叫“傻大妞(SDN)” 的姑娘

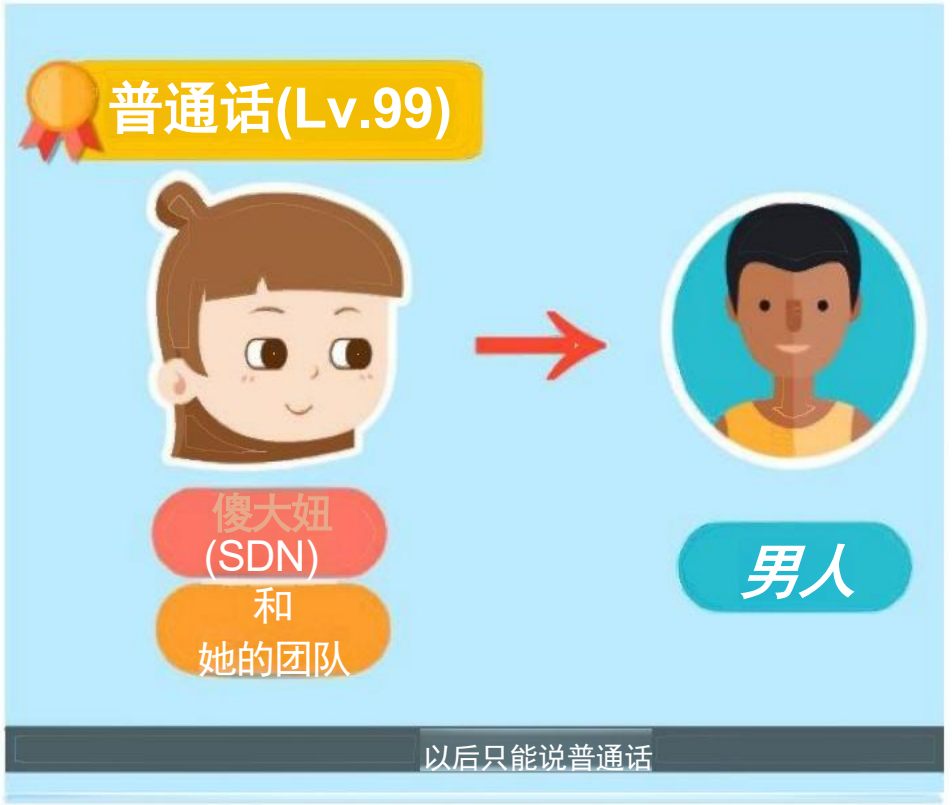
这个傻大妞说

于是

你们这样搬砖是不对的，按照我的方法来吧。
你们这样搬砖是不对的

You
are
fired!
傻大妞把每户人家的女人们都赶回了娘家


傻大妞
(SDN)
和
她的团队
男人
然后告诉每户家庭的男人们


普通话(Lv.99)
傻大妞
(SDN)
和
她的团队
男人
以后只能说普通话

普通话(Lv. 99)

傻大妞
(SDN)
和
她的团队

男人

因为以前每家每户男人女人都是说的自己的方言

傻大妞
(SDN)
和
她的团队

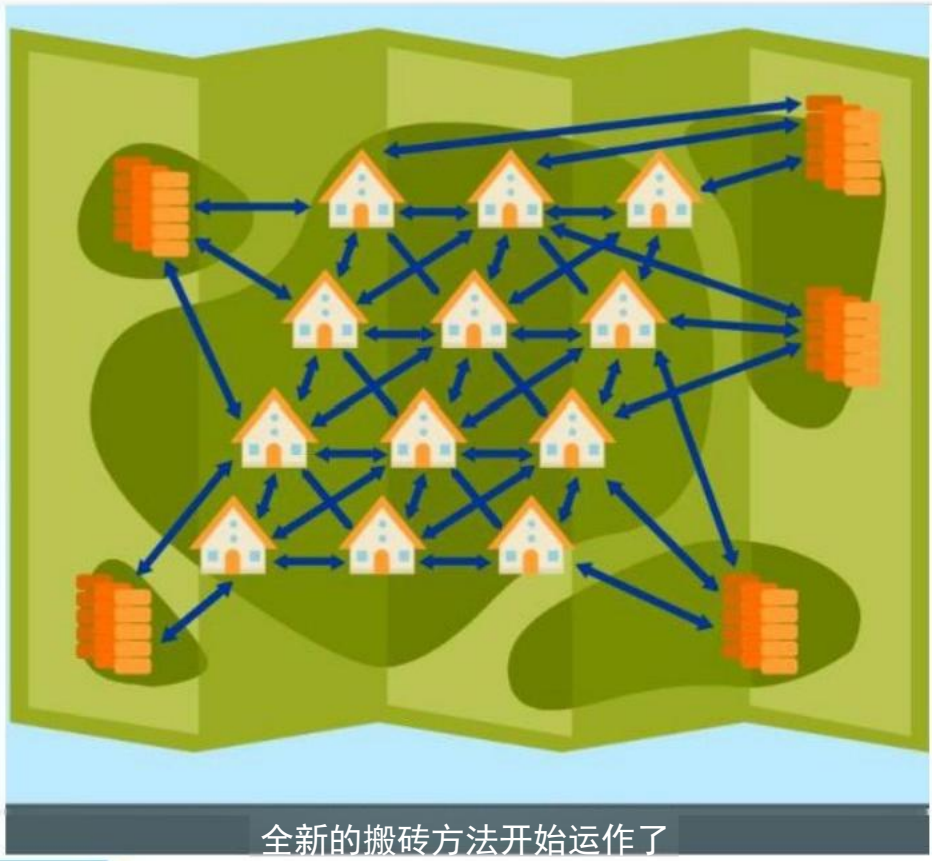
村委会

傻大妞还告诉村委会

傻大妞
(SDN)
和
她的团队
村委会
搬砖方案直接告诉她的团队就行

招商团队
另外指导村委会成立招商团队

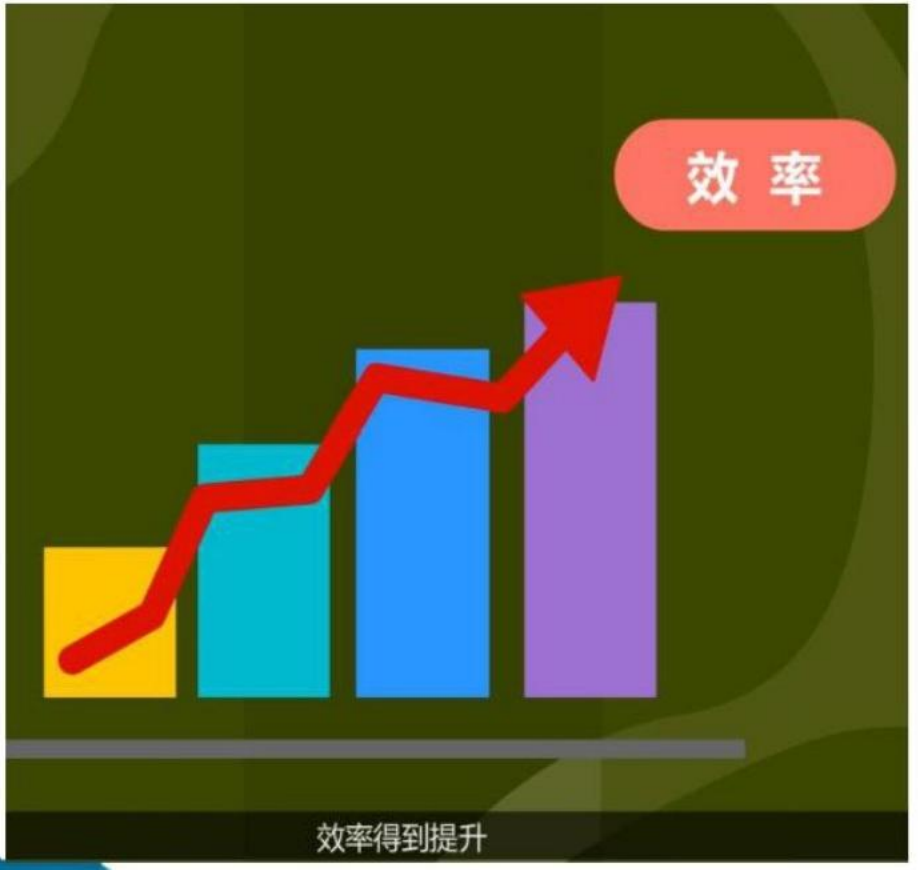
专门对接各大砖厂


全新的搬砖方法开始运作了

因为傻大妞全面接管了男人们

村委会
傻大妞
（SDN）
和
她的团队
所以村委会省事了很多事

村委会
傻大妞
(SDN)
和
她的团队
命令直接传达给傻大妞


效 率
效率得到提升

BOSs
BO5S
BOSs
砖厂老板们更开心了

想往哪搬就往哪搬

故事就愉快滴结束啦

傻大妞
(SDN)

村子：通信网络

砖头：数据

男人：网络设备的转发功能

女人：网络设备的控制功能

砖厂老板：上层用户

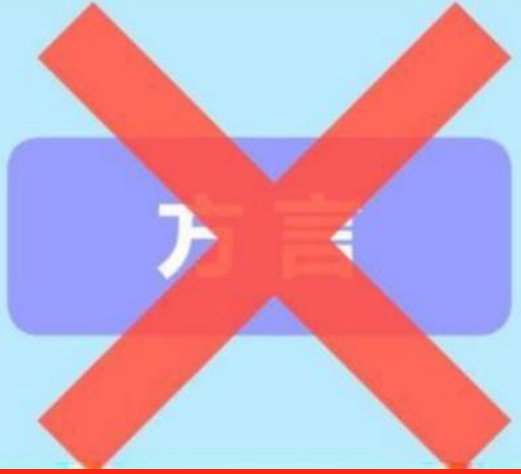
村委会：网络服务提供商

对比我们刚才搬砖故事

SDN带来的好处显而易见

控制和转发分离
男人
傻大妞
(SDN)
女人
男人和女人拆开
— 控制和转发分离

各个硬件厂家不能够
再针对每个硬件设计和安装对应的软件系统



淘汰了方言

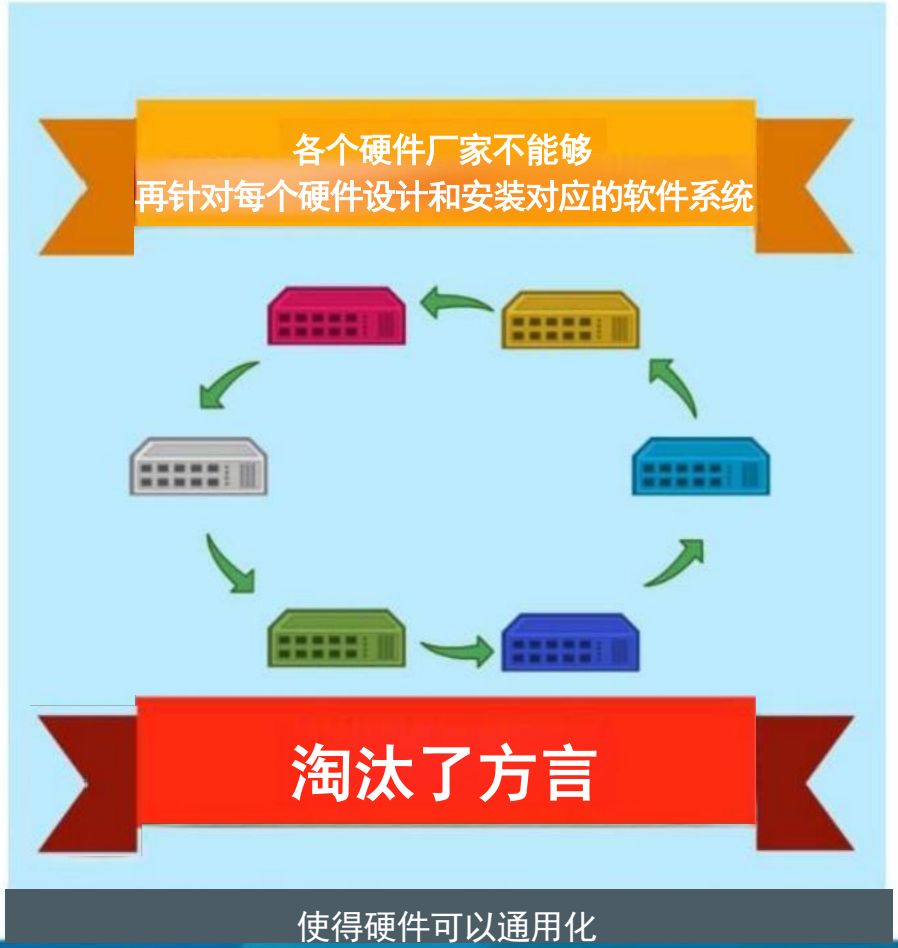
各个硬件厂家不能够再针对每个硬件设计和安装对应的软件系统

各个硬件厂家不能够
再针对每个硬件设计和安装对应的软件系统

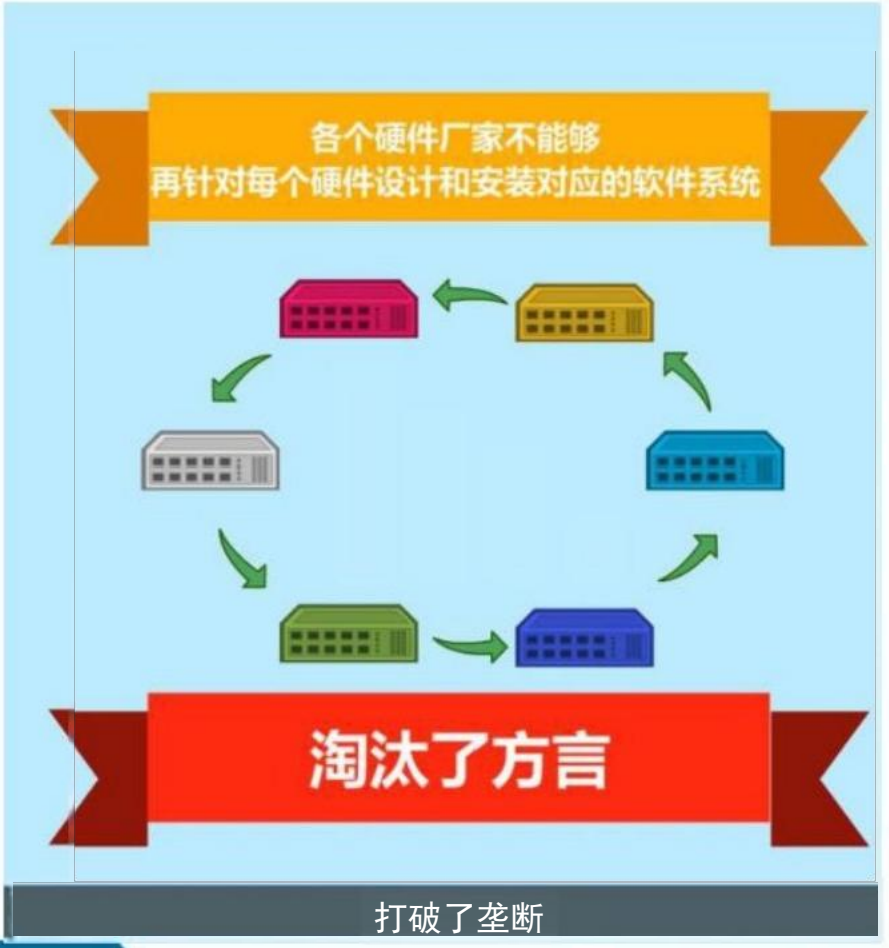
普通话

淘汰了方言

各个硬件厂家不能够再针对每个硬件设计和安装对应的软件系统



使得硬件可以通用化



打破了垄断

二 SDN 控制器进行统一管理

实现了集中控制

降低网络维护难度

村委会不用挨家挨户一个个跑

降低网络维护难度

缩短网络部署周期

砖厂老板们灵活调运砖头

缩短网络部署周期

缩短网络部署周期

砖厂老板们灵活调运砖头

降低运维成本

女人们

女人

传统硬件设备商

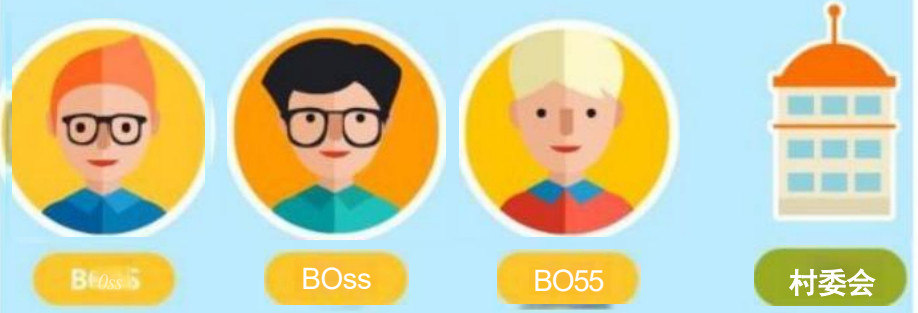
对于那些女人们

女人们

传统硬件设备商

傻 大 妞(SDN)　确实并不是一个好消息

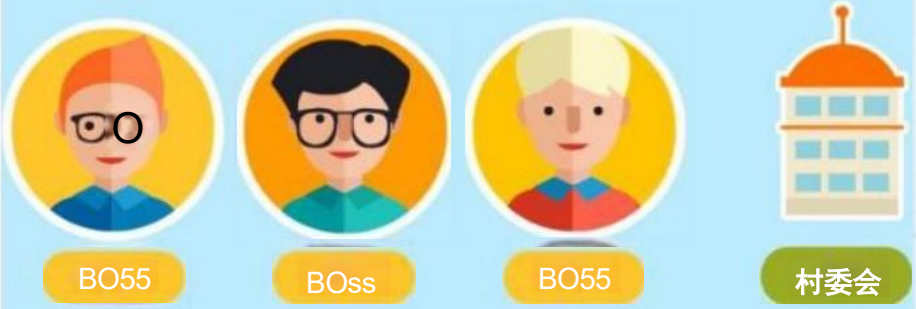
用户&服务提供商



砖厂老板们&村委会

而对于砖厂老板们和村委会

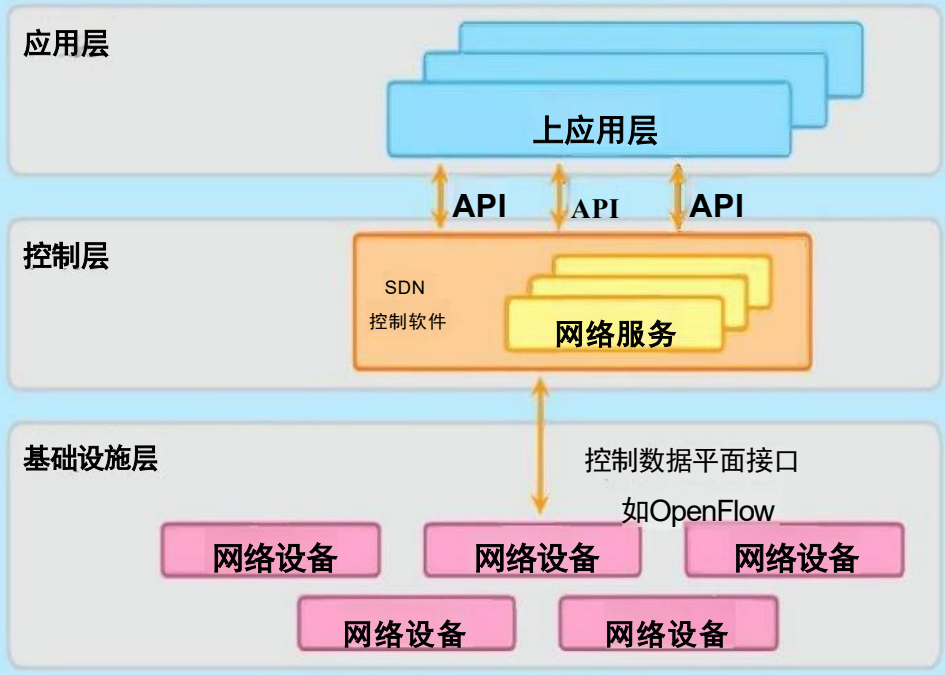
用户&服务提供商



砖厂老板们&村委会

傻大妞(SDN)　肯定是很受欢迎的选择

SDN架构图

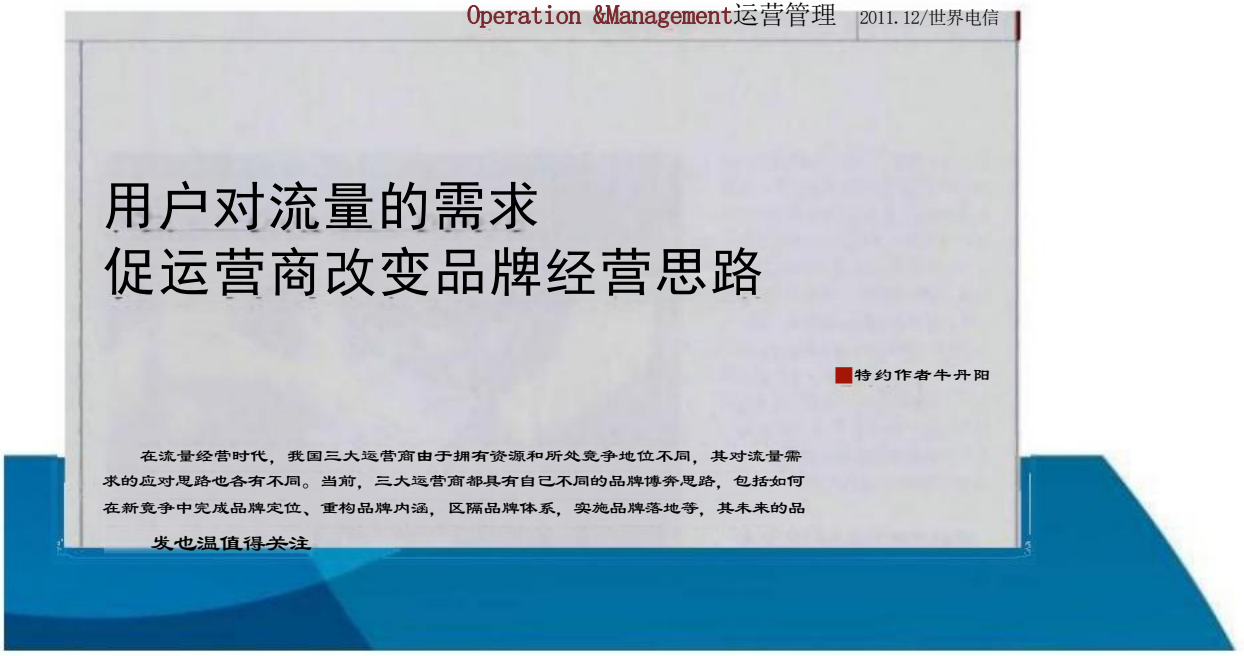


API：应用程序编程接口

肯定是未来网络的趋势

SDN 提出的背景

传统网络的层次结构是互联网取得巨大成功的关键.但是随着网络规模的不断扩大，封闭的网络设备内置了过多的复杂协议，增加了运营商定制优化网络的难度，科研人员无法在真实环境中规模部署新协议.同时，互联网流量的快速增长(到**2018** 年，全球流量将达到1.6×1021字节),用户对流量的需求不断扩大，各种新型服务不断出现，增加了网络运维成本.



当前网络的需求

1　动态的流量模式
2　“IT消费化”
3云服务的兴起
4　“大数据”意味着大带宽

从云到雾再到边缘计算，未来万物互联的世界里，爆炸性增长的流量、层出不穷的未知协议和加密技术，都会导致流量模式的动态性和不确定性。

当前网络的局限

1　复杂性

离散的孤立的协议集导致网络复杂

虚拟化服务器增加网络复杂性

基于IP的网络需要动态配置

2不一致的策略

接入方式、安全问题、QoS、其他鼓励移动用户的策略，可能会企业受到安全漏洞或者其他负面影响。

当前网络的局限

3　无法扩展

流量模式是动态的，不可预测的

计算单元的爆炸式增长、节点间的数据交换达**PB**级

需要满足不同的应用和性能需求

4　对供应商的依赖

供应商的设备生产周期影响网络部署

开放的接口，缺乏标准

即100万GB空间。
(1PB=1000TB,
1TB=1000GB,
1GB=1000MB)

思考：用户业务对网络的需求有哪些?

查资料，分组讨论

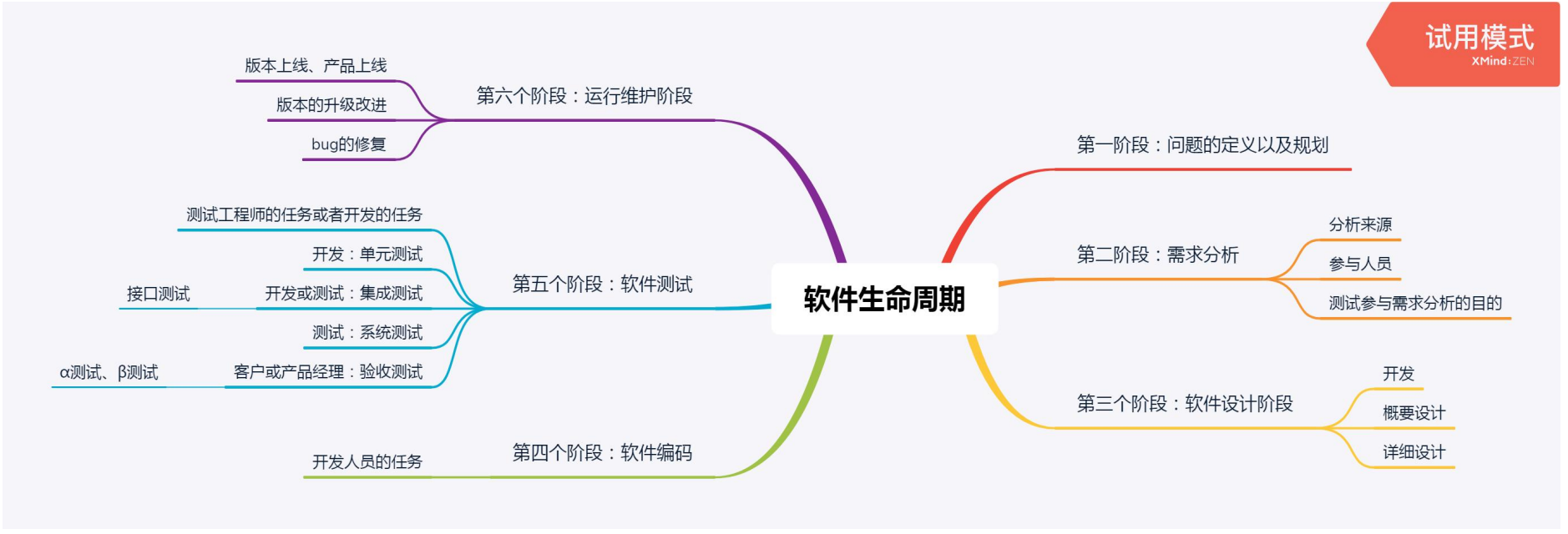
重点:

- 软件生命周期
- 网络生命周期
- 思考生命周期角度下的网络痛点
- 了解SDN基本网络结构、每一层的基本功能
- 了解openflow及流表基本概念
- 新技术思考（开放、自愿）

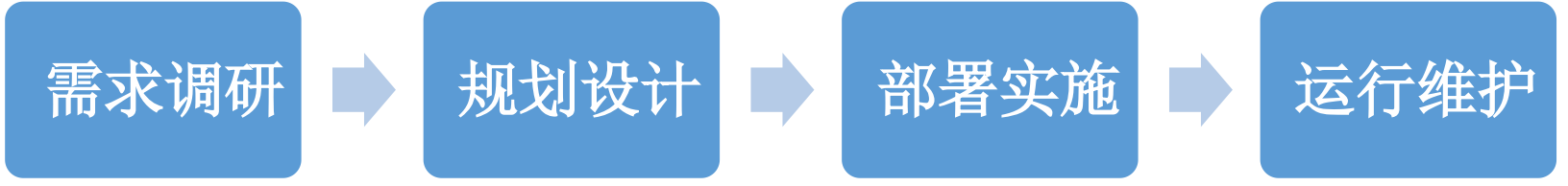
研究方向
协议与演进
频率技术
空口技术
基站技术
天线技术
mMTC 大连接技术
ISAC 感知融合技术
uRLLC 低时延技术
RTBC宽带 实时交互技术
eMBB 大带宽技术
UCBC上行 超带宽技术
无源物联网技术P-IoT
中高速物联网技术
RedCap
通感一体技术
融合定位技术
R17定位增强技术
XR/元宇宙
双发选收技术
低BLER技术
预调度技术
大上行技术
T+F超级上行技术
灵活频谱利用技术FSA
2CC载波聚合技术
频率云化 技术DSS
频率组合 技术
切片技术
毫米波技术
太赫兹
轨道角动量
虚拟大载波技术
灵活带宽管理 技术BWP
软切片技术
硬切片技术
调制方式技术
多址OFMDA 技术
无线子帧配比 技术
信道编码技术
干扰抑制技术
语音编码技术
精准授时
空天地 一体化 组网 技术
6G 白盒 技术
5G LAN 技术
基带 技术
射频 技术
应急 覆盖 技术
固移 融合 技术
VRAN
ORAN
4T4R 双拼 技术
地停 新方案 Lightsite 3.0
Meta AAU
应急 背包 基站
ONU+ Femto
多列阵天线技术
自动可调天线技术
波束自动寻优技术
海洋超远覆盖技术
超表面技术
赋型天线技术

软件产品生命周期?

计划\分析\设计\实现\测试\运维


试用模式
XMind:ZEN
软件生命周期
第一阶段：问题的定义以及规划
第二阶段：需求分析
分析来源
参与人员
测试参与需求分析的目的
第三个阶段：软件设计阶段
开发
概要设计
详细设计
第四个阶段：软件编码
开发人员的任务
第五个阶段：软件测试
测试工程师的任务或者开发的任务
开发：单元测试
开发或测试：集成测试
接口测试
测试：系统测试
客户或产品经理：验收测试
α测试、β测试
第六个阶段：运行维护阶段
版本上线、产品上线
版本的升级改进
bug的修复

网络系统的生命周期



网络系统的生命周期

需求调研

互连范围、互连规模

拟承载的应用

- 对象
- 类型
- 用户规模
- 实现模式
- 服务质量
- 安全要求
- ⋮

规划设计

网络架构与拓扑

IP和路由

安全和Qos策略

体现一定的冗余

根据规划与设计方案进行落地
部署实施
设备
上架
配置
连通
系统
集成
上线
测试

运行维护

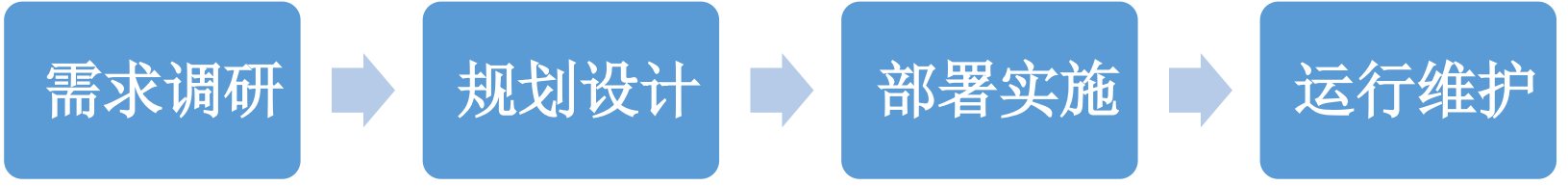
网络状态的监控与管理

- 设备
- 用户
- 流量
- 应用

[发现并排除网络故障与隐患，
确保网络系统可靠有效的运行。]

☞不能再满足扩增或变化的应用需求而需要升级或被取代

网络系统的生命周期



- 需求相对明确（应用数量、每项应用的对象、类型、规模、服务质 量），应用与业务变化较小或可预期
- 基于上述需求特征的规划与设计（拓扑、IP、路由、安全、QoS)
- 强调冗余设计，为可扩展性与业务弹性留下一定的空间
- 基于设备可配置、但不可编程的部署与实施（上架、配置、测试）
- 网络运维的业务弹性响应“捉襟见肘” “应接不暇”

传统网络的局限性（痛点）

传统网络的设计基础

- 需求相对明确与稳定（应用数量、每项应用的对象、类型、规模、服务质量），应用与业务变化较小或可预期
- 基于上述需求特征的规划与设计（拓扑、IP 、路由、安全、QoS）
- 强调冗余设计，为可扩展性与业务弹性留下一定的空间

传统网络面临的挑战

- 多元应用与业务层出不穷
- 相对稳定的规划与部署赶不上快速变化的应用与业务需求
- 分布式的架构、部署与管理
- 厂商和设备的多样性
- 业务弹性响应“捉襟见肘”“应接不暇”

云数据中心网络激发了痛点

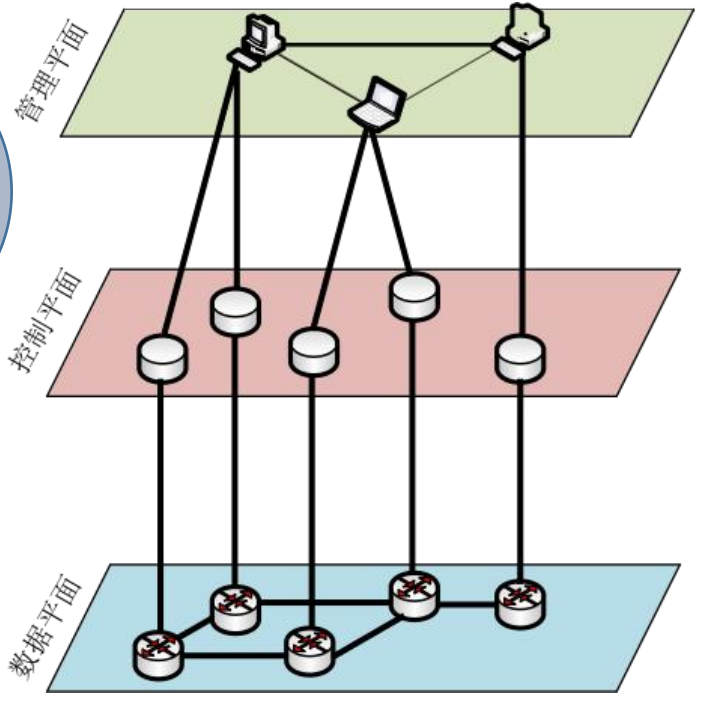
- **动态**：全自动化部署、适配频繁的调整；
- **按需**：各种价位的网络计算资源（存储、带宽）；
- **可靠**：云主机自动恢复能力、可迁移；
- **隔离**：用户之间/内部隔离，私有网络与共享网络。
- **实现**：NFV功能、VPC服务

问题的根源何在？！

- 传统网络的只可配置、不可编程
- 数据平面和控制平面的紧耦合

网络设备的三个功能平面

控制平面：根据不同的协议计算生成各种表项，如指导二层报文转发的MAC表、指导三层报文转发的路由表、指导MPLS报文转发的标签表、决定包是否被转发或过滤的访问控制列表等；



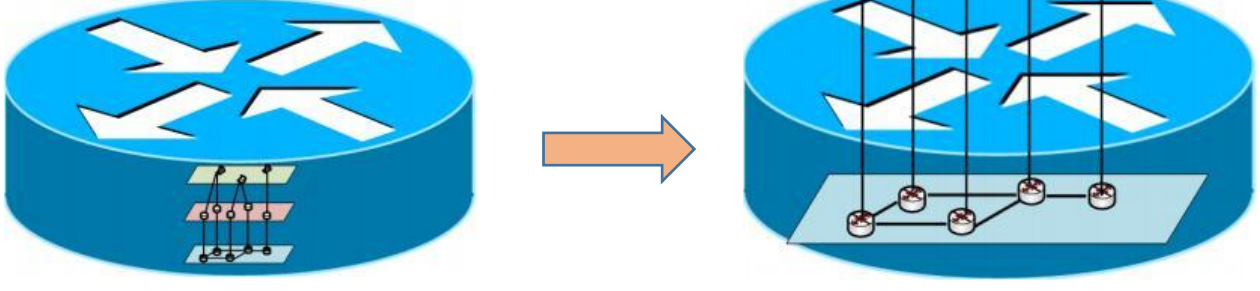
管理平面：负责设备的配置与管理；

数据平面：负责具体报文的转发。

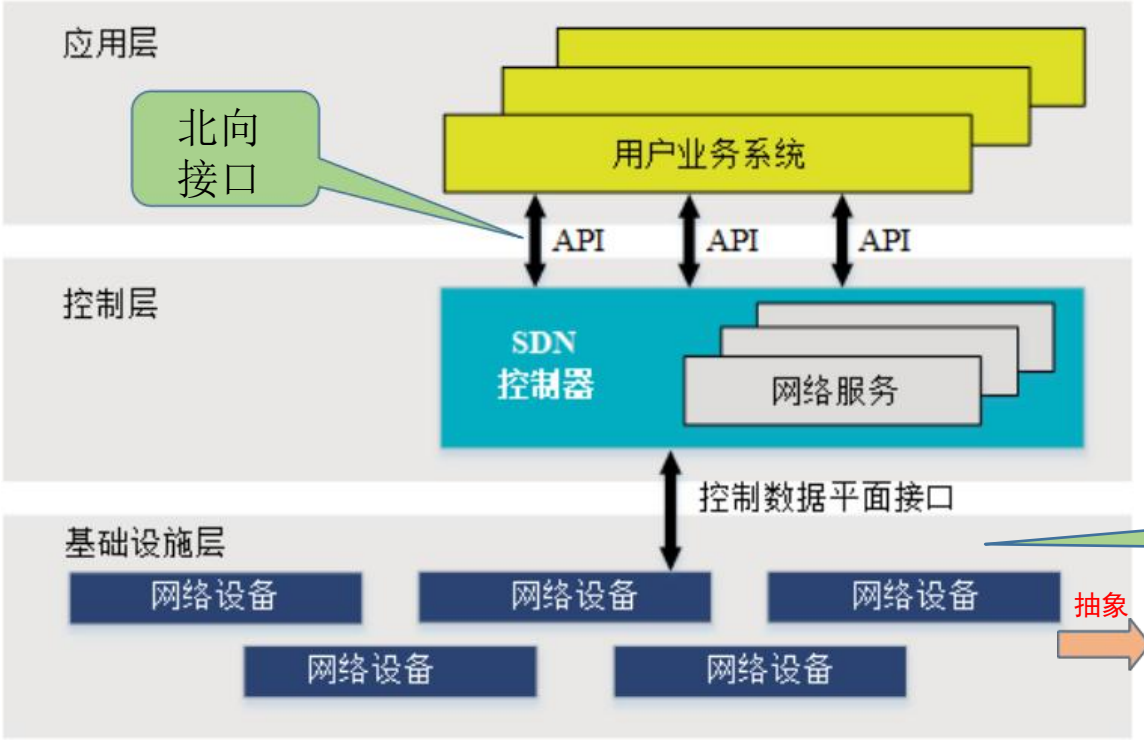
图1.2　网络设备的三个功能平面

解决问题的出路

- ✓ 新的网络架构、新的网络设计
- ✓ 能够弹性响应上层应用的网络可编程
- ✓ 网络设备数据平面和控制平面的解耦



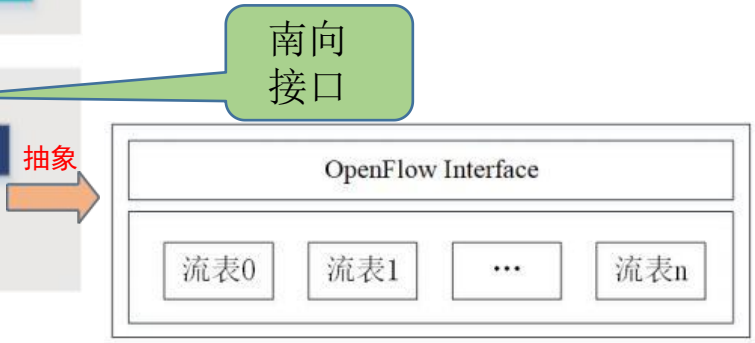
SDN的三层网络架构



. 解耦：数据平面与控制平面的分离；

. 抽象：网络设备抽象为通用的转发模型，将MAC地址表、路由表等抽象成“流表”；

. 可编程：提供开放的可编程接口（北向接口、南向接口）



SDN的起源

2005年，美国国家自然科学基金会（National Science Foundation, NSF）资助启动全球网络创新实验环境（Global Environment Networking Innovations, GENI）计划；

2006年，斯坦福大学主导，联合NSF及多个工业界厂商共同启动Clean-Slate Design for the Internet(Clean-Slate)项目。研究方向包括：网络体系结构，异构应用，异构物理层技术、安全、经济和政策。

TheMckeown Group

SDN师祖：Nick（Name)

OpenFlow: Enabling Innovation in Campus Networks

Nick McKeown
Stanford University

Tom Anderson
University of Washington

Hari Balakrishnan
MIT

Guru Parulkar
Stanford University

Larry Peterson
Princeton University

Jennifer Rexford
Princeton University

Scott Shenker
University of California, Berkeley

Jonathan Turner
Washington University in St. Louis

This article is an editorial note submitted to CCR. It has NOT been peer reviewed.
Authors take full responsibility for this article's technical content.
Comments can be posted through CCR Online.

ABSTRACT

This whitepaper proposes OpenFlow: a way for researchers to run experimental protocols in the networks they use every day. OpenFlow is based on an Ethernet switch, with an internal flow-table, and a standardized interface to add and remove flow entries. Our goal is to encourage networking vendors to add OpenFlow to their switch products for deployment in college campus backbones and wiring closets. We believe that OpenFlow is a pragmatic compromise: on one hand, it allows researchers to run experiments on heterogeneous switches in a uniform way at line-rate and with high port-density; while on the other hand, vendors do not need to expose the internal workings of their switches. In addition to allowing researchers to evaluate their ideas in real-world to experiment with production traffic, which have created an exceedingly high barrier to entry for new ideas. Today, there is almost no practical way to experiment with new network protocols (e.g., new routing protocols, or alternatives to IP) in sufficiently realistic settings (e.g., at scale carrying real traffic) to gain the confidence needed for their widespread deployment. The result is that most new ideas from the networking research community go untried and untested; hence the commonly held belief that the network infrastructure has "ossified".

Having recognized the problem, the networking community is hard at work developing programmable networks, such as GENI [1] a proposed nationwide research facility for experimenting with new network architectures and dis-

SDN之父:Martin

Ethane: Taking Control of the Enterprise

Martìn Casado, Michael J. Freedman,
Justin Pettit, Jianying Luo,
and Nick McKeown
Stanford University

Scott Shenker
U.C. Berkeley and ICSI

ABSTRACT

This paper presents Ethane, a new network architecture for the enterprise. Ethane allows managers to define a single network-wide fine-grain policy, and then enforces it directly. Ethane couples extremely simple flow-based Ethernet switches with a centralized controller that manages the admittance and routing of flows. While radical, this design is backwards-compatible with existing hosts and switches.

We have implemented Ethane in both hardware and software, supporting both wired and wireless hosts. Our operational Ethane network has supported over 300 hosts for the past four months in in Stanford University's network, and this deployment experience has significantly affected Ethane's design.

Categories and Subject Descriptors

C.2.6 [**Computer Communication Networks**]: Internetworking; C.2.1 [**Computer Communication Networks**]: Network Architecture and Design

downtime in multi-vendor networks comes from human-error and that 80% of IT budgets is spent on maintenance and operations [16].

There have been many attempts to make networks more manageable and more secure. One approach introduces proprietary middleboxes that can exert their control effectively only if placed at network choke-points. If traffic accidentally flows (or is maliciously diverted) around the middlebox, the network is no longer managed nor secure [25]. Another approach is to add functionality to existing networks—to provide tools for diagnosis, to offer controls for VLANs, access-control lists, and filters to isolate users, to instrument the routing and spanning tree algorithms to support better connectivity management, and then to collect packet traces to allow auditing. This can be done by adding a new layer of protocols, scripts, and applications [1, 10] that help automate configuration management in order to reduce the risk of errors. However, these solutions hide the complexity, not reduce it. And they have to be constantly maintained to support the rapidly changing and often proprietary management interfaces exported by the managed elements.

SDN大神-SDN的Uncle

- **Scott Shenker-加州大学伯克利分校教授，美国工程院院士**

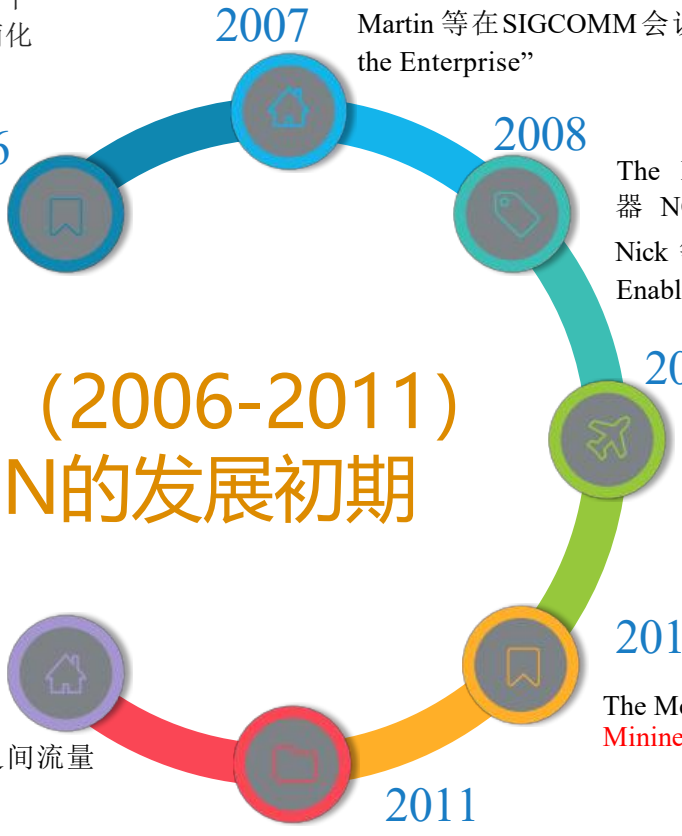
主要贡献:

- SDN开创者之一；
- OpenFlow奠基性论文的作者之一；
- ONF的联合创始人；
- Nicira公司联合创始人；
- 计算机科学界被引用次数最多的论文作者之一

重点:

- SDN的发展历史
- SDN的现状及趋势
- SDN的几个典型应用
- SDN的标准化组织

第一阶段（2006-2011）SDN的发展初期



2006

Martin领导Ethane项目， 目标是提出一个新型的企业网络架构，通过集中控制简化网络模型

2007

Martin联合Nick教授、Scott教授创建网络虚拟化技术创新公司Nicira

Martin 等在SIGCOMM会议上发表论文“Ethane:Taking control of the Enterprise”

2008

The McKeown Group发布了第一个开源SDN控制器 NOX-Classic

Nick 等在SIGCOMM会议上发表论文OpenFlow: Enabling Innovation in Campus Networks

2009

The McKeown Group发布首个基于Python语言的SDN控制器POX，同时发布了OpenFlow v1.0

SDN入选麻省理工科技评论的 “未来十大突破性技术”

2010

The McKeown Group发布开源SDN网络模拟器Mininet

2011

开放网络基金会（Open Networking Foundation, ONF）成立

Google发布应用SDN技术解决数据中心之间流量问题的方案B4

VMware收购Nicira，Juniper收购Contrail

思科发布SDN产品战略ONE

BigSwitch发布开源控制器Floodlight

日本NTT公司发布开源控制起Ryu

SDN的发展

- 第一阶段-SDN的发展初期（2006-2011）：提出了SDN的主要架构、设计思想和实现技术，属于学术界的研究与试验初创阶段，引起了产业界的关注；
- 第二阶段-SDN的发展中期（2012至今）：产业界开始普遍接受SDN的思想。SDN演变成以学院派、网络创新公司、传统网络巨头和运营商为代表的不同技术流派， 同时，出现了更多的SDN控制器、网络操作系统、商用产品和应用案例

第二阶段（2012至今）SDN的发展中期

2018

Cisco 、Apstra等推出基于意图的网络（Intent-based Network)
VMware 推出新的NSX Cloud和NSX SD-WAN
Juniper 推出Contrail SD-WAN
HPE Aruba推出SD-Branch

2017

VMware 宣布其NSX带来10亿美元销售额。
两家重要的SD-WAN创业企业相继被收购。Cisco收购Viptela， VMware收购VeloCloud

2016

开放网络基金会(ONF)与开放网络实验室(ON.Lab)宣布进行合并。
主 推CORD和ONOS
思科发布了全数字化网络架构-DNA

2015

SDN在产业界加快落地，软件定义广域网(SD-WAN)商用产品的出现，使得SDN的实际应用场景不在局 限于数据中心网络

中国联通发布新一代网络架构CUBE-Net2.0白皮书

2014

由斯坦福大学和加州伯克利分校SDN先驱共同成立ON.Lab，推出SDN 开放网络操作系统(ONOS）
Facebook在OCP项目中开放发布自己的Wedge交换机设计细节
Juniper、戴尔、惠普等产商发布支持SDN的White Box交换机
The McKeown Group发布数据平面的高级编程语言框架(P4)
中国电信与华为联合宣布双方合作完成了全球首个运营商SDN商用部署

2013

美国AT&T、英国电信、德国电信等电信网络运营商提出了网络功能虚拟化(NFV)的概念
AT&T启动Domain2.0项目
Cisco 、 Juniper 等 网 络 巨 头 发 起 开 源 项 目 OpenDaylight(ODL)
Cisco发布以应用为中心的基础设施 ACI

SDN发展现状与趋势

. 关注热度上升： 学术界&产业界互为促进

. 应用范围扩大： SD-DC→SD-X（WAN、云&雾、Access, ）

. 市场规模增长

. 技术融合加速

学术界

国外：斯坦福大学（logo）、普林斯顿大学、康奈尔大学等；

国内：清华大学牵头，中科院计算所、北邮、东南大学、北京大学等单位参与开展了类SDN思想的”未来网络体系结构和创新环境”863项目研究。

康奈尔大学

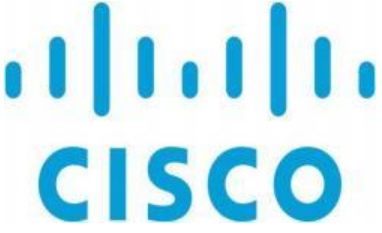
斯坦福大学

普林斯顿大

产业界

以Nicira 和Big Switch为代表的SDN 创业公司不断涌现；

Cisco 、Juniper、HPE、 NEC 、ARISTA、IBM 等厂商先后发布了支持OpenFlow 的SDN 交换机；自主或联合开发SDN控制器；

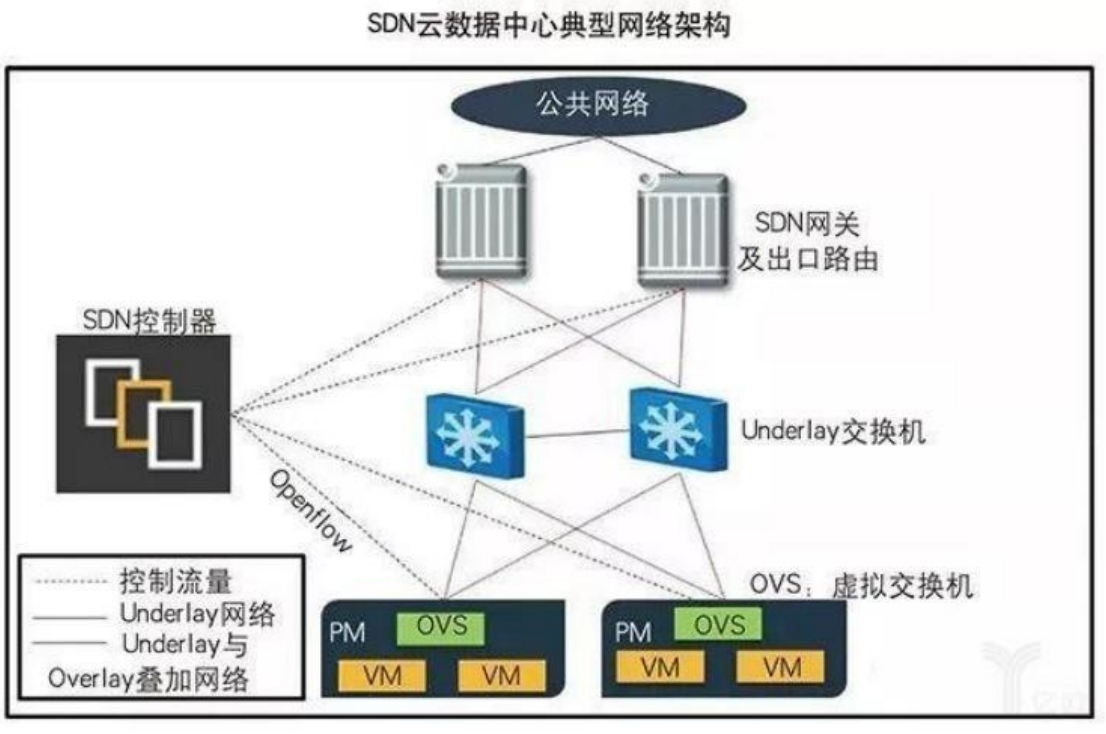


国内的华为、中兴、盛科等设备厂商，移动、联通、电信等运营商，也在SDN产品和解决方案研发上进展迅速。

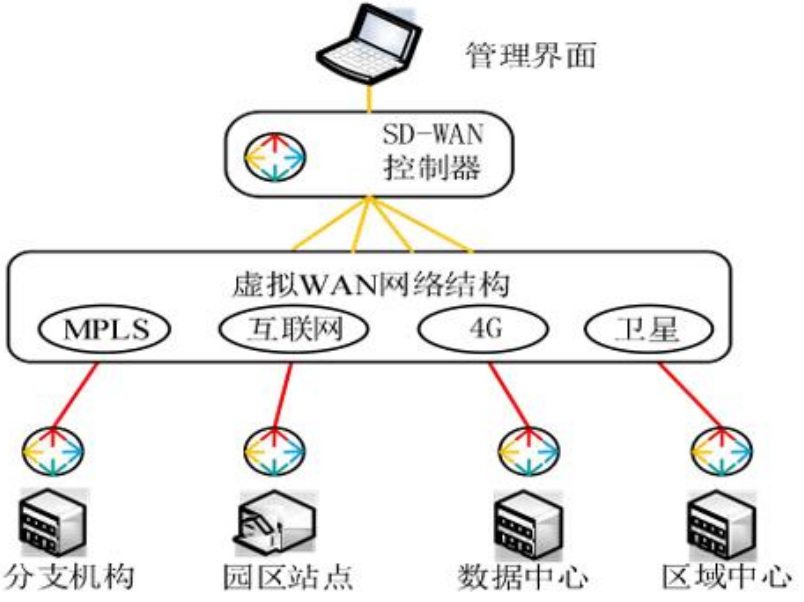
SDN发展现状与趋势

- 关注热度上升： 学术界&产业界互为促进
- 应用范围扩大： SD-DC→SD-X（WAN、云&雾、Access, ）
- 市场规模增长
- 技术融合加速

应用场景-数据中心（SD-DC）



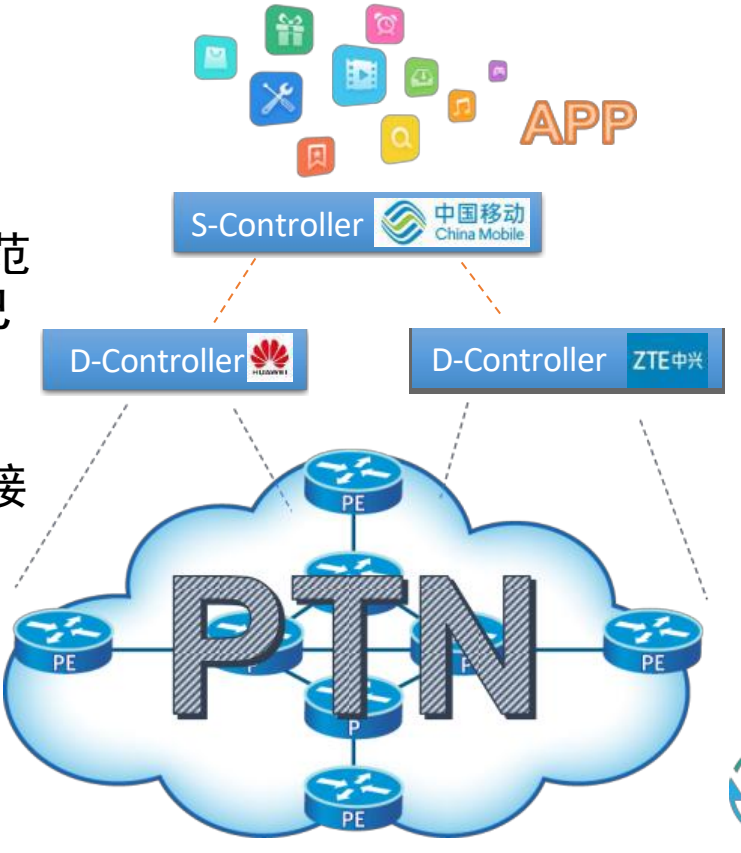
应用场景-广域网（SD-WAN）



一种面向融合S
D-WAN广...网

SD-WAN - 中国移动SPTN

- 保护投资，利用现有传统网络进行上层SDN化；
- 厂商各自基于中国移动提出的北向接口规范开发独立的D-Controller控制器，控制自己的传统设备；
- 中国移动独立自主完成主控制器的S-Controller的开发，利用S-Controller南向接口控制各厂家的D-Controller；
- S-Controller的北向接口提供API实现业务APP的开放接口；



Spirent Assured

应用场景-广域网安全（SD-WAN）

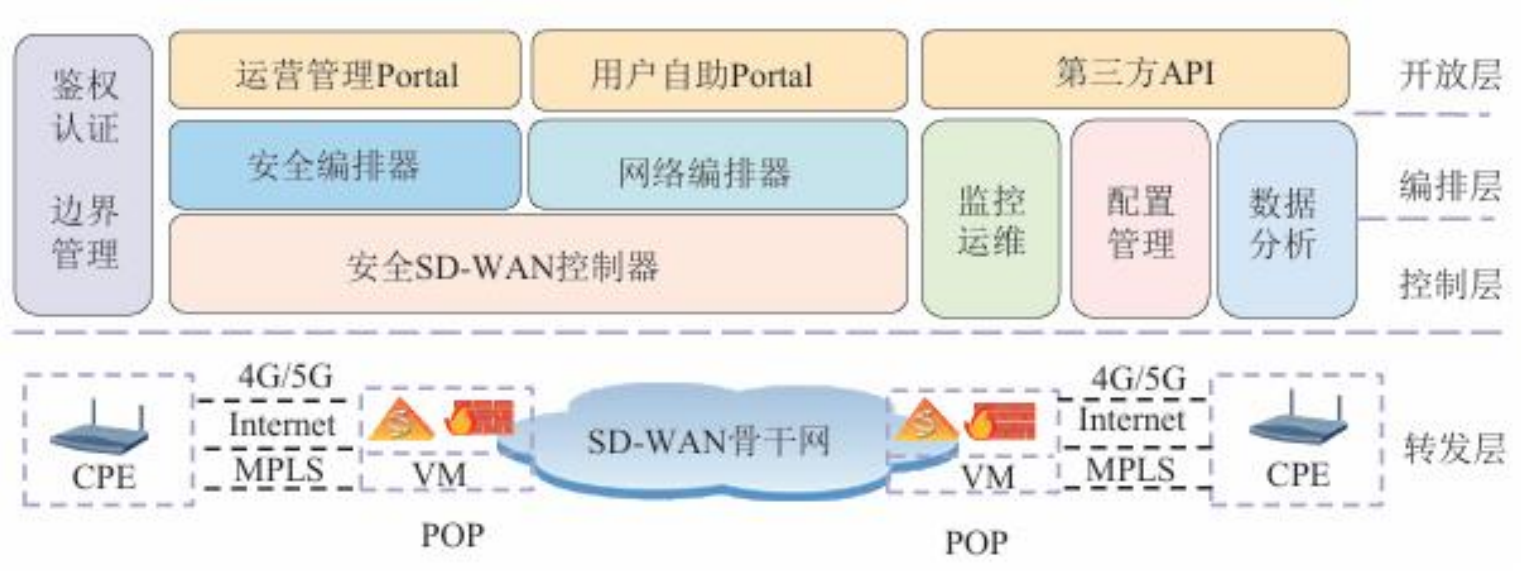


图2　安全SD-WAN的功能架构

应用场景-安全（SD-Security）

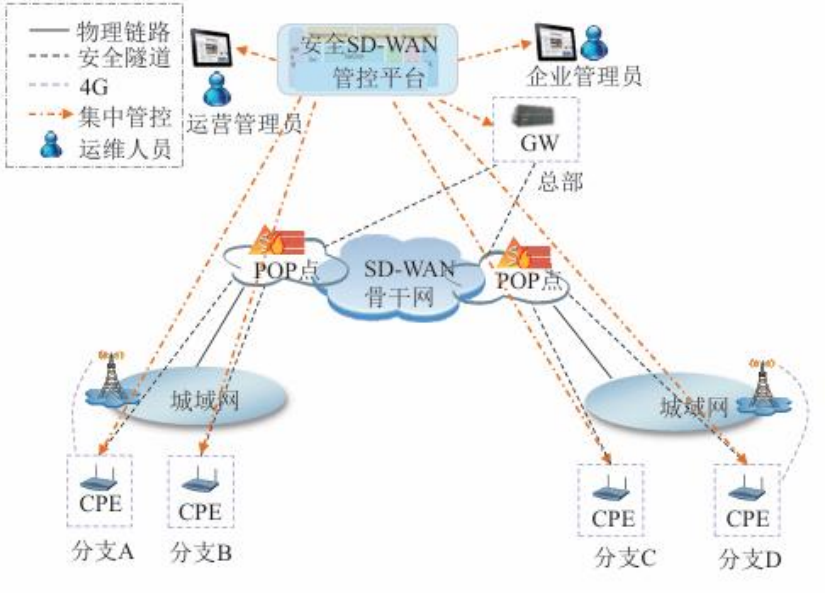


图3　大型企业的安全SD-WAN应用场景

安全SD-WAN
的架构与应用实

应用场景-接入（SD-Acess）

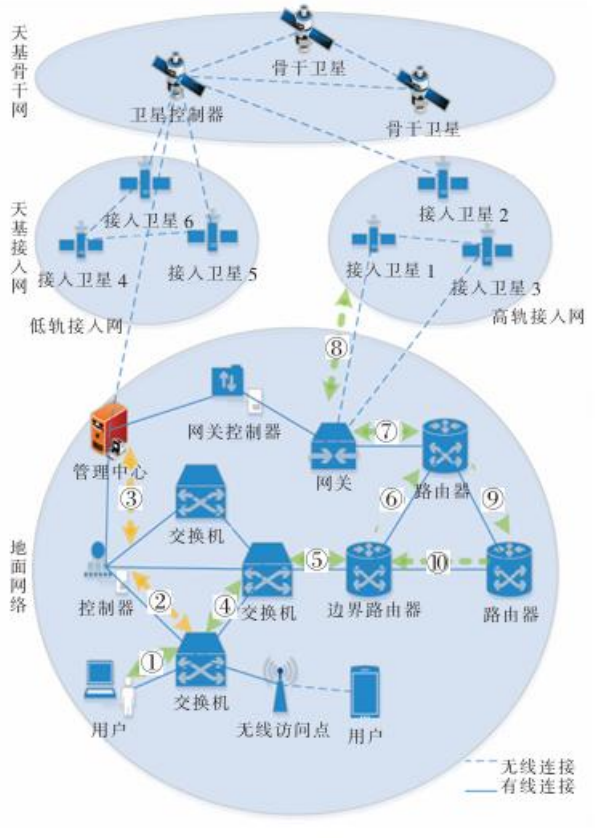
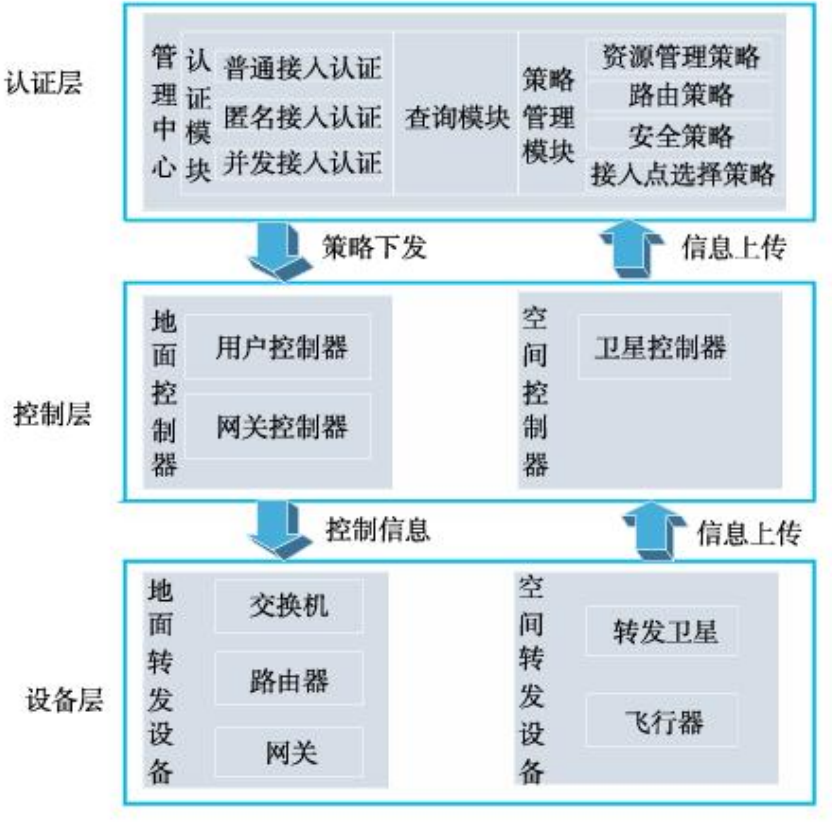


图 4　天地一体化信息网络认证架构拓扑示意图



软件定义天地一体化网络接入

SDN发展趋势

. 关注热度上升：学术界、产业界

. 应用范围扩大： SD-X（DC、WAN、云&雾、Access ）

. 市场规模快速增长

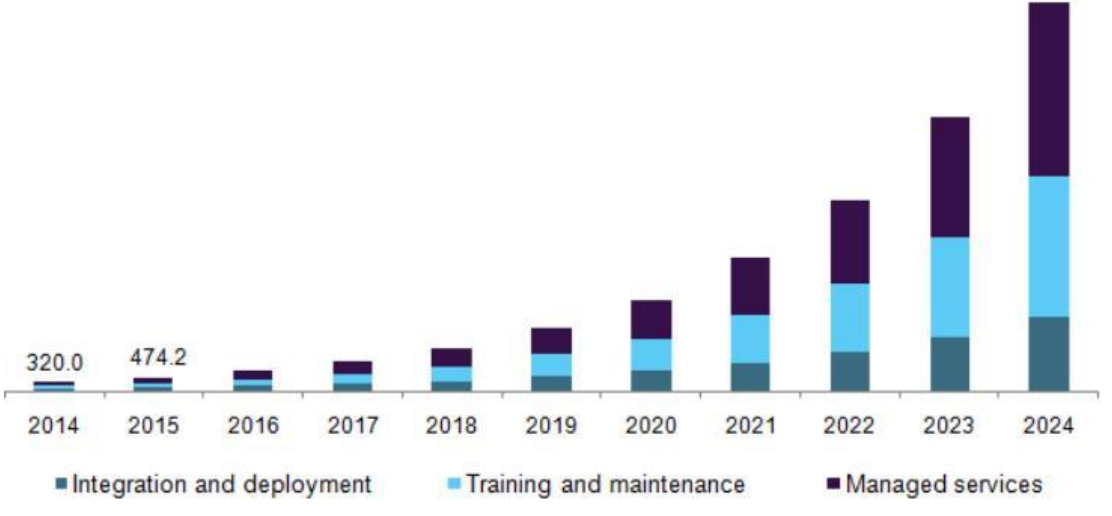
. 技术融合加速

市场占比提升（国内）

相关数据显示：2018年中国 SDN（软件）市场规模达到了13.9亿元。2019年市场规模将达到 20.5亿元，到 2021年，市场规模达到 46.7亿元。2023年,市场规模达到103亿元。

市场占比提升（国外）

Global software defined networking market, by services, 2014 - 2024 (USD Million)



SDN发展趋势

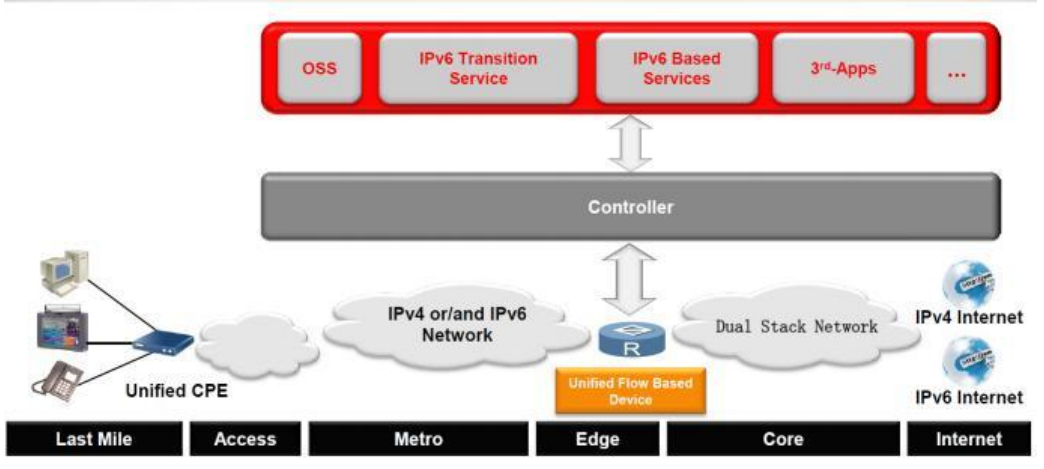
. 关注热度上升：学术界、产业界

. 应用范围扩大： SD-X（DC、WAN、云&雾、Access ）

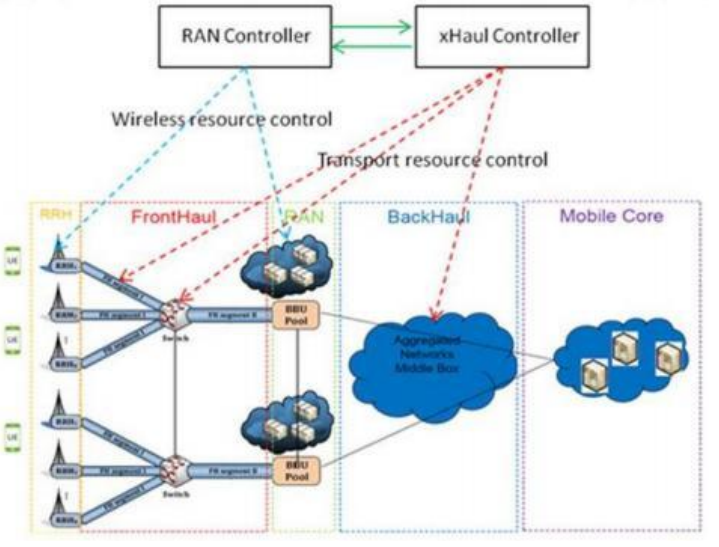
. 市场规模增长

. 技术融合加速

SDN与IPV6、5G加速融合

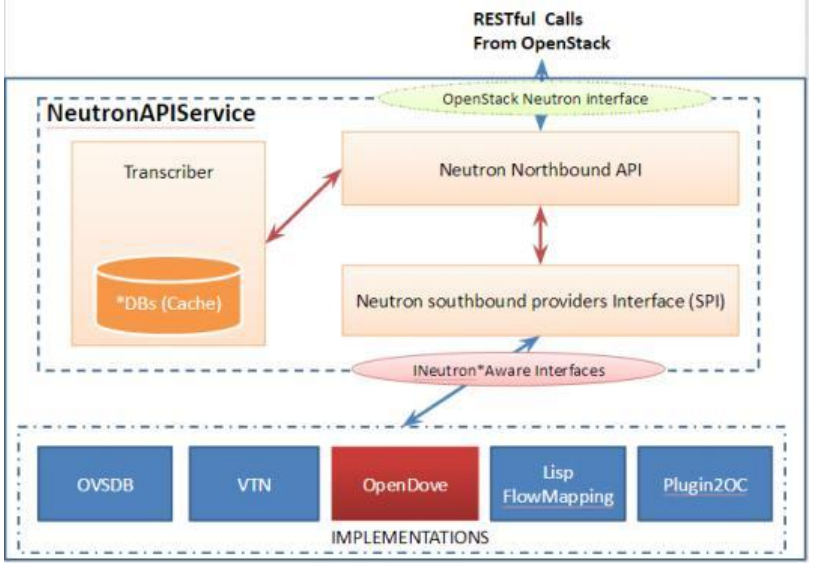


SDN与IPv6技术

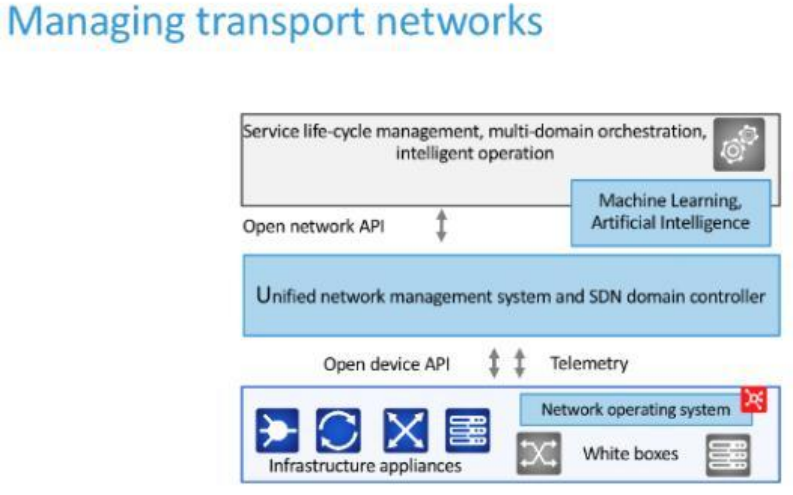


SDN与5G技术

SDN与OpenStack、人工智能加速融合



SDN与OpenStack技术



SDN与人工智能

网络人才新需求

I'm not sure I'd say there is or ever will be a 'SDN engineer'position," says Matthew P. Davy, director of InCNTRE and Indiana University's chief network architect. "But fewer network engineers will be able to live in a world in which they spend 100% of their time using a command

line interface [CLI] to configure network appliances. And there will be fewer network engineers who can perform their job while knowing little to nothing about servers, storage, hypervisors, system administration and scripting."

SDN的标准化组织

开放网络基金会（ONF）(2011)

因特网工程任务组（IETF）(1985)

国际电信联盟电信标准化部门（ITU-T）(1993)

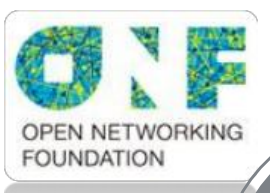
欧洲电信标准化协会（ETSI）(1988)

组织名称	组织性质	关注重点	标准制定方式	应用场景
ONF（Open Networking Foundation，开放网络基金会）	非盈利性机构，代表实际用户	专注于软件定义网络（SDN）的概念推广和技术创新，倡导将网络的控制平面与数据平面分离，推动 SDN 技术在实际网络中的应用	通过发布具有前瞻性的技术理念和规范，如 OpenFlow 协议，引领 SDN 技术发展方向。注重从用户需求出发，鼓励成员共同参与技术研发和标准制定	主要应用于企业网络、数据中心网络等，帮助企业实现更灵活、高效的网络管理和资源配置
IETF（Internet Engineering Task Force，互联网工程任务组）	公开性质的民间国际团体，汇集网络设计者、运营者、投资人和研究人员	聚焦互联网相关技术标准的研发和制定，涵盖从 IP 到通用应用之间的协议层，但不涉及传输硬件（由 IEEE 和 ITU 完成）以及特定应用层协议（如 HTML 和 XML 由 W3C 完成）	内部工作组依据不同研究课题组建，成员通过邮件组交流讨论，每年举行三次会议。产生的文件包括 Internet Draft（互联网草案）和 RFC（请求评论），RFC 更为正式且存档	广泛应用于整个互联网领域，其制定的标准是互联网技术的基础，如 TCP/IP 协议族等，对互联网的稳定运行和发展起到关键作用
ITU-T（International Telecommunication Union - Telecommunication Standardization Sector，国际电信联盟电信标准化部门）	全球性的电信组织，是 ITU 的下属部门	致力于电信领域的标准化工作，涵盖电信网络的各个方面，包括 SDN 的需求、架构、信令等标准的研究和制定	通过研究组开展项目研究，如 SG13 和 SG11 分别对 SDN 的不同方面进行研究。标准理论性较强，注重全球电信网络的互联互通和规范化	主要应用于电信运营商网络，为全球电信网络的建设、运营和管理提供统一标准，确保不同设备和系统之间的兼容性和互操作性
ETSI（European Telecommunication Standards Institute，欧洲电信标准协会）	由欧洲网络运营商组成	侧重于网络技术在运营商场景下的应用和改进，尤其是在 SDN 和 NFV（网络功能虚拟化）方面，希望通过制定标准增强运营商在技术上的话语权	主要工作成果是 NFV 网络功能虚拟化白皮书，对 NFV 的定义、应用场景等进行描述。通过成员间的协作和协商，制定适合运营商需求的技术标准	主要在欧洲及部分全球运营商网络中应用，帮助运营商优化网络架构，实现网络功能的软件化和虚拟化，降低运营成本，提升服务质量

SDN的标准化组织

ONF

最为活跃、最具影响力的SDN标准化组织，SDN标准化和产业化**最主要的推动 力量**



IETF

互联网领域最具权威的技术标准化组织

由网络设备厂商主导，更关注SDN相关功能和技术如何在网络中实现的细节上

ITU-T

研究的SDN的场景对象、相关架构主要针对**运营商网络**的

ETSI

制定支持**虚拟功能硬件和软件基础设施**的要求和架构规范以及发展网络功能的指南

ONF的标准化

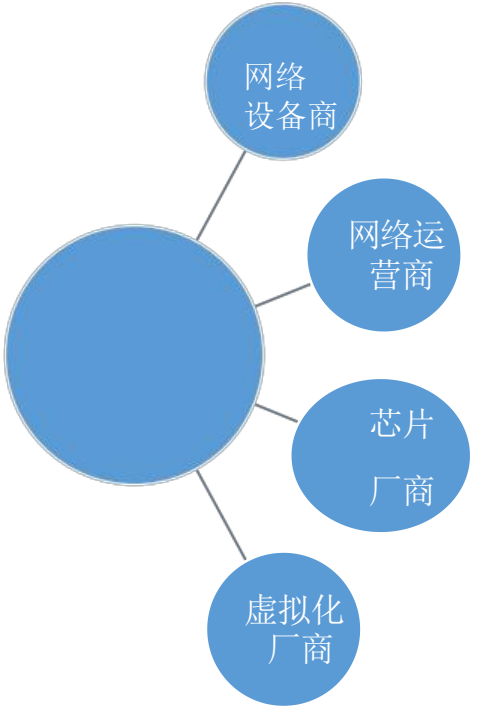
. ONF成立于2011年，是SDN标准化和产业化最主要的推动力量。

开放网络基金会作为仍在发展壮大中的非盈利组织，ONF已经拥有140多家会员，其共同使命是加速开放SDN的部署。ONF推广开放SDN和OpenFlow技术及标准，促进产品，服务，应用，客户和用户市场的发展。位于中国天地互连的“全球SDN测试认证中心（SDNCTC）”是ONF面向亚太地区的权威认证中心。

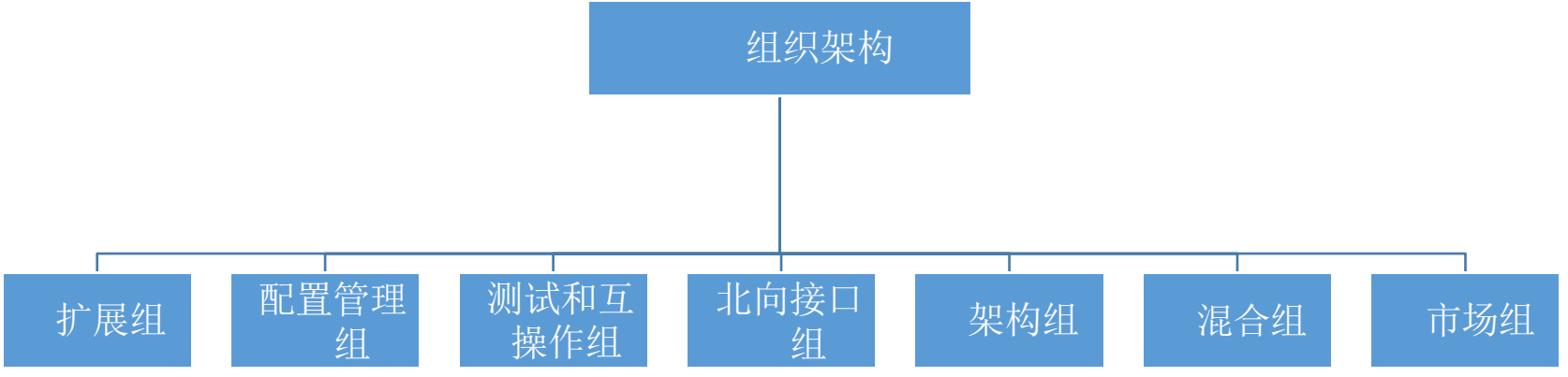
董事会成员

Google
Facebo
ok
Micros
oft
Yahoo
Verizon
德国电
信
NTT
高盛
OPEN NETWORKING
FOUNDATION

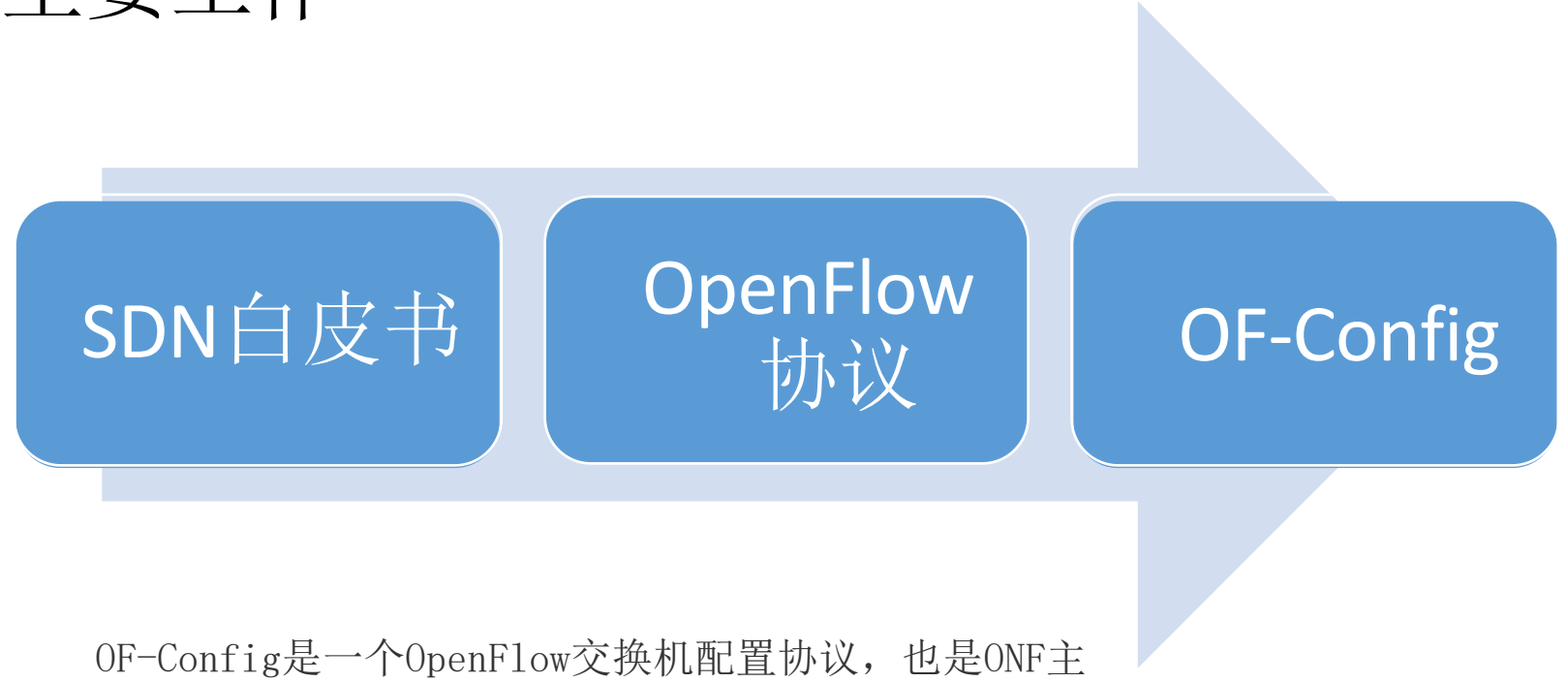
普通成员


网络
设备商
网络运
营商
芯片
厂商
虚拟化
厂商

ONF的组织架构



主要工作



OF-Config是一个OpenFlow交换机配置协议，也是ONF主推的OpenFlow补充协议，很好的填补了OpenFlow协议之外的交换机运维配置等内容。它提供了开放接口用于控制和配置OpenFlow交换机，但不影响流表的内容和数据转发行为。

IETF的标准化

- IETF是全球互联网最具权威的技术标准化组织，负责互联网相关 技术规范的研发与制定
- 网络设备厂商主导，更关注SDN相关功能和技术如何在网络中实现的细节上；

IETF的主要任务是负责互联网相关技术标准的研发和制定，是国际互联网业界具有一定权威的网络相关技术研究团体。IETF大量的技术性工作均由其内部的各种工作组（WorkingGroup，简称WG）承担和完成。这些工作组依据各项不同类别的研究课题而组建。在成立工作组之前，先由一些研究人员通过邮件组自发地对某个专题展开研究，当研究较为成熟后，可以向IETF申请成立兴趣小组（birds of a feather，简称BOF）开展工作组筹备工作。筹备工作完成后，经过IETF上层研究认可后，即可成立工作组。工作组在IETF框架中展开专项研究，如路由、传输、安全等专项工作组，任何对此技术感兴趣的人都可以自由参加讨论，并提出自己的观点。各工作组有独立的邮件组，工作组成员内部通过邮件互通信息。IETF每年举行三次会议，规模均在千人以上。

IETF相关研究工作组

转发与控制分离 组（ForCES）

需求、框架、协议、转发单元模型、MIB等，已发布9个RFC文档

应用层流量优化 工作组（ALTO）

为应用层提供更多的网络信息，完成应用层的流量优化

路由领域公开的研究组（IRSWG）

研究I2RS的问题描述、需求、应用场景与架构模型等，在RFC7921中定义了I2RS支持的SDN系统架构

ITU-T的标准化

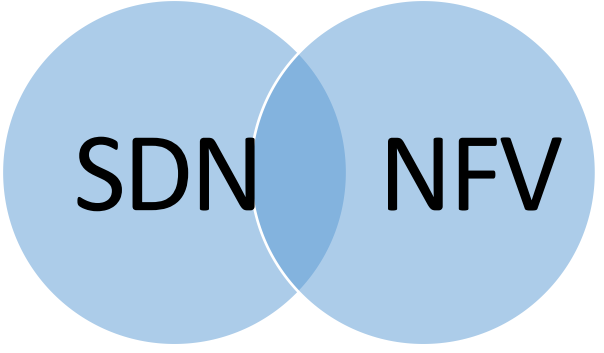
. ITU-T提出了SDN的实现架构。研究的SDN的场景对象、相关架构 主要针对运营商网络的。

ITU-T 在 SG13 组，即包含移动下一代网的未来网络（future networks including mobile and NGN）工作组，设立了 SDN 的研究任务，相关工作在 WP5 组（future network）Q21 研究。

在 2012 年 2 月份的 ITU-T SG13 全会上，Q21 工作组成立了两个项目，即 Y.FNsdn-fm 和 Y.FNsdn，分别面向 SDN 的需求和框架。在 2012 年 11 月举行的 Q21 中间会议上[12]，重点讨论了架构文件 Y.FNsdn，会议认为 Y.FNsdn 需要加快进度，希望在 2013 年底报批该文稿。

ETSI的标准化

. 成立网络功能虚拟化标准工作组（NFV ISG），制定支持虚拟功能硬件和软件基础设施的要求和架构规范以及发展网络功能的指南。



AT&T、英国电信、德国电信、Orange、意大利电信、西班牙电信公司和 Verizon 联合其他 52 家网络运营商、电信设备供应商、IT 设备供应商以及技术供应商，组建成立了 ETSI 网络功能虚拟化行业规范工作组(ISG NFV)。该工作组的首次会议已于 2013 年 1 月 15-17 日在法国 Sophia Antipolis 成功举办，会上，来自 Verizon 通信的 Prodip Sen 博士被选为 ISG 的主席。网络功能的虚拟化旨在通过研究发展标准 IT 虚拟化技术，使得许多网络设备类型能够融入到符合行业标准的大量服务器、交换机和存储设备中去，进而解决这些问题。其中包括在一系列行业标准服务器硬件上运行的软件中执行网络功能。这里的软件可以根据需要在网络中的不同位置硬件上安装和卸载，不需要安装新的硬件设备。ETSI 网络功能虚拟化行业规范工作组(ISG NFV) 将制定支持这些虚拟功能硬件和软件基础设施的要求和架构规范，以及发展网络功能的指南。工作组的工作将视情况整合现有的虚拟化技术和标准，并与其他标准委员会正在开展的工作相配合。首批规范计划将于 2013 年底前完成。ETSI ISG NFV 面向全部 ETSI 成员和非 ETSI 成员单位开放。

目录

SDN 的背景及特性

SDN 的核心技术：OpenFlow

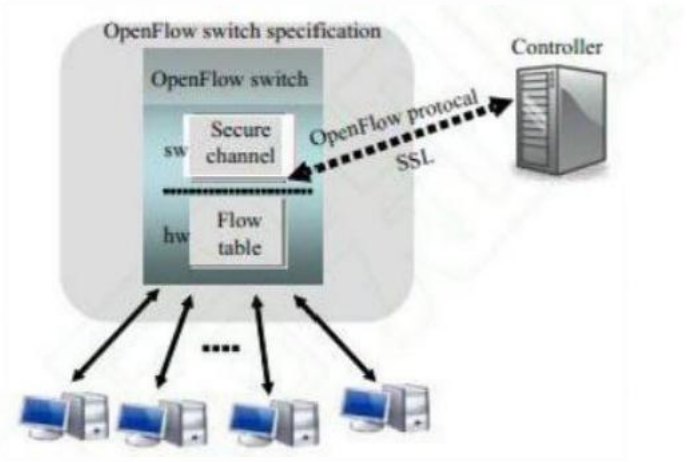
SDN目前的现状

SDN 未来的发展趋势

SDN的设计思想

SDN技术就相当于把每人家里路由器的的管理设置系统和路 由器剥离开，并且集中起来。以前我们每台路由器都有自己的管理系统，而有了SDN之后，一个管理系统可用在所有品牌的路由器上。如果说现在的网络系统是功能机，系统和硬件出厂时就被捆绑在一起，那么SDN 就是Android系统，可以在很多智能手机上安装、升级，同时还能安装更多更强大的手机App(SDN应用层部署)。

SDN 的核心技术：OpenFlow



OpenFlow 起源

OpenFlow由斯坦福大学和加州大学伯克利分校领导的大学联盟所发起，他们的初衷是让研究人员可将企业级以太网交换机作为定制构件用于大学的网络实验。

OpenFlow的核心思想很简单，就是将原本完全由交换机/路由器控制的数据包转发过程，转化为由OpenFlow交换机(OpenFlow Switch)和控制服务器(Controller)分别完成的独立过程。

SDN的核心技术：OPENFLOW

- 这种分布式的实现方式，让控制平面可以和转发平面物理的分离开，控制平面完全可以运行在运算能力更强的服务 器上，而不是路由器。OpenFlow 协议则是分离的两者之 间的通信协议。
- 这一点，使我们回忆起智能网的发展过程。在交换机实现程控用户，程控交换机不仅能完成电路交换的任务，还有许多其他的功能，如三方通话，闹钟，交换控制等等。但是，交换机在完成这些许多的附加功能用户，又显出任务过重的弊病。为此，把各种非交换的功能集中起来，用专门的计算机来实现，就形成了智能网。

SDN的核心技术：OPENFLOW

- 基于OpenFlow 为网络带来的可编程的特性，有研究人员提出了SDN即Software Defined Network(2009年)。在 这个概念中，网络中所有的网络设备被视为被管理的资源， 以此抽象出一个网络操作系统(Network OS)的概念；
- 这个网络操作系统一方面抽象了底层网络设备的具体细节，同时还为上层应用提供了统一的管理视图和编程接口。
- SDN 和OpenFlow 的关系，就像互联网之于TCP/IP协议栈。OpenFlow 是SDN的核心协议，用于控制转发面设备的控制转发，而SDN是上层的界面。

SDN 的核心技术：OPENFLOW

OpenFlow要解决的问题

现代大规模的网络环境十分复杂，给管理带来较大的难度。特别对于企业网络来说，管控需求繁多，应用、资源多样化，安全性、扩展性要求都特别高。因此，网络管理始终是研究的热点问题。对于传统网络来说，交换机等设备提供的可观测性和控制性都十分有限。一方面管理员难以实时获取足够的网络统计信息，另一方面控制手段十分单一，依赖于静态的policy部署。而基于软件定义网络，这两方面的问题几乎迎刃而解。

对于OpenFlow 协议，下一章要进行详细的讲解。

SDN的体系结构

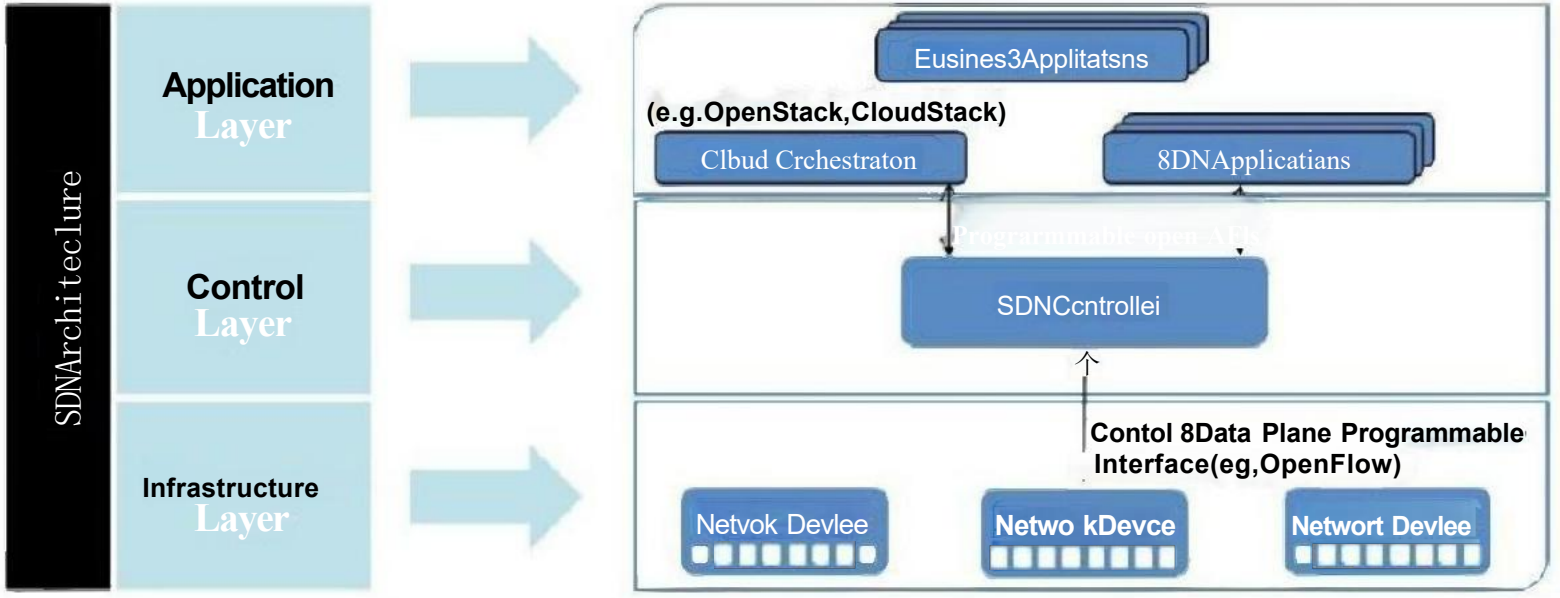
针对不同的需求，许多组织提出了相应的SDN参考架构。SDN 架构最先由ONF（开放网络基金会）组织提出，并已经成为 学术界和产业界普遍认可的架构. 除此之外，欧洲电信标准化组织(European telecommunications standards institute,简称ETSI)提出的NFV架构紧跟着发展起来，该体系结构主要针对运营商网络，并得到了业界的支持. 由各大设备厂商和软件公司共同提出了OpenDaylight，目的是为了具体实现SDN架构，以便用于实际部署.

SDN的体系结构：三个平面

- ONF 组织最初在白皮书中提到SDN体系结构，并于2013年底发布 最新版本，其架构如图1所示.SDN 由下到上(或称由南向北)分为 数据平面、控制平面和应用平面。
- 数据平面与控制平面之间利用SDN控制数据平面接□(control- data-plane interface,简称CDPl)进行通信，CDPI具有统一的 通信标准，目前主要采用OpenFlow 协议。
- 控制平面与应用平面之间由SDN 北向接口(northbound interface, 简称NBI)负责通信，NBI 允许用户按实际需求定制开发。

SDN 的体系架构

- 北向接口，为应用提供编程接口
- 南向接口，设备控制信令，控制设备的转发行为
- 控制器/Controller,对网络的抽象层，NOS 网络操作系统，屏蔽硬件，为应用开发提供开发接口；



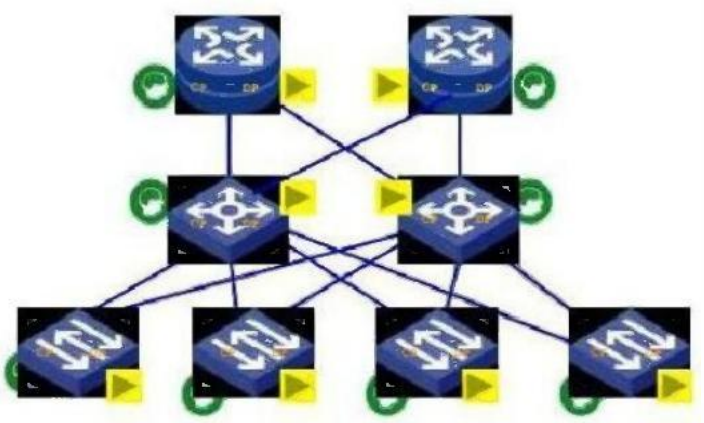
SDN 的三个最基本的特点

- 架构角度：控制平面与数据平面分离，逻辑集中管理(集中到控制器上)；
- 业务角度：通过对控制器，使低层网络被抽象出来网络资源被抽象成服务，实现了应用程序与网络设备的操作系统进行解耦和。应用看到的是网络服务。
- 运营角度：网络可以通过编程的方式来访问，从而实现应用程序对网络的直接影响，一些新型的接口，可以实现传统网络管理不能做到的网络优化。

1、控制转发分离

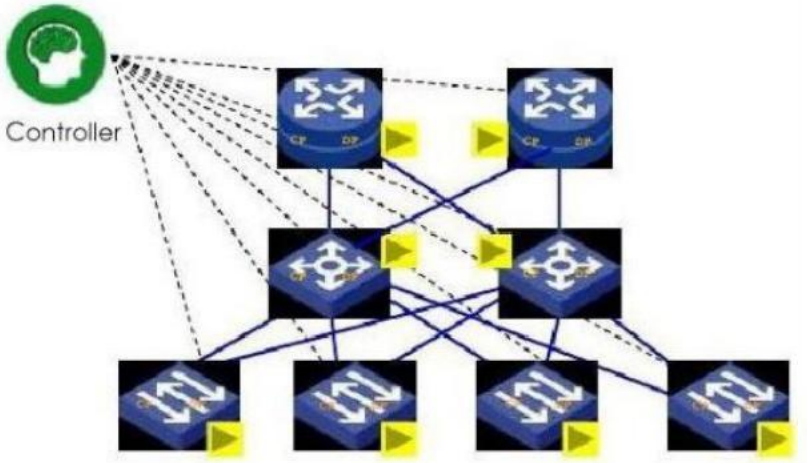
控制平面 (CP)　　转发平面 (DP)

- 传统网络设备的CP 与DP 不分离；
- 设备之间通过控制协议交互转发信息；



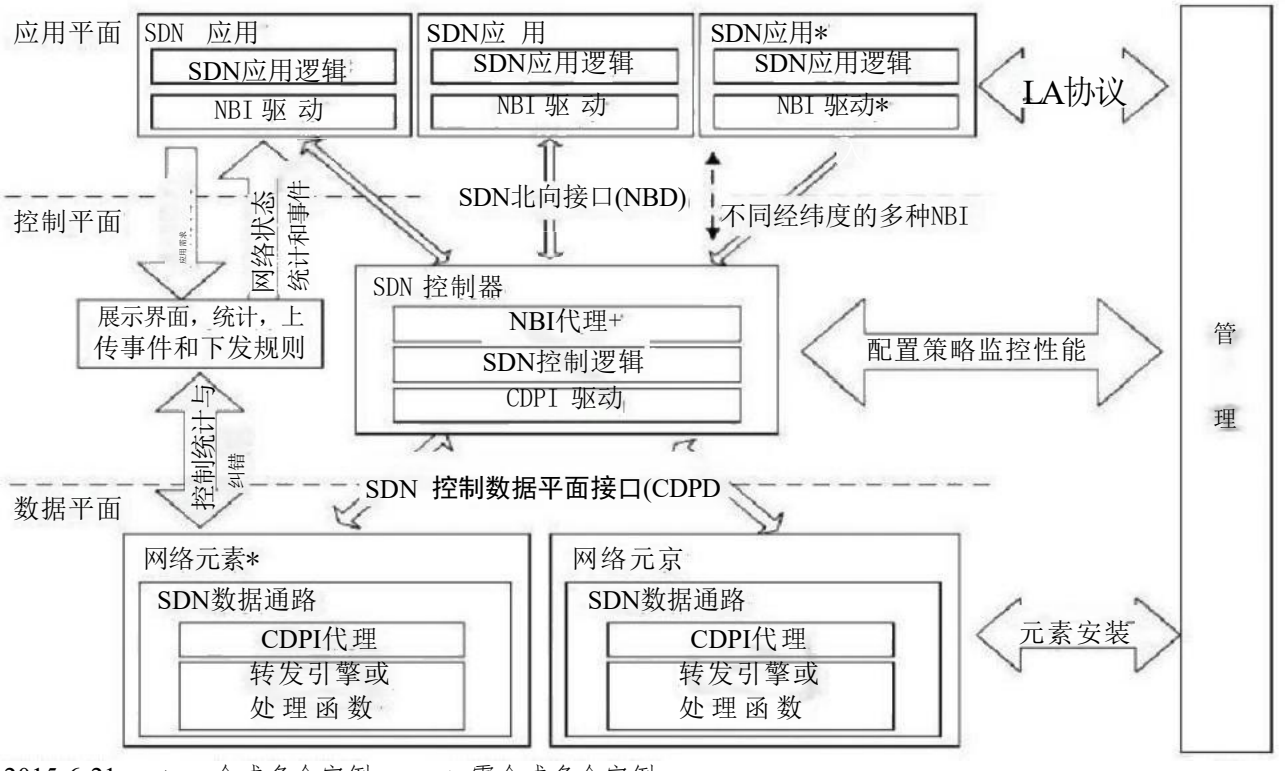
SDN 的思路是将网络设备的控制平面集中上收到Controller;

- 网络设备上指保留转发平面(转发表项)；
- 通过Controller实现网络统一部署和网络自动化；



7

图1 SDN 的体系结构



2015-6-21 +:一个或多个实例 *:零个或多个实例 30

SDN由下到上(或称由南向北)分为数据平面、控制平面和应用平面.

数据平面与控制平面之间利用SDN控制数据平面接口 (control-data-plane interface,简称CDPI)进行通信，CDPI具有统一的通信标准，目前主要采用OpenFlow 协议

控制平面与应用平面之间由 SDN 北向接口 (northbound interface,简称NBI)负责通信，NBI 允许用户按实际需求定制开发.

SDN的意义

现有网络中，对流量的控制和转发都依赖于网络设备实现，且设备中集成了与业务特性紧密结合的操作系统和专用硬件，这些操作系统和专用硬件都是各个厂家自己开发和设计的。而在SDN 网络中，网络设备只负责单纯的数据转发，可以采用通用的硬件；而原来负责控制的操作系统将替换为独立的网络操作系统，负责对不同业务特性进行适配，而且网络操作系统和业务特性以及硬件设备之间的通信都可以通过编程实现。

目录

SDN 的背景及特性

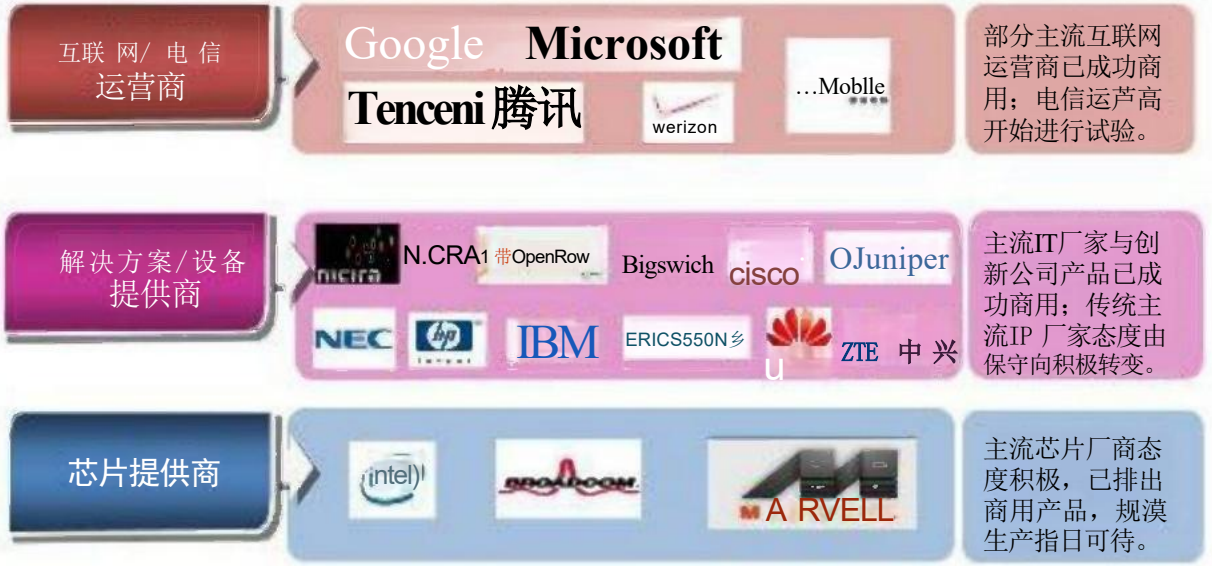
SDN 的核心技术：OpenFlow

SDN 目前的现状

SDN 未来的发展趋势

SDN产业现状

- SDN 已经在解决方案规划、软硬件支持等方面取得长足发展



- 2007年概念提出，斯坦福的教授Nick McKeown提出了Openflow，旨在构建实验的园区网
- 2009年12月，OpenFlow 规范发布了具有里程碑意义的可用于商业化产品的1.0版本
- 2011年Open Networking Fundation(ONF)成立，SDN 产业化程度不断提升
- 2012年4月，Google在ONF Summit 2012上宣布其内部骨干网上已实现SDN 全面部署
- 2012年7月23日，VMWare 宣布以12.6亿美元收购SDN 服务商Niclra Networks

中国SDN与开放网络高峰会议

中 国SDN 与开放网络高峰会议(第一届)

2012年12月6日：北京，中国

以“未来网络的演进之路”为主题的2012中国**SDN** 与开放网络高峰会议在北京隆重召开，本次峰会获得国际组织**ONF**（开放网络基金会）的大力支持，国内三大运营商研究机构齐聚本次SDN 与开放网络高峰会议。参加大会的还有海内外运营商、设备商及业界专家学者近500余人，共同探讨SDN、开放网络等相关热点。

2013中国SDN 与开放网络高峰会议(第二届)

2013年8月29日-30日：北京，中国

将于2013年8月29-30日在北京举办，众多国内外运营商、厂商及业界专家学者将云集于此，共同探讨SDN、开放网络等相关主题。2013中国SDN 与开放网络高峰会议由中国SDN 与开放网络高峰会议组委会主办，大会组委会由中国通信标准化协会、中国互联网协会、开放数据中心联盟、云计算开放联盟、中关村云计算联盟、中关村下一代互联网产业联盟、中国电信、清华大学等支持单位共同建立。

专注于SDN

开放网络的旗帜性峰会

TMT

2013中国SDN与开放网络高峰会议

China Open Networking &SDN Conference 2013

2013.08.29-08.30

中 北京实酒店

SDN目前应用分析

可用性	· 转发性能可满足数据交换容量、性能要求
建设成本	· 目前看，两年内专门用于openflow的交换机不会普遍生产 • Openflow 仍以传统设备额外支持的方式作为实现手段，因此openflow转发设备的成本应不会大幅低于现有设备，有可能还会更高 • Controller是SDN重要的组成部分，其成本比较昂贵
可维护性	· 对于拓扑发现、故障检测及倒换、各类告警须满足电信级要求 · 目 前SDN的拓扑发现、OAM协议尚未完善
可扩展性	· 节点规模可扩展，能与现有网络兼容
安全性	· 能够保证不同安全级别流量的隔离

SDN目前应用范围

分域的网络架构	• SDN应仅在同一个域内有效，如运营商某省网或数据中心内部、互联网络 • 不同域之间仍通过IP网络公有协议互通
流量调优的工具	• SDN利用应用层(如云平台管理软件、网管系统)和控制层之间的API接口，实现应用对网络的动态调度 • 通过应用层与控制层接口的开发，实现基于负载情况**自动的、精细的**流量调整
定制化需求快速实现的方案	• OpenFlow协议应用于控制与转发设备之间，使控制面和转发面分离 • 定制化需求的开发仅通过OpenFlow Controller升级支持，较现有方式，无需各厂家路由器、交换机设备逐一支持，开发周期缩短

SDN目前存在的问题

向题 一：
SDN 成熟了吗？

口控制层与基础设施层之间的接口(OpenFlow)缺乏拓扑发现机制，目前对隧道技术支持有限，扩展性问题未有实际验证；QoS、安全、DPI等方面还在研究中。

口目前有关SDN更多的研究和产品是基于OpenFlow 实现控制转发分离，但是SDN的根本价值在于应用层与控制层的“关联”，这部分的研发和标准化工作远不如OpenFlow 成熟。

向题二：
网管增强可以替代SDN?

口根据厂家调研，SDN 所解决的场景通过改造数据网管对多厂家的支持基本都能够做到，只是改造的效果和代价有待评估。

向题三：
SDN 是否可以平清演进?

口产业界目前采用最多的是IETF比较成熟的协议像BGP、ISIS、OSPF 等。

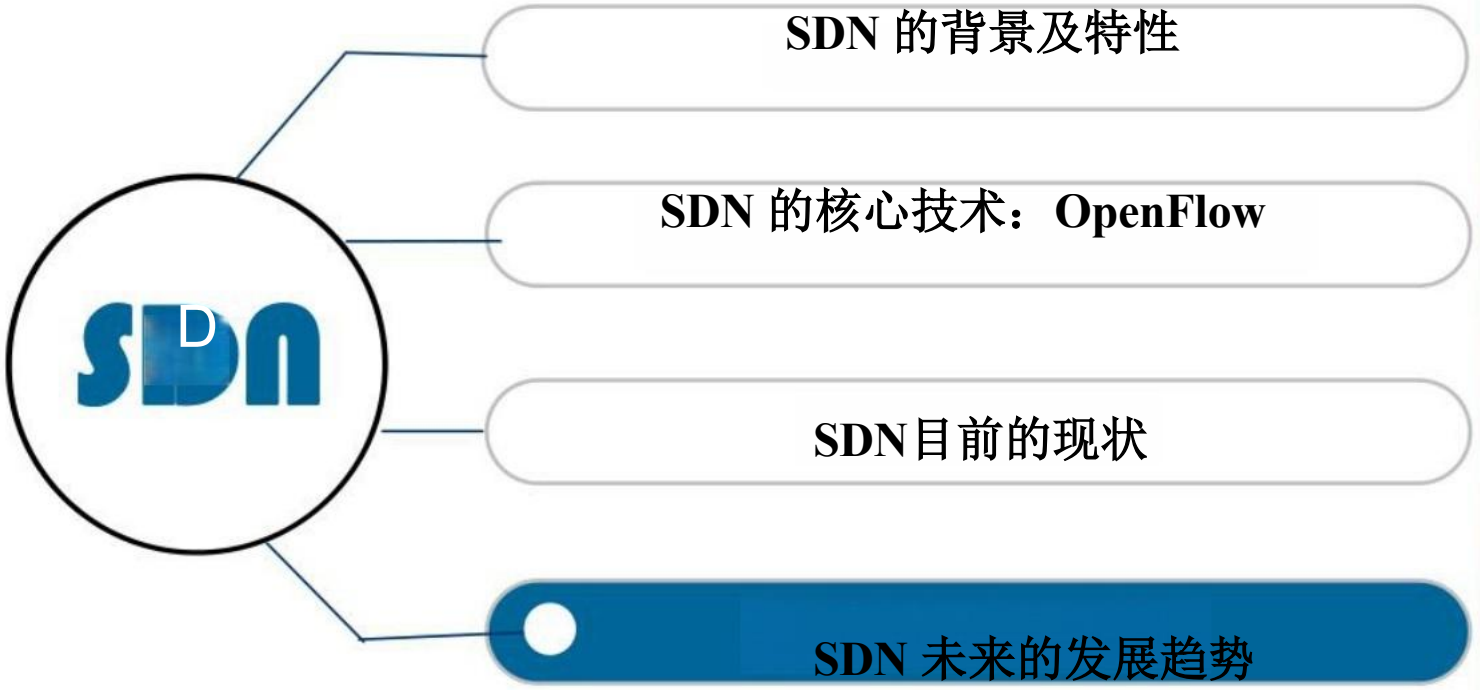
技术演进方向上存在未知

仍存替代性方案

技术标准仍在完善

结论：标准技术仍有缺陷，产业仍未完全成熟，技术发展方向正在收敛过程中。

目录



SDN 的未来趋势

目前还是SDN的发展初期。拥有强大IT 资源和顶尖人才 的企业(谷歌和Facebook等)正在利用SDN的优势。主流IT 企 业应该稍等一段时间，SDN 应用程序(管理脚本等)和最佳做 法将需要几年十年才会出现。SDN 技术正在迅速发展，并且 需要学习曲线，而大多数网络管理者和VAR/SI 通道所不具备 的。

SDN 将会改变网络架构、编程和管理的方式。网络将会 变得更具敏捷、灵活和节约成本。然而，像很多IT创新技术 一 样 ，SDN需要一些时间来发展。

SDN未来的市场

- 2023 年，中国 SDN 行业市场规模为 103.1 亿元，同比增长 45.01%。据贝哲斯咨询预测，到 2030 年全球软件定义网络市场规模将达到 1024.59 亿元，在预测期间年复合增长率预估为 44.6%。中国 SDN 市场规模也有望从 2025 年的约 480 亿元人民币增长至 2030 年的超 1200 亿元，年均复合增长率维持在 20% 以上。
- SDN 目前已广泛应用于数据中心、电信网络、园区网等领域，未来，SDN 技术将深度融合人工智能和机器学习技术，实现网络的智能化管理和自动化运维，加速与云计算、边缘计算的融合，进一步拓展在物联网和工业互联网等领域的应用场景。

明德新民 | 止于至善

第二章 SDN的基本原理

学院：软件学院 任课教师：申凌峰

河南大學

目 录

CONTENTS

- 01 SDN的定义
- 02 SDN的体系结构
- 03 SDN的核心设计思想

Part 01

SDN的定义

SDN的定义

关于SDN的定义，实际上并没有一个统一的标准答案。在SDN技术的发展过程中，曾从不同角度对SDN给出了不同的描述，归纳起来，可分为狭义SDN和广义SDN。

狭义SDN的定义是由开放网络基金会(ONF)提出的。ONF在2012年发布的SDN白皮书中指出，SDN是一种控制与转发分离且直接可编程的新兴网络架构，它是网络虚拟化的一种实现方式，通过核心技术OpenFlow协议将设备的控制平面与数据平面分离，以实现更加灵活的网络流量控制，使网络变得更加智能。该定义说明了SDN 网络架构的新特点，颠覆了传统网络的架构，但将 SDN实现技术局限于特定的协议OpenFlow，因此存在明显的狭隘性，不能为SDN的技术发展与实现提供更大的空间。

随着SDN的发展，人们在接受SDN核心设计思想的基础上，对SDN的认识不再局限于狭义SDN，逐渐形成了广义SDN的概念。与狭义SDN相比，广义SDN在网络架构、南向接口 (southbound interface) 协议和应用场景等方面都有更大的扩展。

SDN的定义

网络架构的多样性

不再局限于ONF提出的基于OpenFlow的三层架构，还出现了IETF提出的基于I2RS的SDN架构以及业界目前广泛应用的Overlay架构。

应用场景的扩大

从早期的软件定义数据中心，到后来的软件定义广域网、软件定义安全、软件定义接入、软件定义存储，软定义一切(software define anything) 的时代正在到来。

南向接口协议的扩展

除了OpenFlow协议之外，与不同的SDN网络架构相适应，新增了NETCONF、OVSDB、OpFlex、PCEP、XMPP、I2RS等多种选择。

未来发展趋势

伴随软件定义一切的发展，SDN的架构、协议与内涵还将进一步扩展与丰富。

SDN的定义

表 2.1　狭义 SDN 和广义 SDN 主要区别

主要区别	SDN 概念	
	狭义 SDN	广义 SDN
网络架构	ONF 基于 OpenFlow 三层架构	ONF 基于 OpenFlow 三层架构，IETF 基于 I2RS 的 SDN 架构，Overlay 架构
南向接口协议	OpenFlow	OpenFlow、NETCONF、OVSDB、OpFlex、PCEP、XMPP、I2RS 等
应用场景	软件定义数据中心	软件定义广域网、软件定义安全、软件定义接入、软件定义存储，软件定义一切

Part
02

SDN的体系架构

ONF定义的SDN架构

IETF定义的SDN架构

Overlay网络架构

SDN的体系架构-ONF定义的SDN架构

尽管在SDN发展过程中，网络架构有所变化与扩展，但ONF 2012 年在其发布的SDN白皮书中所给出的三层框架仍然是最基本的描述，该框架依据SDN功能进行逻辑分层，自上而下分别为应用层、控制层和基础设施层，如图2.1所示。

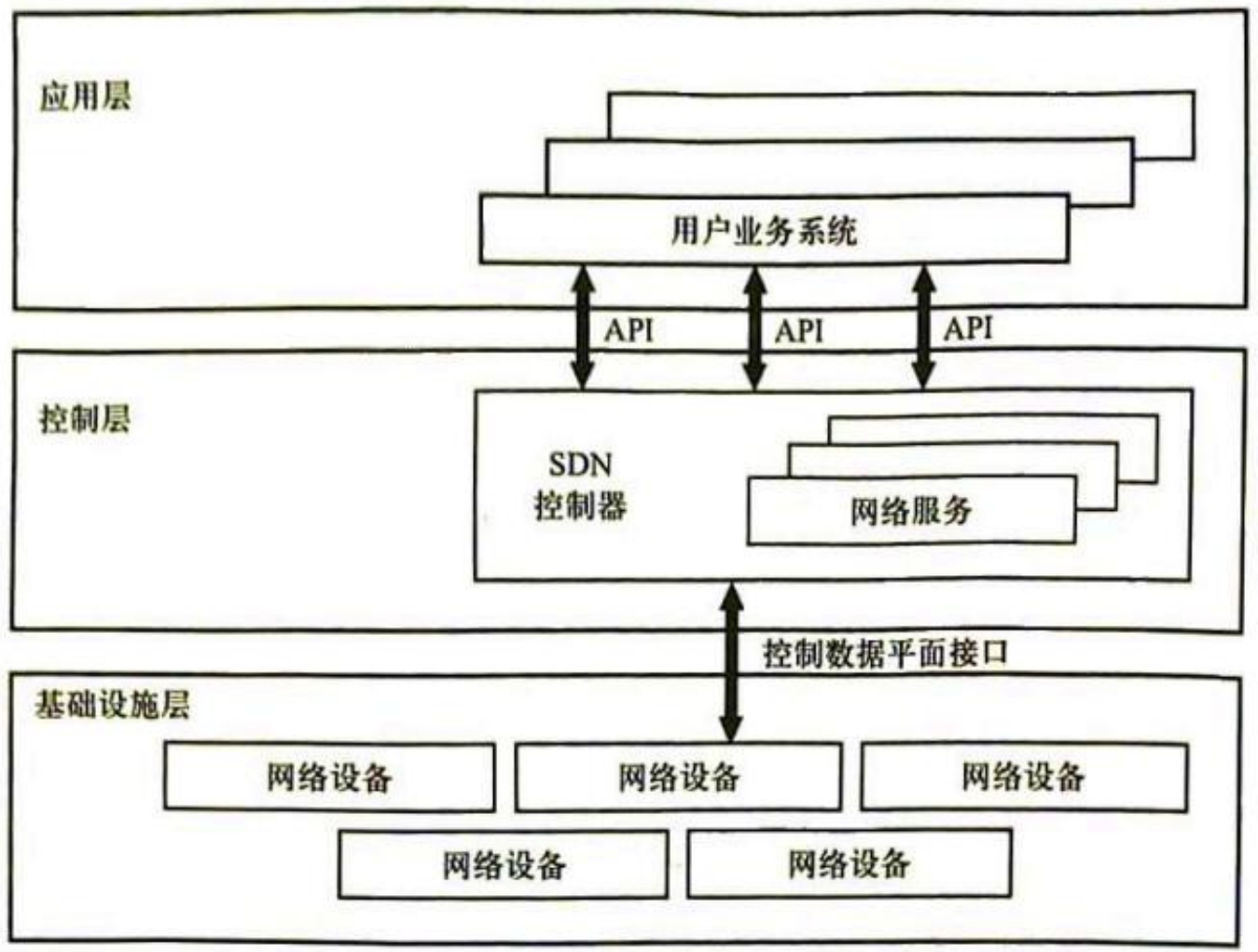


图 2.1　ONF 描述的 SDN 基本框架

SDN的体系架构-ONF定义的SDN架构

1）应用层

应用层主要包括实现用户业务的各种网络应用。相应的应用程序通过调用控制层SDN控制器所提供的应用程序接口，实现对基础设施层中网络设备的配置、管理与控制，使之能够为对上层应用与业务提供弹性支持。

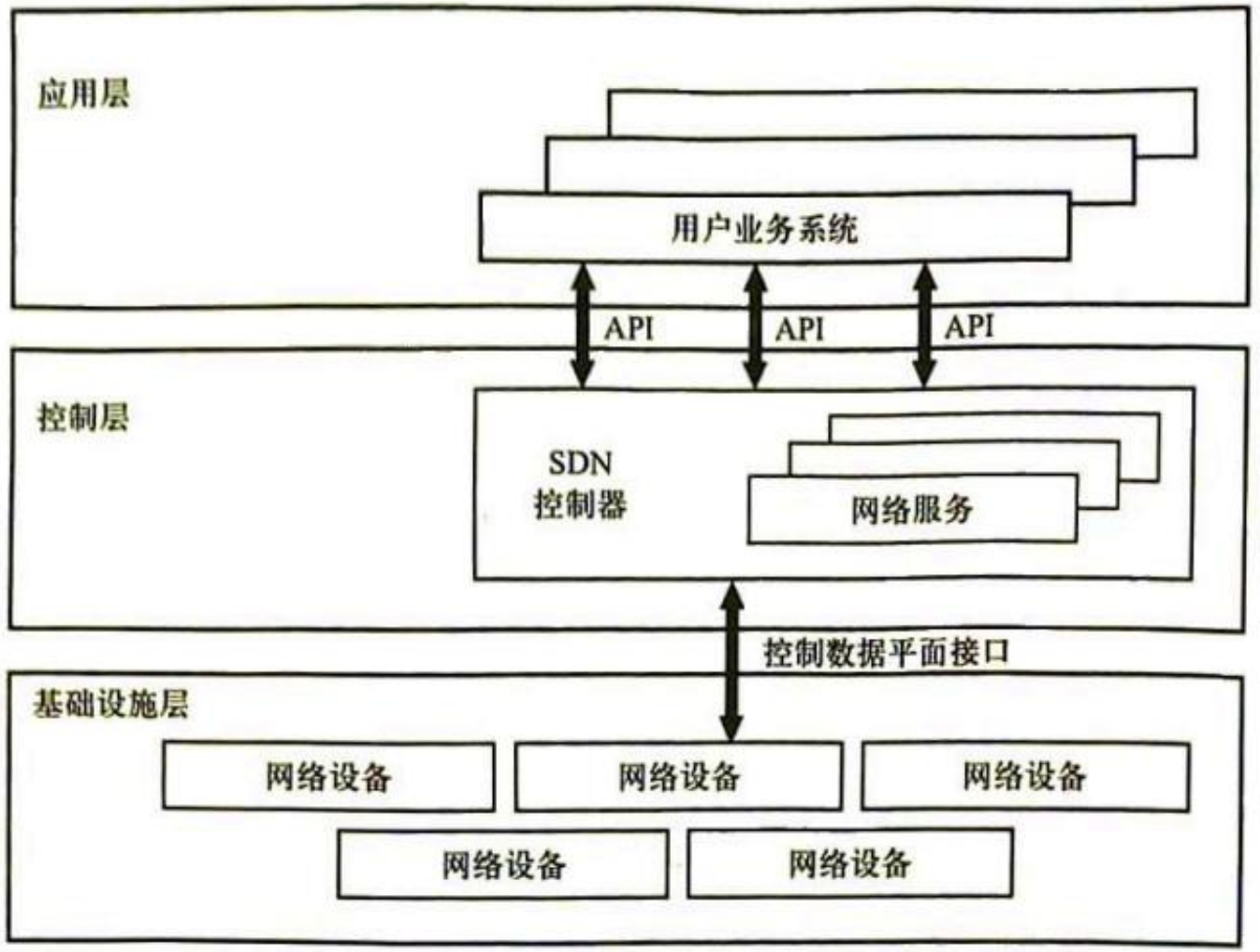


图 2.1　ONF 描述的 SDN 基本框架

SDN的体系架构-ONF定义的SDN架构

2）控制层

控制层是框架中承上启下的一层，其核心是基于软件的SDN控制器。SDN控制器作为网络运行逻辑的集中控制点，相当于SDN网络的“大脑”，它一方面通过API为上层网络应用提供编程接口，另一方面通过控制数据平面接口(con-trol-data-plane interface, CDPI)对基础设施层的网络设备进行统一的配置、管理与控制。

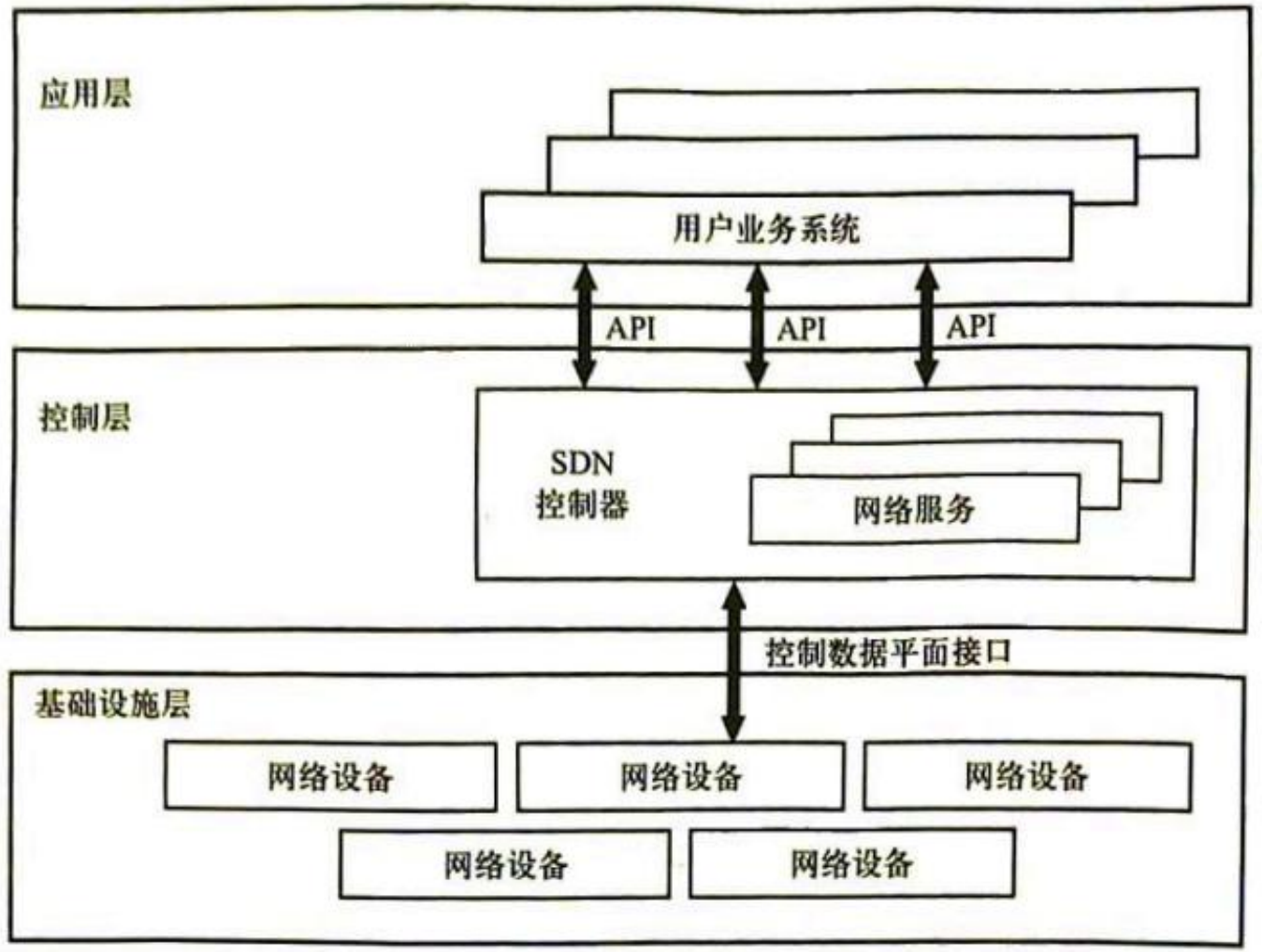


图 2.1　ONF 描述的 SDN 基本框架

SDN的体系架构-ONF定义的SDN架构

3）基础设施层

该层由基于硬件或基于软件实现的网络设备构成。这些设备通过CDPI接收来自控制器的指令，并根据指令完成网络数据包的处理；同时，它们也通过CDPI向控制器反馈网络配置和运行状态等信息。

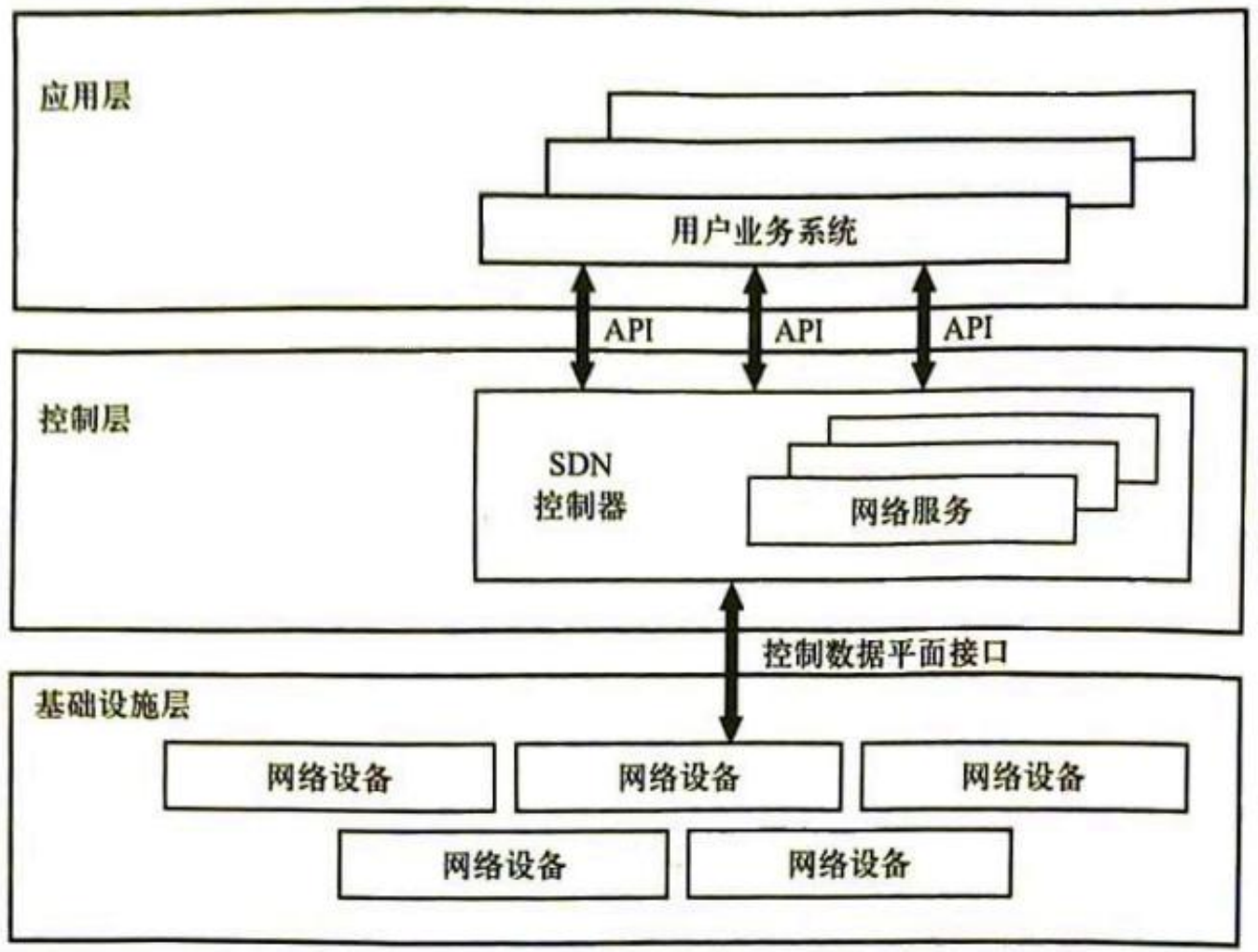


图 2.1　ONF 描述的 SDN 基本框架

SDN的体系架构-ONF定义的SDN架构

以三层基本框架为基础，加入对SDN运行机理与功能实现逻辑的描述，ONF还给出了SDN体系结构。该体系结构包括应用平面、控制平面、数据平面和管理平面四个平面。

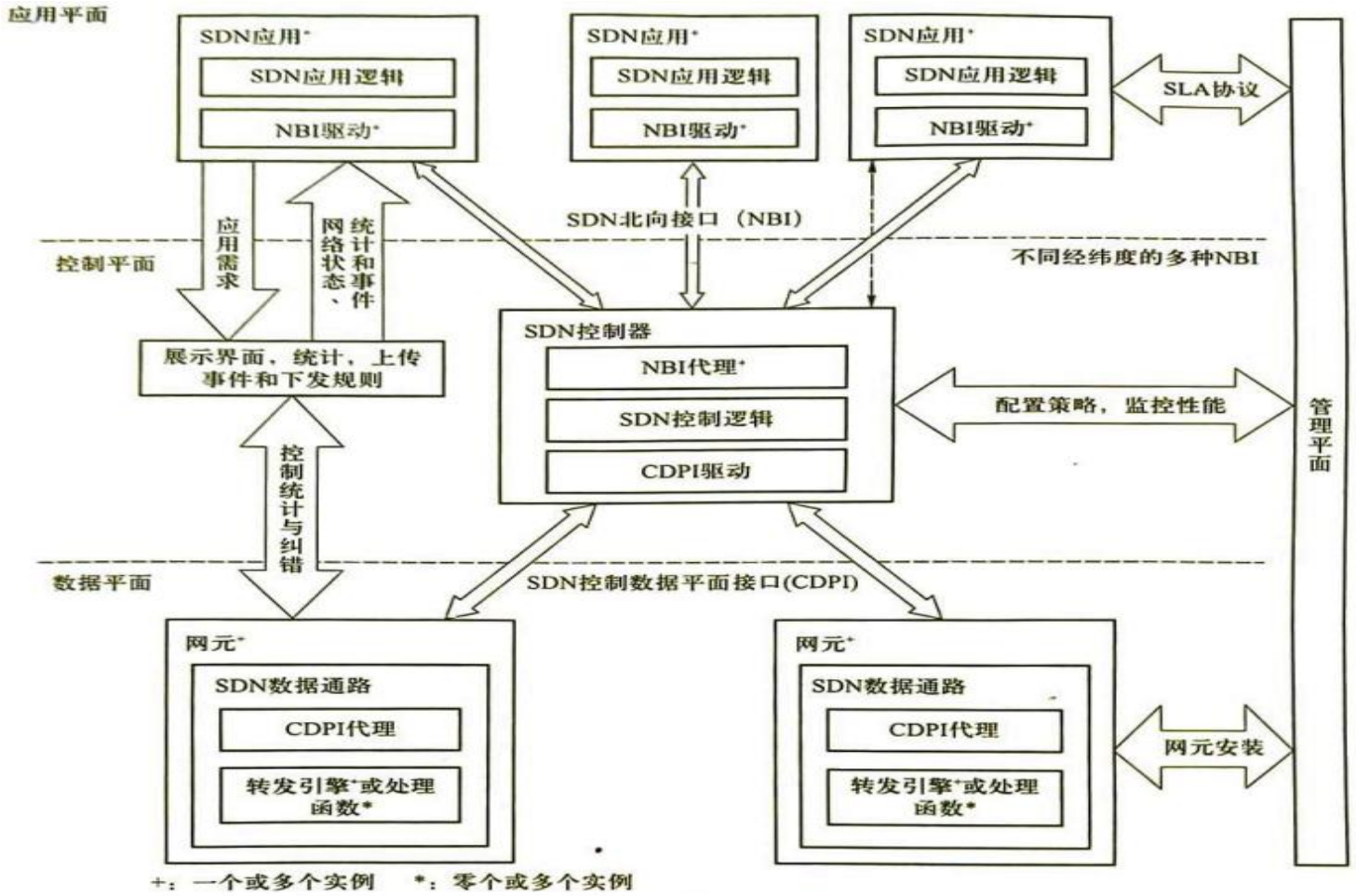


图 2.2 ONF 定义的 SDN 体系架构

SDN的体系架构-ONF定义的SDN架构

1）应用平面

该平面与图2.1中的应用层相对应，由一个或多个应用构成。每个应用包括SDN应用逻辑与北向接口(northbound interface, NBI)驱动，SDN应用逻辑由不同应用所对应的业务或功能来决定，SDN应用依托NBI 驱动，通过 SDN 北向接口与控制平面的SDN 控制器进行交互。SDN 应用还通过封装 API 对外提供北向接口代理。

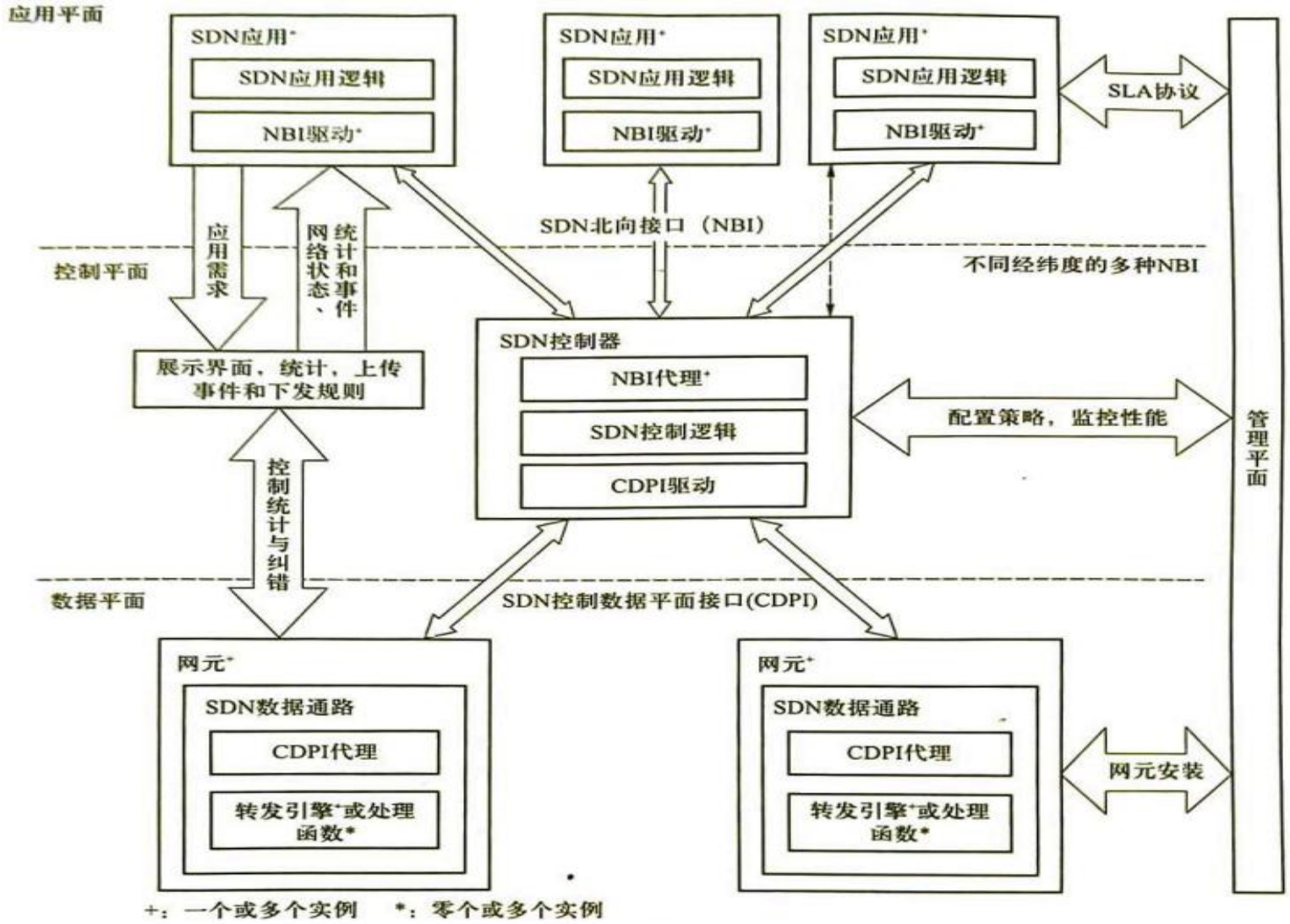


图 2.2 ONF 定义的 SDN 体系架构

SDN的体系架构-ONF定义的SDN架构

2）控制平面

该平面与图2.1中的控制层相对应，由一组控制器组成。每个控制器包括NBI代理、SDN控制逻辑(control logic)和CDPI驱动三部分。一方面，控制器借助NBI代理，通过北向接口与应用平面的SDN应用(NBI驱动)进行交互，实现从北到南的应用需求发布，以及从南到北的底层网络反馈，包括底层网络的抽象模型和相应的状态、事件及统计信息。另一方面，控制器依托CDPI驱动，通过控制数据平面接口实现与数据平面的交互，包括将SDN应用层需求以SDN转发处理规则的形式下发到数据平面的SDN数据通路(data path)，以及从数据平面获取不同网元的运行状态及事件反馈信息。

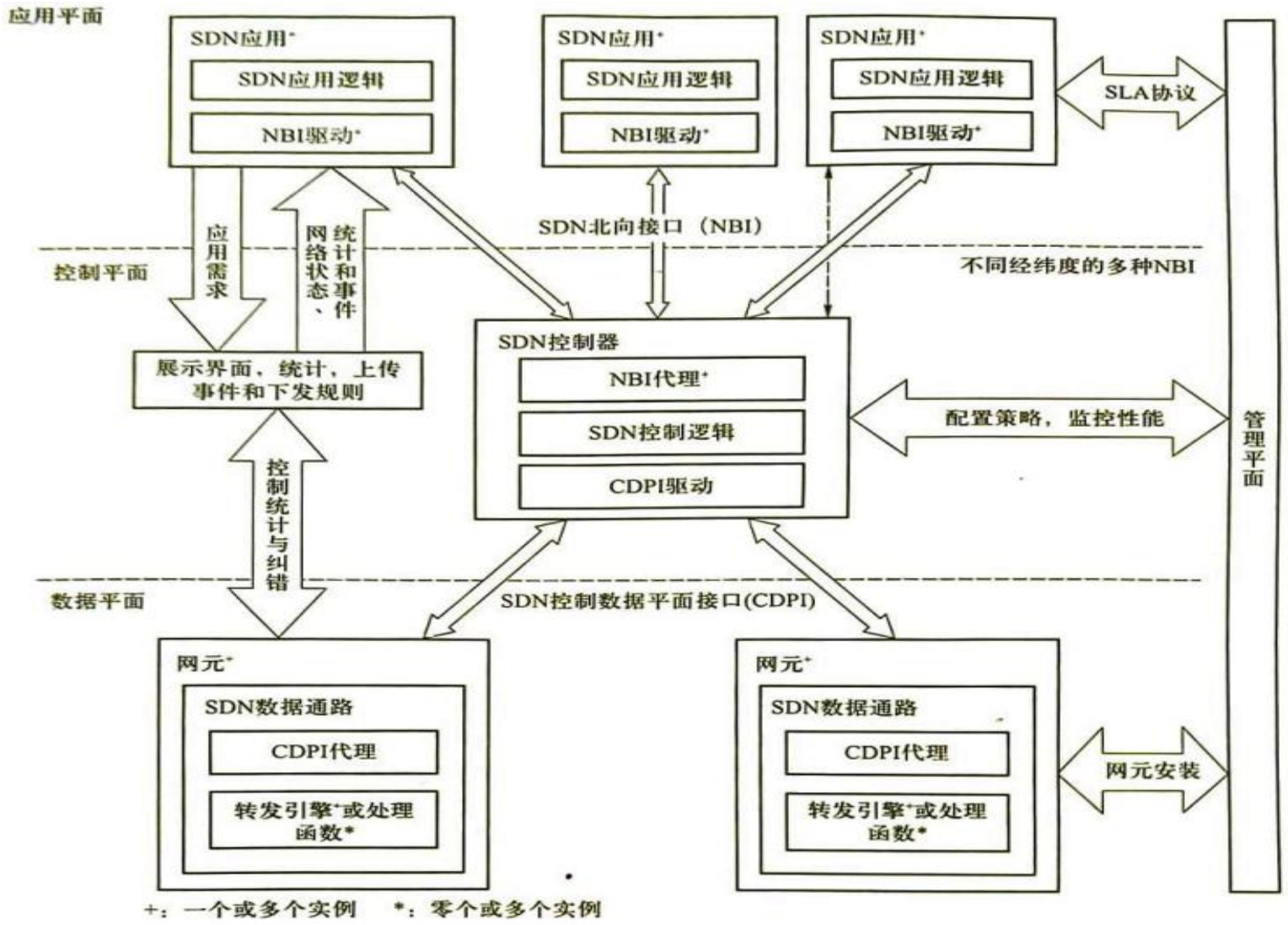


图 2.2 ONF 定义的 SDN 体系架构

SDN的体系架构-ONF定义的SDN架构

3）数据平面

该平面与图2.1中的基础设施层相对应，由若干网元(net element)构成。每个网元包含一个或多个SDN数据通路，每条SDN数据通路对应或被视为一个逻辑上的网络设备，其负责数据包或流的转发处理，但没有转发决策与控制能力。一个SDN数据通路包含CDPI代理、转发引擎(forwarding engine) 表和处理功能，用于从控制平面获取特定的转发规则，实现相应的转发功能， 并向控制平面反馈网元的运行状态。

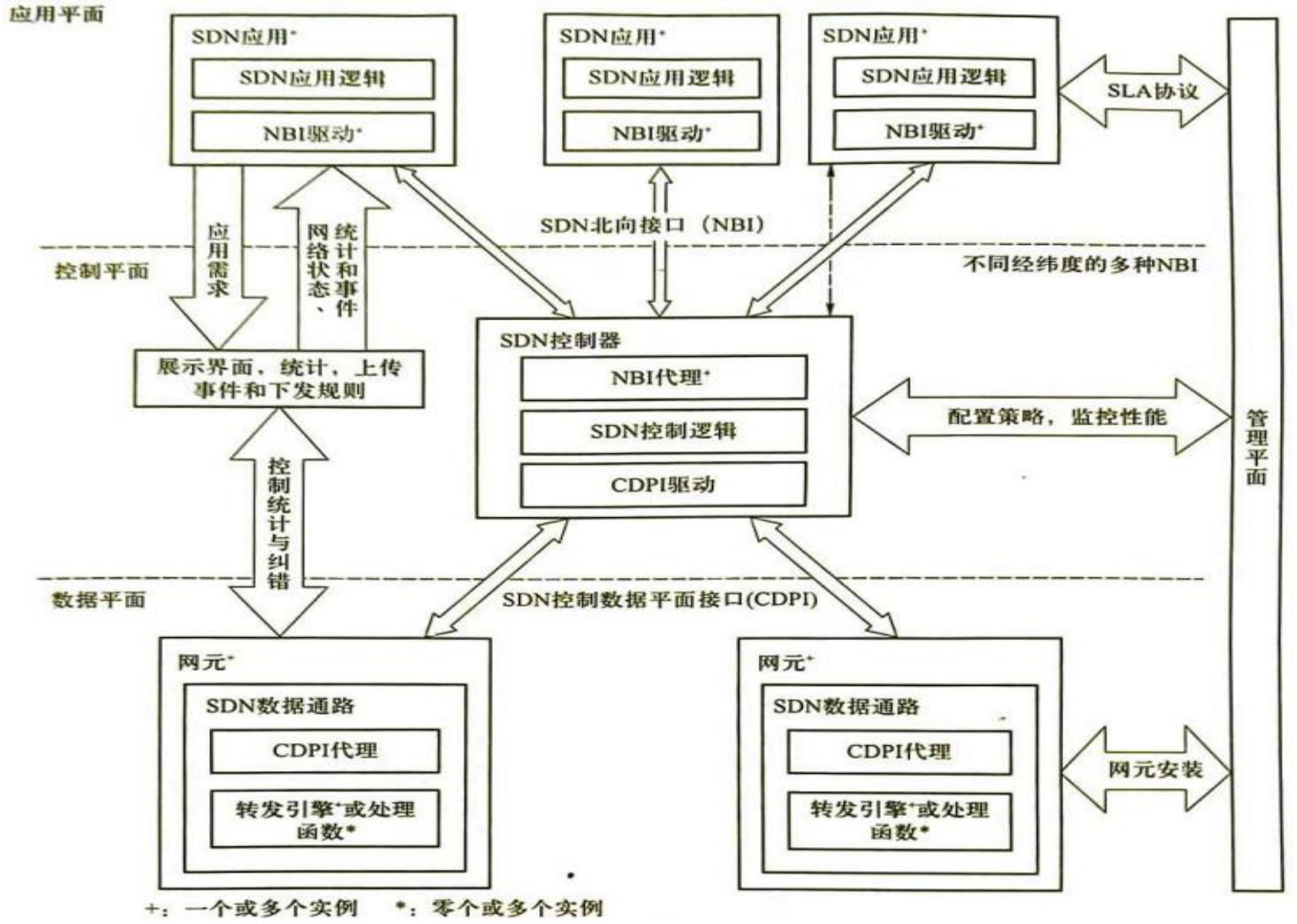


图 2.2 ONF 定义的 SDN 体系架构

SDN的体系架构-ONF定义的SDN架构

4）管理平面

该平面贯穿应用平面、控制平面、数据平面，每个应用、SDN控制器和网元都为管理平面提供相应的功能接口，通过该平面提供或实现一些通用的管理功能。例如，在应用平面，签订服务等级协定(service level agreement，SLA)；在控制平面，指定SDN控制器、定义SDN控制器、配置控制器策略、监控控制器性能等；在数据平面，实现网元安装、初始化配置等。

(application control plane interface, ACPI) -NBI

(control data plane interface, CDPI,CDPI)-SBI

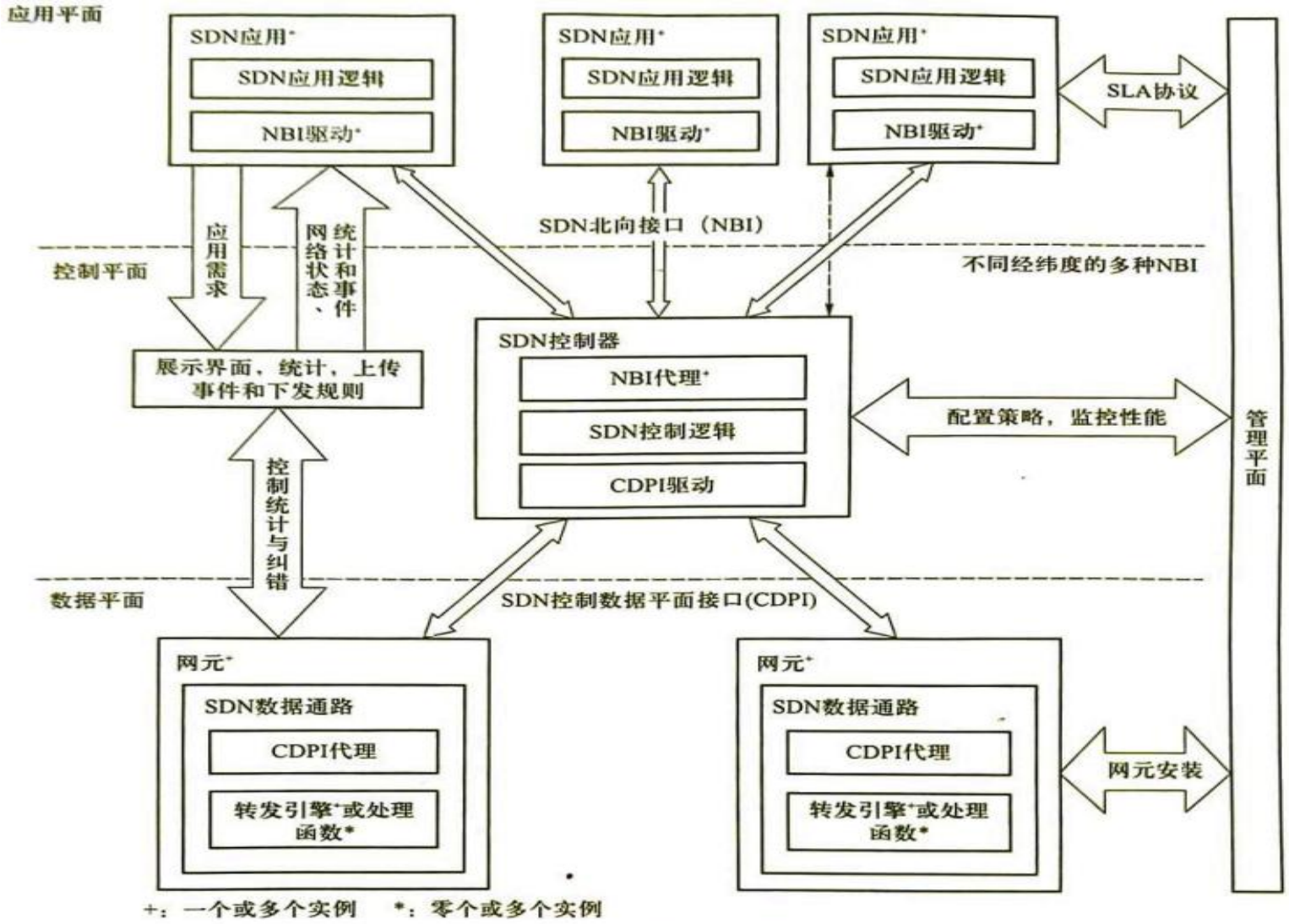


图 2.2 ONF 定义的 SDN 体系架构

SDN的体系架构-IETF定义的SDN架构

前面介绍的SDN架构是一种全新的网络架构，而事实上，要将现有的网络更换成这种新型架构存在很多困难。许多网络设备与解决方案供应商、网络服务提供商希望通过改进现有网络的方式来提供一种新的SDN解决方案，既能重用现有网络设备、路由协议及IP网络技术，又能具备SDN网络集中式软件控制带来的灵活性。全球最具权威的技术标准化组织因特网工程任务组(IETF)在SDN架构方面做了很多探索与研究，与 ONF 相比较，IETF的相关工作更多由网络设备厂商主导，因此更加关注、聚焦在与SDN相关功能和技术实现上，以及如何实现对现有网络协议与技术的重用。IETF于2012年设立了路由系统接口(I2RS)工作组，并于2013年成立了专门的SDN工作组，致力于I2RS协议的标准化工作，并相继发布了RFC 7920和 RFC 7921等相关协议文档。I2RS研究草案提出的SDN系统架构，如图2.3 所示，这种解决方案实质上是一种利用现有设备API的SDN架构。

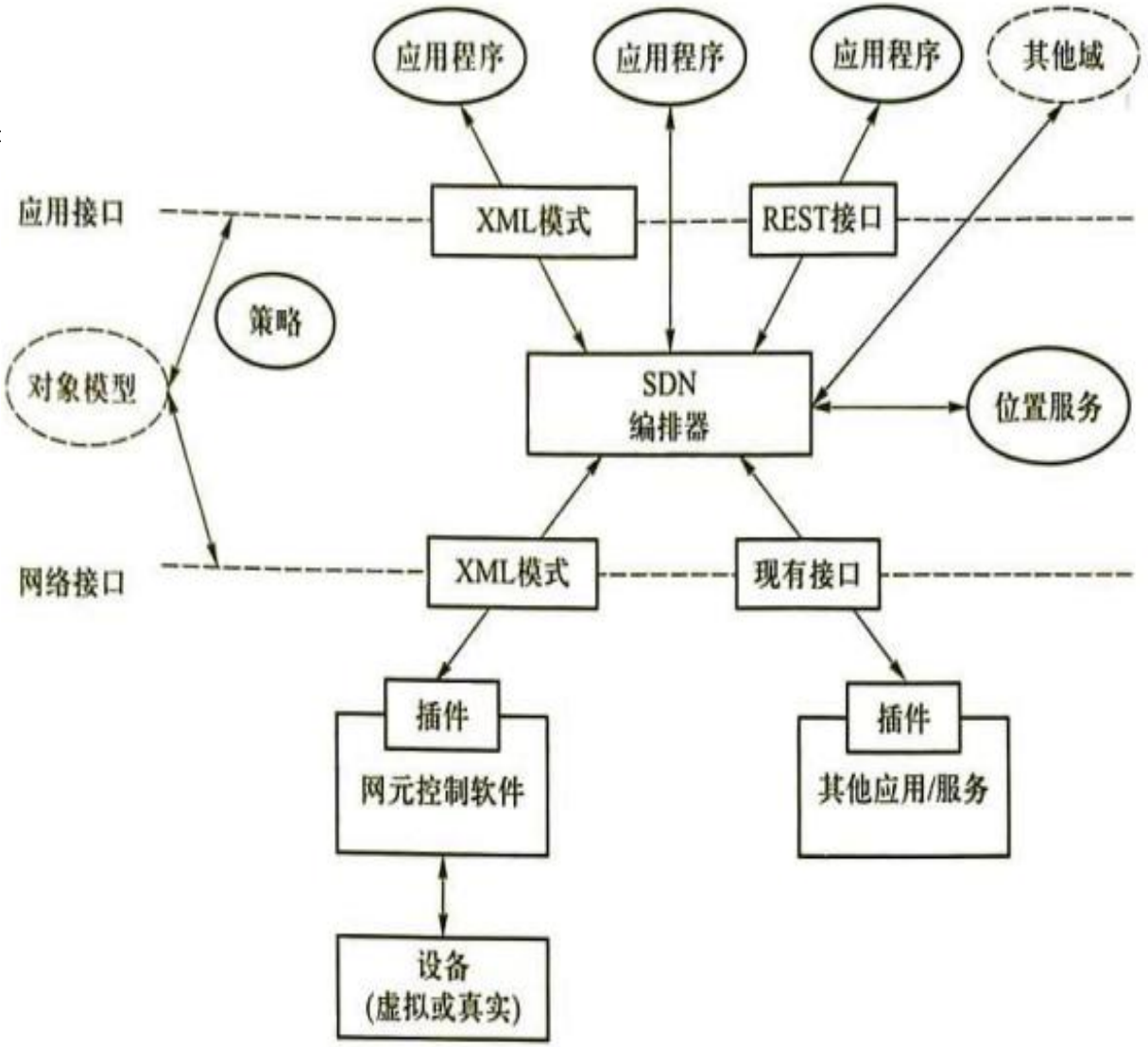


图 2.3　IETF 的 I2RS 支持的 SDN 体系结构

SDN的体系架构- IETF定义的SDN架构

在由I2RS支持的SDN体系结构中，自下而上分别是设备、插件(plug-in)、SDN编排器(orchestrator)、应用程序。位于插件和SDN编排器之间的接口称为网络接口，这些接口包括现有接口、XML模式(XML schema，一种描述XML文档结构的标准)等，其中，现有接口包括命令行接口(command line interface,CLI)、简单网络管理协议(simple network management protocol，SNMP)提供的接口等；位于SDN编排器和应用程序之间的接口称为应用接口，这些接口通常包括XML模式、REST(representational state transfer，表述化的状态转移)API等。REST API是一种基于HTTP协议的跨网络调用的接口，关于REST API将在6.2节详细介绍。

从实现思想上来看，这一体系结构尽量保留和重用现有各种路由协议和IP网络技术，网络设备通过增加插件的方式实现能力开放，而不是直接采用Open-Flow 进行能力开放，I2RS的路由系统需要发布网络拓扑和状态，通过网元数据计算选路，并将相关结果传递给各网元的控制平面。同时，通过在网络设备与应用层之间增加SDN编排器实现能力开放的封装，网络程序员通过SDN编排器提供的REST API等北向接口进行应用程序的开发。

SDN的体系架构- Overlay网络架构

Overlay 网络最早是由Martin Casado 博士创立的Nicira公司（后被VMware收购）提出的一种技术架构，它是在对基础网络不进行大规模修改的前提下，实现应用在网络上的承载，并能与其他网络业务分离，且以基于IP的基础网络技术为主。“Overlay”一词在中文中没有既定的译法，一部分技术读物将其称为“覆盖网络”。通常，“Overlay”是指根据某种特定需要，对底层(Underlay)网络使用特定协议或技术进行抽象、封装之后建立起来的逻辑网络层。Overlay覆盖在底层网络之上，给出Underlay中的数据传输决策或转发规则，控制并决定Underlay中的数据传输行为，属于虚拟逻辑网络。在实际数据通信过程中，数据仍然需要借助底层网络所提供的基础设施，沿着物理网络路径进行传输。Overlay与Underlay之间的关系，有点类似于网络拓扑中的逻辑拓扑与物理拓扑之间的关系。

SDN的体系架构- Overlay网络架构

图2.4 给出了一个使用某二层协议建立Overlay网络的示意图。其中，Underlay 是一个由两台路由器、三台交换机和三台服务器主机组成的网络设施，分别属于网络层、数据链路层和应用层的节点设备，Underlay中设备之间的连线代表设备之间存在三层连接。但是，在Overlay中只出现了三台交换机，而未出现Underlay 网络中的那两台路由器，这说明三台交换机之间已经使用某种封装协议建立起了通信隧道，而两台路由器没有使用Overlay 网络交换机所采用的封装协议，从而不能与邻居交换机设备建立通信隧道。

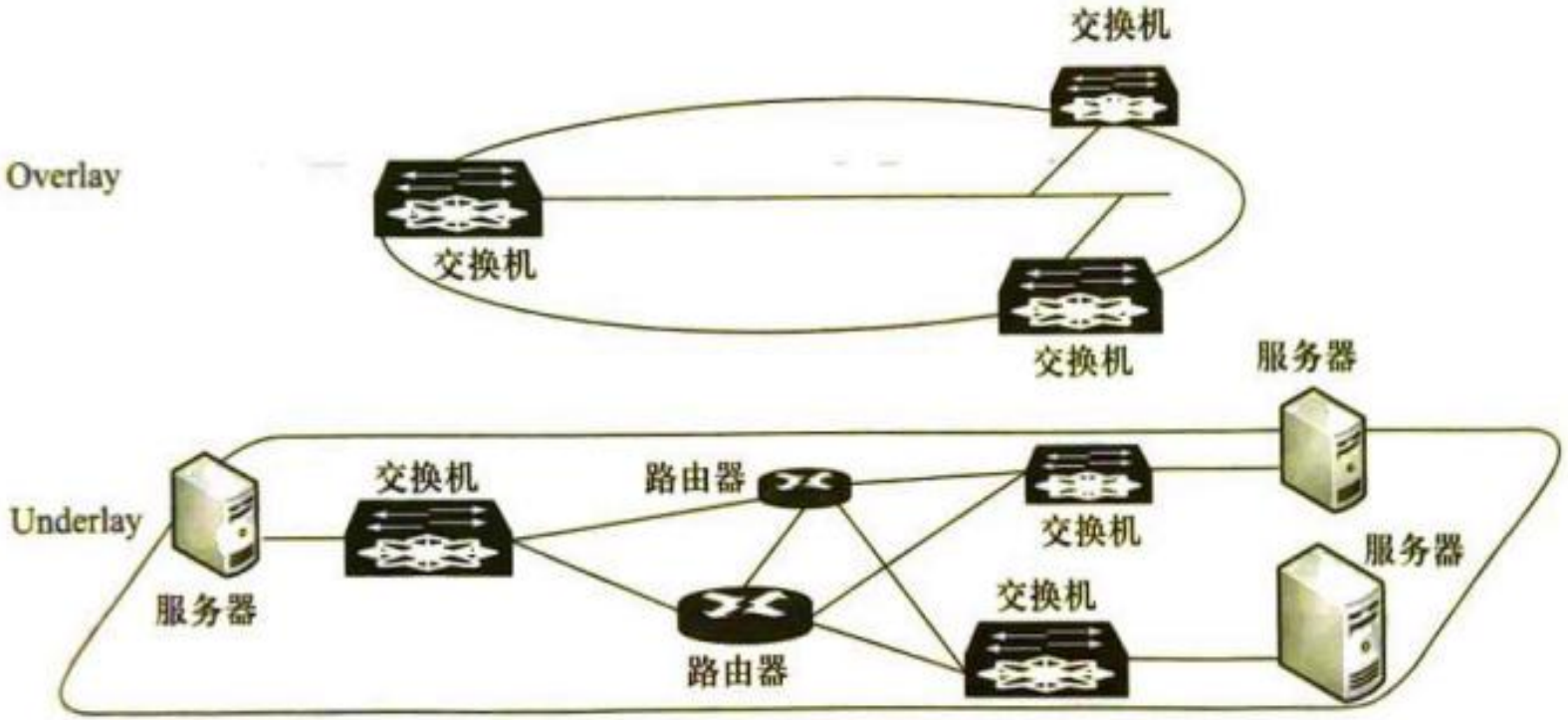
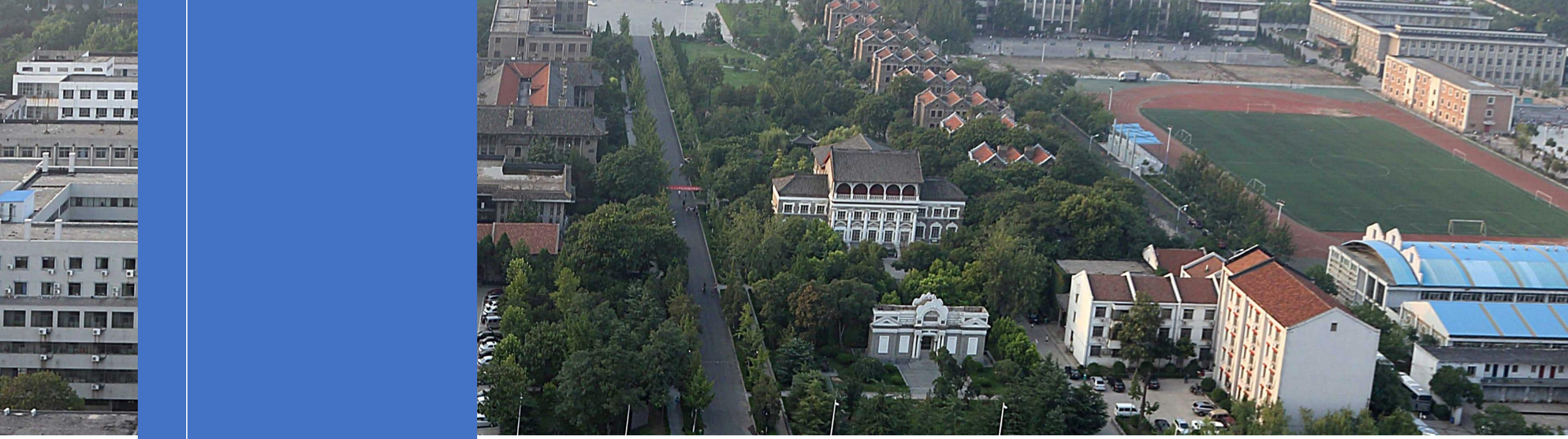


图 2.4　Overlay 示意图

由此可知，Overlay网络并不是某种特定技术的代名词，而是指代任何一种在底层网络基础上，通过某种封装协议建立逻辑连接的方式，所以属于物理网络之上的逻辑网络。Overlay 网络作为在 Underlay 网络基础上针对某种协议实现的抽象，它与这种协议的OSI分层模型无关。

这也就是说，从OSI分层的角度上看，Overlay给出的逻辑抽象层不一定在OSI分层上高于Underlay。这个特性说明，在OSI分层较低的数据头部外层重新封装OSI高层的头部，就可以实现为较低分层建立隧道的目的。在实际应用中，利用Overlay技术可以建立更符合业务意图的通信。



Part
03

SDN的核心设计思想

控制平面与数据平面的解耦

网络能力的抽象

网络可编程

SDN的核心设计思想

尽管不同的SDN体系结构存在差异，但从本质上说，它们通过对网络架构与实现方式的变革实现了对传统网络的颠覆。本节以ONF给出的SDN网络架构为基础，从控制平面与数据平面解耦、网络能力抽象、网络可编程等三个角度介绍SDN的核心设计思想，读者从中可以体会SDN与传统网络的本质区别。

SDN的核心设计思想-控制平面与数据平面的解耦

控制平面与数据平面解耦(decoupling)是SDN架构区别于传统网络体系架构的重要标志。控制平面负责上层的控制决策，数据平面负责数据的交换转发。在传统网络的分布式架构中，每台网络设备都要提供事先定义的控制平面与数据平面，而且两者密不可分，具有强烈的互依赖性。SDN通过将控制平面和数据平面解耦，两个平面不再直接相互依赖，两个平面之间通过遵循相同协议或标准的开放接口进行通信。这种设计思路为实现网络逻辑的集中控制提供了可能与前提条件，并且简化了数据平面上的网络设备。同时，两个平面的分离使得各自可以独立完成体系结构和技术的发展演进，有利于促进技术创新与发展，并可以在两个平面之间提供更大的技术兼容性。

SDN的核心设计思想–控制平面与数据平面的解耦

虽然控制平面和数据平面解耦对SDN很重要，但也给SDN带来了新的问题与挑战，主要表现在以下几方面。

（1）控制平面的可扩展性问题。

控制平面与数据平面分离后，原来分布式的控制平面被集中化。关于网络计算，分布式与集中式是两种非常有代表性的设计思路，分布式设计的重要优势在于其可扩展性与容错性强，而集中式设计容易因为集中控制点的原因，导致网络可扩展性与容错性下降。也就是说，随着SDN底层网络规模与上层应用或业务的扩大，控制平面的服务能力有可能成为网络性能的瓶颈。因此，控制平面的可扩展性问题是SDN控制平面与数据平面分离后必须考虑和面对的问题之一。

（2）分布式状态一致性问题。

在传统网络中，网络状态的一致性是由分布式协议来保证的。以网络路由为例，每个路由器通过自身控制平面与数据平面的耦合，获得对网络局部状态的认知，运行相同路由协议的分布式路由器之间通过交换路由协议数据包，实现对网络状态或路由信息的一致认知，这种一致性称为路由收敛。在SDN中，当数据平面和控制平面分离后，控制平面中的多个控制器如何交换路由信息，以便为不同的上层应用提供其所需的最佳数据传输服务，并确保数据平面或基础设施层的运行效率，是一个重要的挑战。

（3）数据平面的可用性问题。

传统网络的数据平面具有很高的可用性，而与数据平面耦合在同一物理设备上的控制平面，会快速、实时响应数据平面的请求，保障网络的稳定性。但SDN将数据平面和控制平面分离后，控制平面需要通过南向接口向数据平面下发数据转发策略与规则，实施必要的交互，这就导致控制平面服务能力不足，增大接口通信交互延迟及数据平面的传输延迟，严重时可能会产生数据平面的可用性问题。

SDN的核心设计思想-网络能力的抽象

抽象是计算科学的核心思想之一，在计算机系统中，抽象无处不在。例如高级程序设计语言就是对机器语言的抽象，今天的程序员可以不必理解机器语言，利用高级程序设计语言就能简单地定义复杂操作。在SDN架构中，基于软件方法进行网络功能与行为的抽象既重要又必要。从用户的视角来看，可以把SDN看成是一个分层系统，底层硬件是通用的转发设备，中间层是由软件实现的控制器或网络操作系统，上层是网络应用程序，并且在各层次上实现进一步的抽象。在SDN系统中有三种重要的网络能力与行为抽象，分别是转发抽象、分布抽象和配置抽象。

SDN的核心设计思想-网络能力的抽象

转发抽象(forwarding abstration)：是指借助对底层硬件实现的屏蔽，使转发行为与基础设施层的设备无关。转发抽象使网络程序员可以不必了解具体硬件设备的知识，就能指定必要的转发行为。

分布抽象(distribution abstration)：通过位于网络操作系统(network operation system，NOS)中逻辑集中的分布层(distribution layer)，屏蔽控制平面中多个逻辑控制器对数据平面不同物理转发设备所进行的分布式控制实现细节，包括控制命令的下发、转发事件的上传、设备和链路状态的收集等。分布抽象为高层网络程序员提供了全局网络视图，不用关心分布式控制的底层实现细节。

配置抽象(specification abstration)：指基于简化抽象模型，通过网络程序设计语言实现网络配置与行为的表达，并由控制平面将程序设计语言表达的配置指令与行为规则下发到数据平面，从而将抽象配置映射为物理配置。

SDN的核心设计思想–网络可编程

说到网络可编程，传统网络工程师对其的认识可能还停留在网络设备配置阶段。传统网络中的路由器、交换机和防火墙等设备通常为用户提供了命令行界面(CLI，也称命令行接口)和图形用户界面(graphical user interface, GUI) 两种管理接口，用户可以借此对设备进行手工配置和操作，但其所能提供的管理功能受限于设备设计与生产时所赋予或定义的功能，而且这种手工配置与操作模式的效率较低下，严重影响了网络业务的部署进度。简单网络管理协议(SNMP)有助于提升网络管理的便捷性与效率，但不能解决设备不可编程或难以编程所带来的根本性制约。

在介绍SDN可编程能力之前，先简单回顾几种常见的传统网络管理模式。

SDN的核心设计思想–网络可编程

1.命令行界面(CLI)

每个厂商的设备都提供了某种形式的命令行界面，用户通过命令行界面实现设备的配置。网络设备的CLI类似于UNIX操作系统的命令行提示符，它是基于ASCII字符的系统，实际上是一个有效的语法分析器，当按下回车键后，系统基于字符串的标记实施相应的动作。用户在本地可以通过USB或串口线等方式连接设备后进行CLI配置，在远端则可以通过远程登录协议(telnet protocol)或安全外壳(secure shell， SSH)等协议实现远程访问CLI。

CLI提供了多种用户模式，如配置模式、查询模式、安全模式等，在不同的模式下，用户可以进行不同的配置操作。尽管不同厂商提供的CLI所能实现的功能大同小异，但是相互之间却是不兼容的，CLI至今没有实现标准化。

CLI 实现网络可编程的主要方式是通过UNIX脚本与设备的CLI进行交互，目前被广泛应用的脚本语言包括Perl、Expect、UNIX Shell 以及Python等。根据所要完成的配置任务，需要预先编制相应的脚本，然后通过Telnet或SSH方式连接到设备，在通过必要的认证后，脚本程序就可以自动执行了。这种编程实现方式在执行效果上与用户输入命令相当，并节省了人工配置的时间，但是缺乏交互性，不是应用程序或用户友好型的编程方式；同时，实现脚本编程前，需要熟悉设备的命令行，这依赖于设备的型号、厂家和版本等，因此，编程人员一般都是熟悉设备命令的网络管理员。

SDN的核心设计思想–网络可编程

2. 简单网络管理协议(SNMP)

SNMP是IETF发布的、基本的网络管理协议，SNMP主要有SNMPv1、SNMPv2c、SNMPv3三个版本，目前仍然被广泛应用于监测网元设备的状态和性能特性。

SNMP 协议标准提供了定义管理信息的框架和交互管理信息的协议，SNMP基于客户-服务器(client-server， C/S)模型，为管理员提供了一个网络管理平台(network management station，NMS，又称为管理站)，负责网管命令的发出、数据存储及数据分析等。被监管的设备上运行一个SNMP代理(agent)，代理实现设备与管理站之间的SNMP通信，实现结构如图2.5所示。

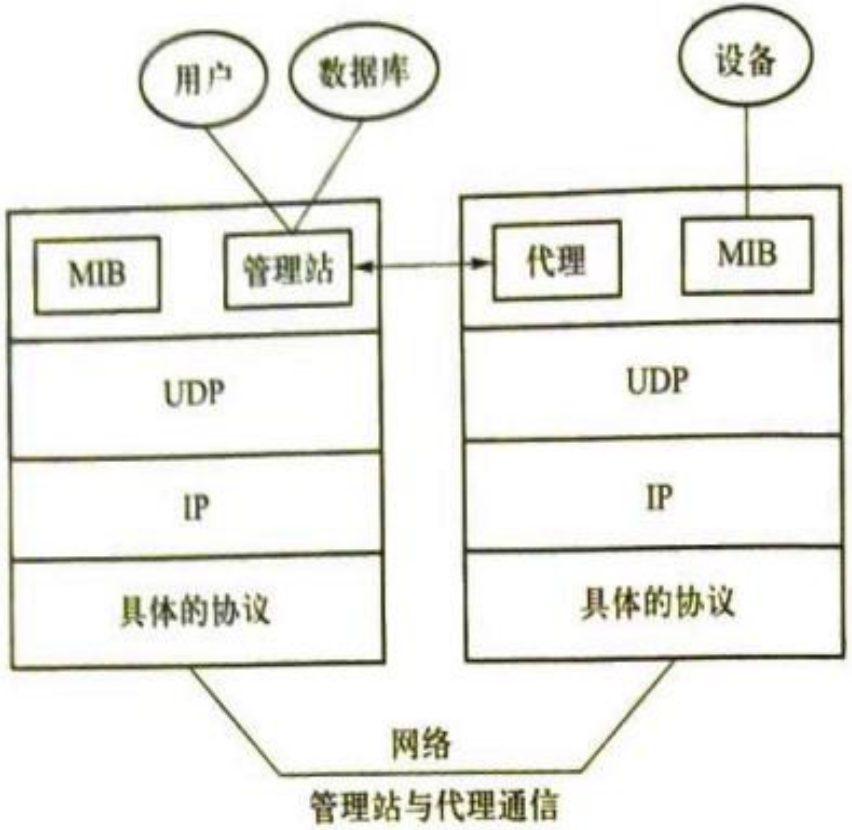


图 2.5 SNMP 协议的实现结构

SDN的核心设计思想-网络可编程

管理站与代理通过管理信息库(management information base，MIB)实现接口统一，MIB定义了设备中的被管理对象。管理站和代理都实现了相应的MIB对象，使得双方可以识别对方的数据，从而实现通信。管理站向代理申请MIB中定义的数据，代理识别后，将管理设备提供的相关状态或参数等数据转换为MIB定义的格式，应答给管理站，完成一次管理操作。网络管理涉及的数据包括设备的配置、参数、状态等信息，涉及的操作包括读取(Get)和设置(Set)；同时，因为网络设备众多，为了能及时获取设备的重要状态，还要求设备能主动汇报重要状态，这就是主动告警(Trap)功能。

要实现SNMP编程，需安装支持SNMP协议的开发包，如SNMP++、Net-SNMP等，SNMP++提供C++应用程序接口；Net-SNMP 提供C和 Perl(practical extraction and reporting) 的应用程序接口，这些开发包均提供了用于SNMP应用程序开发的、完整的应用程序接口。

SDN的核心设计思想-网络可编程

01

本质上来说，传统网络中基于CLI、SNMP等方式所实现的网络编程方式非常初级，只是为了增强网络设备管控能力和效率而提供的一种辅助方法。随着传统网络规模与复杂度的增加，网络管理者需要更高级的编程方式。他们希望这种可编程性是基于整个网络的，而不只是基于某一设备的，通过对网络功能的抽象，实现网络功能的可编程。例如，通过软件方式，收集并根据实时链路负载情况，实现对分布式路由器转发策略的自动配置。在SDN出现之前，业界就已经对这类网络的可编程进行了相关研究，其中，最具代表的是主动网络(active network)。主动网络改变了传统网络被动传输信息的模式，其主要思想是开放网络节点的编程接口，允许网络节点在用户数据上根据用户需求执行相应的计算。在主动网络中，开发者可以把具体的代码下发到交换设备，或者在数据分组中添加可执行代码， 以提高网络的可编程性。主动网络由于各种原因未得到广泛的应用，但是为SDN的可编程设计提供了借鉴。

02

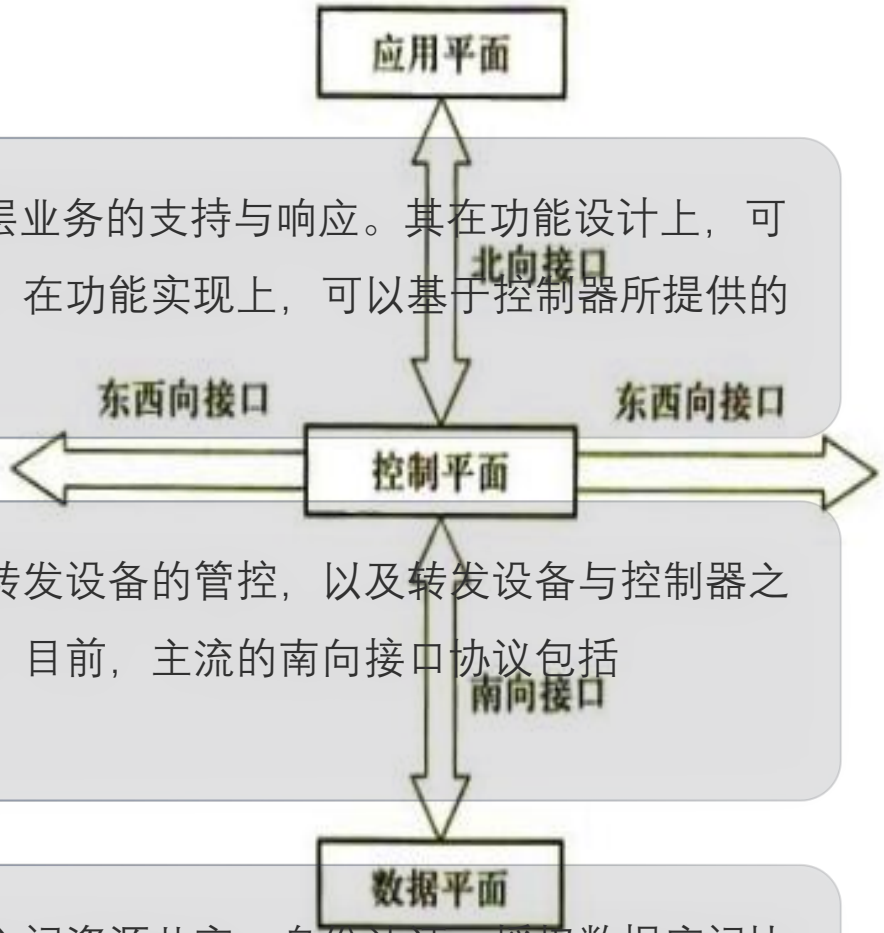
网络编程接口是实现网络可编程的关键，它通过逻辑与模型抽象、接口协议、API等方式为网络可编程提供对通道与接口的支持。以ONF的SDN网络架构为例，SDN的编程接口主要包括北向接口、南向接口和东西向接口。

SDN的核心设计思想-网络可编程

北向接口是SDN应用平面与控制平面之间的接口，主要用于控制器对应用层业务的支持与响应。其在功能设计上，可根据应用层不同业务或应用的需要，为其提供底层网络的逻辑抽象与模型；在功能实现上，可以基于控制器所提供的各种接口加以实现。

南向接口是控制平面与数据平面之间的接口，主要用于实现控制器对底层转发设备的管控，以及转发设备与控制器之间的协调工作。控制器与转发设备之间的消息交互通过南向接口协议实现。目前，主流的南向接口协议包括OpenFlow、XMPP、PCEP、I2RS、LISP等。

东西向接口是控制平面中分布式控制器之间的通信接口，主要负责控制器之间资源共享、身份认证、授权数据库间协作及保持控制逻辑一致性等工作，用于实现底层基础设施透明化的多控制器组网策略。目前，东西向接口协议尚未形成统一标准。



明德新民　　止于至善

《软件定义网络技术》
Software Defined Networking (SDN)

主讲人:黄辉
email:huanghui@wzu.edu.cn

SDN的主流技术架构

ONF定义的基于OpenFlow的架构

特点：1、转发与控制分离；
2、标准化转发面。

优点：易于流量调度

IETF提出的技术架构

特点：1、开放网络设备能力；
2、标准化API。

优点：1、充分利用现有设备；
2、快速实现。

NICIRA提出的Overlay技术架构

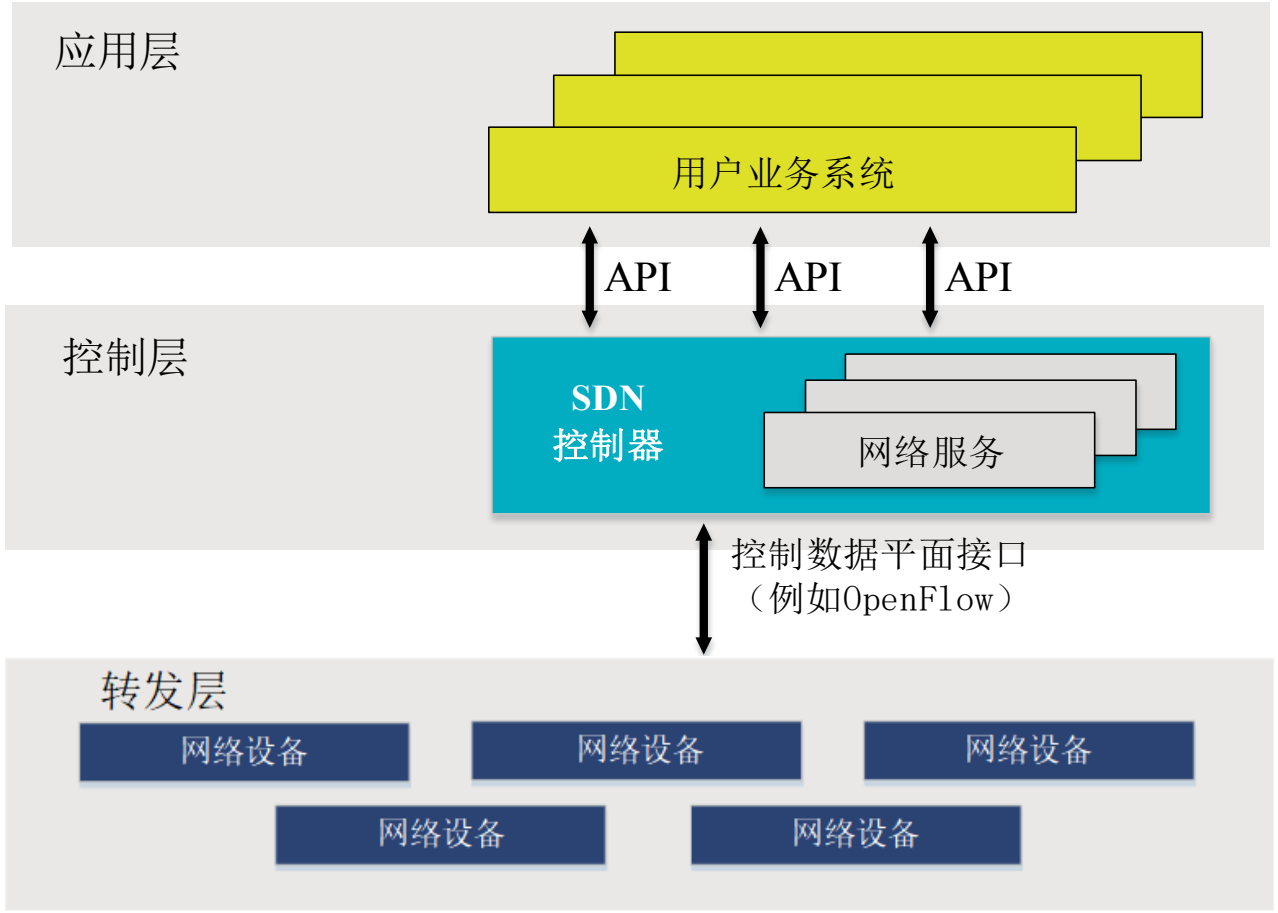
特点：1、网络边缘软件化；
2、Overlay技术。

优点：1、与物理网络解耦；
2、部署灵活。

ETSI提出的NFV技术架构

NFV与SDN技术互补；
但有本质区别，严格来说不能算是。

ONF定义SDN基本架构

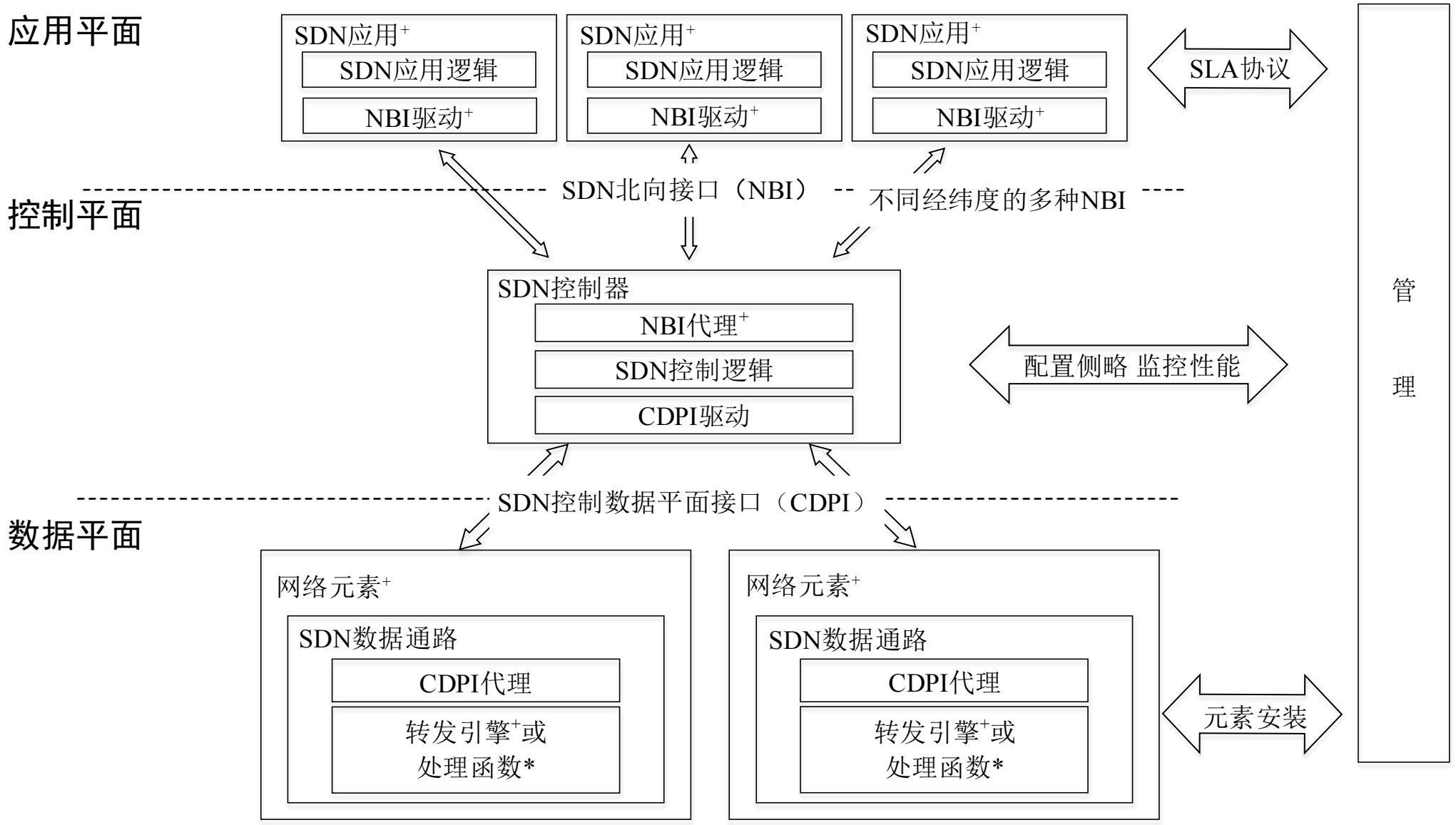


3. 实现了网络流量的灵活控制，使网络变得更加智能。

2. 网络虚拟化的一种实现方式，其核心技术OpenFlow

1. 一种新型网络创新架构，实现了网络设备控制与转发的分离

ONF定义SDN体系架构



+：一个或多个实例　　*：零个或多个实例

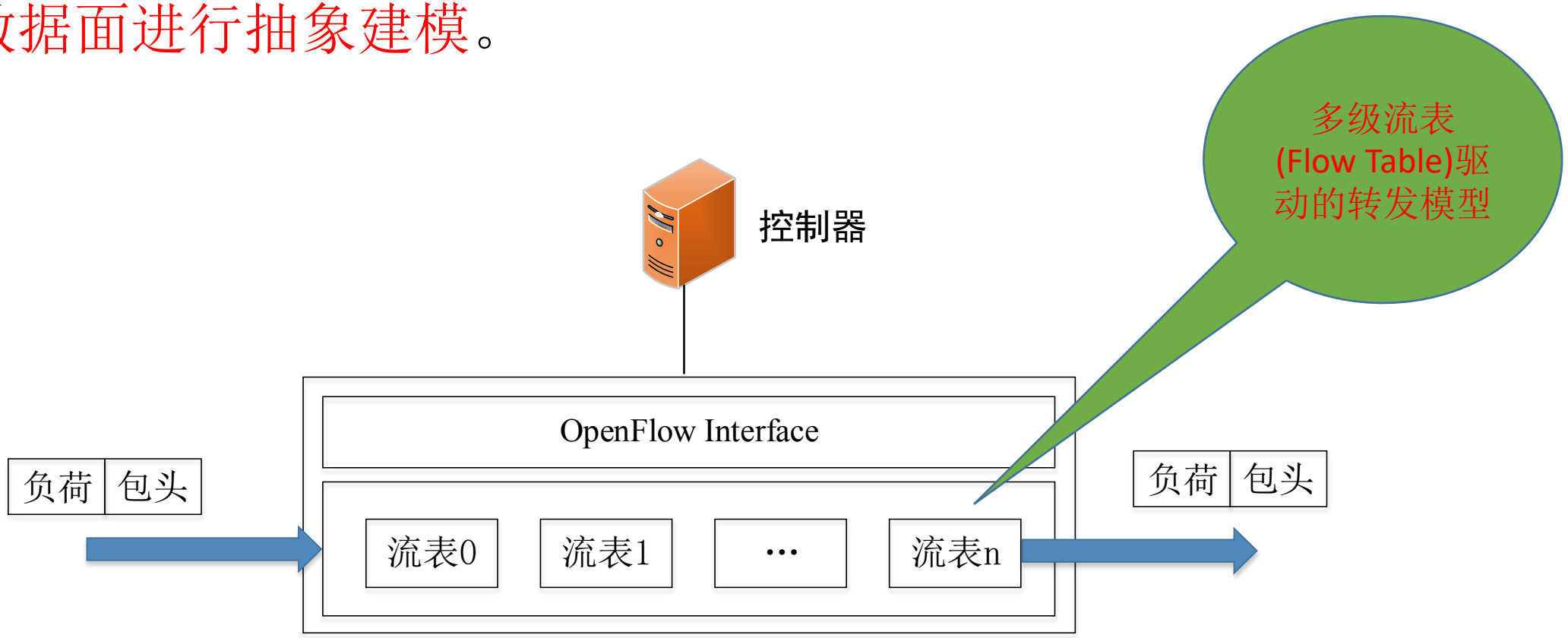
四个平面

数据平面

- 若干网元（Network Element）：包含一个或多个SDN 数据路径（ Datapath ）；
- SDN Datapath：逻辑上的网络设备，负责转发和处理数据；
  - 控制数据平面接口（Control Data Plane Interface, CDPI）代理、转发引擎（Forwarding Engine）表和处理功能（Processing Function）

数据面（转发面）关键技术

- 对数据面进行抽象建模。



控制平面

- 北向接口（Northbound Interfaces，NBI）代理、SDN控制逻辑（Control Logic）和控制数据平面接口驱动（CDPI Driver）；
- 两个任务：
  - 将SDN应用层请求转换到SDN Datapath;
  - 为SDN应用提供底层网络的抽象模型（状态或事件）。

控制面关键技术

- 控制器，网络操作系统（NOS）或网络控制器。
- 智能、核心均在SDN 控制器中实现
- 开源SDN控制器： NOX、POX、FloodLight、RYU、OpenDayLight、ONOS等。

应用平面

- SDN应用逻辑与北向接口（ NBI ）驱动；
- 通过北向接口与SDN控制器交互；

应用逻辑

- 应用交付能力（负载均衡、访问控制、应用加速）：
  - 降低系统的开支和成本；
  - 网元的虚拟化和集中化控制；
  - 网络快速部署，故障快速发现与解决；
  - 更高的智能，自动化运作，应用可感知的网络

管理平面

- 静态的工作：网元初始化配置，指定控制器、定义控制器及应用的控制范围。

两大接口

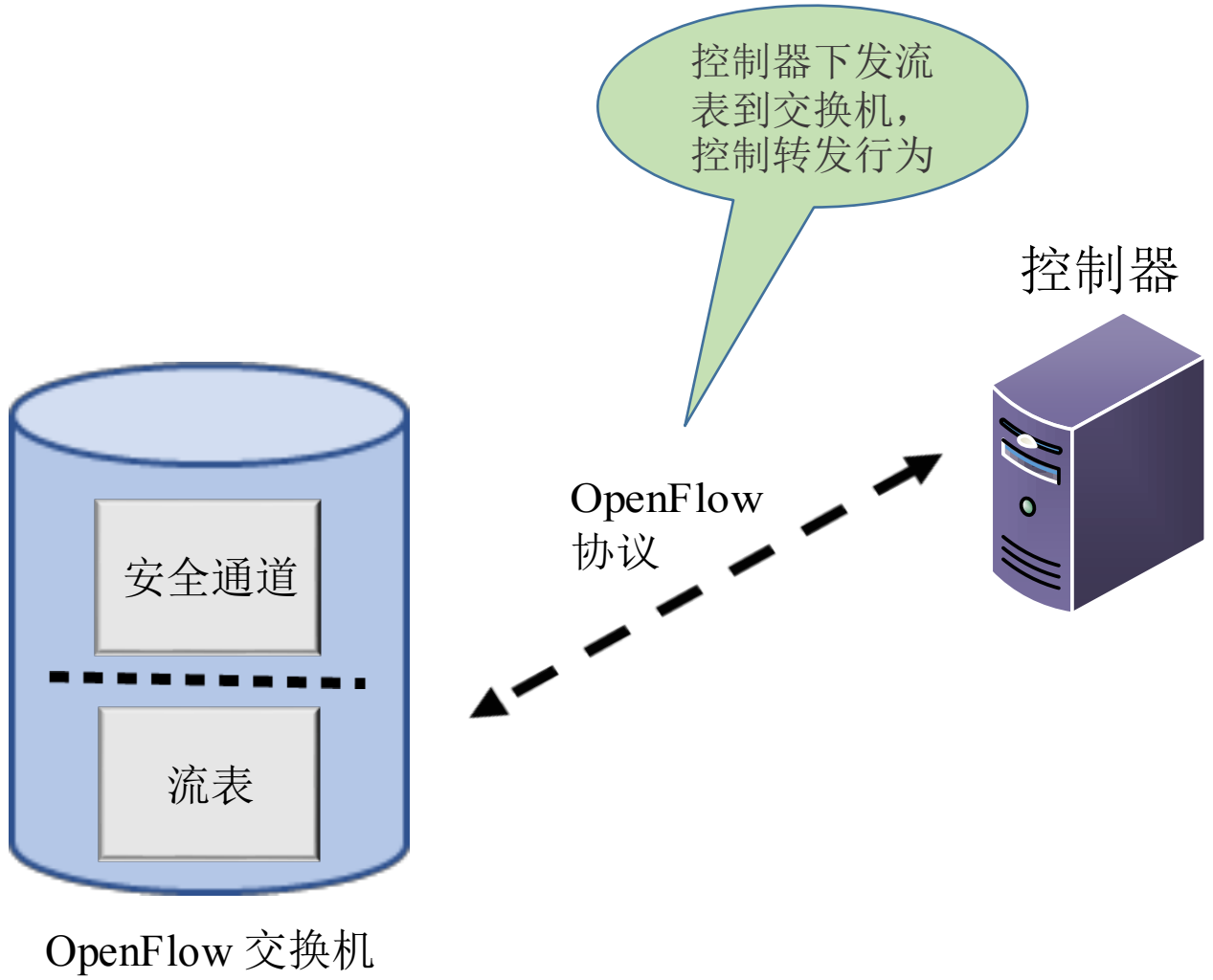
南向接口

- 控制平面和数据平面之间的接口（CDPI）。
- 功能：转发行为控制、设备性能查询、统计报告、事件通知等。

- ONF体系架构：标准化的南向接口协议（Openflow），不依赖于底层具体厂商的交换设备。

南向接口关键技术

- 转发面开放协议：允许控制器控制交换机的配置以及相关转发行为。
- ONF定义的转发面开放协议：Openflow协议


控制器下发流
表到交换机，
控制转发行为
控制器
OpenFlow
协议
安全通道
流表
OpenFlow 交换机

北向接口

- 应用平面与控制平面之间的接口（NBI），向应用层提供抽象的网络视图，使应用能直接控制网络的行为。
- 开放的、与产商无关的接口。

北向接口关键技术

- SDN北向接口设计：控制器将网络能力封装后开放接口，供上层业务调用。
- REST API成为SDN北向接口的主流设计

- 交换机状态采集、静态流表推送、防火墙策略等多种类型的接口。

URI	方法	描述	参数
/wm/core/switch/all/<statType>/json	GET	获取跨交换机的聚合状态	statType: port, queue, flow, aggregate, desc, table, features
/wm/core/switch/<switchId>/<statType>/json	GET	获取每台交换机的工作状态	switchId: Valid Switch DPID (XX:XX:XX:XX:XX:XX:XX:XX) statType: port, queue, flow, aggregate, desc, table, features
/wm/core/counter/<counterTitle>/json	GET	获取控制器所辖交换机的全部流量计量信息	counterTitle: "all" or something of the form DPID_Port#OFEventL3/4_Type

谢谢！

《软件定义网络技术》
Software Defined Networking (SDN)

主讲人:黄辉
email:huanghui@wzu.edu.cn

SDN的核心思想

- 解耦（Decoupling ）；
- 抽象（Abstraction）；
- 可编程（Programable）

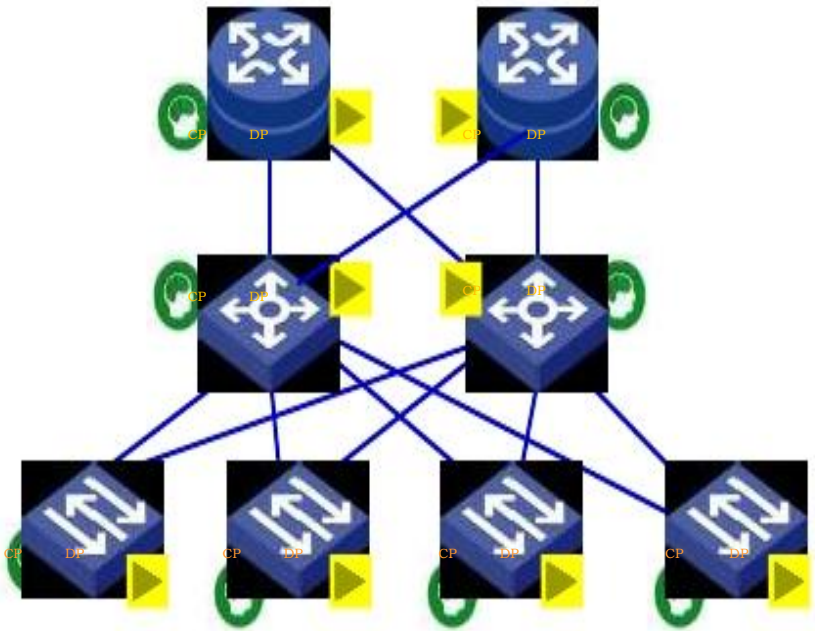
数据平面与控制平面的解耦

- 通过解耦合，控制平面负责上层的控制决策，数据平面负责数据的交换转发，双方遵循一定的开放接口进行通信；
- 实现网络逻辑集中控制的前提；
- 两个平面独立完成体系结构和技术的发展演进，有利于网络的技术创新与发展。

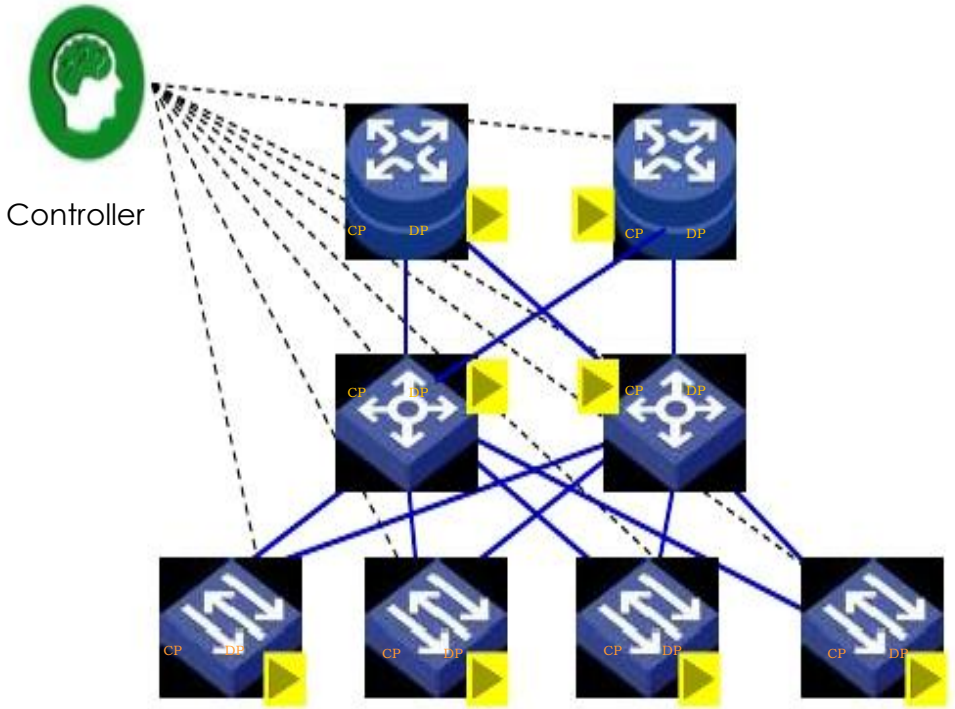
控制与转发分离

控制平面（CP）　　转发平面（DP）

- 传统网络设备的CP与DP 不分离；
- 设备之间通过控制协议交互转发信息；



- SDN 将网络设备的控制平面集中上收到Controller;
- 网络设备上只保留转发平面（转发表项）；
- 软件可以实现灵活的控制面功能满足用户多元化需求
- 硬件成为简单哑资源，专注转发；



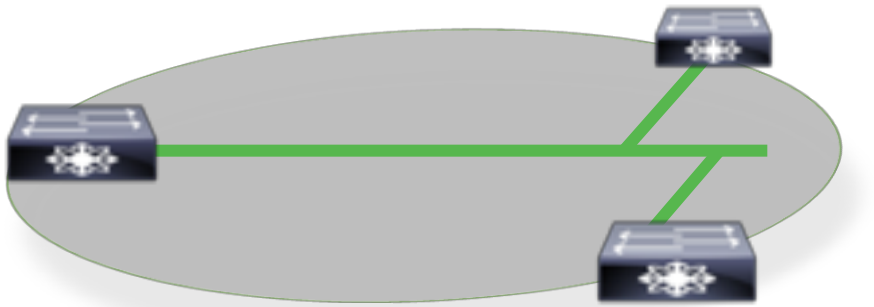
解耦带来的问题与挑战

- 网络规模的扩大，单一控制器成为网络性能的瓶颈；
- 保持分布式网络节点状态的一致性，是一个重要的挑战；
- 响应延迟，导致数据平面的可用性问题。

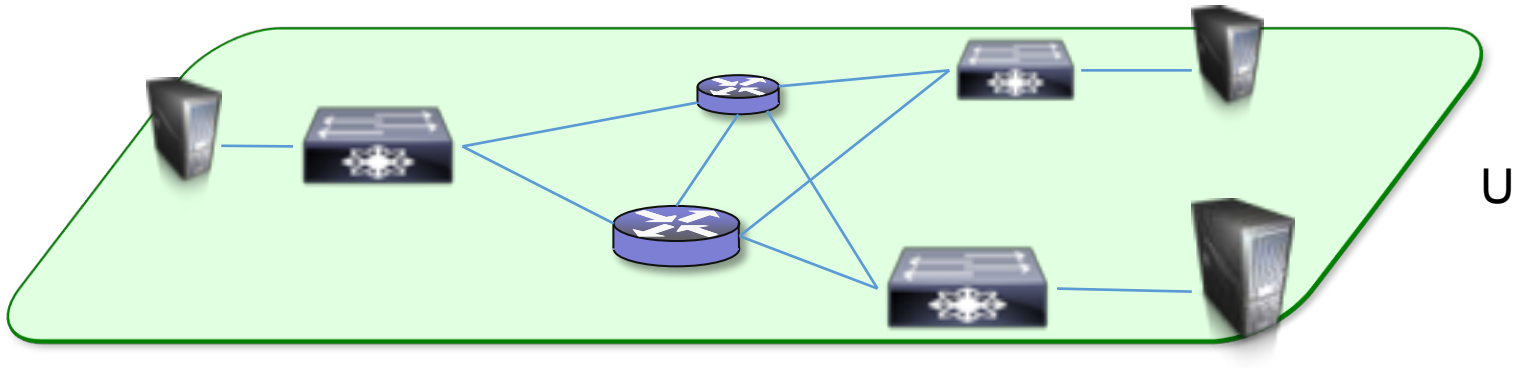
网络功能的抽象

- ONF网络架构实现转发抽象、分布状态抽象和配置抽象：
  - 转发抽象（forwarding abstration）：隐藏了底层的硬件实现，转发行为与硬件无关；
  - 分布状态抽象（distribution abstration）：屏蔽分布式控制的实现细节，为上层应用提供全局网络视图；
  - 配置抽象（specification abstration）：网络行为的表达通过网络编程语言实现，将抽象配置映射为物理配置。

- Overlay网络架构实现对基础网络设施的抽象。



Overlay网络

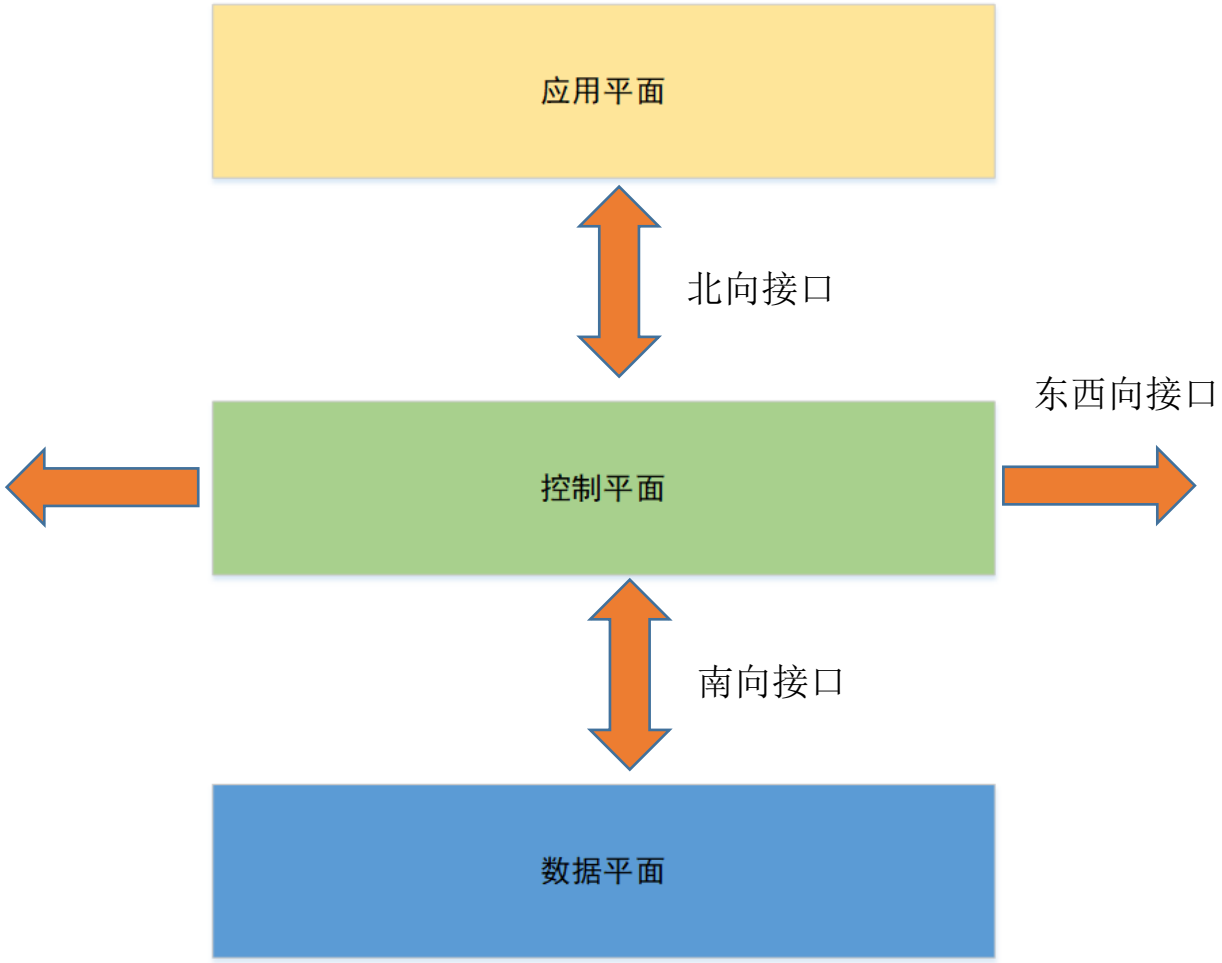


Underlay网络

网络可编程

- 传统网络的管理接口： CLI、SNMP等，是初级的网络编程方式；
- 网络管理者需要基于整个网络的，而不是基于某一设备的可编程，
- 网络可编程相关研究：主动网络（Active Networking）、4D架构

SDN开放可编程接口



SDN可编程接口

- 网络可编程接口：
  - 北向接口：REST（Representational State Transfer） API，RESTCONF协议
  - 南向接口：OpenFlow、OF-Config、NETCONF、OVSDB、XMPP、PCEP、I2RS、OPFlex等协议；
  - 东西向接口：负责控制器之间的通信，未形成统一标准。
- 数据平面可编程技术：Intel主导的DPDK、斯坦福大学主导的P4(Programming Protocol-independent Packet Processors)等

谢谢！

明德新民 | 止于至善

第三章 SDN数据平面

学院：软件学院 任课教师：申凌峰

河南大學

目 录

CONTENTS

- 01 数据平面概述
- 02 OpenFlow交换机模型
- 03 SDN交换机
- 04 通用可编程数据平面

Part
01

数据平面概述

传统网络数据平面　　SDN数据平面

数据平面的概述-传统网络数据平面

在传统网络中，网络设备的控制平面和数据平面虽然从功能逻辑上说相互独立，但它们存在于同一网络设备中，即使分别运行于独立的处理器或板卡之上，彼此之间仍然是紧耦合的关系，两个平面的功能及依赖关系在设备设计与生产时就被预定义了，且不提供基于可编程的功能扩展或变更，这种紧耦合状态导致设备功能与适用场景受限，当上层应用出现快速变化、多元化、服务质量差异化等情况时，难以提供其所需要的响应弹性和部署可扩展性。

数据平面的概述-传统网络数据平面

以路由器为例，数据平面的架构如图3.1所示，其主要功能是执行控制平面所给出的网络控制逻辑，主要处理流程包括数据包的解析与拆封、数据包转发策略匹配、数据包转发调度等。

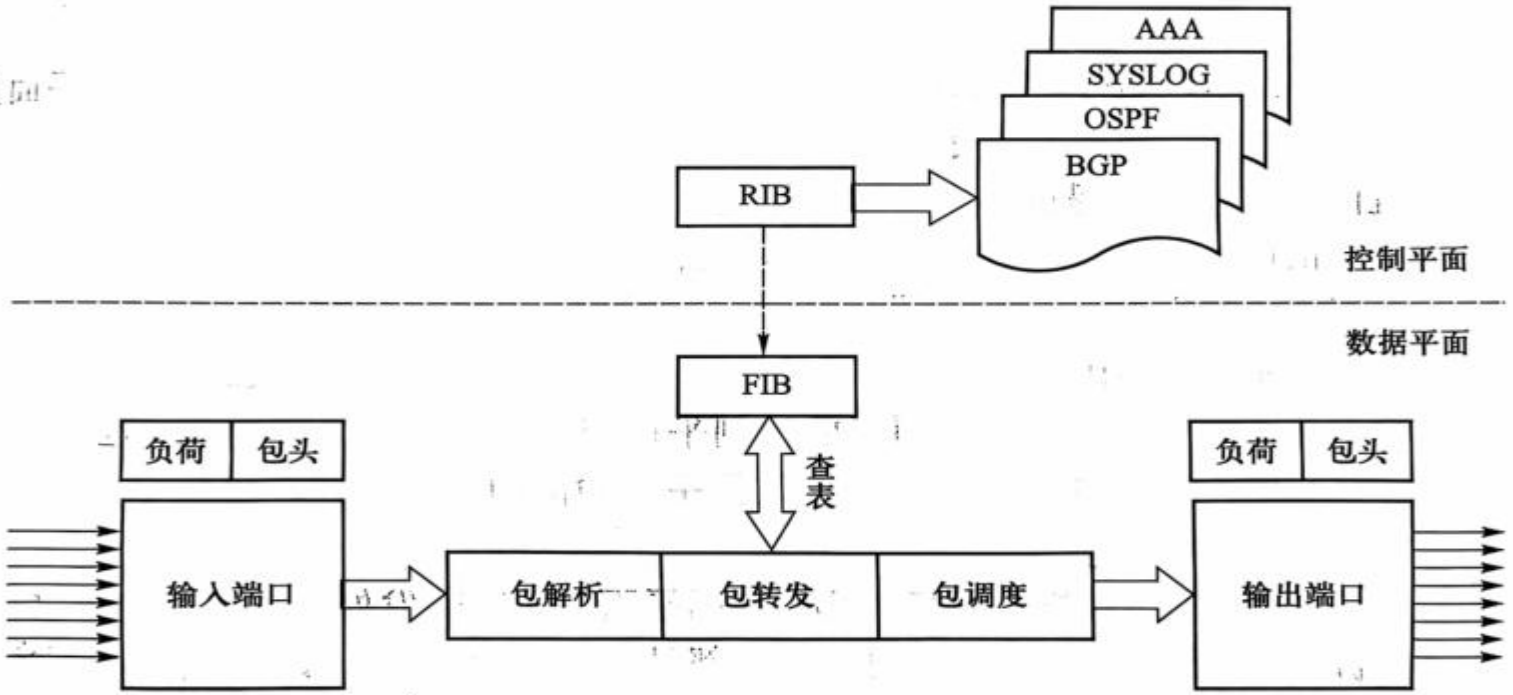


图 3.1　传统典型网络设备（路由器）的数据平面

https://www.bilibili.com/video/BV1bPgremEN3/?spm_id_from=333.337.search-card.all.click&vd_source=363a1a6da8510506c2a4be69351ff671

数据平面的概述-传统网络数据平面

传统网络设备数据平面的相应功能模块在设备生产时就已经固定，只能在预定义或预置的功能框架或管理指令集下进行有限的用户配置，不支持用户编程自定义。因此存在以下几方面的限制。

(1)只能识别、解析设计芯片时预先定义的报文类型。

(2)只能匹配特定的报文字段。

(3)只能对特定的报文执行特定的动作。

单纯从数据平面的数据传输与转发功能看，SDN与传统网络的数据有高度的相似性，均具有数据包解析、转发策略匹配、包转发调度等功能且两者的处理流程也基本一致。但是，SDN通过将数据平面与控制平面解耦并提供可编程功能，使SDN在数据平面的存在形态、运行机理和特性等方面有了很大的变化。

- **存在形态：**SDN数据平面独立存在于基础设施层设备中。
- **运行机理：**SDN数据平面中所有转发处理模块（包括数据句解析、转发策略匹配和转发调度等）都是与协议无关和通用可编程的。
- **运行特性：**SDN数据平面对上层应用与业务具有很高的可扩展性与灵活性。

数据平面的概述- SDN数据平面

斯坦福的The McKeown Group于2008年提出了基于OpenFlow标准的交换机(OpenFlow switch)模型，该模型是实现通用可编程数据平面的一次尝试，如图3.2所示。

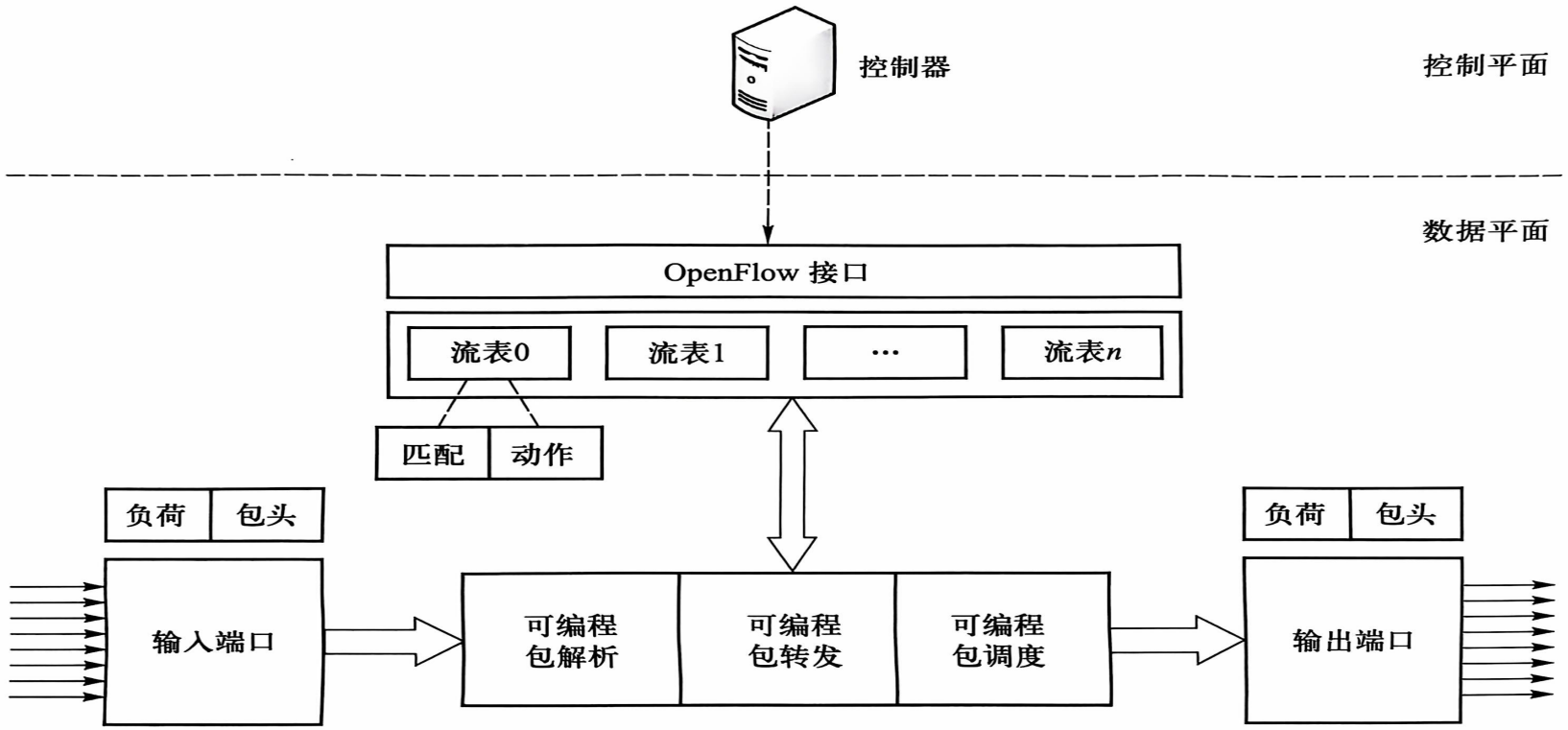


图 3.2　SDN 的数据平面

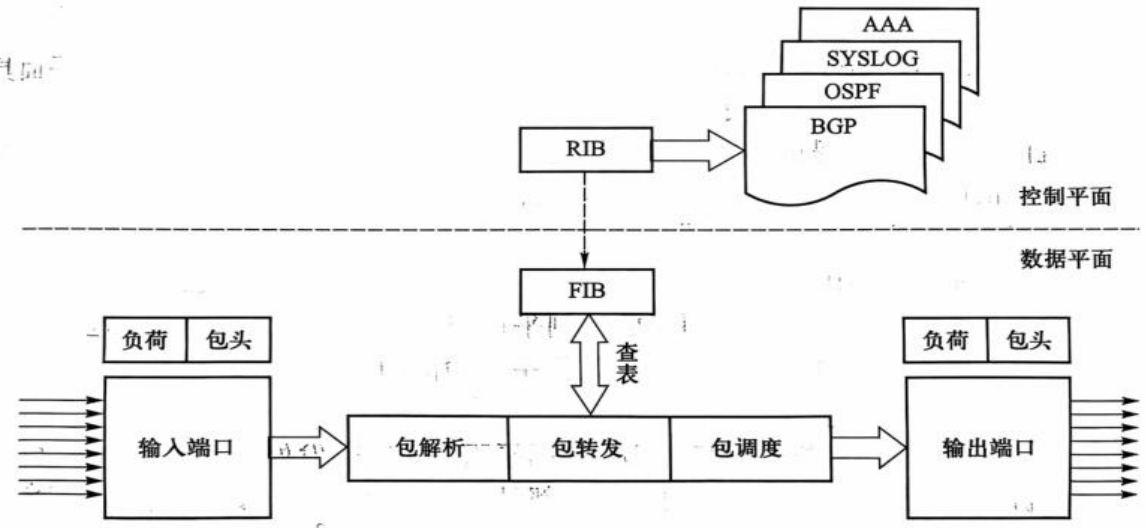


图 3.1　传统典型网络设备（路由器）的数据平面

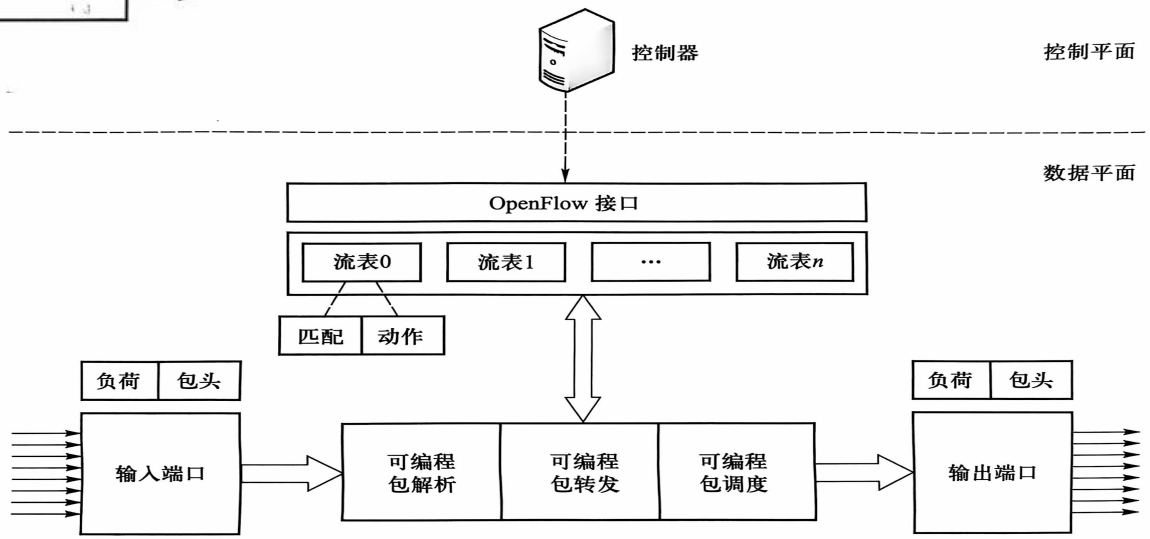


图 3.2　SDN 的数据平面

Part 02

OpenFlow 交换机模型

OpenFlow 概述

流表

计量表

OpenFlow 交换机功能架构

组表

流表匹配

各版本及主要特征性如表3.1所示：

OpenFlow v1.0：每个OpenFlow交换机中只有一张流表，并且只能同一台控制器进行通信，流表的维护也是通过控制器下发相应的OpenFlow消息来实现。流表由多个流表项组成，而每个流表项就是一个转发规则。流表项由匹配字段、计数器和动作组成，同时， v1.0只支持IPv4。

OpenFlow v1.1：支持多级流表，形成流水线处理流表匹配架构，这一架构能避免单流表过度膨胀问题，也能更好地利用硬件内部固有的多表特性。v1.1版本和 v1.0版本存在不兼容问题，而后续版本都是在v1.1版本基础上发展而来的。

OpenFlow v1.2：不再采用固定长度的结构，而是采用TLV(Type/Length/Value)结构来定义匹配字段，称为OpenFlow可扩展匹配(OXM)，协议有更灵活的可扩展性，既能增加关键字匹配字段，也能节省流表空间。同时，该协议允许一台交换机与多台控制器连接以增强可靠性，通过控制器命令消息变换角色。更重要的是，从 v1.2开始支持IPv6协议。

版 本 号	发布时间	主要特性
v1.0	2009.12	单流表，只支持 IPv4
v1.1	2011.02	支持多级流表、组表、MPLS 和 VLAN 标签
v1.2	2011.12	支持可扩展匹配、多控制器、IPv6 基本头部
v1.3	2012.04	支持计量表、IPv6 扩展头部
v1.4	2013.10	支持流表同步机制、光端口
v1.5	2014.12	支持出向流表

OpenFlow v1.3:是长期支持的稳定版，支持的匹配关键字增加到40个，足以满足网络应用的需要；增加了计量表，以此控制关联流表的数据包传输效率；同时允许交换机和控制器根据自身需要协商所采用的OpenFlow版本。

OpenFlow v1.4：主要是增加了流表同步机制，能让多个流表共享相同的匹配字段，而且可以定义不同的动作。此外，新增加了Bundle消息，确保控制器下发一组完整消息或同时向多个交换机下发消息的状态的一致性。

由于OpenFlow v1.0版本和其他各版本之间不兼容，因此开放网络基金会将OpenFlow v1.0和OpenFlow v1.3版本作为长期支持的稳定版本。

OpenFlow 交换机模型- OpenFlow 交换机模型架构

OpenFlow 规范不仅定义了控制器与交换机之间的通信协议，同时还描述了OpenFlow交换机的功能架构，以OpenFlow v1.0为例，其交换机架构如图3.3所示。

在该架构中，至少包括3个组成部分。

(1)流表(flow table)：由若干流表项(flow entry)构成，每个动作(action)关联一个流表项，指示交换机如何进行流的处理。

(2)安全通道(secure channel)：OpenFlow交换机通过安全通道与远端控制器连接，负责控制器与交换机之间的交互。

(3)OpenFlow协议：定义了一种南向接口标准，为控制器与交换机之间的通信提供了一种开放标准。

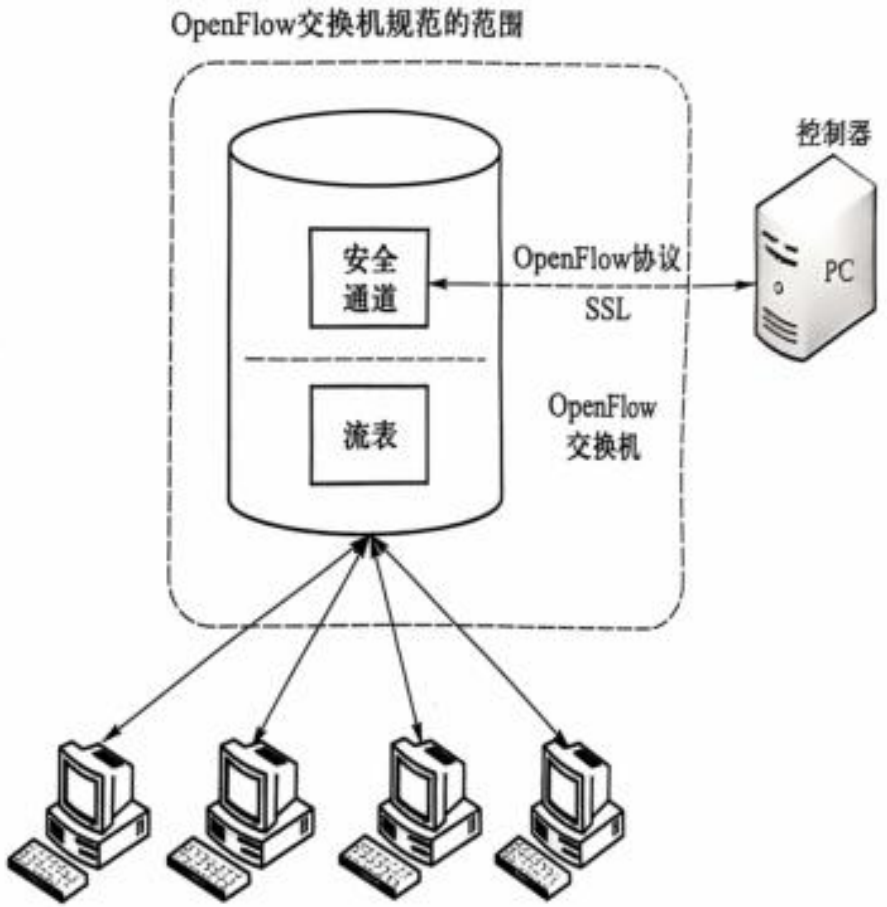
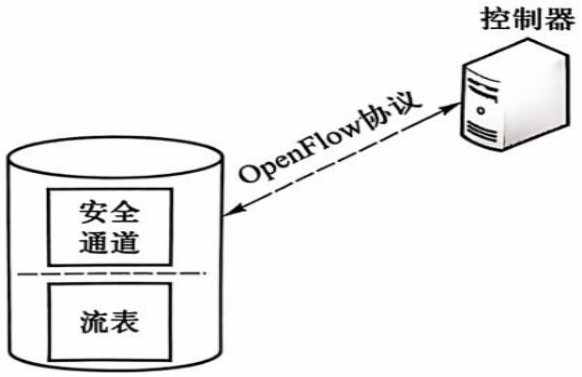


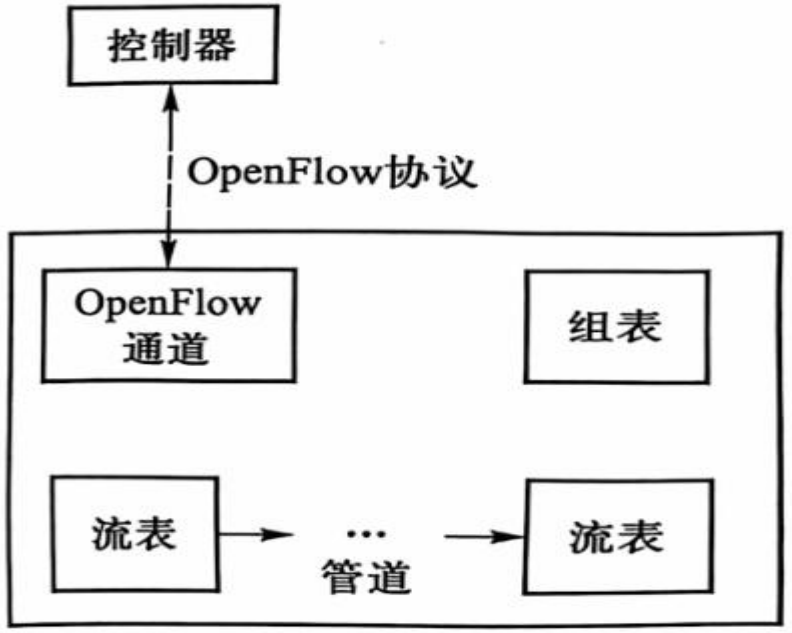
图 3.3　OpenFlow v1.0 交换机架构

OpenFlow 交换机模型- OpenFlow 交换机模型架构

以上是OpenFlow提出之初的功能架构，在OpenFlow规范的演变过程中，交换机的架构发生了较大的变化，右图a给出了OpenFlow v1.0规范的交换机架构，该架构体现了简单的单流表设计，数据包的匹配在这张流表中完成，这张流表承载了传统网络OSI模型中第4~7层的大部分功能。



(a) OpenFlow v1.0



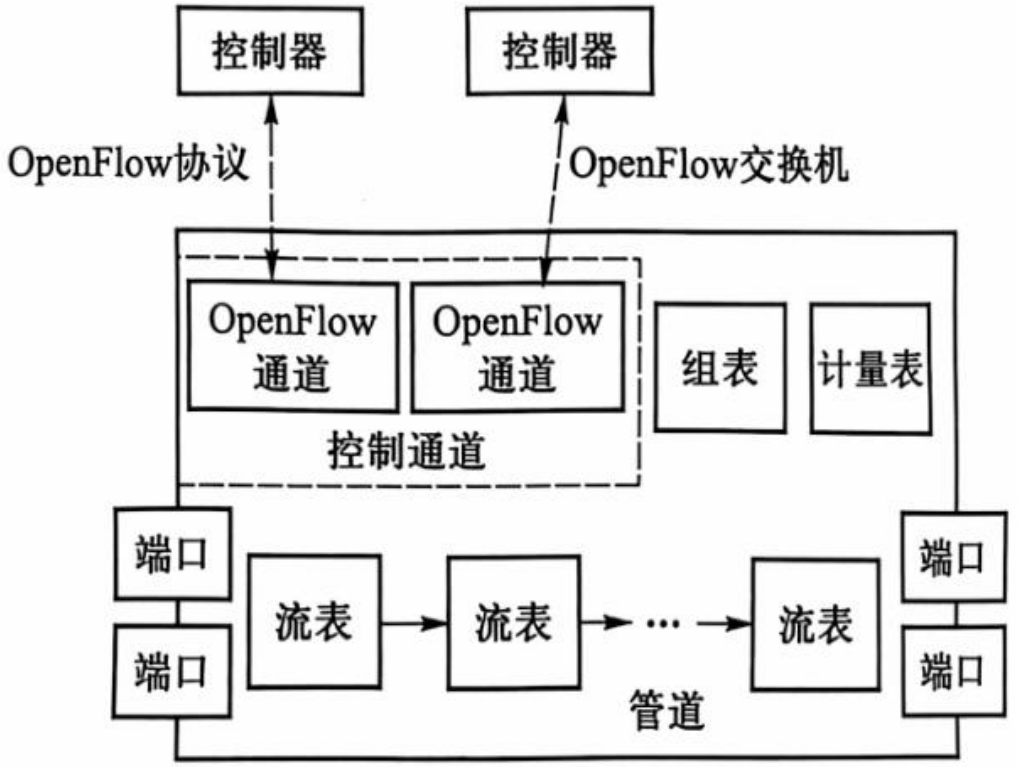
(b) OpenFlow v1.1~v1.4

随着网络需求越来越多样化，流表变得十分臃肿，对硬件性能的要求也越来越高。为解决这一问题，OpenFlow v1.1设计了“多级流表+组表”的交换机架构，左图3.4(b)显示的是OpenFlow v1.1~v1.4所沿用的“多级流表+组表”的交换机架构，多张流表串联起来形成了流水线(pipeline)，数据包的匹配在流水线的处理过程中完成。

OpenFlow 交换机模型- OpenFlow 交换机模型架构

右图3.4(c)是OpenFlowv1.5的交换机架构，该架构中流水线处理包括入向(ingress)匹配和出向(egress)匹配两个阶段。

在OpenFlow版本演进过程中，交换机在功能架构上的主要变化体现在核心部件和处理流程两方面，在核心部件方面，在流表的基础上增加了组表、计量表；在流程处理上，由单流表的处理演变成多级流表的流水线处理方式。后续将对OpenFlow交换机的核心部件以及流表匹配处理进行讲解。



(c) OpenFlow v1.5

图 3.4　OpenFlow 版本规范对应的交换机架构变化

OpenFlow 交换机模型- 流表

OpenFlow交换机模型采用基于流的数据转发机制。在该机制中，每个数据分组都属于某个流。流是对同一网络中具有某种共同特征或属性的数据分组所进行的抽象，指从一个网络端 （或一组端点）向另一个网络端点（或一组端点）传输的一组数据分组。流是单向的，两组端点相同、方向相反的分组属于不同的流。流在设备中以流表项的形式体现。

流表则是特定流的转发策略表项的集合，是OpenFlow交换机中对数据转发逻辑的抽象，也是OpenFlow交换机的核心数据结构。简单来说，流表就是由一些具有优先级顺序的流表项构成的。流表与传统网络设备中的MAC地址表、路由表类似，但又有所不同，它可以包含更多层次与类型的网络特征信息，因为一个OpenFlow交换机可以集交换机、路由器、防火墙等功能于一身，以对上层应用与业务部署提供灵活的响应。因此，流表使得SDN网络设备变得更加通用，并且具有可编程性。

1. OpenFlow v1.0 流表项

下面以OpenFlow v1.0版本为例，介绍流表项的组成。在OpenFlow v1.0版本中，每个OpenFlow交换机仅有一张流表，每张流表存储着一系列的流表项，所有经过交换机的数据分组都要先匹配流表，如果找到匹配项，则根据流表项的动作实施相应的数据包处理；否则，通过安全通道转发给控制器。

在OpenFlow v1.0中，流表项由3个部分组成，分别是包头域、计数器、动作表，如图3.5所示。

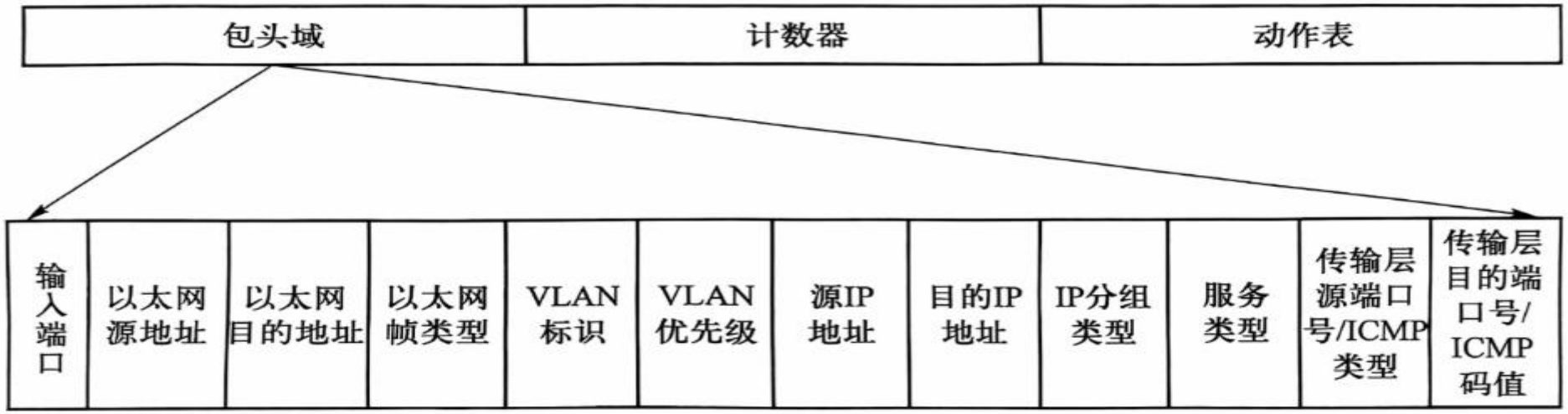


图 3.5 流表项的结构

OpenFlow 交换机模型- 流表

1）包头域

包头域是用于和交换机接收到的数据包进行匹配的元组，包含12个字段，称为12元组(12-tuple)，这些字段涵盖了OSI网络模型中第2~4层的网络特征。每一个元组中的数值可以是一个确定的值或者是ANY，ANY表示支持匹配任意值。另外，如果交换机能够在IP地址相关元组上支持子网掩码，将有助于实现更精确的匹配，每个字段具体介绍如表3.2所示

表 3.2 包头域各元组简介

字 段	位数/b	适用范围	备 注
输入端口	未规定	所有数据分组	数值表示输入端口，从 1 开始
以太网源地址	48	有效端口收到的数据分组	
以太网目的地址	48	有效端口收到的数据分组	
以太网帧类型	16	有效端口收到的数据分组	OpenFlow 交换机需要与标准以太网和 IEEE 802.2 中的类型匹配
VLAN 标识	12	帧类型为 0x8100 的数据分组	
VLAN 优先级	3	帧类型为 0x8100 的数据分组	VLAN PCP 字段
源 IP 地址	32	IP 和 ARP 分组	可划分子网
目的 IP 地址	32	IP 和 ARP 分组	可划分子网
IP 分组类型	8	IP 和 ARP 分组	仅使用 ARP Opcode 的低 8 位
服务类型（ToS）	6	IP 分组	高 6 位为（ToS）
传输层源端口号/ICMP 类型	16	TCP、UDP 和 ICMP 分组	如果分组类型是 ICMP，低 8 位用于标识 ICMP 类型
传输层目的端口号/ICMP 码值	16	TCP、UDP 和 ICMP 分组	如果分组类型是 ICMP，低 8 位用于标识 ICMP 码值

2）计数器

计数器主要对交换机中的每张流表、每个数据流、每个设备端口及每个转发队列进行维护，用于统计数据流量的相关信息。例如，针对每张流表，可统计当前活动的表项数、数据包查询次数、数据包匹配次数等；针对每个数据流，可统计接收的数据包数、字节数、数据流持续时间等；针对每个设备端口，除了统计接收的数据包数、发送数据包数、接收字节数、发送字节数等指标之外，还可以对各种错误发生的次数进行统计；针对每个队列，可统计发送的数据包数和字节数，还有发送时的溢出(overrun)错误次数等。

3）动作表

OpenFlow流表项的动作表用于指示交换机如何对接收到的匹配数据包进行处理。

流表项的动作分为必备动作(required action)和可选动作(optional action)两种类型。其中，必备动作是所有OpenFlow交换机默认支持的，而可选动作是交换机所能支持的可选动作种类（由交换机告知控制器）。OpenFlow流表动作的列表如表3.3所示。

表 3.3 OpenFlow 流表动作列表

动作类型	具体动作	动作说明
必备动作	转发（forward）	转发数据包到物理端口或以下几类虚拟端口：LOCAL、ALL、CONTROLLER、IN_PORT、TABLE
	丢弃（drop）	丢弃所有匹配的数据包
可选动作	转发（forward）	转发数据包到两类虚拟端口：NORMAL、FLOOD
	入队（enqueue）	转发数据包到端口绑定的队列，提供基本的QoS 支持
	修改包头（modify-field）	修改数据包的包头，可以是 12 元组中的任意字段

3）动作表

流表项支持的动作主要包括转发匹配的数据包或者丢弃这个数据包，最常见的情况是为数据包指定一个物理端口(physical port)进行转发。除了物理端口外，在OpenFlow v1.0中，还定义了几类虚拟端口(virtual port)：LOCAL、ALL、CONTROLLER、IN_PORT、TABLE。各虚拟端口的作用如下。

- **LOCAL**：转发给本地的网络栈。
- **ALL**：转发给所有输出端口，但不包括输入端口。
- **CONTROLLER**：封装数据包并转发给控制器。
- **IN_PORT**：从输入端口发出。
- **TABLE**：该端口只适于从控制器发送到交换机的数据包。
- **NORMAL**：该端口如果接收到数据包，将按照传统交换机的转发逻辑进行处理。
- **FLOOD**：该端口将数据包洪泛到生成树链路上，不包括输入端口。

2. OpenFlow 流表项的结构演变

随着OpenFlow协议的版本迭代，流表项的结构随之有所更新与变化。各版本流表项结构的变化如图3.6所示。

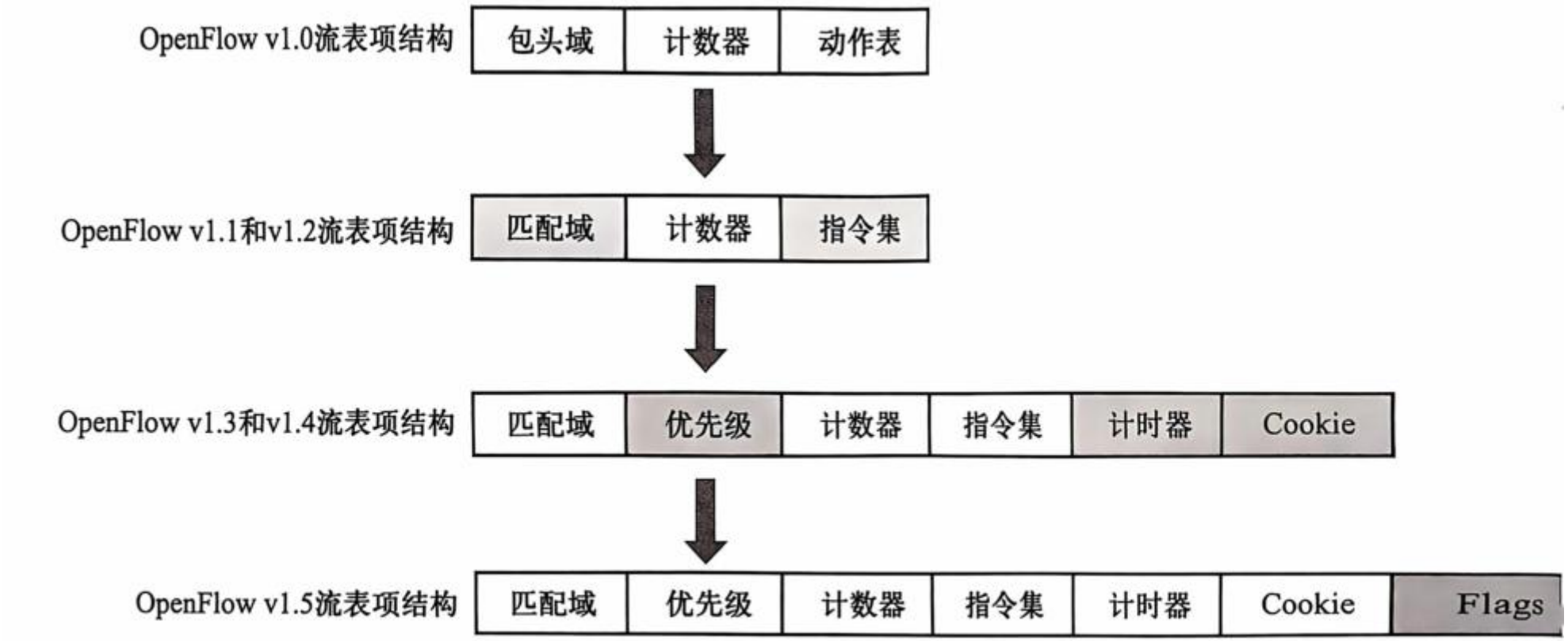


图 3.6　OpenFlow 各版本流表项结构

组表(group table)是OpenFlow vl.1版本中新增的模块。组是OpenFlow为数据包指定在多个流中执行相同操作集的一种高效方法。一个组表由多个组表项(group entry)组成，组表项的结构如图3.7所示。

图 3.7　组表项的结构

(1)组表号(group identifier)：32位无符号整数，是组表项的唯一标识。
(2)组表类型(group type)：决定了组表的语义。
(3)计数器(counter)：被该组表项处理过的数据包的数量。
(4)动作桶(action bucket)：有序列表，包含动作集合及其参数。

OpenFlow交换机提供了更高级的数据包转发特性，可实现组播、负载均衡、容灾备份和聚合等功能。

计量表是OpenFlow v1.3版本中提出的，计量的定义以流为基础，每个计量表由多个计量表项(meter entry)组成，计量表项的结构如图3.8所示。

图 3.8　计量表项结构

(1)计量表号(meter identifier)：32位无符号整数，是计量表项的唯一标识。
(2)计量带(meter band)：无序集合，指明带宽速率以及处理数据包的行为。
(3)计数器(counter)：被该计量表项处理过的数据包数量。

1. 单一流表匹配处理

在OpenFlow v1.0中，每个OpenFlow交换机只有一张流表。单一流表匹配相对简单，相当于单一数据库表的查找与匹配过程。如图3.9所示，在数据包进入OpenFlow交换机后，首先进行包头的解析，然后进入匹配阶段。

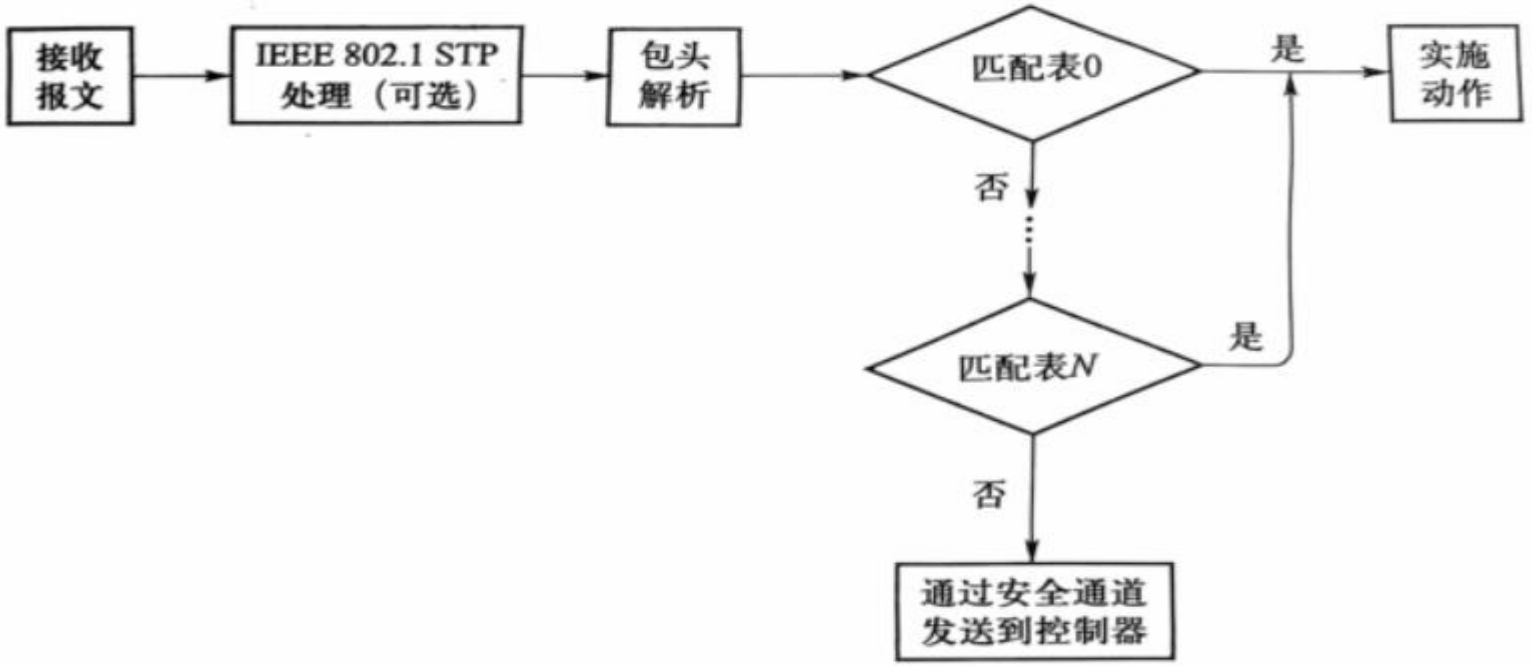


图 3.9　数据包匹配处理流程图

OpenFlow 交换机模型- 流表匹配

包头解析是为了得到数据包的12元组。

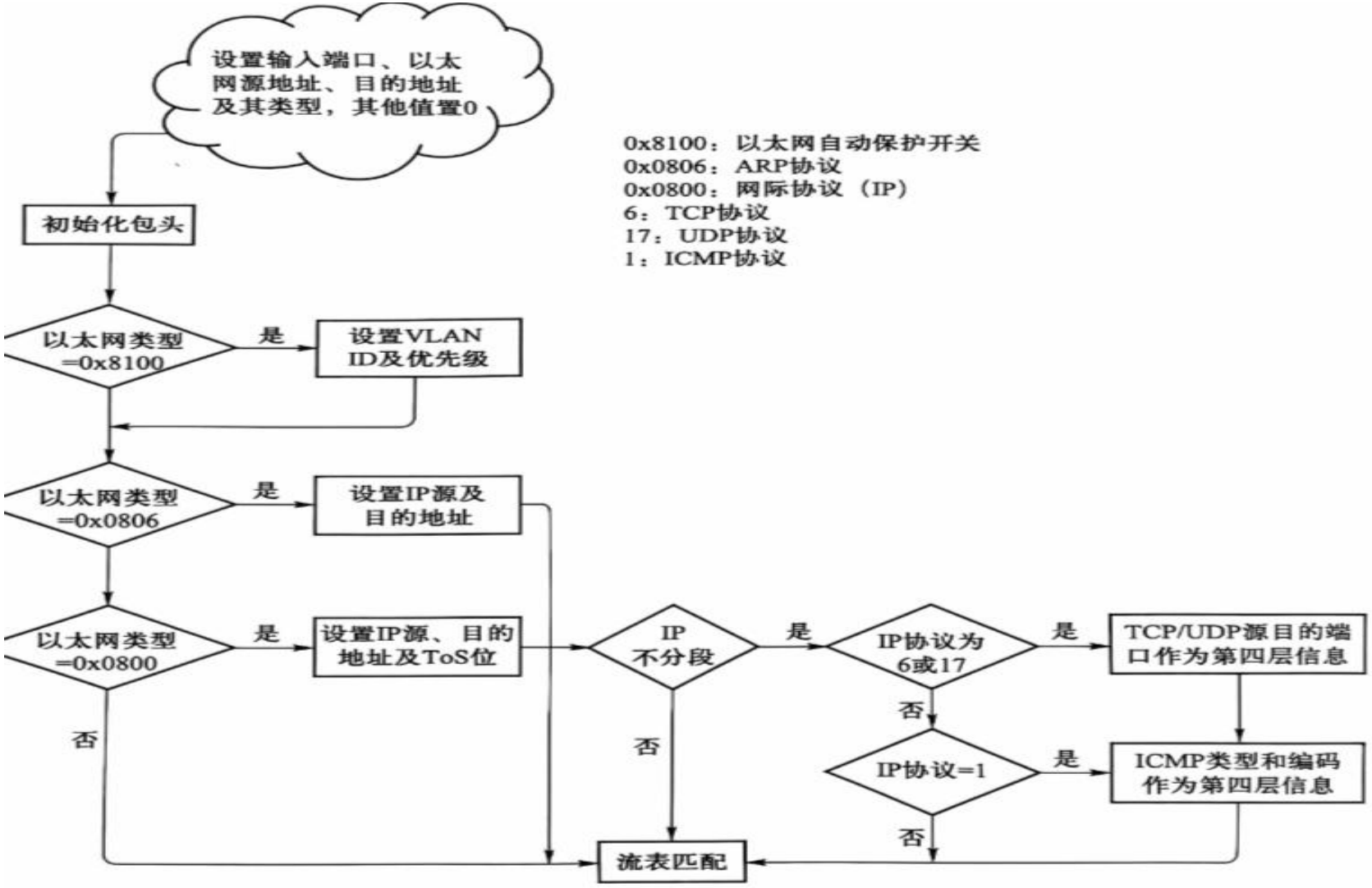


图 3.10　数据包头解析过程图

2. 多流表匹配处理

OpenFlow v1.0的单流表处理模式简单、易实现，但在网络需求变得越来越复杂时，各种策略放在同一张流表中就显得十分臃肿，而且影响查找与匹配效率。因此，从OpenFlow v1.1开始，提出了多流表(multiple flow table)架构。在多流表架构中，数据包在交换机中的匹配是以流水线的方式处理的，如图3.11所示，并在后续版本中沿用了这一处理方式。

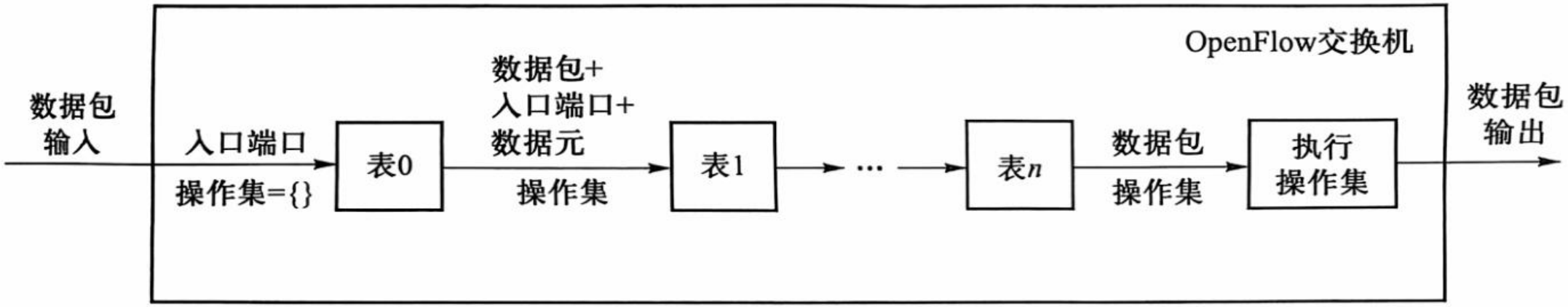


图 3.11　多流表匹配的流水线处理过程

3. 流表项匹配实例分析

例3.1

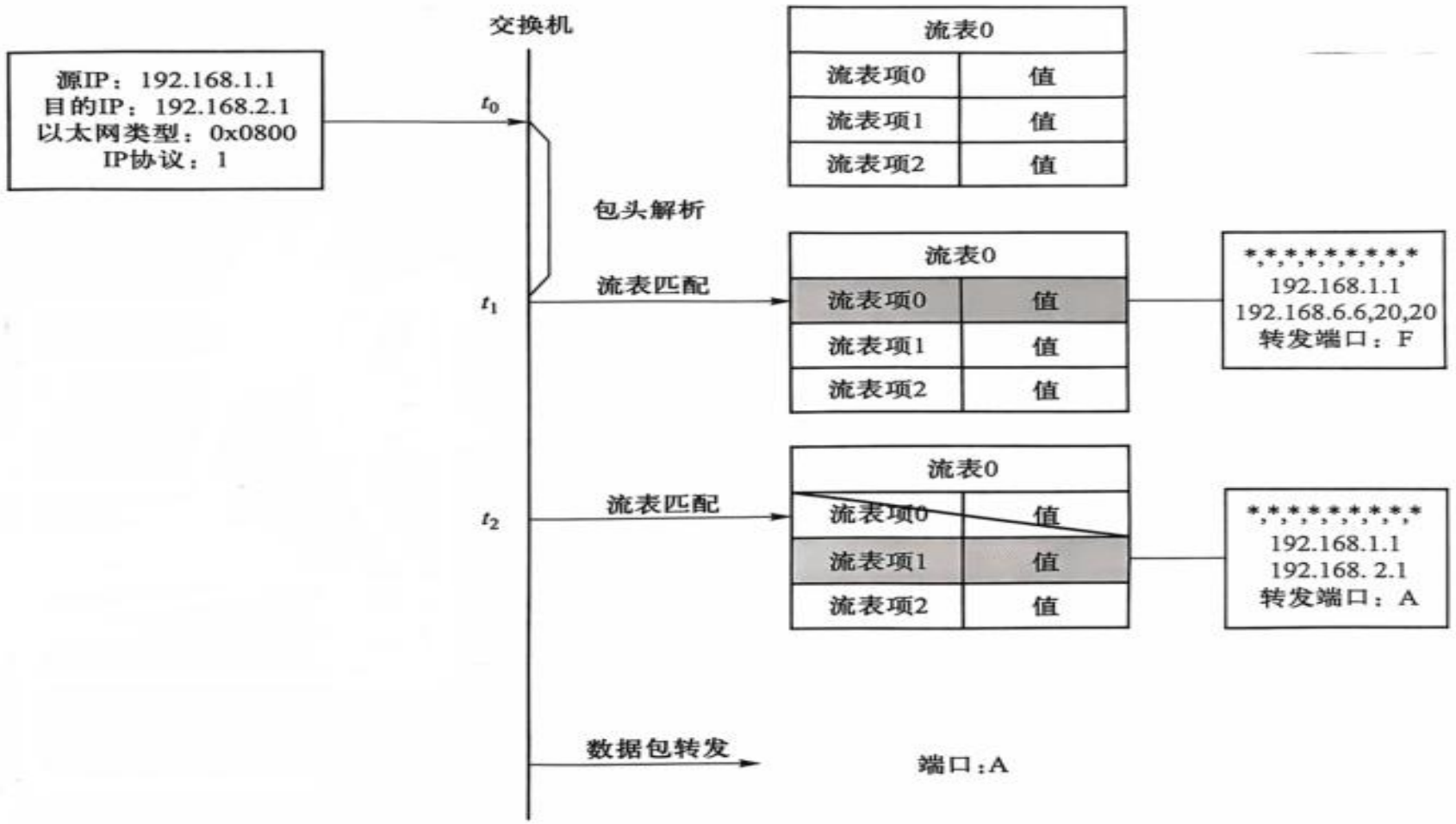
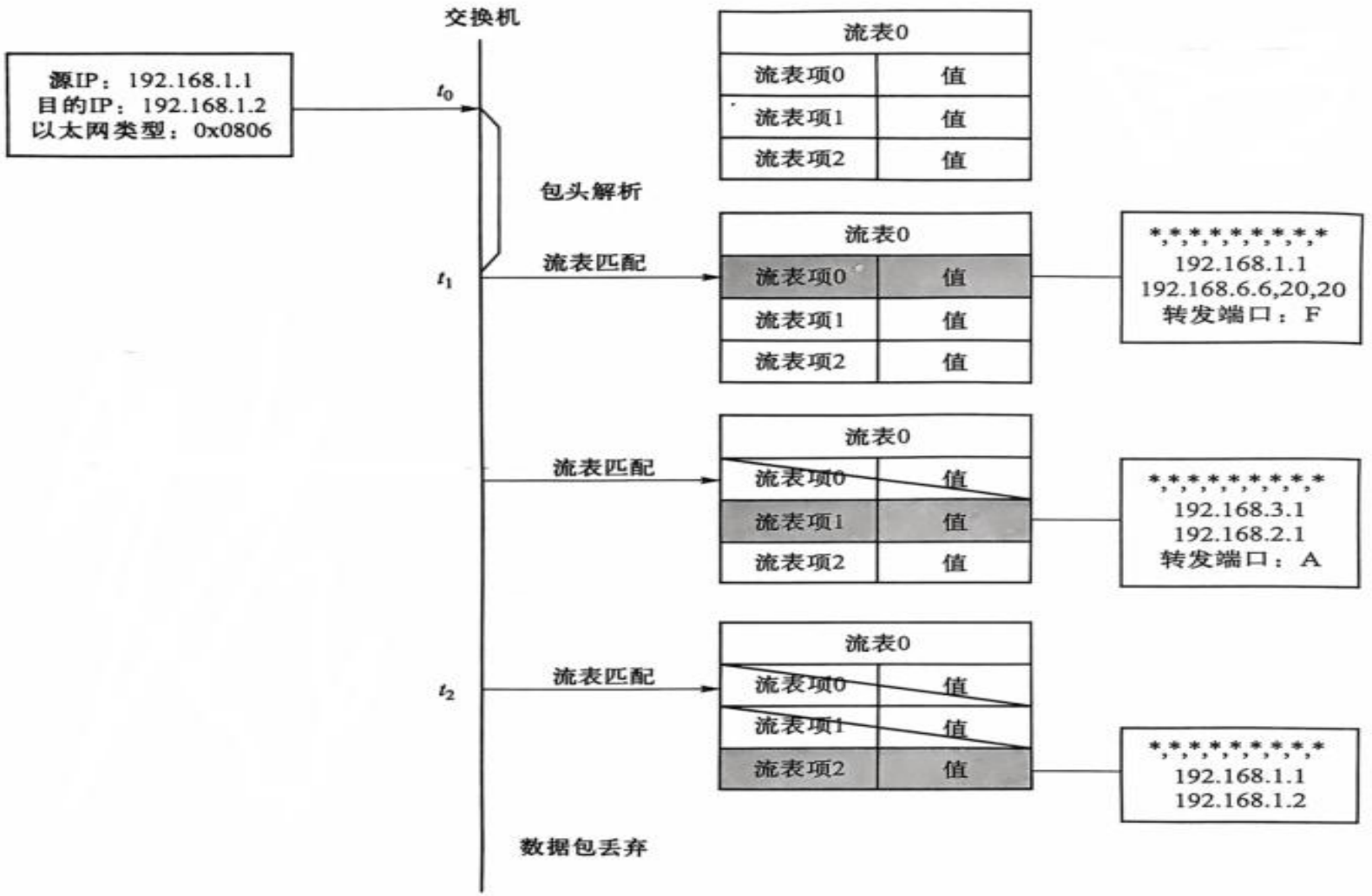


图 3. 12　ICMP 数据包流表匹配过程示意

3. 流表项匹配实例分析

例3.2



3. 流表项匹配实例分析

例3.3

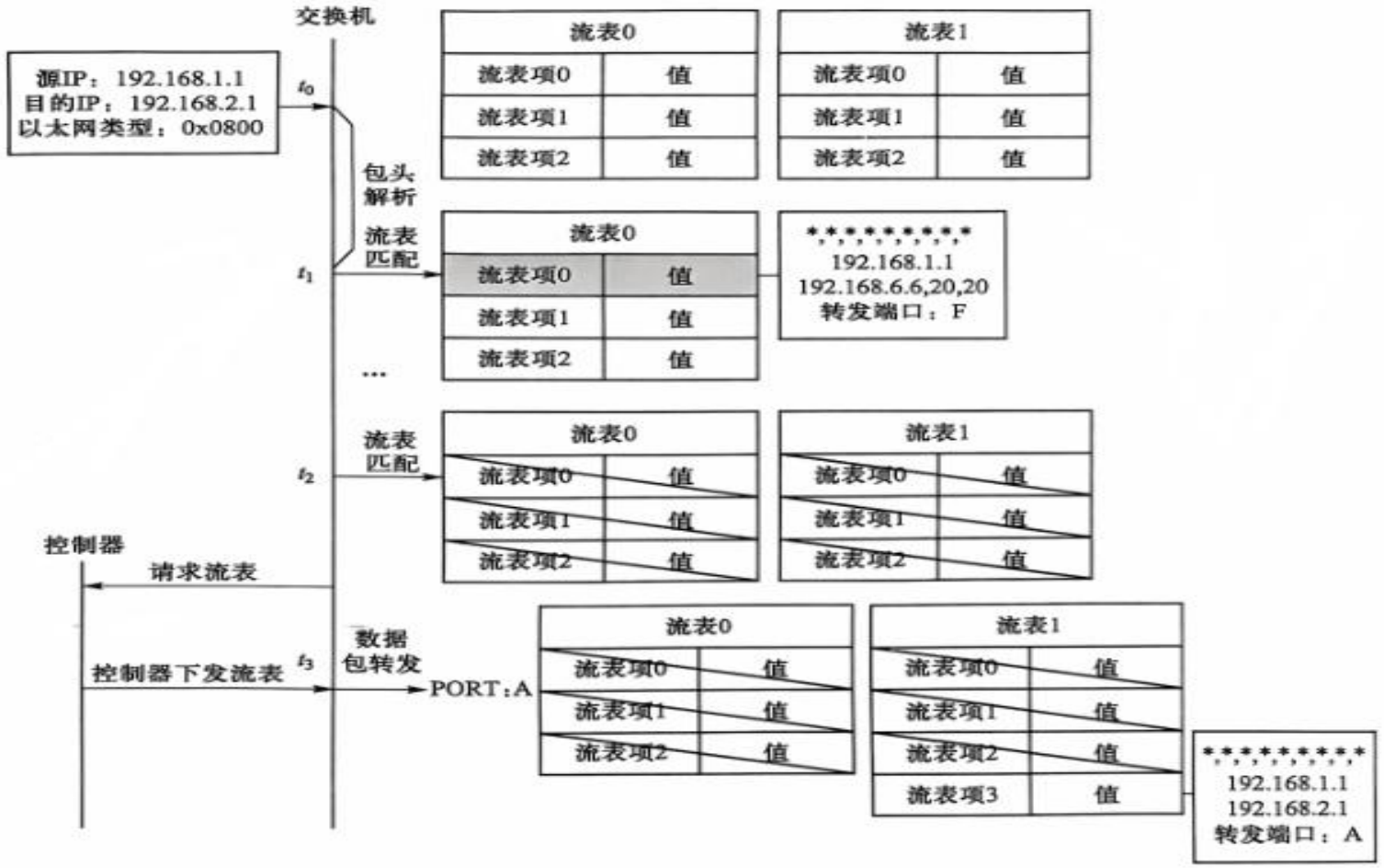
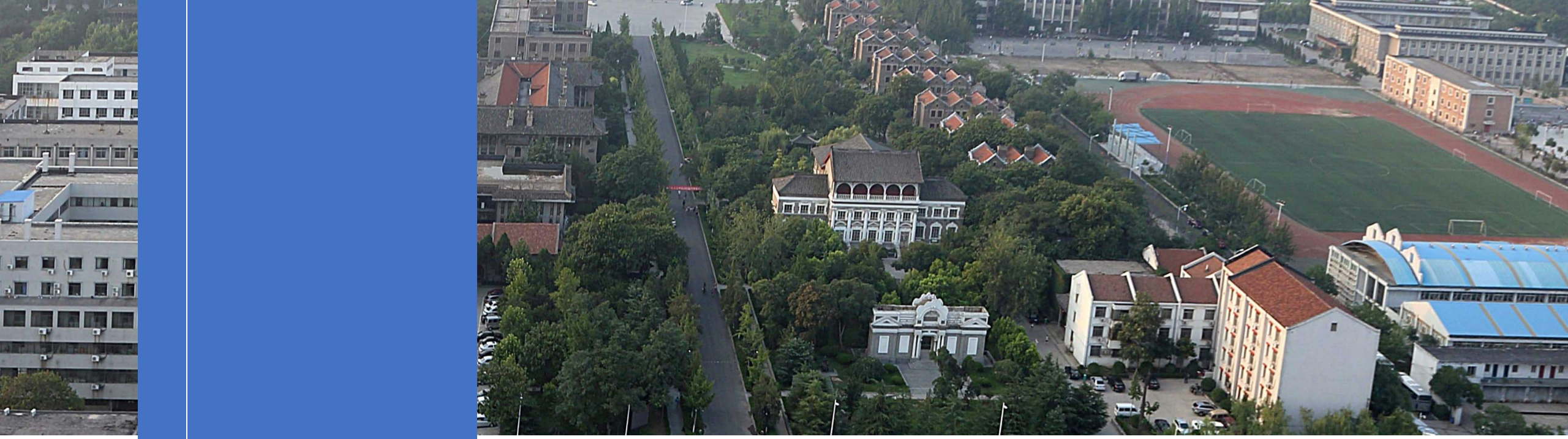


图 3.14 多流表匹配过程示意



Part 03

SDN交换机

交换机芯片概述

交换机芯片产品

SDN交换机产品

传统网络设备中用于实现数据平面交换功能的常用芯片类型主要有通用CPU、专用集成电路(application specific integrated circuit, ASIC)、现场可编程门阵列(field programmable gate array, FPGA)和网络处理器(network processor, NP)。各种类型芯片的比较如下表3.4所示。

表 3.4 常用交换芯片比较

芯片类型	优　　点	缺　　点	应 用 场 景
通用 CPU	易扩展、通用性强	处理性能较低	应用于网络设备的控制和管理
ASIC	性能高、处理能力很强	不易扩展、研发周期长	应用于实现各种成熟的协议
FPGA	支持反复擦写、可编程	处理能力有限	应用于科研和验证
NP	可编程、可进行复杂的多业务扩展	性能不及 ASIC	应用于路由器、防火墙等协议更为复杂的网络设备

由于SDN数据平面架构改变了传统的数据转发处理与运行机制，因此，如何设计专门满足SDN交换设备的交换芯片是业界需要解决的重要问题。

基于开源技术的SDN交换设备而言，目前在数据平面的主要技术是Open-Flow，但OpenFlow对交换芯片的设计要求与传统芯片技术有根本性的差异，其所要求的灵活匹配规则、流表和多级流表的架构，对交换芯片的设计提出了很大的挑战。

基于私有协议或技术开发的SDN交换设备以及具有SDN功能特性的网络设备而言，因为目前关于SDN体系结构并没有统一的国际标准或行业事实标准可供遵循，所以不同厂商的SDN交换产品在硬件架构与实现方式上存在较大的差异，也就难以为其提供通用的交换芯片。目前，基本上采用厂商专门定制的方式进行芯片的选择与设计。

1. 博通StrataXGS系列交换芯片

2011年，全球有线和无线通信半导体创新解决方案的领导者博通公司为满足新一代运营商以太网汇聚平台对带宽、可扩展性和效率的要求，推出了高性能StrataXGS交换解决方案。

StrataXGS解决方案中包括Trident和Tomahawk 两个系列的芯片，分别应用于不同的用户场景。其中Trident3对交换芯片架构做了较大的改进，支持可编程的报文处理，具体功能特性如下：

(1)采用了5级可编程的流水线来增加包处理的灵活性。

(2)通过可配置数据库对报文的解析、查找和修改过程进行编程。

(3) 支持新一代的Overlay 协议。

(4)通过编程可实现增强网络遥测。

2. 英特尔 Barefoot Tofino系列交换芯片

Tofino芯片基于通用可编程数据平面PISA架构，并支持数据平面编程语言P4。最新的Tofino2 ASCI芯片采用7nm制造工艺，单个芯片上最多可支持32个400千兆以太网端口(400GbE)，提供高达12.8Tbps的交换处理能力，可满足大型数据中心、云和服务提供商网络的高速大容量交换需求。

Tofino2分为U、M和H三个子系列。其中，U系列针对高容量(high capac-ity)和高特征(high feature)环境，具有交换处理能力强、延时低等特点；M系列针对的是主流企业和数据中心Spine-Leaf（脊与叶）架构；H系列针对缩小的超大规模环境进行了功率和效率优化，这些环境具有较低的复杂性并且需要低延迟。

3. 盛科系列芯片

盛科网络是国内一家以太网交换核心芯片供应商，其产品致力于SDN和白牌交换机在运营商、企业网以及数据中心网络的部署和应用。其产品及其主要功能如表3.5所示。

表 3.5 盛科主要芯片产品与功能

功能与应用领域	芯片型号	
	CTC5160	CTC7148
SDN 功能	支持 OpenFlow v1.3，32 KB 哈希流表	大象流检测，基于 LAG/ECMP 的流量自愈，实时网络延时监控，可视化网络管理，支持 VXLAN、VXLAN-GPE、NVGRE 及 GENEVE
应用领域	城域网接入/汇聚，PTN/IP RAN，企业网接入/汇聚	企业网，城域以太网/PTN/IP RAN，XGPON / NGPON 线卡、IP 存储

伴随着OpenFlow技术的不断成熟以及交换芯片的支持，设备厂商推出了各种支持OpenFlow的交换机商用产品，包括物理交换机和虚拟交换机。考虑SDN网络与传统网络的兼容性问题，多数厂商推出的物理交换机都是混合模式的交换机。而不是纯SDN交换机。混合模式的交换机就是在传统交换设备的基础上增加对OpenFlow协议的支持。本小节对主流物理交换机与虚拟交换机做一简单介绍。

1. 物理交换机

物理交换机又称硬件交换机。根据交换芯片的类型，SDN物理交换机根据采用的芯片不同主要分为以下三类。

1）基于ASIC的SDN交换机

又可细分为基于ASIC的SDN品牌交换机和基于ASIC的SDN白盒交换机。

2)基于NP的SDN交换机

网络处理器(NP)是一种可编程的处理器，利用众多并行运转的微码处理器进行复杂的多业务扩展，适于实现创新或未成熟的业务。但性能和ASIC相比仍存在差距，多用于路由器、防火墙等协议中更为复杂灵活的网络设备。

3)基于FPGA的SDN交换机

FPGA是一种门列阵芯片，它支持反复擦写，可以通过编程改变电路结构，以实现不同的网络功能，但其处理能力有限，难以实现大规模的网络转发，因此FPGA主要应用于科研和验证。

2. 虚拟交换机

虚拟交换机由于硬件交换机的研发投入较高，而虚拟交换机通过软件实现，因此成本更低、配置更为灵活，虽然性能与硬件交换机有一定差距，但基本能满足中小网络的要求。虚拟交换机又称软件交换机。目前已经实现的开源虚拟交换机主要有Open vSwitch (OVS)、Indigo、 Ofsoftswitch13、LINC等。

OVS是目前使用最为广泛的主流开源交换机，它由NiciraNetworks发，是一款产品质量级的多层虚拟交换机，遵循Apache 2.0许可证。 主导研运行在虚拟化平台（例如KVM、Xen、VirtualBox）上，为动态变化的端点提像OVS可以二层交换功能，可很好地控制虚拟网络中的访问策略、网络隔离、流量监控等。

2. 虚拟交换机

图3.15是OVS各部分的构成与接口。OVS包含三个重要的组件：数据库服务、守护进程以及转发通路。

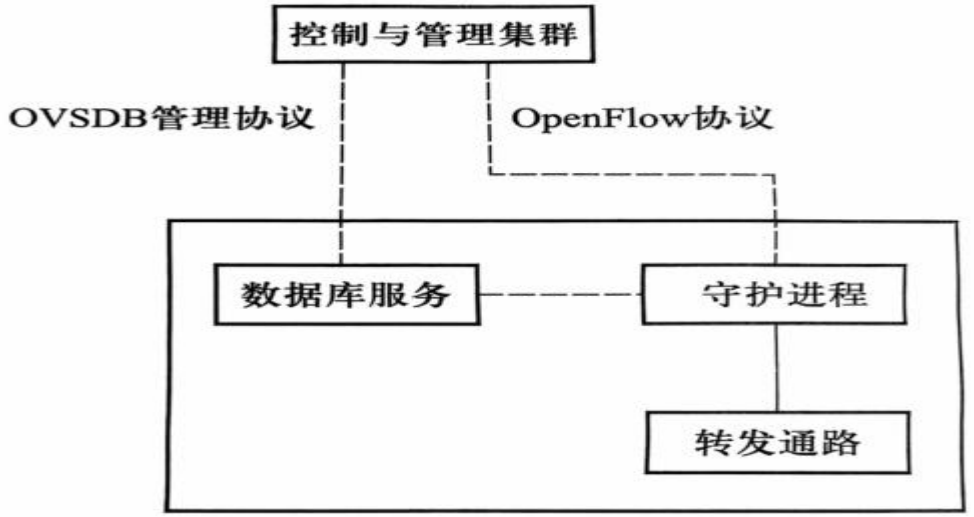
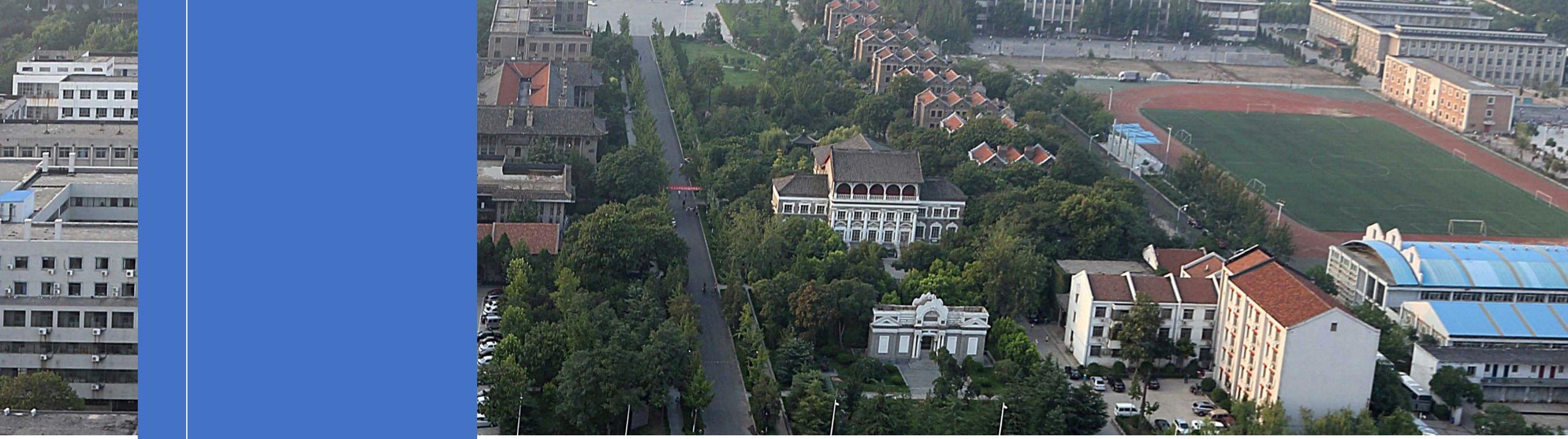


图 3.15 OVS 各部分的构成与接口

(1)数据库服务：OVS的数据库服务进程，用于存储虚拟交换机的配置信息（比如网桥、端口等），为控制器和守护进程提供OVSDB操作接口。

(2)守护进程：OVS的核心组件，负责保存和管理控制器下发的所有流表，为OVS的内核模块提供流表查询功能，并为控制器提供OpenFlow协议的操作接口。

(3)转发通路：OVS的内核模块，缓存某些常用流表，并负责数据包转发，当遇到无法匹配的报文时，该模块将向守护进程发送Packet-in请求，获取报文处理指令。



Part
04

通用可编程数据平面

协议无关的交换机架构

数据平面编程语言

通用可编程数据平面-协议无关的交换机架构

为进一步寻求协议无关的通用可编程数据平面，学术界和产业界做了大量的工作。其中最具代表性的是Nick McKeown等人在 OpenFlow 交换机转发模型基础上，于2013年提出的可编程的“协议无关交换机架构”(protocol-independentswitch architecture, PISA）。如图3.16所示，PISA是一种具有完全可编程数据包解析器和通用“匹配-动作”单元的高速数据包处理器架构。

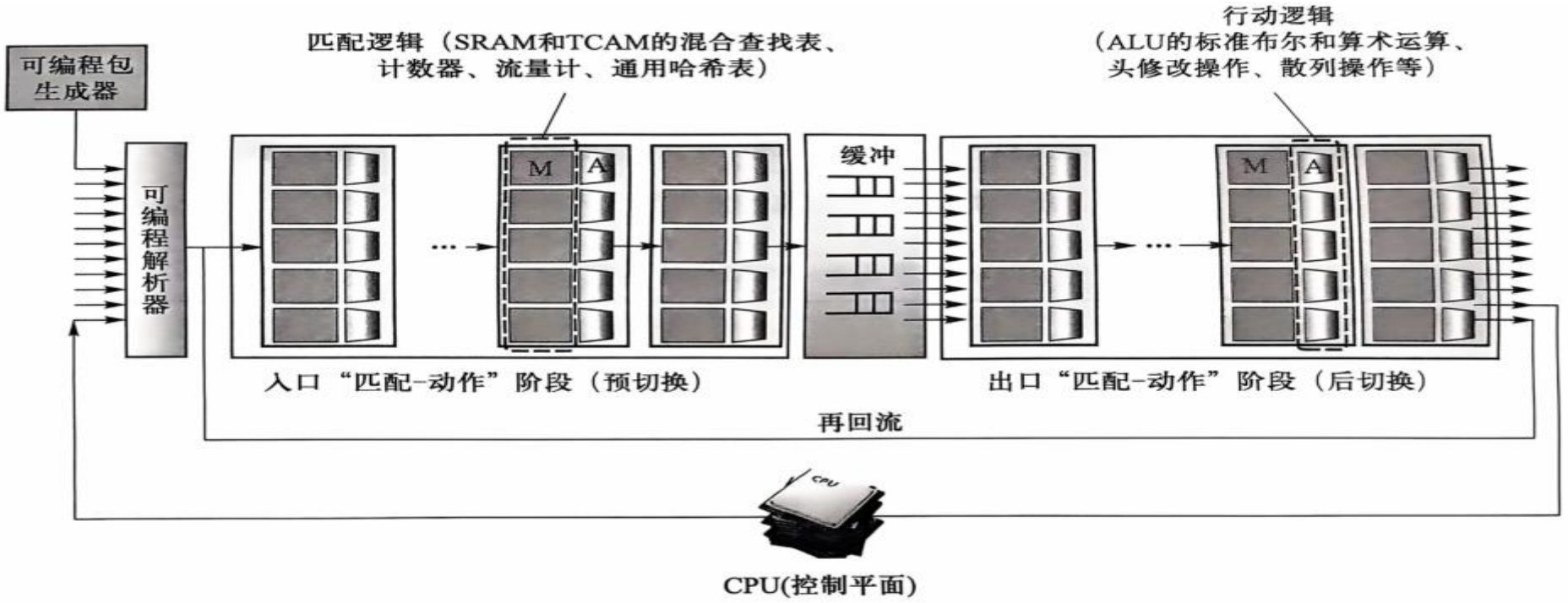


图 3.16　协议无关的交换机架构（PISA）

其中，可编程解析器提供了协议无关的数据包协议解析处理功能，改进了OpenFlow交换机在支持新网络协议方面的不足；PISA中包含了大量通用“匹配-动作”单元，每个单元包括“匹配”逻辑和“行动”逻辑，这些“匹配-动作”逻辑是通过静态随机存储器(static random access memory. SRAM)、三态内容寻址存储器(ternary content ad- dressable memory， TCAM)、寄存器、通用哈希表(hash table)和支持通用数据包处理指令集的计算逻辑单元来实现的。与OpenFlow交换机模型相比，解析器和“匹配-动作”单元只有在被编程后才能做具体的数据包处理工作，PISA芯片在没有配置前不会实现任何数据平面协议，因此PISA是协议无关的架构。

除了可编程解析器和通用的“匹配-动作”流水线。为了提高可编程性，PISA还提供了一条回流路径使一些特殊数据包能够被多次反馈到解析器和流水线上。编译器也可以使用这个回流功能来实现传统芯片因只有固定的查找修改单元而无法完成的复杂逻辑。

要对PISA或其他类型的可编程数据包处理器进行编程，就需要一种专用的数据平面编程语言。P4就是一种协议无关的数据包处理编程语言，它支持用户自定义匹配字段、协议解析过程和转发过程，从而能实现真正意义上的协议无关可编程数据平面。

P4提供了简单易理解的编程结构V1Model，如图3.17所示，从中可以看出，数据包的处理过程像一条流水线一样，它由5个模块组成，如表3.6所示。

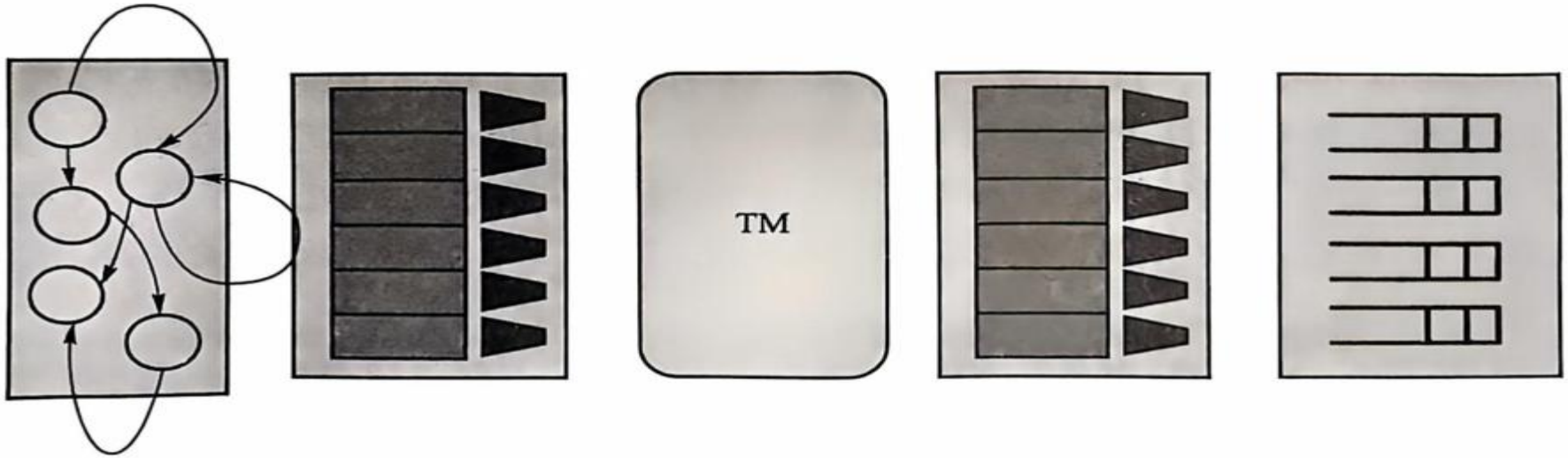


图 3.17　P4 的抽象转发模型 V1Model

表 3.6　V1Model 各模块作用

模　　块	作　　用
解析器（parser）	根据预先定义的报文头部格式对数据包进行解析，并提取头部的各个字段。在解析器中根据报文头部的协议字段匹配下一步需要解析的协议头部。例如，若以太网帧头中的协议字段是 0x0800，则下一步需要解析的是 IP 分组头部；如果是 0x0806，则下一步需要解析的是 ARP 报文，会进行不同的处理
流水线入口（ingress）	流水线部分，数据处理逻辑的入口

续表

模　　块	作　　用
流管理（traffic manager，TM）	定义核心的处理逻辑，并且对一些元数据进行更新
流水线出口（egress）	流水线部分，数据处理逻辑的出口
重组（deparser）	用于对数据包进行重组，因为数据包在处理过程中经历了分解和处理，所以最后转发的时候需要重组数据包

1. P4程序定义

(1)包头(header)：定义一系列头部区域的顺序和结构。它包含区域长度的规范，约束了区域数据的取值。例如，对于IPv4协议分组头部的定义，头部的定义需要精确到位，在没有选项字段时，可以按如下结构定义IPv4分组头部。

```
header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    bit<32>   srcAddr;
    bit<32>   dstAddr;
```

1. P4程序定义

((2)解析器(parser)：定义如何识别数据包内的包头和有效的包头顺序。通过在头部中的协议字段判断下一步解析过程。

```
parser MyParser(packet_in packet, out headers hdr, inout metad
    inout standard_metadata_t standard_metadata) {
        //将解析开始的状态转移到解析以太包头的状态
        state start {
            transition parse_ethernet;
        }
        //以太网包解析过程
```

1. P4程序定义

```
state parse_ethernet{
    packet.extract(hdr.ethernet);  //根据定义的数据结构提取以太包头
    //根据协议类型选择下一个状态
    transition select(hdr.ethernet.etherType){
        0x0800: parse_ipv4;        //如果是 0x0800,则转换到 parse_ipv4 状态
        default: accept;           //默认是接收,进入下一步处理
    }
}
//ip 包解析过程
state parse_ipv4{
    packet.extract(hdr.ipv4);   //根据定义的数据结构提取 IP 包头
    transition accept;          //最终状态转移到 accept,数据包头部提取结
}
}
```

1. P4程序定义

(3)表(table)：“匹配-动作”表是执行数据包处理的机制。P4程序定义了在表内可以匹配的字段和可以执行的动作。P4代码中对表的定义大致如下，其中，key定义了使用哪个字段、使用哪种方式进行匹配；actions定义了该表中有多少动作是可以被执行的；size表示流表可以容纳多少流表项；default_action表示无匹配项时的默认动作。

```
table ipv4_lpm {
    key = {
        hdr. ipv4. dstAddr: lpm;
    }
    actions = {
        ipv4_forward;
        drop;
        Notion;
    }
    size = 1024;
    default_action = NoAction( );
}
```

1. P4程序定义

(4)动作(action)：通过一系列预先定义的协议无关的元操作，构造复杂的自定义动作。这些复杂的动作可以在“匹配-动作”表中使用。例如数据包丢弃动作，内部实现方式可以自定义实现。

```
action drop( ) {
    mark_to_drop( );
}
```

1. P4程序定义

(5)控制程序(control program)：控制程序决定了“匹配-动作”表处理数据包的顺序。一个简单又必要的程序描述了“匹配-动作”表之间的控制流。如果需要应用表并决定处理顺序，需要进行如下步骤，在这个过程中还会判断数据包是否需要进行处理等。。

```
apply {
    if (hdr.ipv4.isValid() && hdr.ipv4.ttl> 0) {
        ecmp_group.apply();
        ecmp_nhop.apply();
    }
}
```

2. P4程序开发示例

下面通过实例说明如何使用P4语言编写程序。在如图3.18所示的模拟网络环境中，要求实现最简单的SDN交换机转发功能。

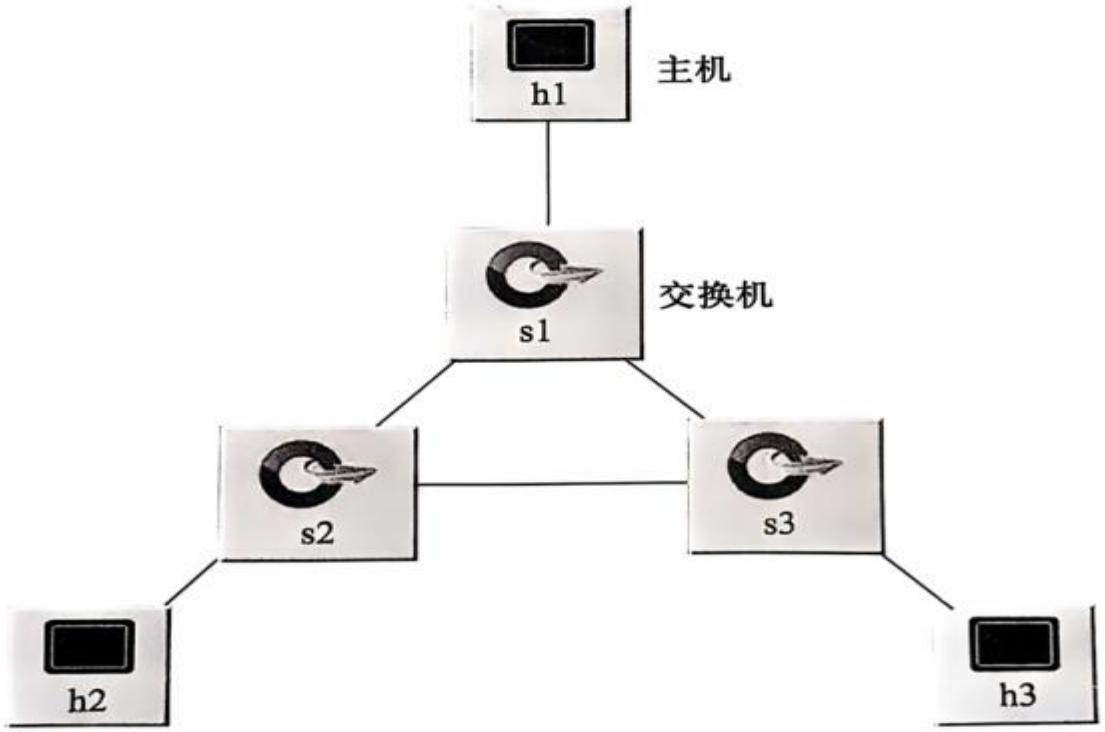


图 3.18　网络拓扑示意图

2. P4程序开发示例

在P4程序编写之前，首先需要根据任务需求，明确P4程序中所应该提供的具体功能及其实现逻辑。

第一步，定义和设计数据包的头部。
第二步，定义解析器模块，以明确交换机解析数据包头的过程。
第三步，计算和验证校验和。
第四步，数据包的“入口-匹配”操作处理。
第五步，数据包输出处理。
第六步，校验和的重新计算。
第七步，数据包的重新封装。
代码的最后，还要加入逻辑执行部分。

下面是根据上述步骤所编写的详细代码及注释。

```
/* -*- P4_16 -*- */
/*相关模块的导入以及常量的定义*/
#include <core. p4>
#include <v1model. p4>
const bit<16> TYPE_IPV4 = 0x800;
```

/*第一步：按照任务实际需求，自定义数据包头部，以达到灵活操纵数据包处理的目的*/

```
/ ********** H E A D E R S  ****
typedef bit<9>   egressSpec_t;
typedef bit<48> macAddr_t;
typedef bit<32> ip4Addr_t;
/ * 定义二层数据帧头 * /
header ethernet_t {
        macAddr_t dstAddr;
        macAddr_t srcAddr;
        bit<16>    etherType;
}
```

通用可编程数据平面-数据平面编程语言

```
/*定义 ip 分组头*/
header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}
struct metadata {
    /*  empty  */
}
/*定义 headers,由二层数据帧头与三层 IP 分组头组成*/
struct headers {
    ethernet_t    ethernet;
    ipv4_t        ipv4;
}
```

```
/* 第二步:定义 PARSER 模块,根据定义好的 HEADERS 提取数据包头部信息 */
/****************** P A R S E R  ***************************/
parser MyParser(packet_in packet,
        out headers hdr,
        inout metadata meta,
        inout standard_metadata_t standard_metadata) {
            state start {                          //将解析开始的状态转移到解析以太包头的状态
                transition parse_ethernet;
            }
        state parse_ethernet{                      //以太网包解析过程
        packet.extract(hdr.ethernet);              //根据定义的数据结构提取以太包头
        transition select(hdr.ethernet.etherType){      //根据协议类型选择下一个状态
            0x0800: parse_ipv4;                    //如果是 0x0800,则转换到 parse_ipv4 状态
            default: accept;                       //默认是接收,进入下一步处理
            }
        }
        state parse_ipv4{                          //IP 包解析过程
            packet.extract(hdr.ipv4);              //根据定义的数据结构提取 IP 包头
            transition accept;                     //最终状态转移到 accept,数据包头部提取结束
        }
}
```

/*第三步：校验和验证，本例中暂时省略*/

```
control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
    apply {  }
}
```

通用可编程数据平面-数据平面编程语言

```
/*第四步:定义一个用于转发的流表,定义匹配域和动作*/
/*********  INGRESS  PROCESSING  *****************/
control MyIngress(inout headers hdr,
    inout metadata meta,
    inout standard_metadata_t standard_metadata) {
    action drop() {                    //定义丢弃动作
        mark_to_drop();                //内置函数,将当前数据包标记为即将丢弃的数据包
    }
    action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) { //定义转发动作
        standard_metadata.egress_spec = port; //输出的端口从控制面传递的参数中获取
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;      //数据包源地址改为目的地址
        hdr.ethernet.dstAddr = dstAddr;        //数据包新目的地址从控制面获得
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;       //数据包生存时间减 1
    }
    /*定义用于转发的流表*/
    table ipv4_lpm {
        key = {                                //流表的匹配域
            hdr.ipv4.dstAddr: lpm;  // lpm 说明匹配的模式是 Longest Prefix Match
                                    // 即最长前缀匹配
        }
        actions = {
            ipv4_forward;
            drop;
            Notion;
        }
        size = 1024;
        default_action = NoAction();
    }
    /*上述都是声明定义过程,这里是真正的数据处理逻辑*/
    apply {
        ipv4_lpm.apply();
    }
}
```

/*第五步：本例中可以省略Egress过程*/

```
/*********  EGRESS  PROCESSING  ***************/
control MyEgress(inout headers hdr,
      inout metadata meta,
      inout standard_metadata_t standard_metadata) {
      apply {  }
}
```

```
/*第六步:校验和计算,校验和计算由高度抽象的内置函数直接完成*/
/*****    CHECKSUM    COMPUTATION    **********/
control MyComputeChecksum(inout headers hdr, inout metadata meta) {
    apply {
        update_checksum(
            hdr.ipv4.isValid(),
            hdr.ipv4.version,
            hdr.ipv4.ihl,
            hdr.ipv4.diffserv,
            hdr.ipv4.totalLen,
            hdr.ipv4.identification,
            hdr.ipv4.flags,
            hdr.ipv4.fragOffset,
            hdr.ipv4.ttl,
            hdr.ipv4.protocol,
            hdr.ipv4.srcAddr,
            hdr.ipv4.dstAddr },
            hdr.ipv4.hdrChecksum,
            HashAlgorithm.csum16);
    }
}
```

```
/*第七步:数据包重组,数据包重组这一过程同样有高度抽象的内置函数帮助完成*/
/************  D E P A R S E R  ************/
control MyDeparser(packet_out packet, in headers hdr) {
    apply {                    //数据包重组过程需要注意顺序
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
    }
}

/*整段代码在交换机中的运行过程*/
/***************  S W I T C H  ***************/
V1Switch(
MyParser(),
MyVerifyChecksum(),
MyIngress(),
MyEgress(),
MyComputeChecksum(),
MyDeparser()
) main;
```

至此，关于交换机转发功能的P4代码已经完成，但是SDN交换机尚不能正，常工作。因为交换机还需要控制平面来控制数据包的转发处理，机下发流表。在本例中，通过加载文件的形式来载入流表。P4-tutorial中会通过例如，为sl交JSON文件的形式将JSON文件中的流表项载入到虚拟交换机中。例如，为s1交换机添加的流表项部分如下：

```
{
    "table" : "MyIngress. ipv4_lpm",
    "match" : {
      "hdr. ipv4. dstAddr" : ["10. 0. 1. 1", 32]
    },
    "action_name" : "MyIngress. ipv4_forward",
    "action_params" : {
      "dstAddr" : "00:00:00:00:01:01",
      "port" : 1
    }
}
```

文件中流表与数据包输入处理过程中定义的流表一致，流表中的动作就是在P4代码中定义的动作。在运行过程中，当交换机匹配到流表中的流表项之后，根据流表项中的动作来处理这些数据包。在模拟数据包发送的过程中，可以使用Python的scapy库，在该库中提供了构建数据包、抓包、解析数据包等功能。利用scapy库可以自定义数据包格式和内容来实现简单的数据模拟，验证P4程序的运行效果。

P4语言语法简单，程序结构清晰。一旦理解P4程序的工作模式，开发工作较为容易。在网上也有不少P4学习资源，如P4官方的示例教程中包括了基于P4实现的隧道、负载均衡、源路由、ECN等例子。P4语言目前仍然保持着良好的发展势头。基于P4语言的芯片（如Tofino）已经开始逐渐在市场上使用，包括复杂的云网络环境。基于数据平面可编程思想的P4，正在推动着SDN领域不断前行。

明德新民　　止于至善

明德新民 | 止于至善

第四章 南向接口协议

学院：软件学院　　任课教师：申凌峰

河南大學

目录

CONTENTS

01 南向接口协议概述
02 OpenFlow协议
03 OF-Config
04 NETCONF协议
05 OVSDB管理协议
06 其他南向接口协议

Part
01

南向接口协议概述

协议概述

关于南向接口在第2章SDN体系结构中已经有所介绍，它是SDN控制层（控制平面）与基础设施层（数据平面）实现交互的接口。自北向南，控制平面的控制器需要以某种方式，如ONF体系结构中所采用的流表，或IETF的I2RS结构中所采用的插件，向数据平面的交换设备下发转发策略与规则；自南向北，数据平面的交换设备需要向控制器上传转发状态、事件及其统计信息。为此，需要对通过南向接口交互的消息类型、消息格式和交互规程等进行定义，从而产生了南向接口协议。南向接口协议是为了实现控制平面控制器和数据平面交换机之间信息交互而设计的协议。简单来说，它是控制器与交换机之间的通信协议，如图4.1所示。

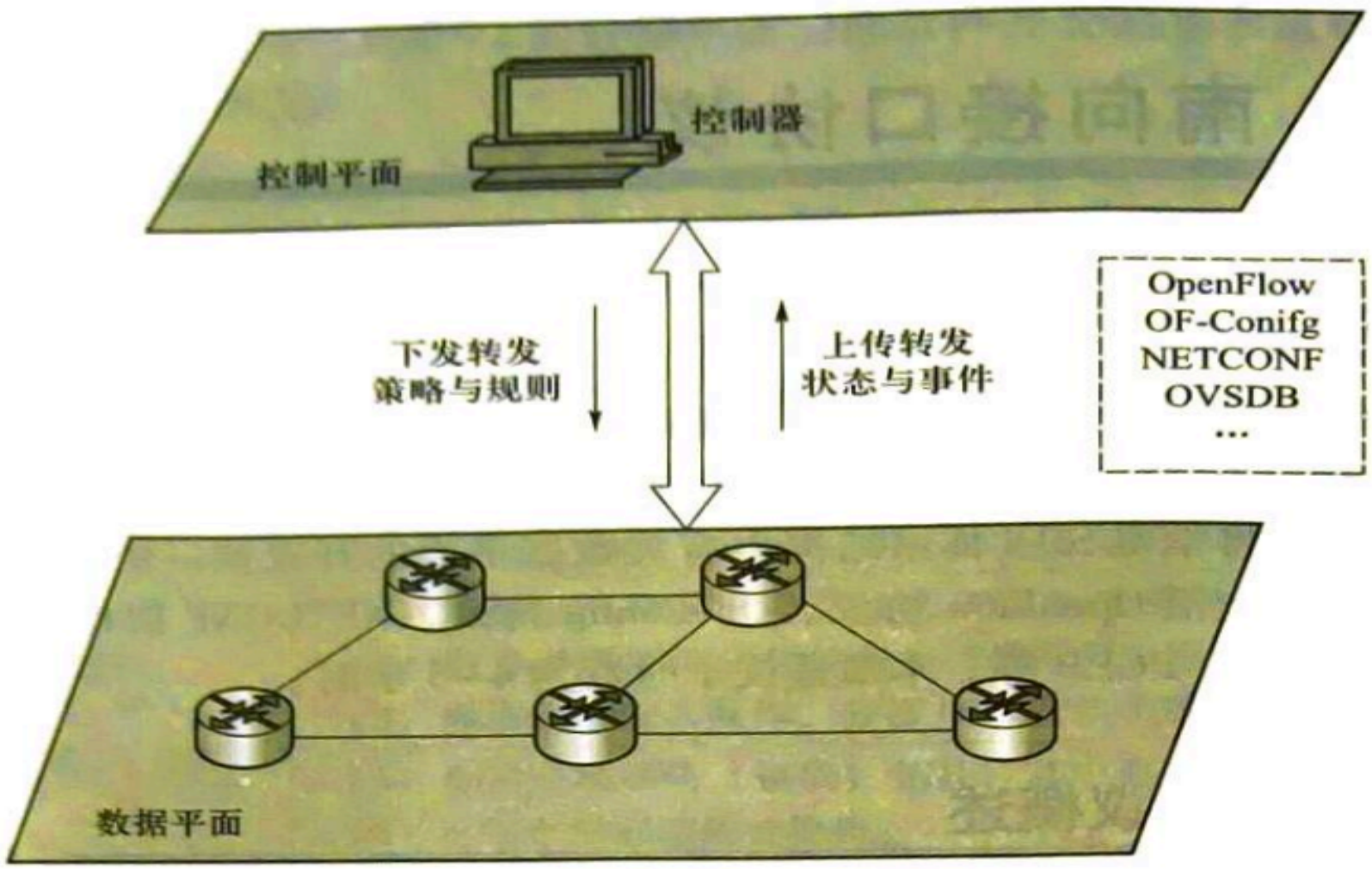


图 4.1　南向接口协议示意图

协议概述

目前，已经实现的南向接口协议很多，主要有ONF发布的OpenFlow、OF-Config， IETF 发布的NETCONF、XMPP、PCEP 、I2RS，VMware 提出的OVSDB.思科提出的OpFlex等。同为南向接口协议，由于设计出发点和问题关注点的不同，不同的南向接口协议会存在设计目标与功能定位上的差异。归结起来，主要有以下三类。

第一类用于实现数据平面与控制平面的信息交互。向上收集交换机信息，比如交换机的特性、配置信息、工作状态等；向下下发控制策略，指导数据平面设备的转发行为。OpenFlow、XMPP、I2RS和OpFlex都属于此类协议。

第二类用于实现网络的配置与管理。也就是说，这类协议专注于控制器与交换机之间有关设备配置与管理信息交互的实现。OF-Config、

第三类主要面向流量工程(traffic engineering，TE)的实现。流量工程是指根据各种数据业务流量的特性选取传输路径的处理过程，用于平衡网络中的不同交换机、路由器以及链路之间的负载。支持流量工程的实现，需要收集基础设施层的拓扑与链路、流量与负载等信息。以实现路由计算为例，需要收集底层链路的拓扑、带宽、开销和连通状态等信息。PCEP就属于此类协议。

表 4.1　南向接口协议的目标与功能定位

协议名称	制定或发布者	设 计 目 标
OpenFlow	ONF	用于 OpenFlow 交换机与控制器之间的信息交互，提供双方通信的标准，适于支持 OpenFlow 交换机与控制器之间的通信
OF-Config	ONF	用于 OpenFlow 交换机的配置与管理，提供控制器连接设置、加密、队列、端口等配置能力，支持双向认证、可靠传输、事务管理等服务，适合作为 OpenFlow 的伴侣协议
NETCONF	IETF	用于网络设备的配置与管理，提供获取、添加、修改或删除网络设备的配置数据等功能，适于对非 OpenFlow 设备的配置与管理
OVSDB	VMware/IETF	用于 Open vSwitch 的配置与管理，提供对设备数据库的增、删、改、查等操作，适于对 Open vSwitch 交换机的配置与管理
XMPP	IETF	用于即时通信的协议，具有良好的可扩展性，对现有网络设备的兼容性较好，适合作为 OpenFlow 的一种替代或者补充协议
PCEP	IETF	用于流量工程中路径计算的通信协议，可以与 OpenFlow 结合使用，在流量工程的应用场景中实现传统网络和 SDN 网络的统一管理和调度
I2RS	IETF	用于控制器与非 OpenFlow 设备之间的通信协议，适于 IETF 定义的 SDN 体系结构中的南向接口协议
OpFlex	思科	用于思科 ACI 体系中的策略控制协议，提供了双向的通信能力，适于支持 OpFlex 协议的控制器与设备之间的通信

Part
02

OpenFlow协议

1. Controller-to-Switch 消息

Controller-to-Switch 消息由控制器发起，发送给交换机。其主要用于管理和获取交换机状态，并可能会要求交换机回复对应的消息。此消息的功能特性可进一步细分为以下几类。

(1) Features：用于获取交换机的特性或属性信息。在建立传输安全会话时，控制器主动向交换机发送Features-request消息，请求交换机的特性信息，交换机通过Features-reply消息回复自身所支持的功能特性。

(2) Configuration：用于设置或查询交换机的配置信息，若为设置配置信息，交换机只需执行信息；若为查询配置信息请求，交换机需给予应答。

(3) Modify-state：用于修改交换机的状态消息，如用于增加、删除和修改交换机的流表项、组表项及端口状态。

(4) Read-state：用于读取与收集交换机的各种状态信息，包括配置信息、统计信息和功能特性。

(5)Send-packet：用于向交换机的指定端口发送数据包，在OpenFlow v1.1以后的版本中，该类型改成Packet-out消息，用于响应Packet-in消息的处理。Packet-out 消息必须携带一个 Buffer_id 来定位缓存在交换机上的数据包，此外，该消息还需携带用于指导数据处理的动作集，如果动作集为空，丢弃数据包。

(6) Barrier：用于保证消息执行的顺序。Barrier 消息会在所有的前置消息完成后才处理。

2.Asynchronous 消息

Asynchronous 消息通过异步方式由交换机上传给控制器，用于传递网络事件和交换机状态的变化更新，如告知控制器新数据包的到达和交换机状态的改变等。其细分的功能特性如下。

(1)Packet-in：如果数据包在流表中没有匹配项，或者匹配的流表项中有指定端口为 CONTROLLER 的动作时，交换机将发送Packet-in 消息给控制器。如果交换机有足够的缓存用于存储发往控制器的数据包，则Packet-in 消息只需要将包头中的控制信息（默认128B）和交换机缓存标识（Buffer_id）发送给控制器，控制器在Packet-out消息中可以根据Buffer_id处理交换机缓存中对应的数据包；如果交换机不支持缓存（或者缓存容量不足），则将整个数据包封装进Packet-in 消息发给控制

(2) Flow-removed：用于交换机向控制器主动发送流表被移除的信息。当交换机中的流表项因超时或修改等原因被删除时，会触发该消息的发送。

(3) Port-status：用于交换机向控制器主动发送端口状态变化的信息。当交换机的端口状态发生变化时(Up/Down)，会触发该消息的发送。

(4) Error：用于交换机向控制器主动发送运行错误的信息。例如，当交换机产生运行时错误时，会通过消息包主动告知控制器所发生的错误信息。

3.Symmetric 消息

Symmetric 消息可以由控制器或交换机任意一方发送，而无须对方的许可或请求。其细分的功能特性如下。

(1)Hello：用于控制器和交换机双方建立安全连接时，交换连接握手消息。 (2)Echo：用于测量链路延迟与带宽，或用于保持活动连接。双方均可主动发送Echo-request消息给对方，接收者需要回复Echo-reply消息。

(3) Vendor：用于厂商自定义的消息扩展，以为交换机生产厂商提供额外的定制功能预留空间。

OpenFlow 协议数据包由 OpenFlow 消息头(OpenFlow header)和 OpenFlow消息体(OpenFlow message)两部分组成。其中，OpenFlow 消息的头部结构是固定的，包含4个字段(64b)数据结构，如图4.2所示。其中，Version 代表协议的版本号；Type 代表消息类型，消息类型是一个以OFPT_为前缀的枚举类型常量，详见OpenFlow 规范文件；Length代表数据包的长度；Xid代表数据包的 Transaction_ID，Reply 消息会使用和 Request 消息中相同的Transaction_ID。

消息格式

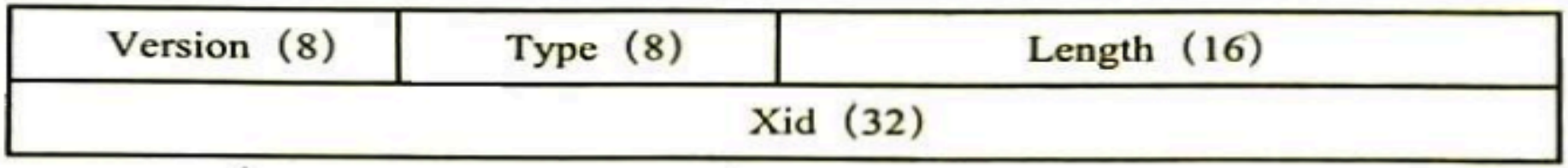


图 4.2　OpenFlow 消息头格式

下面是伪程序语言描述的 OpenFlow 消息头的结构体：

```
struct ofp_header
{
    uint8_t Version;        /* 协议版本号 */
    uint8_t Type;           /* 消息类型 */
    uint16_t Length;        /* 数据包的长度,包括消息头 */
    uint32_t Xid;           /* 数据包的 Transaction_ID */
};
```

1. Hello 消息Hello 消息(OFPT_HELLO)用于控制器与交换机在建立连接时协商协议版本，该消息只包含OpenFlow消息头，如图4.3所示，Version表示发送方所支持的OpenFlow协议的最高版本号，双方选取Hello消息中最低版本的协议作为通信协议。通过在字段 Type 中指定该消息的类型为 OFPT_HELLO 来表明其消息类型。

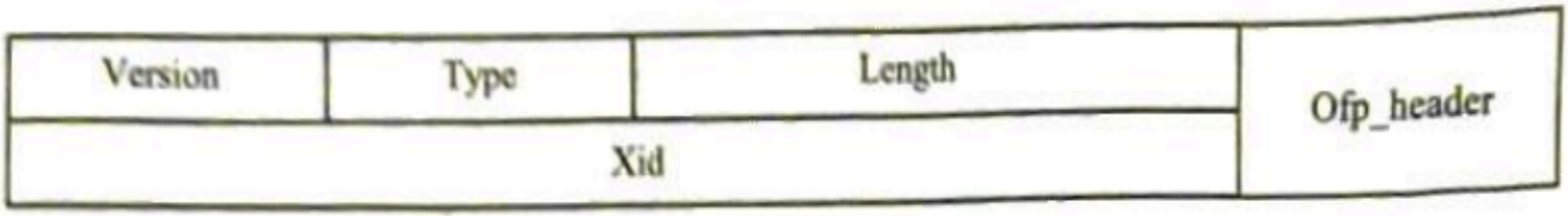


图 4.3　OFPT_HELLO 消息格式

2. Features 消息

该消息包括 Features-request和 Features-reply两种不同的格式。其中，Features-request 在OpenFlow连接建立后，由控制器向交换机发送，请求获取交换机的特性信息，包括交换机的数据通路ID、缓冲区数量、端口及端口属性等。Features-request 消息只包含 OpenFlow 消息头，通过在字段 Type 中指定该消息的类型为OFPT_FEATURES_REQUEST来表明其消息类型。

交换机收到 Features-request 消息后，通过 Features-reply 消息进行回应。该消息包括OpenFlow 消息头和Features-reply消息体两大部分，其格式如图4.4所示。通过在 OpenFlow 消息头字段 Type 中指定该消息的类型为 OFPT_FEATURES_ REPLY 来表明其消息类型。

N_tables 用以给出交换机的流表数量，Capabilities 用于说明交换机支持的特殊功能，Actions 表示交换机可支持的动作，Ofp_phy_ports 给出交换机物理端口属性的列表描述。Ofp_phy_ports可列举的端口属性包括端口编号(Port_no)、端口硬件或 MAC 地址(Hw_addr)、端口名称(Name)、端口配置(Config)、端口状态(State)和端口物理属性(Curr、Advertised、Supported、Peer)。

消息格式

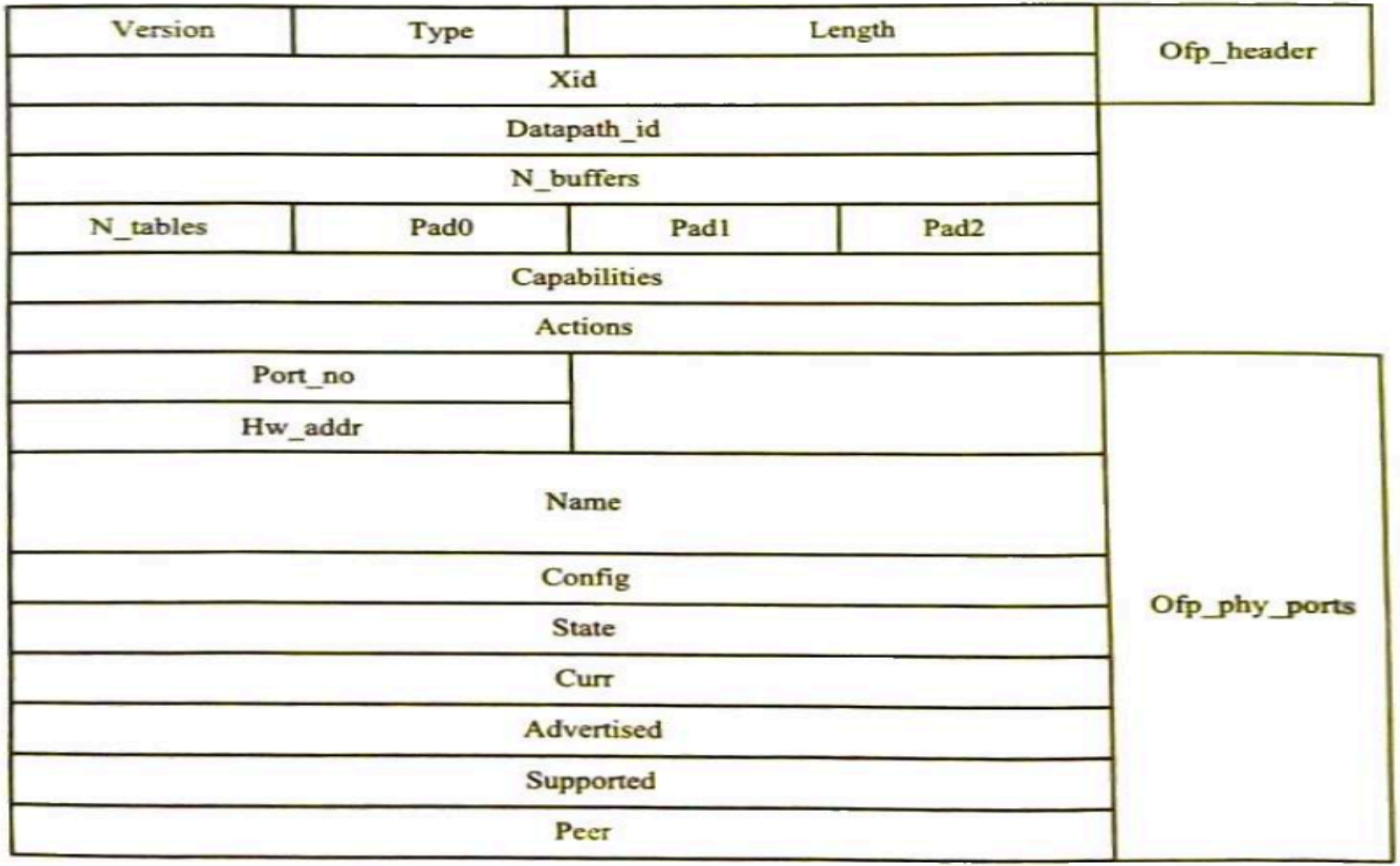


图 4.4 OFPT_FEATURES_REPLY 消息格式

3. Configuration 消息

该消息包括Set-config、Get-config-request和 Get-config-reply三种格式，分别通过在OpenFlow消息头字段Type中指定相应的消息类型来表示，分别为OFPT_SET_CONFIG、OFPT_GET_CONFIG_REQUEST 和OFPT_GET_CONFIG_REPLY。控制器通过Set-config 消息(OFPT_SET_CONFIG)对交换机进行配置，通过 Get- config-request 消息(OFPT_GET_CONFIG_REQUEST)获取交换机的配置参数，交换机则通过 Get-config-reply 消息(OFPT_CET_CONFIG_REPLY)对控制器发出的 Get-config-request 消息进行回应。OFPT_GET_CONFIG_REQUEST只包括OpenFlow 消息头。OFPT_SET_CONFIG和OFPT_GET_CONFIG_REPLY_消息除了OpenFlow 消息头，还包括Flags和Miss_send_len两个字段，如图4.5所示。Flags用于指示交换机处理IP分片数据包，Miss_send_len用于指示当交换机无法处理的数据包到达时，将数据包发给控制器的最大字节数。

消息格式

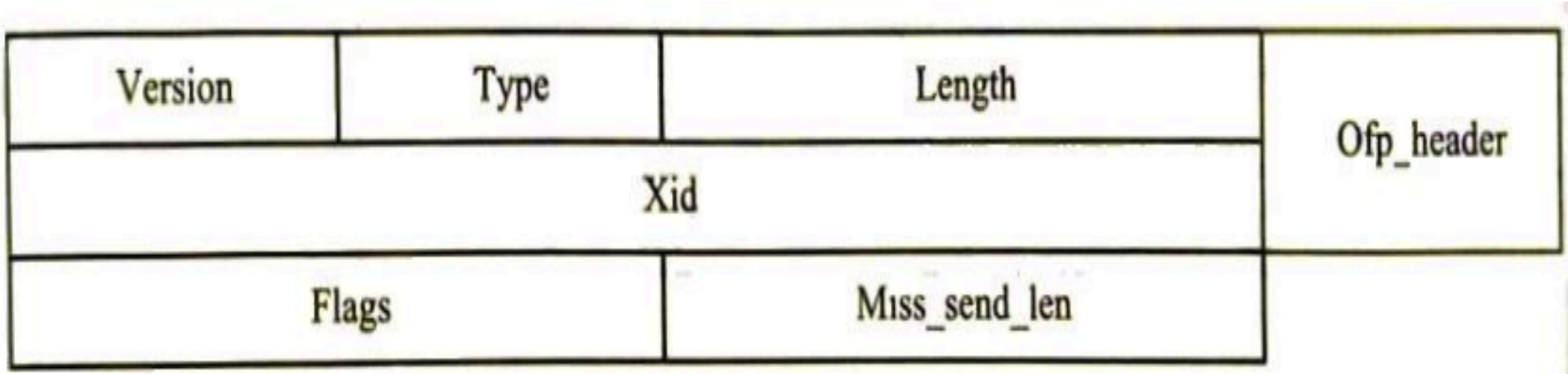


图 4.5　OFPT_SET_CONF 和 OFPT_GET_CONFIG_REPLY 消息格式

4. Packet-in消息

该消息被交换机用于向控制器发送包接收的信息。但只有当数据包在交换机的流表中没有匹配项，或者匹配的流表项中所指定端口为CONTROLLER 的动作时(Output=CONTROLLER)，交换机才发送Packet-in 消息(OFPT_PACKETIN）给控制器，以提示控制器进行相应的转发决策处理。Packet-in 消息格式如图4.6所示，在OpenFlow 消息头字段Type中需指定为OFPT_PACKET_IN类型。消息体部分中，字段Buffer_id 标明数据包在交换机的缓存区ID，Total_len给出Data 部分的长度，In_port 说明交换机的输入端口，Reason 用以说明触发 Packet-in事件的原因，Pad用于填充。

5. Modify-state 消息

控制器利用 Modify-state 消息(OFPT_FLOW_MOD)对交换机的流表进行配置，包括增、删、改等操作。Modify-state 的消息格式如图 4.7 所示，其消息头字段Type对应的类型OFPT_FLOW_MOD。消息体部分又进一步划分出Ofp_match、Ofp_actions_header等功能区域。其中，Ofp_match是流表的匹配域，包括了12元组的字段，In_port表示输入端口，Dl_src 和 Dl_dst 分别是 IP 源地址和目的地址，Dl_vlan 和 Dl_vlan_pcp分别是 VLAN ID 和 VLAN 优先级，Dl_type 表示以太网帧类型，Nw_tos 给出 IP 服务类型，Nw_proto 表示 IP 协议，Nw_src 和 Nw_dst分别是MAC源地址和目的地址，Tp_src 和Tp_dst分别是TCP/UDP 源端口号和目的端口号。此外，还包括了位掩码通配符Wildcards字段，用于指定被掩码的字段位，未被掩码的字段位参与

在上述Ofp_match流表匹配域之后，字段Cookie是控制器定义的流表项标识符；Command 用以说明具体的Flow-mod（流表编辑）消息类型，包括ADD、MODIFY、MODIFY-STRICT、DELETE、DELETE-STRICT 五种类型；Idle_timeout表示流表项的空闲超时时间；Hard_timeout 给出流表项的最大生存时间；Priority表示流表项的优先级；Buffer_id给出交换机的缓冲区ID；Out_port 给删除流表的Flow-mod 消息提供额外的匹配参数；Flags 表示标志位。最后的Ofp_actions_header由 Type、Len等字段组成，用以提供Action列表。

Version	Type	Length	Ofp_header
Xid			
Buffer_id			
Total_len		In_port	
Reason	Pad	Data[0]	

图 4.6　OFPT_PACKET_IN 的消息格式

Version	Type	Length	Ofp_header
Xid			
Wildcards			Ofp_match(40B)
In_port			
Dl_src			
Dl_dst			
		Dl_vlan	
Dl_vlan_pcp	Pad0	Dl_type	
Nw_tos	Nw_proto	Pad0	Pad1
Nw_src			
Nw_dst			
Tp_src		Tp_dst	
Cookie			
Command		Idle_timeout	
Hard_timeout		Priority	
Buffer_id			
Out_port		Flags	
Type		Len	Ofp_actions_header
Pad0	Pad1	Pad2	Pad3

图 4.7　OFPT_FLOW_MOD 的消息格式

6. packet-out 消息

控制器通过发送 Packet-out 消息(OFPT_PACKET_OUT)指示交换机如何对数据包进行处理。OFPT_PACKET_OUT消息的格式如图4.8所示。其中，Buffer_id表示交换机中缓冲区ID号，In_port给Packet-out消息提供额外的匹配信息，Actions_len

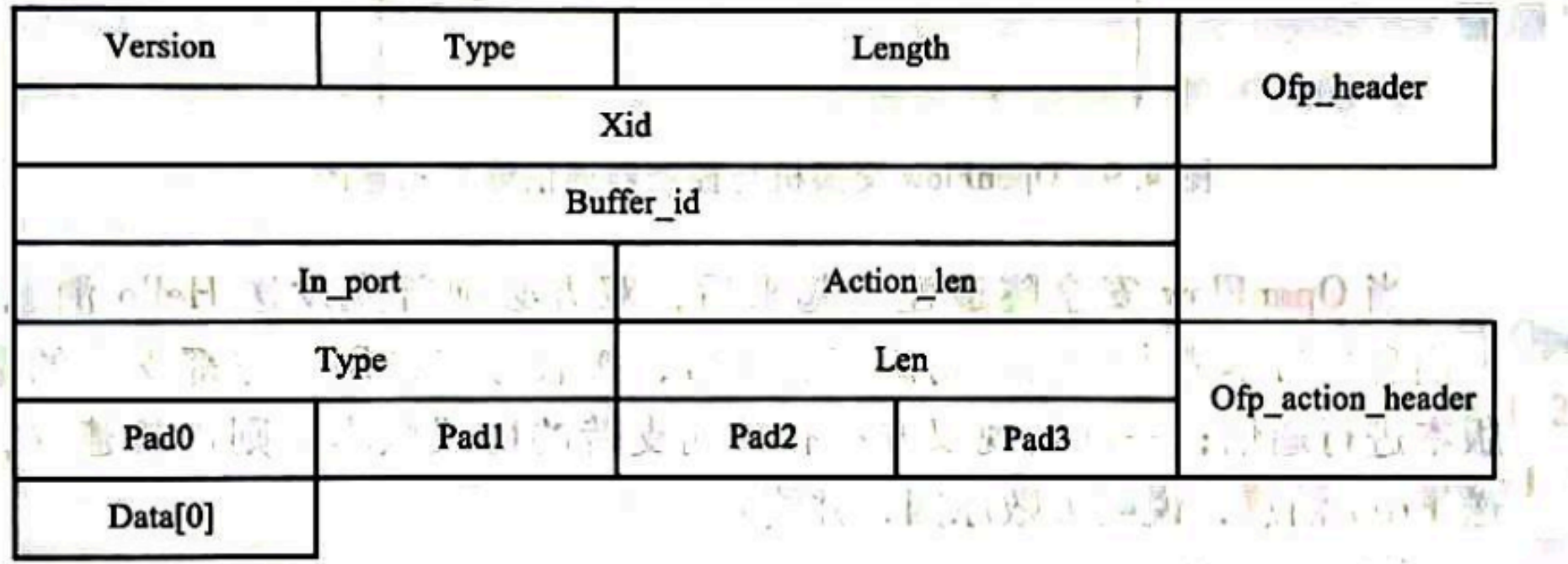


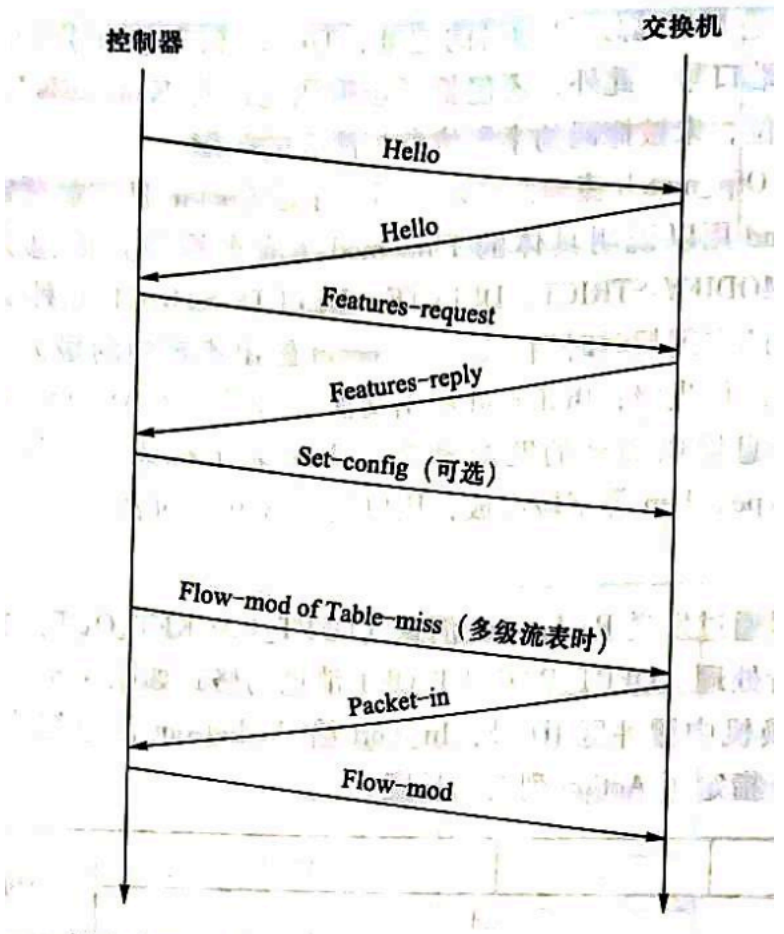
图 4.8　OFPT_PACKET_OUT 消息格式

通信流程

1.建立连接

控制器开启TCP的6633端口等待交换机的连接，交换机启动时，尝试连接到指定控制器的6633端口，为保证安全性，双方需要认证；控制器与OpenFlow交换机建立TLS隧道后，隧道中传送的都是控制协议消息，因此隧道中的所有流量转发都无须查询交换机中的流表。

当 OpenFlow 安全隧道建立起来后，双方必须首先发送Hello消息给对方，该消息携带本方支持的最高协议版本号，接收方将采用双方都支持的最低协议版本进行通信；一旦发现双方拥有共同支持的协议版本，则连接建立，否则发送Error消息，说明失败原因，并终止连接。



2.获取交换机特性

控制器向OpenFlow 交换机发送 Features-request消息，请求OpenFlow交换机上传自己的详细参数。OpenFlow 交换机收到请求后，向控制器发送Featuresreply消息，详细汇报自身参数，包括支持的buffer数目、流表数以及Actions等。

3.配置交换机控制器

通过Set-config消息下发配置参数。由于交换机的配置管理也可以通过OF-Config协议来实现，因此这里的Set-config是可选操作。配置后，控制器可通过Get-config-request 消息请求获取OpenFlow交换机修改后的配置信息。Open-Flow 交换机通过 Get-config-reply 消息向控制器发送当前的配置信息。

4.正常通信状态

交换机配置完成后，进入正常的通信状态。如果OpenFlow 版本支持多级流表，控制器还需要下发Table-Miss 流表项到交换机。当数据包匹配流表出现失败或匹配到Table-Miss时，交换机将发送Packet-in 消息到控制器，控制器可通过回复Packet-out或Flow-mod 消息，指导交换机对相关数据流的处理。

5.中断连接

当交换机与控制器之间的连接发生异常时，OpenFlow 交换机通常会尝试连接备份控制器。若多次尝试均失败，OpenFlow交换机将进入紧急模式，并重置所有的TCP 连接。此时所有包将匹配指定的紧急模式表项，而其他所有正常表项将被从流表中删除。当交换机刚启动时，默认

Part 03

OF-Config

OF-Config 概述

OF-Config作为OpenFlow 协议规范中专门的交换机配置协议，它是OpenFlow的伴侣协议。OF-Config 目前有3个版本，分别是版本1.0、1.1和1.2。OF-Config v1.0 是 OpenFlow v1.2 的伴侣协议，OF-Config v1.1 是 OpenFlow v1.3 的伴侣协议，OF-Config v1.2 可作为 OpenFlow v1.3 及其之前所有版本的伴侣协议。图4.10 给出了 OF-Config与OpenFlow协议及OpenFlow交换机之间的关系。由 图 4.10 可见，OF-Config和 OpenFlow 协议都是用于与交换机进行交互的协议，但存在本质上的区别。OpenFlow 协议是控制平面的接口协议，控制器通过OpenFlow 协议对 OpenFlow 交换机流表的转发进行指导；而OF-Config 协议属于管理平面的接口协议，其设计动机是实现 OpenFlow 交换机的远端配置，Open-Flow 配置点通过 OF-Config 协议完成 OpenFlow 交换机的配置与管理工作。从某种意义上讲，相当于OF-Config将有关交换机配置与管理功能从OpenFlow 协议中专门抽取出来，并进行了有针对性的设计，从而减轻了OpenFlow协议的负担，使其能够专注于控制平面对数据平面包转发逻辑的处理与信息交互。

OF-Config协议在设计实现中，对交换机进行了功能组件的逻辑抽象。首先，将支持 OpenFlow 协议规范交换机及其操作上下文(operation context)抽象为OpenFlow 使能交换机(OpenFlow capable switch)。其次，将 OpenFlow 使能交换机中所运行的 OpenFlow 交换机实例抽象为OpenFlow 逻辑交换机(OpenFlowlogical switch)，每个OpenFlow逻辑交换机都被分配了一定的资源，如端口、队列等。一个OpenFlow交换机相当于一个物理或虚拟的网元设备，其中可以包含一个或多个OpenFlow 逻辑交换机。

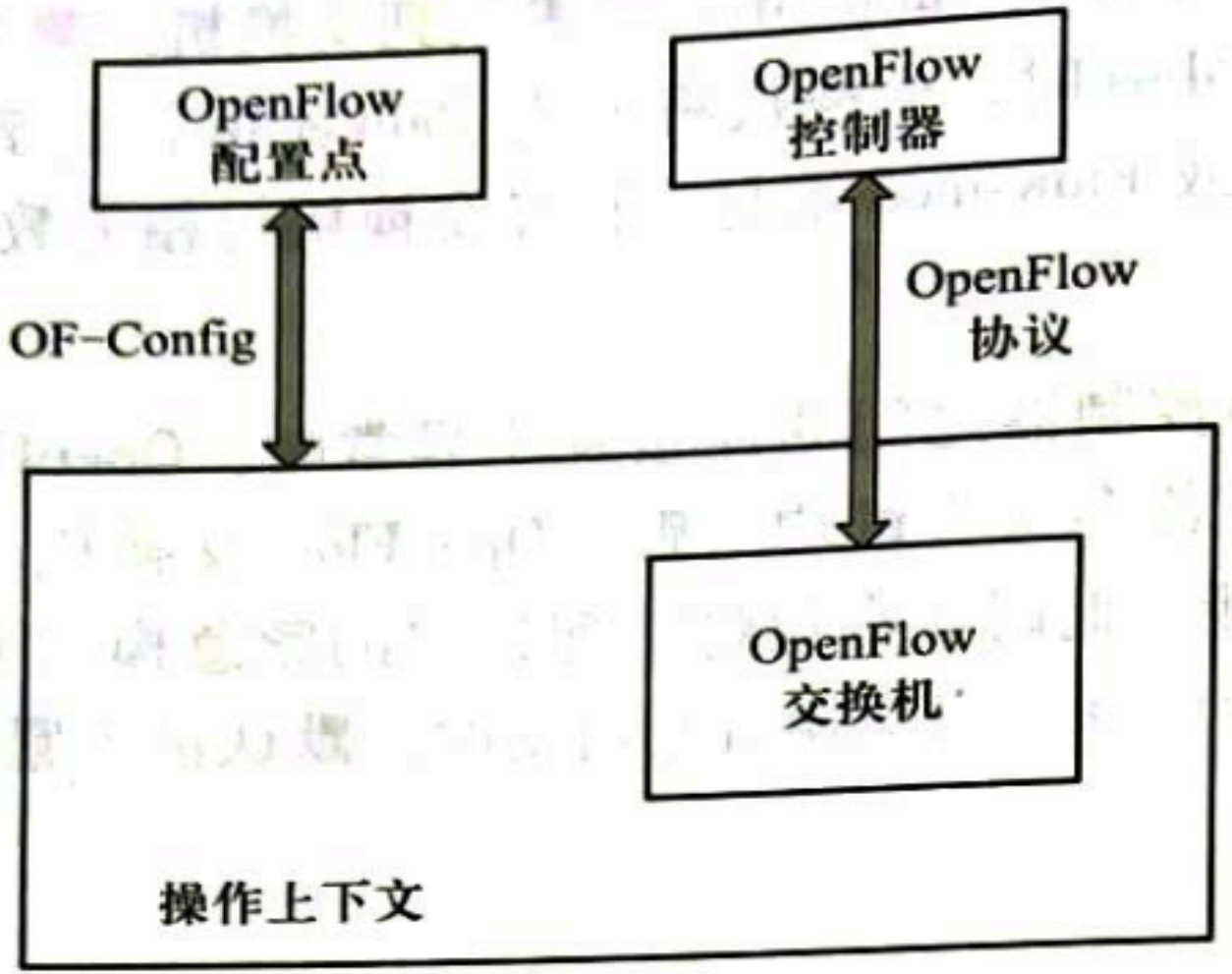


图 4.10　OF-Config 与 OpenFlow 协议及 OpenFlow 交换机之间的关系

OF-Config 概述

图 4.11以 OF-Config v1.2 协议为范本，进一步描述了 OF-Config 各组件及其与OpenFlow 协议之间的关系。在管理平面中，提供OF-Config服务的功能点被称为 OpenFlow 配置点(OpenFlow configuration point)。OpenFlow 配置点通过发送OF-Config 消息到OpenFlow使能交换机实现对交换机的远程配置，包括对其中每个 OpenFlow 逻辑交换机的配置。控制平面中的控制器通过 OpenFlow 协议对OpenFlow 使能交换机中所运行的一个或多个 OpenFlow 逻辑交换机，分别进行转发策略或规则的控制，一个控制器可控制多个OpenFlow 逻辑交换机，一个OpenFlow 逻辑交换机

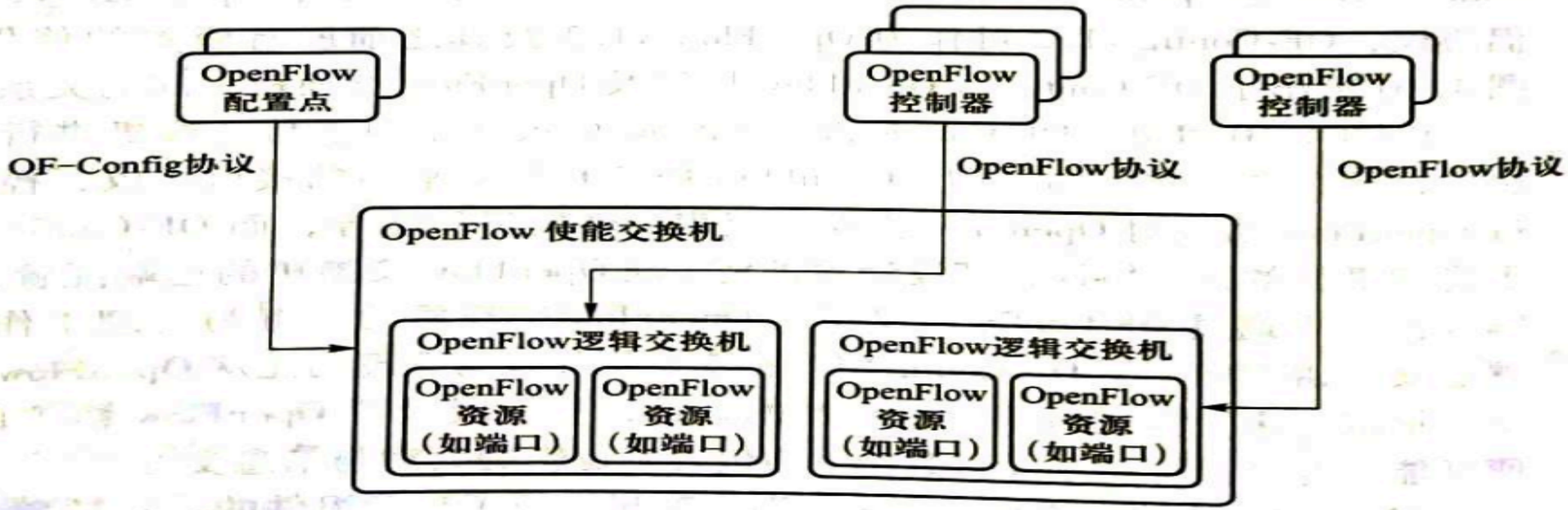


图 4.11　OF-Config v1.2 协议规范文件中各组件及其关系

Part
04

NETCONF协议

简单网络管理协议(SNMP)是传统网络实现管理功能的重要协议，通过在SNMP 管理服务（代理）端和被管理设备（客户端）进行SNMP信息的交互实现对网络设备配置信息、状态信息等的管理。但该协议在复杂网络配置、传输效率、安全性等方面存在明显的不足。为此，2003年IETF 成立了 NETCONF工作组，其目的在提供一个全新的基于XML的网络配置协议。2006年关于NETCONF 核心规范的RFC 4741 发布，2011 年发布 RFC 6241，RFC 6241是目前最新的RFC文件。事实上，NETCONF 协议在SDN诞生之前就已经提出，并非是为SDN专门设计的，而当SDN出现后，由于NETCONF在复杂网络下的强大、灵活的管理与配置能力，使其成为一个适合SDN的协议。 NETCONF 协议提供了一套管理网络设备的机制，包括增加、修改、删除网络设备的配

(1)提供用户配置管理的保护锁定机制。当一个管理用户操作时，其他管理用户不能进行有关增加、修改、删除等更新操作，以防止多用户操作所产生的冲突。

(2)提供针对整个系统配置数据的直接操作，因此，其配置的对象是整个网络，而不是单个设备，且定义了查询过滤功能，实现对某个数据子集的操作。

(3)具有更好的扩展性。协议模型采取分层定义，各层之间相互独立，当对协议中的某一层进行扩展时，能够最大限度地不影响其上层协议。协议采用XML实现，提供了基于可编程的编码管理能力可扩展性，并具有良好的系统兼容性。

(4)提供实现更加灵活的安全功能(特性)。通过对SSH、TLS等现有安全协议的支持与利用，为数据传输提供安全保证，同时不与某个具体的安全协议绑定，具有更大的选择与使用灵活性。

(5)区分配置数据和状态数据的操作功能集。配置数据(configuration data)是对网络设备进行配置的数据，如端口设置、VLAN 创建的数据；状态数据(state data)是反映设备状态的数据，如端口状态(Up/Down)、VLAN数量等。NETCONF协议分别

协议架构

NETCONF 协议沿用了 SNMP 所采用的客户-服务器结构，协议模型采取分层架构、定义，在逻辑上划分为四层，自下而上分别是安全传输层、消息层、操作层和内容层，如图4.14所示。

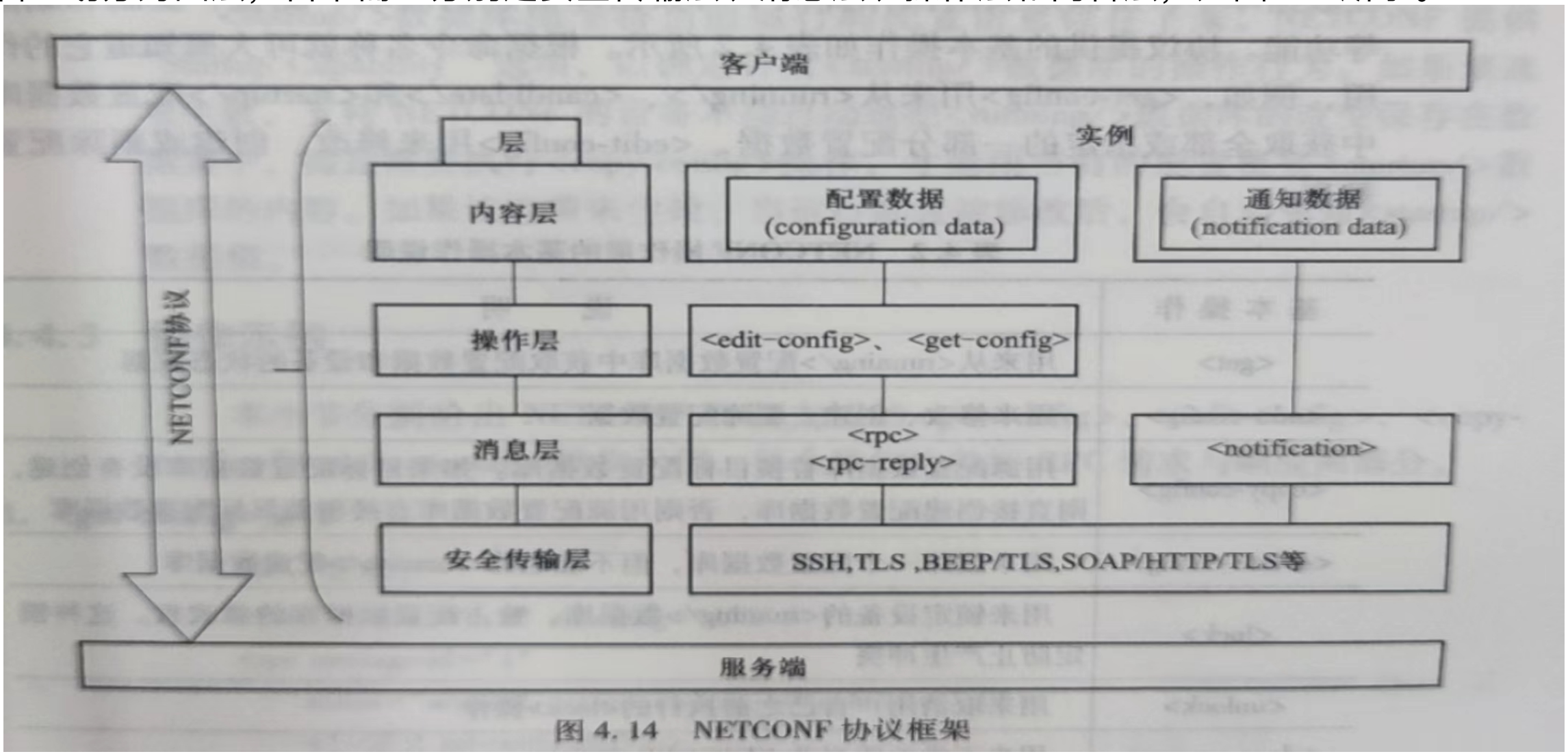


图 4.14 NETCONF 协议框架

1.安全传输层

安全传输层用于为NETCONF连接提供安全、可靠的传输功能，涉及序列号、认证、数据完整性和安全保障等机制。安全传输层承载在SSH、TLS等协议基础上，也就是说可以选择SSH、TLS等现有传输安全协议建立安全连接。当一个安全连接建立后，客户端与服务器会通过Hello 消息进行一次 NETCONF 消息交互，用于确认双方支持的能力。

2.消息层

消息层用于为编码RPC与通知(notification)提供一个简单、独立于传输层协议使用<pe>的框架机制。借用RPC协议的通信机制，RPC是一种通过网络从远程计算机程序上请求服务，而不需要了解底层网络技术的协议。NETCONF和<rpc-reply>元素来提供独立于承载协议的NETCONF请求和响应，，完成对设备的配置和管理，在交换Hello。消息后，NETCONF客户端向服务器端发送<rpc>消息对服务器端设备进行配置管理，服务器端对每个<rpc>请求发送<rpc-reply>连行响应。

3.操作层

操作层是NETCONF协议的核心，用于为协议提供基于RPC的操作集。该层定义了一个底层操作的集合用以配置和获取设备信息，而设备信息一般存放在数据库中，所以定义的操作提供了对数据库信息的获取、配置、复制和删险等功能。协议提供的基本操作如表4.2所示。根据命令名称就可大概知道它的作用，例如，<get-config>用来从<running/>、<candidate/>和<startup/>配置数据库中获取全部或指定的一部分配置数据，<edit-config>用来修改、创建或删除配数据。

表 4.2 NETCONF 操作层的基本操作说明

基本操作	说明
<get>	用来从<running/>配置数据库中获取配置数据和设备的状态数据
<edit-config>	用来修改、创建、删除配置数据
<copy-config>	用源配置数据库替换目标配置数据库。如果目标配置数据库没有创建，则直接创建配置数据库，否则用源配置数据库直接覆盖目标配置数据库
<delete-config>	用来删除一个配置数据库，但不能删除<running/>配置数据库
<lock>	用来锁定设备的<running/>数据库，独占配置数据库的修改权。这种锁定防止产生冲突
<unlock>	用来取消用户自己之前执行的<lock>操作
<close-session>	用来正常关闭当前 NETCONF 会话
<kill-session>	用来强制关闭另一个 NETCONF 会话，只有管理员用户才有此权限

4.内容层

内容层提供了关于网络管理所涉及配置数据的描述，使用YANG（一种数据建模语言，详见RFC6020）进行建模。配置数据主要涉及<running/>、<candidate/>和<startup/>数据库等。

1) <running/>

<running/>数据库用以描述目前在设备上运行的生效配置，包含当前设备上可用的所有概念状态信息。对<running/>数据库使用<get>操作可以获得有关运行的状态信息、统计信息和配置参数，应用<get-config>指令操作只能获得配置数据。

2) <candidate/>

<candidate/>数据库用来存放可能被调用或生效的候选（备选）配置信息，对<candidate/>数据库的任何改变都不会立即直接影响网络设备。管理员需要使用<commit>操作来运行<candidate/>数据库上变化的配置，并使这些变化的配置生效而成为<running/>数据库的一部分。<commit>操作成功执行后，

3)<startup/>

<startup/>数据库用于将当前运行的配置信息保存下来。NETCONF提供“Startup Capability”选项，以确定针对<startup/>数据库的操作行为。如果该选项生效，支持NETCONF的设备不会自动地把<running/>数据库的改变保存在数据集中，而是需要执行<copy-config>操作，才能用当前的配置覆盖<startup/>数据库的内容。如果该选项未生效，当运行配置被修改后，会自动更新<startup/>数据集。

Part
05

OVSDB管理协议

1. OVSDB 数据格式的描述

OVSDB管理协议使用JSON作为数据格式的描述语言。JSON是JavaScriptobject notation 的英文缩写，它是用于结构化数据序列化的一种文本格式，由RFC 4627定义。作为基于 JavaScript 对象表示法语法的一个子集，JSON是一种轻量级的数据交换格式，采用完全独立于语言的文本格式，易于阅读与编写。JSON 解析器和JSON 库支持许多不同的编程语言。下面简单介绍JSON的语法格式。 JSON数据的书写格式是：名称/值对。例如，"firstName":"John"，等价于JavaScript 语句：firstName ="John"。

JSON值可以是数字（整数或浮点数）、字符串（在双引号中）、逻辑值(true或 false)、数组(在方括号中)、对象(在花括号中)及null。

JSON对象在花括号中书写，对象可以包含多个名称/值对，例如，|"first-Name":"John"，"lastName":"Doe"|，等价于JavaScript 语句：

firstName ="John";

lastName ="Doe";

JSON数组在方括号中书写，数组可包含多个对象，例如：

```
{
    "employees": [
        {" firstName" :"John"," lastName":" Doe" },
        {" firstName":"Anna","lastName":"Smith" },
            {" firstName":"Peter"," lastName":"Jones"} ] }
```

在上面的例子中，对象"employees”是包含三个对象的数组。每个对象代表一条关于某人姓和名的记录。

不同于普通的JSON 实现，OVSDB的JSON实现有以下限制：字符串中不能存在null字节，仅支持UTF-8编码。

OVSDB 使用了JSON下述简化术语：<string>,<id>，<version>，<boolean>. <number>,<integer>,<json-value>, <nonnull-json-value>, <error>。

2. OVSDB 数据模式

OVS配置数据库由一系列的表组成，每张表由若干列和行组成。OVSDB数据库模式(database schema)由JSON对象表示，JSON 对象所包含的成员如表 4.3 所示，其中 tables 所涉及的 OVSDB表模式JSON对象所包括的主要成员如表4.4所示，OVSDB列模式JSON对象所包含的主要成员如表4.5所示。

表 4.3　JSON 对象表示的 OVSDB 数据库模式

名　称	值	可选项	说　明
name	<id>	必选	数据库整体标识
version	<version>	必选	数据库模式版本
cksum	<string>	可选	实现定义的校验和
tables	{<id>:<table-schema>,…}	必选	说明表名和表模式的 JSON 对象

表 4.4　JSON 对象表示的 OVSDB 表模式

名　称	值	可选项	说　明
columns	{<id>:<column-schema>,…}	必选	包含表格的 UUID、版本信息等
maxRows	<integer>	可选	表示表格的最大行数
isRoot	<boolean>	可选	用以说明表格内是否存在强依赖关系
indexes	[<column-set> *]	可选	用于标识表格列

表 4.5　JSON 对象表示的 OVSDB 列模式

名　称	值	可选项	说　明
type	<type>	必选	存储于该列的数据类型
ephemeral	<boolean>	可选	数据是否持久化
mutable	<boolean>	可选	数据是否可修改

线路协议

线路协议用于为用户通过远程调用的方式管理OVSDB提供支持。OVSDB线路协议基于JSON-RPC v1.0规范实现，JSON-RPC是一种无状态轻量级的远程过程调用协议，该规范定义了与数据传输相关的数据结构及相应的处理规则，可以用于进程内通信、套接字、HTTP或各种消息通信环境。JSON-RPC使用JSON作为数据格式。在JSON-RPC v1.0规范中，每个RPC请求对象包括方法名称、参数和请求ID三部分。

例如，当发起远程调用RPC请求时，向服务器端传输的数据格式如下：

{"method":"sayHello", "params":["Hello JSON-RPC"]，"id"：1}

其中，method 是调用的RPC方法名称；params是传递给方法的参数数组；id是标识符，用于标识一次远程调用过程。

当服务器端收到调用请求，完成相应的RPC方法调用后，将处理结果返回给调用方，返回使用的数据格式如下：

{"result":"Hello JSON-RPC", "error":null, "id":1}

其中，result 表示返回结果，如果出错result 必须为null；error表示出错，如果请求成功，则 error必须为null；id表示为此响应对应的请求。

OVSDB管理协议要求OVSDB服务器必须能实现所有的OVSDB PRC方法，OVSDB 客户则必须能实现Echo及其他所有必要的方法，表4.6是常见的OVSDBRPC方法。

表 4.6　常见 OVSDB RPC 方法

RPC 方法	作 用 说 明
List Database	列出通过管理协议连接所能获取的数据库名称列表
Get Schema	根据数据库名称获取<database-schema>
Transact	引发数据库服务器对给定的数据库以指定顺序执行一系列操作
Cancel	通知数据库服务器立即完成或取消指定的 Transact 操作请求
Monitor	通过请求 Table 或其子集的变化通知，使得客户端可以对数据库中的表或其子集进行复制
Update Notification	在 Monitor 请求之后，服务器将 Update 通知发送给客户端，告知所监控表的变化情况
Monitor Cancellation	Monitor_cancel 请求通知服务器取消之前发出的有问题的 Moniter 请求
Lock Operation	支持客户端对数据库进行锁定操作，包括三类 RPC 方法：lock、steal、unlock
Locked Notification	通知客户端，其之前提交的 lock 请求已授权
Stolen Notification	通知客户端，其之前获取的锁定权限已失效，已经被另一个客户端取代了所有权
Echo	服务器端和客户端用于确认连接的保活性

在表4.6中，方法Transact 主要用于引发（发送或定义）数据库服务器对给定数据库以指定顺序所执行的一系列操作。Transact成员：

(1)"method":"transact"，成员method指定了 RPC 的方法是transact；

(2)"params":[<db-name>, <operation>*]，成员params参数指定了该方法在调用时被操作的数据库<db-name>及对应的操作行为 <operation>。 (3)"id": <nonnull-json-value>.成员id是一个唯一的请求标识。

OVS数据库操作主要指上述参数operation中所可以提供或执行的数据库操作。表4.7给出了所能支持的数据库操作，每一个OVSDB数据库操作都是个JSON对象。

表 4.7　OVSDB 数据库操作列表

RPC 数据库操作	说　　明
Insert	在指定表中插入一行
Select	在表中查找满足指定条件的数据
Update	更新表中满足指定条件的数据
Mutable	更新表中满足指定条件的数据，可单独更新列数据
Delete	从表中删除满足指定条件的数据
Wait	触发器
Commit	数据强制持久化后提交
Abort	强制终止 Transact
Comment	为数据库管理员提供操作的备注信息
Assert	如果指定的客户端没有锁定权限，终止 Transact

Part
06

其他南向接口协议

XMPP

XMPP是可扩展消息处理现场协议(extensible messaging and presence protocal)的简称，其中包含了一组用于即时消息传递、状态、多方聊天、语音和视频呼叫、协作、轻量级中间件、内容联合和通用的XML数据路由的技术规范。XMPP 协议并非专门为SDN 设计的，它的前身是Jabber，是一个在开源社区产生的网络即时通信协议，主要用来构建大规模即时通信系统、游戏平台、协作空间及语音和视频会议系统。大多数社交媒体采用了该协议，SDN控制器Open-Contrail 采用其作为南向编程接口协议。

XMPP 基于 XML 技术，目前由因特网工程任务组(IETF)负责其标准化工作。最新的技术规范文件由RFC 6120(XMPP Core)、RFC 6121(XMPP IM)和RFC 7622(XMPP address format)定义。

XMPP 采用典型的客户-服务器架构，而不同于一些流媒体应用中所采用的P2P Client-to-Client 的架构。如图4.15所示，在该网络架构下，两个客户端之间的消息交互要通过服务器，因为服务器承担了大量的信息交互功能，所以客户端的功能得到了简化。

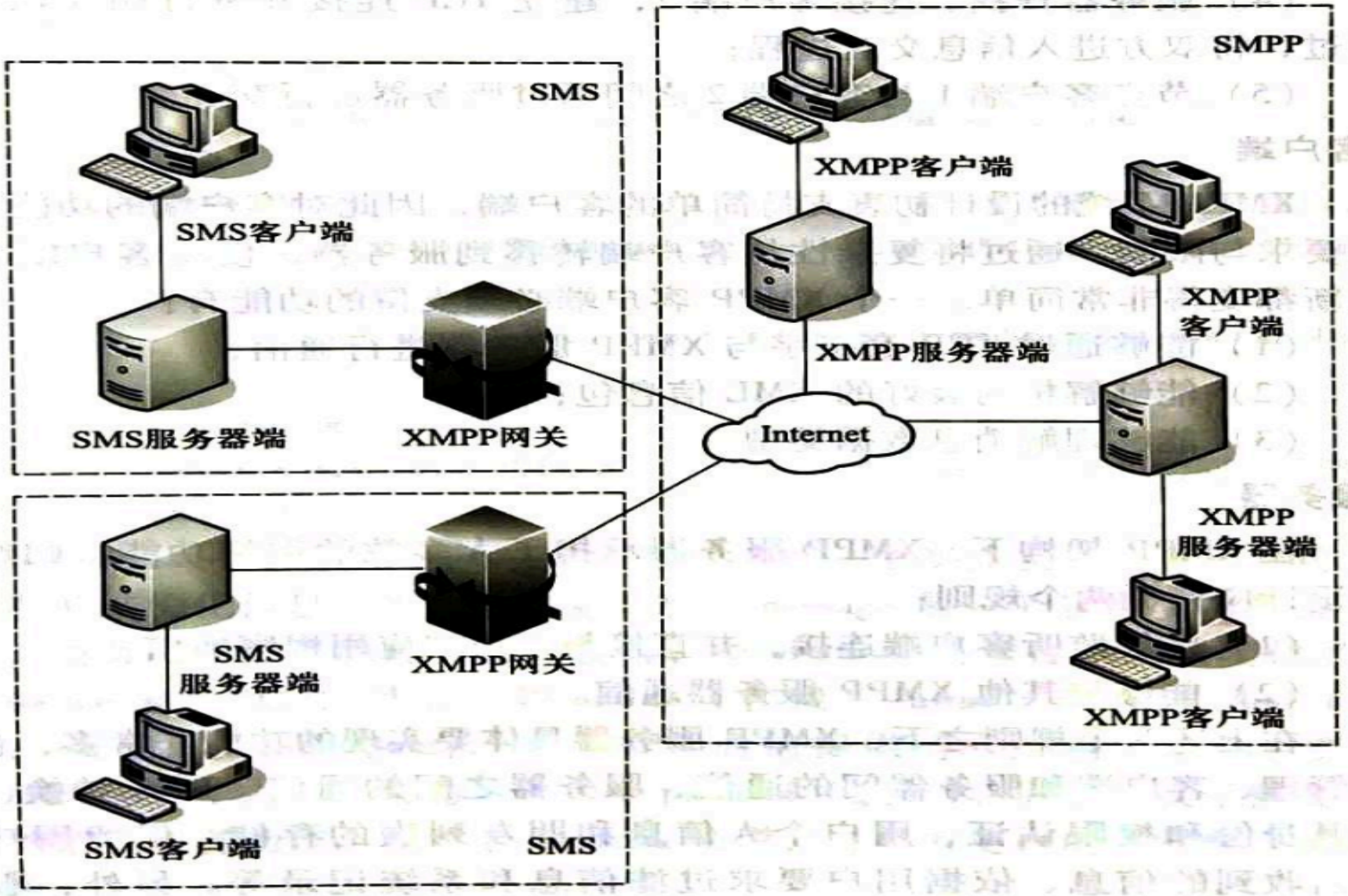


图 4.15　XMPP 协议网络架构图

XMPP 中定义了三个角色：XMPP 客户端、XMPP 服务器、XMPP 网关，通信可以在任意两个角色之间双向发生，服务器同时承担了客户端信息记录、连接管理和信息路由的功能，网关承担着与异构即时通信系统的互联互通，异构系统可以包含SMS（短信）、MSN、ICQ等网关，这大大提高了XMPP 的灵活性和可扩展性。

就两个同构XMPP客户端节点之间的通信而言，以客户端1和客户端2为例，其交互实现的流程如下：

(1)客户端1通过TCP/IP套接字请求与服务器建立TCP连接，采用的TCp端口为注册端口5222;

(2)服务器端利用本地文件夹系统中的证书对客户端1进行认证，若认证通过，双方进入XMPP协议信息交互过程，所传输的数据采用XML格式：

(3)客户端1节点指定所要通信的客户端2目的地址，由服务器告知目标状态；

(4)服务器查找、连接客户端2，建立TCP连接并进行相互认证，若认证通过，将双方进入信息交互过程；

(5)节点客户端1与客户端2之间通过服务器进行交互。

1. XMPP 客户端

XMPP 系统的设计初衷支持简单的客户端，因此对客户端的功能只有非常少的要求与限制。通过将复杂性从客户端转移到服务器，使得客户端功能实现与更新都变得非常简单。一个XMPP客户端必须支持的功能有：

(1)能够通过TCP套接字与XMPP服务器进行通信；

(2)能够解析封装好的XML信息包；

(3)能够理解消息数据类型。

2. XMPP服务器

在XMPP架构下，XMPP服务器承担了大多数的网络功能，归结起来其功能设计应遵循两个规则：　(1)能够监听客户端连接，并直接与客户端应用程序通信；

(2)能够与其他XMPP服务器通信。

在上述两个规则之下，XMPP服务器具体要实现的功能非常多，包括处理会话管理、客户端和服务器间的通信、服务器之间的通信、DNS转换、用户注册及其身份和权限认证、用户个人信息和朋友列表的存储、保留用户离线状态时所收到的信息、依据用户要求过滤信息和系统记录等。另外，服务器还被要求能够通过附加服务实现扩展，如完整的安全策略、同意服务器组件的连接或客户端选择、提供通向其他消息系统的网关。为了简化XMPP服务器功能协议开发与实现，上述具体的功能实现采用了模块化设计，构成多个不同的代码包。

3.XMPP网关

XMPP 协议架构的突出特点是能够提供不同即时通信网络之间的兼容性或互操作性，即能够作为网关为不同即时通信系统之间交换信息和用户在线状况提供支持。因为不同即时通信系统所采用的协议不同，XMPP 和其他系统之间的互操作必须通过协议转换来实现，实现这个特殊功能的服务器端在XMPP 架构里称为网关。通过网关，XMPP 实现和 AIM、ICO、IRC、MSN Messager、RSS 0.9和 Yahoo Messager 等非XMPP 消息系统之间的通信。

XMPP中定义了3个顶层XML元素：message、presence、iq。XMPP流由两个XML文档组成，通信的每个方向均有一个文档，这些文档有一个根元素<stream:stream>，<stream:stream>的子元素由可路由的节以及与流相关的顶级子元素构成。

<message>元素用于从一个实体向另外一个实体发送消息，并可以传输任何类型的结构化信息，不保证传输可靠，message是一个非常基础的推(push)模型，通常用于消息的传递与交换等；<presence>元素提供网络实体的可访问性、presence 表明用户的状态，如 online（在线）、away（离开）、dnd（请勿打扰）等；<iq>表示 Info/Query，为XMPP 通信提供请求及响应机制，类似于GET/POST/PUT方法。iq只能包含一个payload（载荷），并且定义了需要由服务器处理的请求或者动作。相对于message来说，iq具有更好的可靠性，因其要求收到回应。

如前所述，XMPP 并非专门为SDN 设计的。由于XMPP具有良好的可扩展性，对现有网络设备的兼容性较好，因此，在SDN中，XMPP可以作为OpenFlow的一种替代或者补充协议。

PCEP

PCEP是路径计算单元协议(path computation element protocol)的简称，其由 IETF 的 PCE 工作组于2006年提出，主要是为了提供（支持） MPLS 网络坡间流量工程等应用，以支持集中化的路径计算。

在PCEP的协议架构中，定义了两个核心功能模块：PCC(path computationclient)和PCE (path computation element)，PCC部署在路由器等数据平面节点上，PCE服务器则部署在控制平面上，实现路径计算单元，负责计算来自PCC的路径计算请求，PCEP则是为PCE和PCC之间通信而提出的相关通信协议。

为了支持集中的路径计算，控制平面上还需要有TED(traffic engineering database）功能模块，主要负责收集网络拓扑和当前的带宽、资源利用率等链路信息，但PCEP 中并不包括这些信息的交互，而是由TED通过路由协议或由网管的其他机制获取。PCE 从 TED 中获取链路和网络参数并计算一条满足约束条件的最佳路径。

PCEP 协议主要包括两部分：RFC 4657 说明PCE和PCC之间通信的要求，RFC 5440 说明通信协议的具体实施标准。PCEP 协议定义了 PCC与 PCE交互过程中的消息报文格式及其相关作用，其中包括会话初始化、请求回复、安全管理、状态通知、出错反馈等。PCEP协议描述的一次会话主要分为以下4个阶段，如图4.16所示。

第一个阶段为会话的初始阶段。首先在PCE和PCC间通过三次握手建立TCP连接。然后在TCP连接基础上建立PCEP会话，完成双方会话参数的协商。该阶段包括的消息类型为用于 PCE和 PCC或者 PCE和PCE之间的消息开启的Open消息，以及用于保持维护该次会话的Keepalive消息。

第二阶段为路径计算的请求阶段。发送PCE和PCC之间或PCE和PCE之间进行路由计算的请求消息，即发送包括请求路由的详细信息、路由的约束条件以及其他特殊的约束条件等信息的PCReq请求消息。

第三阶段为路径计算的响应阶段。PCE根据TED链路信息，结合约束条件进行路径计算，并发送包括详细路由、节点信息、带宽等信息的响应消息。

第四阶段为会话的关闭阶段。用于处理Close结束消息、异常和错误等信息。PCEP支持多种设备和路由协议，因此在流量工程领域得到了广泛的应用。

PCEP并非专门为SDN设计的，其编程能力不够强大，也不甚完善。但是， PCEP协议和OpenFlow结合使用，可以在流量工程的应用场景中实现传统网络和SDN 网络的统一管理和调度。PCEP因此被认为是一种广义上的SDN南向接口协议。

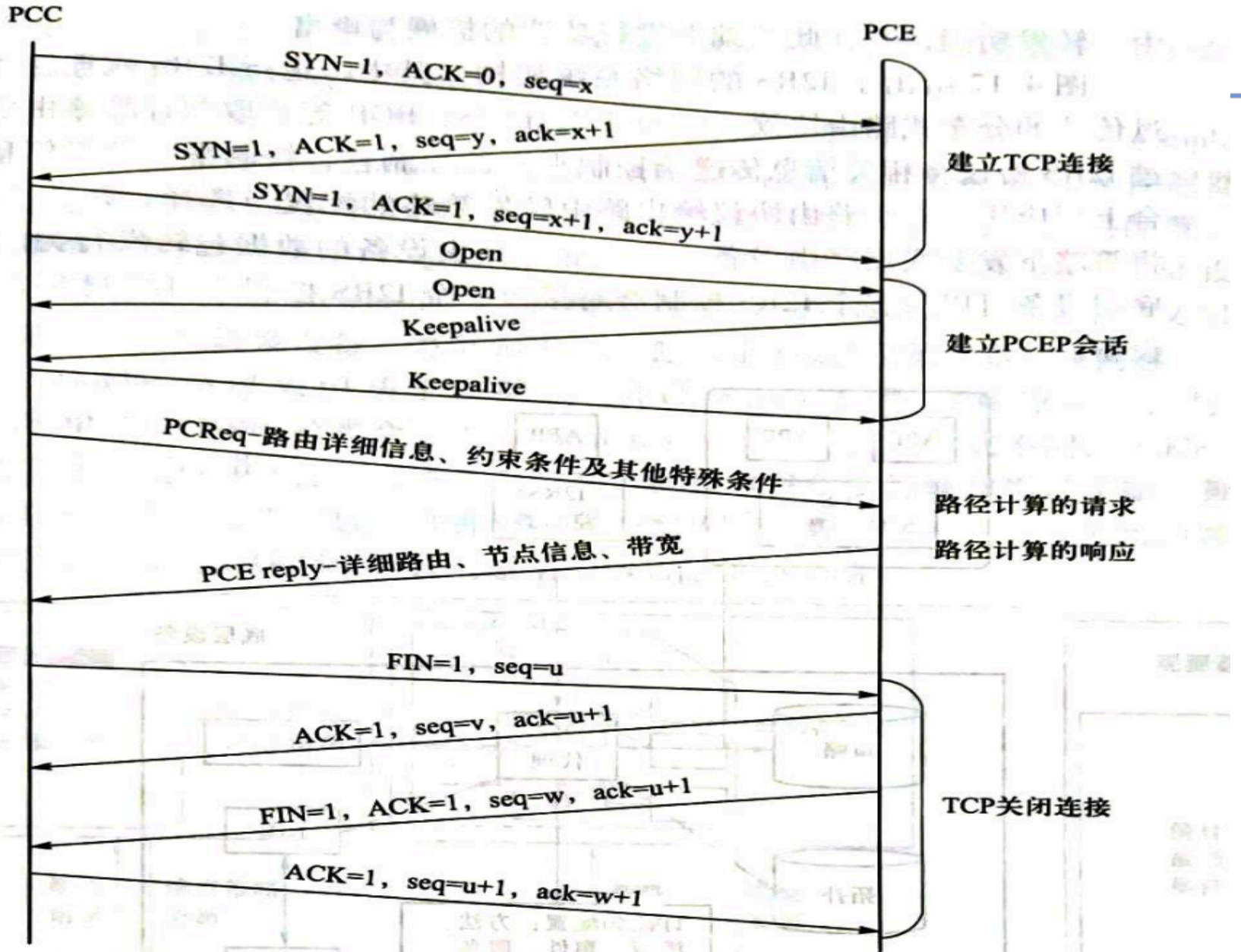


图 4.16　PCEP 协议一次会话交互过程

I2RS是路由系统接口的简称，是IETF主推的SDN南向接口协议。IETF于2013年成立了专门的SDN 工作组，致力于I2RS协议的标准化工作，并相继发布了RFC 7920 和 RFC 7921等相关协议文档。I2RS的核心思想是在传统网络设备的路由及转发系统上定义开放接口，使外部应用或控制实体可读取路由器中的信息，从而可基于拓扑变化、流量统计等信息动态下发路由状态、策略到转发设备上，以支持网络的可编程能力。I2RS基本保留了传统网络设备的结构和路由、转发功能，并在此基础上进行功能的扩展与丰富。

图4.17给出了I2RS的网络系统架构。其中，运行I2RS过传统的分布式路由协议，如 OSPF、IS-IS、BGP等获取并计算路由信息，并通过I2RS协议将相关信息传递给控制平面的控制器，控制平面负责信息收集，结合上层应用需求和路由协议给出路由转发策略的配置与选择，并通过I2RS代理将策略下发至底层路由设备，指导底层路由设备的数据包转发行为。一个I2RS底层设备可以与多个I2RS控制器对接，一个I2RS控制器可控制多个I2RS 底层设备。

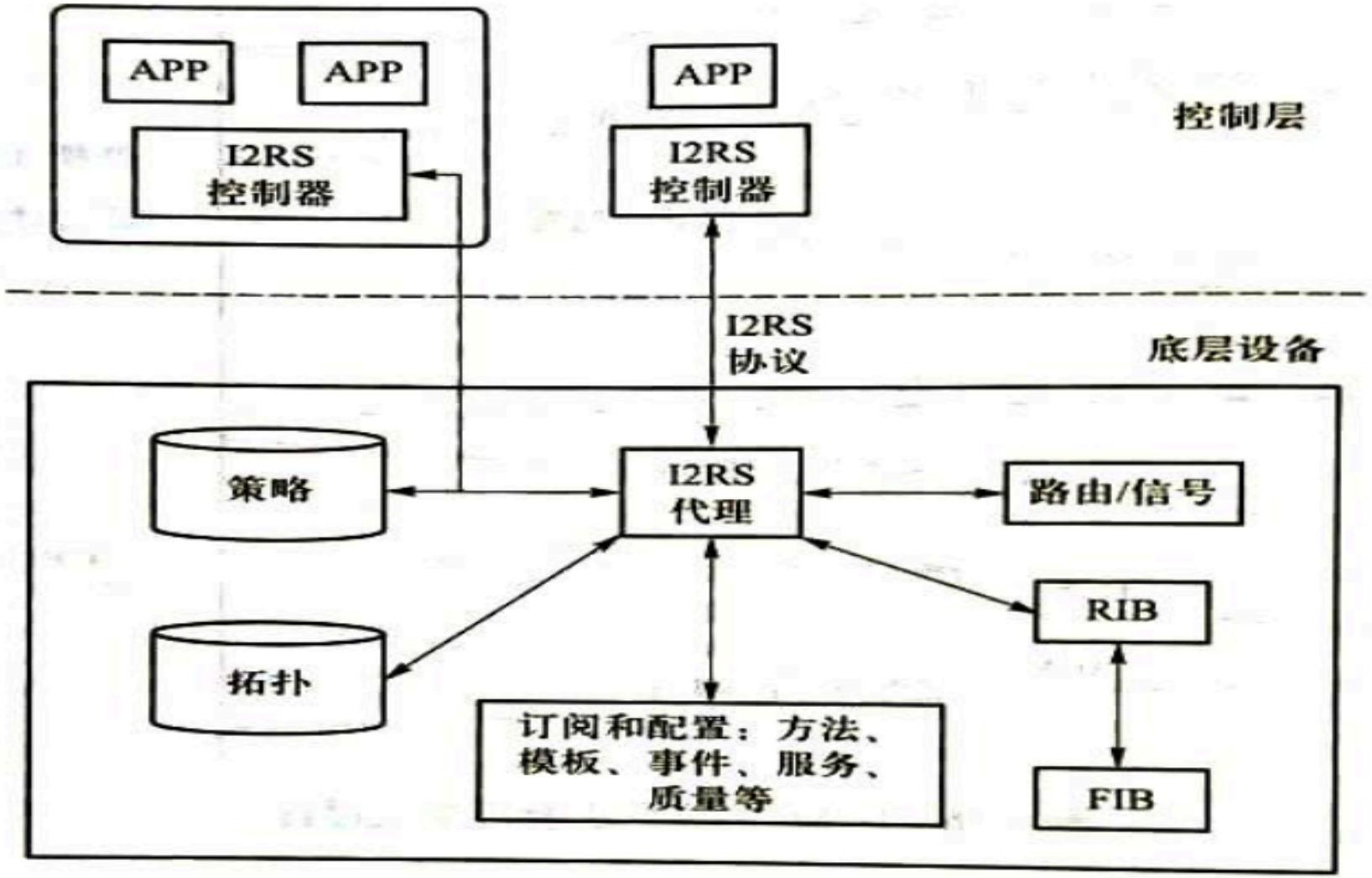


图 4.17　I2RS 的系统结构

从 I2RS 的设计来看，它借鉴了狭义 SDN 中的控制层与基础设施层概念，但其底层设备仍然保留了部分控制平面的功能，路由计算仍然以分布式形式在每台底层设备上进行，只是把路由策略选择与配置、流量统计等控制功能集中到I2RS控制层，这与狭义SDN中将数据平面和控制平面完全分离的设计思想有很大的不同，因此I2RS也被认为是一种广义的SDN南向接口协议。

OpFlex

OpFlex是思科公司提出的南向接口协议。面对SDN技术的加速落地，思科提出了“以应用为中心的基础设施”(ACI)SDN解决方案。OpFlex是ACI内部的策略控制协议，思科开放了OpFlex的部分源码，并向IETF提交了协议草案，以实现OpFlex协议的标准化。

在ACI体系中，控制平面提供了应用策略基础设施控制器(applicationpolicy infrastructure controller,APIC)。APIC的作用类似于狭义SDN 控制器，其向上通过GUI/CLI/REST API/Python Seripts 向应用层的管理员提供配置、管理和策略下发的接口，向下通过 OpFlex 协议将策略推送给 ACI Leaf/vLeaf 设备中的策略基元(policy element，PE)模块，ACI Leaf/vLeaf 设备相当于狭义SDN 中的基础设施层转发设备。关于ACI的详细介绍可以参见第7章相关内容。

OpFlex协议采用基于声明式(declarative)的配置管理方式，与 CLI/OVSDB/NETCONF 这类命令式(imperative)的配置管理方式不同，OpFlex将策略实现的复杂性从控制器中剥离出来，控制器只负责管理和推送策略，策略的实现由转发设备来完成，这种声明式的配置管理方式在一定程度上可以减轻控制平面的压力。OpFlex协议的通信模型如图4.18所示。

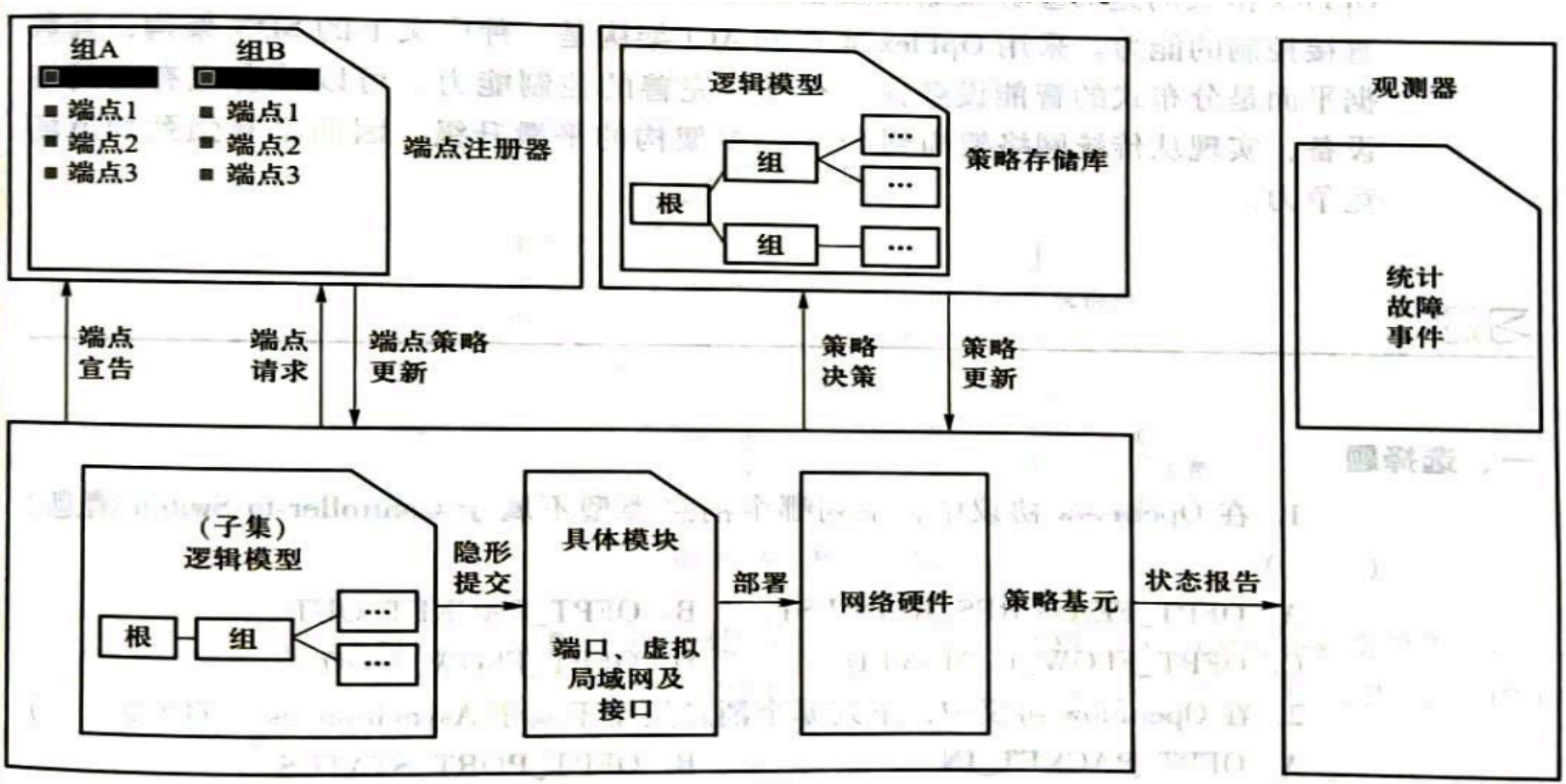


图 4.18 OpFlex 协议的通信模型

在 OpFlex 协议通信模型中，端点注册器(endpoint registry，ER)存储各端点(endpoint，EP)的信息，并根据策略存储库(policy repository， PR)中所存储的策略对各端点进行分类，形成不同的端点组(endpoint group，EPG)。策略注册器用于存储不同的业务或应用策略，策略基元负责监测各端点并在转发设备本地执行策略。观测器(observer)负责监测转发设备的状态信息。上述协议功能角色中，策略基元分布在ACI的各个Leaf/vLeaf上，相当于各个转发设备本地；端点注册器、策略存储库和观测器则是逻辑上集中的，位于APIC集群中。

当出现新的端点时，端点会向端点注册器宣告并提出请求，端点注册器根据存储在策略存储库中的策略将新端点添加到指定的端点组中，并将新的端点信息反馈给策略基元。在出现数据转发时，策略基元根据策略存储库中的策略完成数据的转发，并向观测器发送状态报告，收集统计信息、错误和事件。

OpFlex 协议可以通过XML或者JSON来实现，承载于RPC之上，提供了双向的通信能力。目前，开源控制器 OpenDaylight (ODL)已经支持 OpFlex 协议，虚拟交换机OVS也可以通过部署 OpFlex 代理来支持 OpFlex协议。本质上OpFlex 擅长的是对业务策略和设备的管理，却并不具备对转发设备的转发进行直接控制的能力。采用OpFlex 实现的ACI架构是一种广义上的SDN架构，其数据平面是分布式的智能设备，具有非常完善的控制能力，可以兼容现有的网络设备，实现从传统网络架构到SDN 网络架构的平滑升级，因而具有强劲的市场竞争力。

明德新民 | 止于至善

明德新民 | 止于至善

第五章 SDN控制平面

学院：软件学院 任课教师：申凌峰

河南大學

目 录

CONTENTS

- 01 SDN控制器概述
- 02 开源控制器
- 03 商业控制器
- 04 基于控制器的编程

Part 1

SDN控制平面

在SDN架构中，控制器是核心，它是连接底层交互设备与上层应用之间的桥梁。一方面，控制器通过南向接口协议实现对底层网络交换设备的集中管理与状态监测，为数据平面提供转发决策和流量调度；另一方面，控制器通过北向接口向上层应用开放多个层次的可编程能力，允许用户根据特定的网络应用场景灵活地定制相应的运行与管理策略，并为开发人员提供灵活的开发接口，为用户提供方便使用的操作接口。

图5.1给出了一种典型的控制器体系架构。该架构分为两大部分，其中左边为控制器的控制功能部分，自下而上分为4层，分别为协议适配层、基础网络层、内置应用层和北向接口层；右边为控制器的管理功能部分，称为配置管理层，包括用户界面管理、集群管理、软件管理等功能。

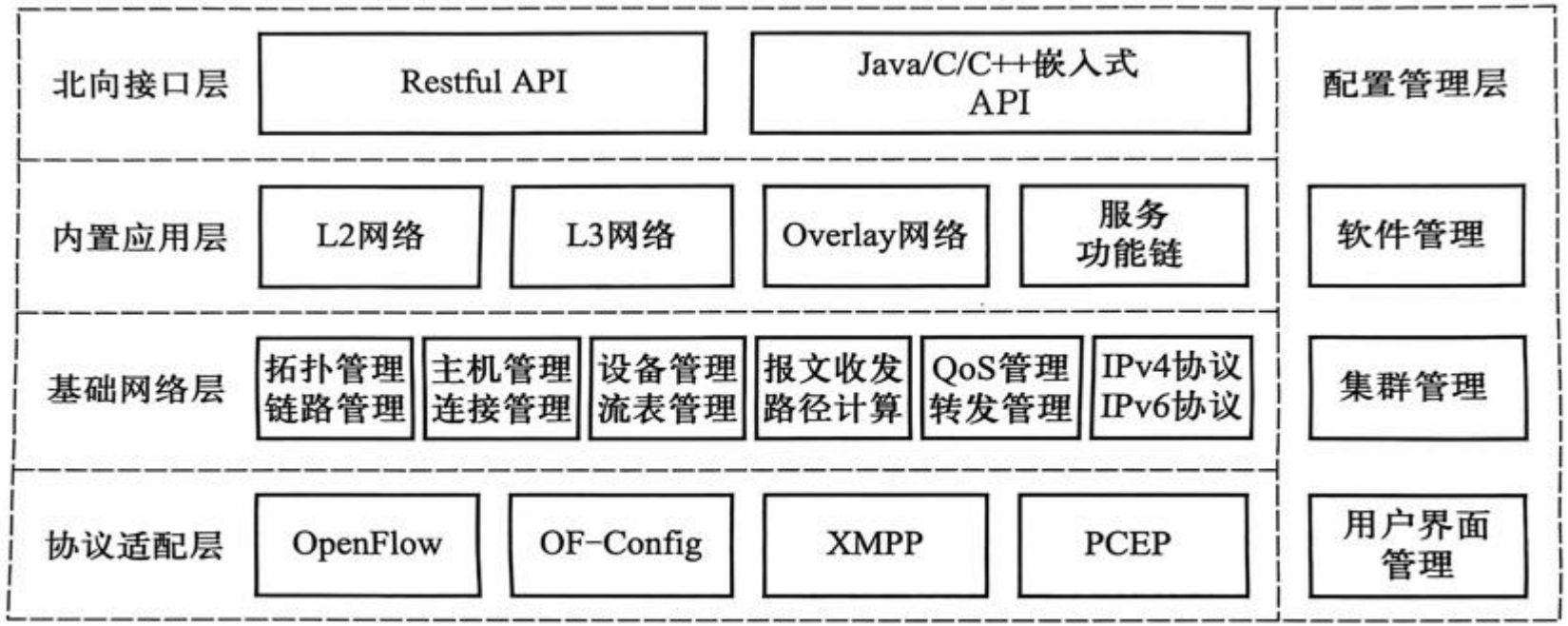


图 5.1　典型控制器的体系架构

图5.1 典型控制器的体系架构：

(1)协议适配层：主要实现对底层多种协议的适配，包括支持多种南向接口协议。

(2)基础网络层：这是该体系模型中必不可少的核心层，用于实现关键的网络控制逻辑，包含控制器内部的实现逻辑和一些底层的网络实现逻辑。

(3)内置应用层：用于提供一些常用的基础功能包。

(4)北向接口层：用于提供与应用层交互的北向接口。

(5)配置管理层：该层实现包括软件管理、集群管理和用户界面(user in-erface，UI)管理等功能。

在SDN控制器的众多功能中，南向网络控制是起核心作用的基础功能，涉及链路发现、拓扑管理、策略制定、表项下发等。其中链路发现和拓扑管理是控制器利用南向接口的上行通道对底层交换设备上报信息进行统一监控和统计；而策略制定和表项下发则是控制器利用南向接口的下行通道对网络设备进行统一控制。下面对这些核心功能进行介绍。

1. 链路发现

链路发现是指控制器利用南向接口的上行通道，收集底层交换设备上报的链路状态相关信息，并经过统计和分析获得对底层链路的发现及其状态信息。

目前，在SDN链路发现的技术实现上，主要采用链路层发现协议(linklayer discovery protocol， LLDP)，该协议定义在IEEE 802.1ab中，是一个二层协议，广泛应用于网络设备的发现，其中也包括SDN网络。

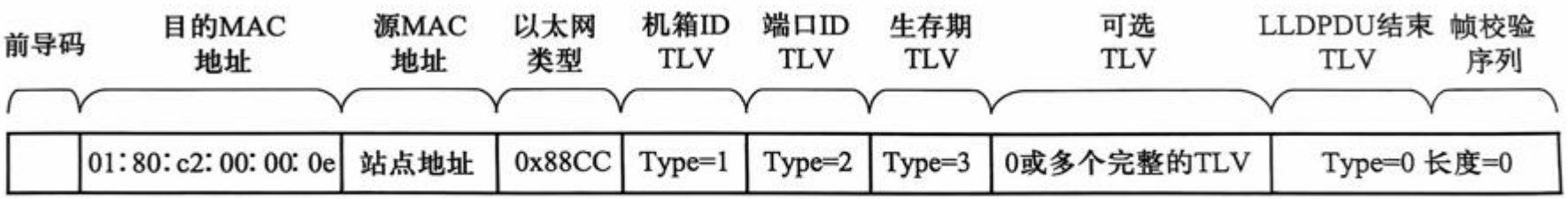


图 5.2 LLDP 报文结构

(1)前导码：用于同步发送端与接收端的传输信号。
(2)目的MAC地址：用于指明下一跳MAC地址。
(3)源MAC地址：用于记录源MAC地址。
(4)以太网类型：用以将LLDP报文与其他MAC数据帧加以区分。
(5)机箱ID：用于描述设备信息。
(6)端口ID：描述发送端口信息。
(7)生存期：表示该LLDPDU的有效时间。
(8)可选：附加信息。
(9) LLDPDU结束：作为LLDPDU结尾。
(10)帧校验序列：校验数据帧，判断传输过程中是否出错或被篡改。

图5.3LLDP链路发现流程

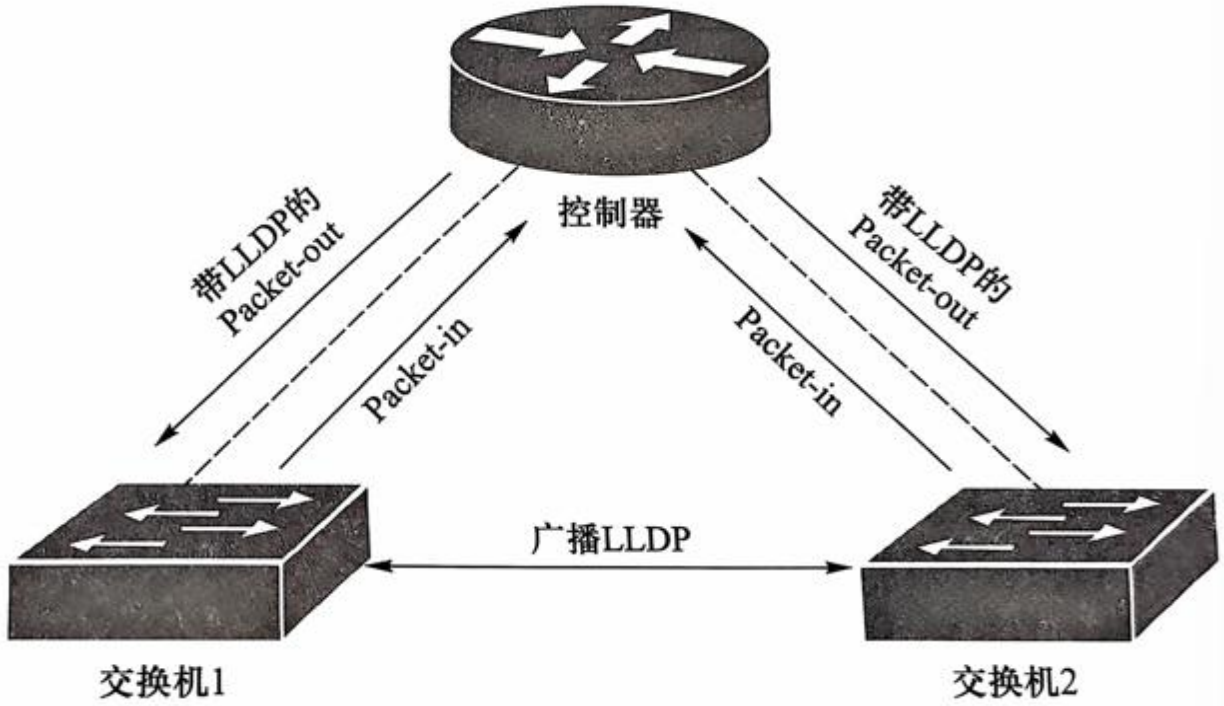


图 5.3　LLDP 链路发现过程示意图

2. 拓扑管理

拓扑管理为了随时监控和采集网络中SDN交换机的信息，及时反馈网络的设备工作状态和链路链接状态，以形成对网络拓扑及其变化的准确认识。

拓扑管理还有一项主要工作：在随时更新SDN交换机及链接状态的同时，对各种逻辑组网信息进行记录。

3. 策略制定

南向网络控制中的核心功能之一是策略制定，交换流表生成算法是影响控制器智能化水平的关键因素，控制器需要针对不同网络层次的传输需求，制定相应的转发策略并生成对应的流表项。

控制器流表生成算法的计算结果即转发策略需要通过下发流表项来告知，有两种方式：主动下发与被动下发。

主动下发是指，控制器在交换机还没收到数据包前，预先给交换机下发流表，告诉交换机如何处理到来的数据包，包括第一个数据包。

被动下发是指，控制器不主动给交换机下发流表，等到交换机有数据流到来，才向控制器发送问询信息，等待下发流表，控制器在收到问询信息后，再下发相应的流表。

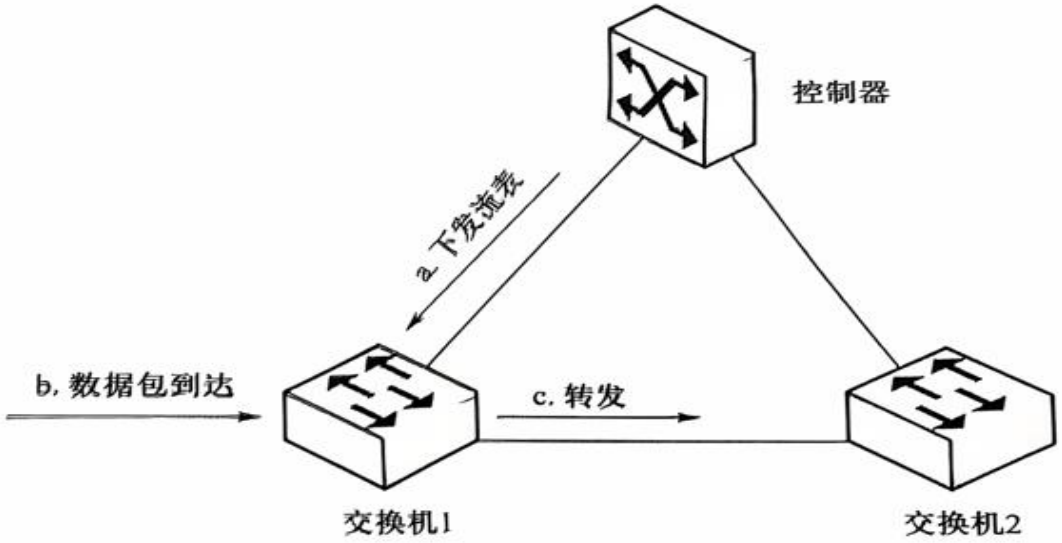


图 5.4　控制器主动下发流表示意图

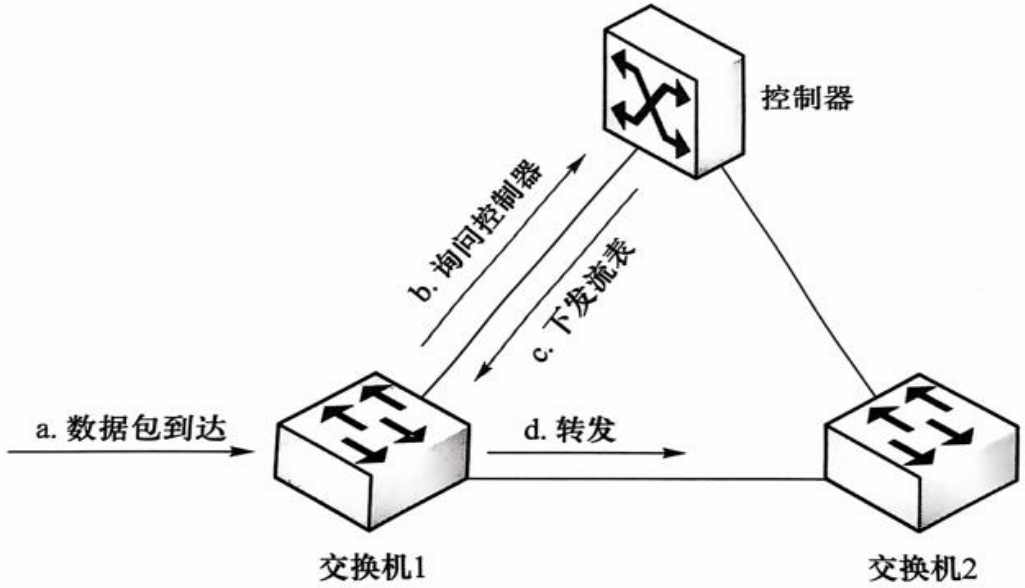


图 5.5　控制器主动下发流表示意图

控制器负责整个SDN网络的集中化控制，对于把握全网资源视图、改善网络资源交付具有极其重要的作用。但控制能力的集中化，也意味着控制器的可靠性与安全性成为全网的瓶颈。

鉴于控制器对整个SDN网络的核心作用，有必要考虑采用多个SDN控制器组成的分布式集群，以有效避免单一控制器节点所存在的上述性能问题，提高可靠性与可扩展性。虽然关于多个控制器之间沟通和联系的东西向接口目前还没有公认标准，但一些非常成熟的集群技术可应用到SDN网络中来解决上述难题。例如，以控制器软件化为依托的服务器集群方案。

控制器的软件化使服务器可以作为控制器软件的运行载体，从而控制器集群可以基于服务器集群来搭建和实现。鉴于控制器对于SDN网络的重要核心作用，控制器集群的设计具有一些特殊性，例如，要支持向正在运行的集群中增加新的控制器，以改善扩展性，同时要保存失效控制器对应的交换机状态，以保证可靠性。考虑控制器掌控着全网的SDN交换设备，通过分布式协作方式确保网络状态的一致性尤其重要。

如图5.6所示：基于控制器集群的SDN架构

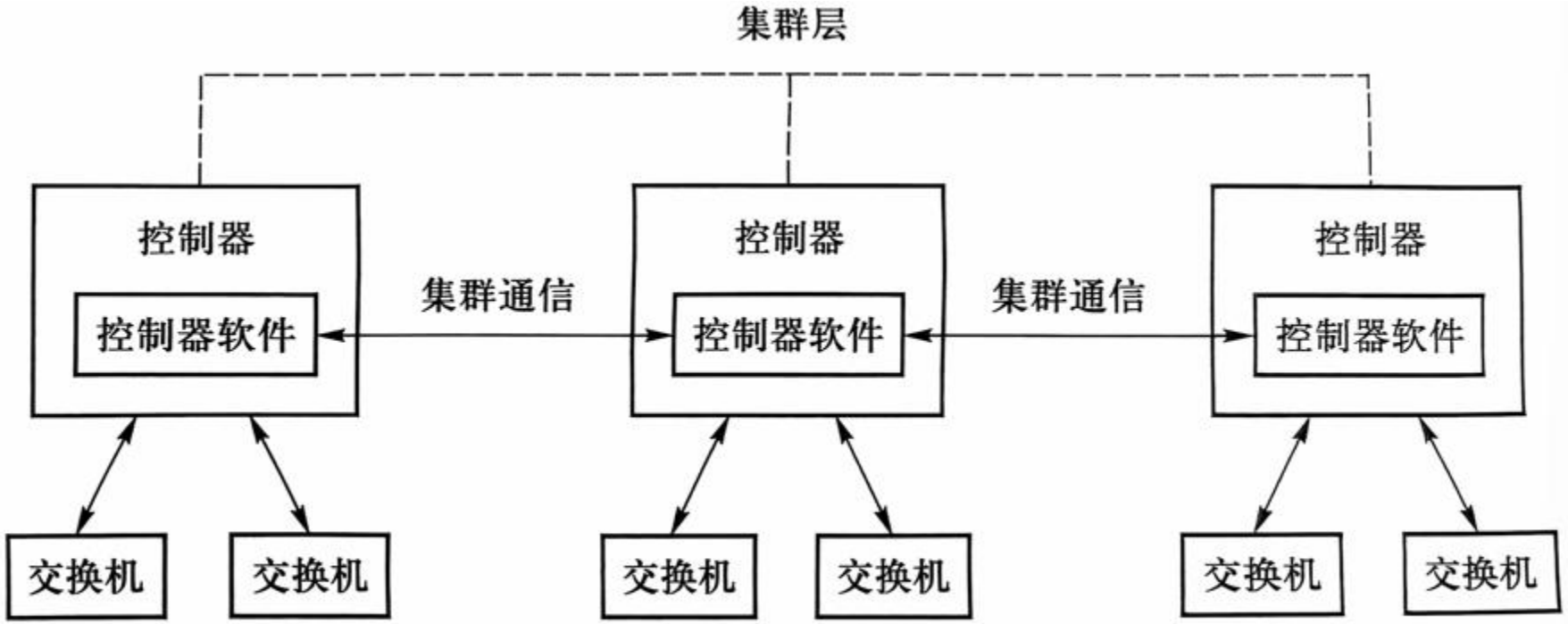


图 5.6　基于控制器集群的 SDN 架构

1.网络功能

控制器对于网络功能的支持是评估的首要因素。

2. 响应时间

性能主要关注在处理相同或类似功能的情况下，控制器在响应能力上的不同。性能评测主要涉及吞吐量(throughput)和延时(delay time)两个关键指标。

3.可扩展性

可扩展性主要关注对网络规模增大的适应能力。在此，网络规模包括了底层基础设施和上层应用。

4.可靠性

控制器作为SDN的核心，其自身的可靠性至关重要。除了强化单台控制器的可靠性之外，可以考虑采用控制器集群来提升可靠性。另外，控制器能否为所管控的网络提供足够的容错性或冗余备份也是其可靠性的重要体现。

5.可编程性

网络可编程性是SDN的重要特征，因此，控制器的可编程能力也是需要考虑的重要因素。控制器的可编程性包括控制器内部编程和控制器外部编程。控制器内部编程是指基于控制器的基础网络层接口，开发网络应用程序；控制器的外部编程是指基于控制器的北向接口，开发网络应用程序。

6.安全性

控制器是SDN的核心，故其安全性也是重要的评估指标因素。

除上述关键因素外，控制器接口的丰富性、兼容性、易用性等也是需要考虑的因素。

Part
02

开源控制器

开源控制器

目前，已经实现的SDN开源控制器非常多。若按实现的程序设计语言进行分类，主要有三类：第一类为C或C++语言系列，如Mul、Trema和NOX控制器；第二类为Python语言系列，如POX和Ryu控制器；第三类为Java语言系列，如Beacon、Floodlight、OpenDaylight和ONOS.

表5.1给出了主流开源控制器及其实现语言。

表 5.1　主要开源控制器列表

名称	实现语言	特 征 简 介
Mul	C	由 Kulcloud 公司开发，内核是一个基于 C 语言的多线程基础架构，用于托管应用的多层级北向接口
Trema	Ruby/C	由 NEC 开发，具有模块化的框架
NOX	C++	由 Nicira 开发，是业界第一款 OpenFlow 控制器，是众多 SDN 研发项目的基础
POX	Python	由 Nicira 开发，是 NOX 的纯 Python 实现版本，支持控制器原型功能的快速开发
Ryu	Python	由 NTT 开发，能够与 OpenStack 平台整合，具有丰富的控制器 API，支持网络管控应用的创建
Beacon	Java	由斯坦福大学开发，采用跨平台的模块化设计，支持基于事件和线程化的操作
Floodlight	Java	由 Big Switch Networks 开发，基于 Beacon，是企业级的 OpenFlow 控制器
OpenDaylight	Java	主要由设备厂商驱动，如思科、IBM、HP、NEC 等
ONOS	Java	由开放网络实验室 ON. Lab 推出

1. NOX

NOX是SDN发展史上第一个控制器，由Nicira开发，并于2008年发布。NOX的底层使用C++编写，支持OpenFlow v1.0协议，提供C++版本的Open-Flow v1.0的API，具有高性能的异步网络1VO。

NOX的功能架构可分为核心层和外围组件两部分。核心层主要提供对组件、事件的管理以及与底层网络的通信接口。外围组件主要用于实现基本的网络功能，组件内部的网络业务逻辑基于OpenFlow的API进行开发。

NOX基于组件的模式促进了模块化的开发，提高了功能的可扩展性和可重用性，从而影响了后来的很多控制器设计方案。按照功能定位，NOX组件可以分为核心组件和应用组件两种类型。

1. NOX

使由Nicira开发，与SDN网络控制相关的组件主要有以下几种。

(1)拓扑发现组件：用于网络拓扑推算的LLDP发现应用。

(2)拓扑组件：用于在控制器内存中记录当前网络中所有处于活动状态的链接信息，并根据连接情况变化及时更新。

(3)路由组件：用于数据转发路径的计算，它将尽可能根据路由算法和应用业务需求，保证最佳转发路径的选择，并与拓扑组件关联以维护路由信息。

(4)监控组件：用于周期性地向所有与控制器相连的交换机发送查询信息，以获知交换机工作状态是否正常。

(5)认证组件：用于在控制器内存中记录网络中所有经过认证的主机和用户信息，并利用一个三元组作为网络标识对相关信息进行索引。这个三元组由链路层地址、网络层地址和访问点（如数据通路、端口对等）组成。

作为世界上第一个SDN控制器，NOX在SDN发展初期得到了广泛的应用。但由于采用C++开发难度非常高，影响了NOX开发的效率和推广，为此Nicira公司于2011年推出了NOX的增强版POX。

2. POX

POX采用Python 语言编写，其基本结构如图5.7所示。

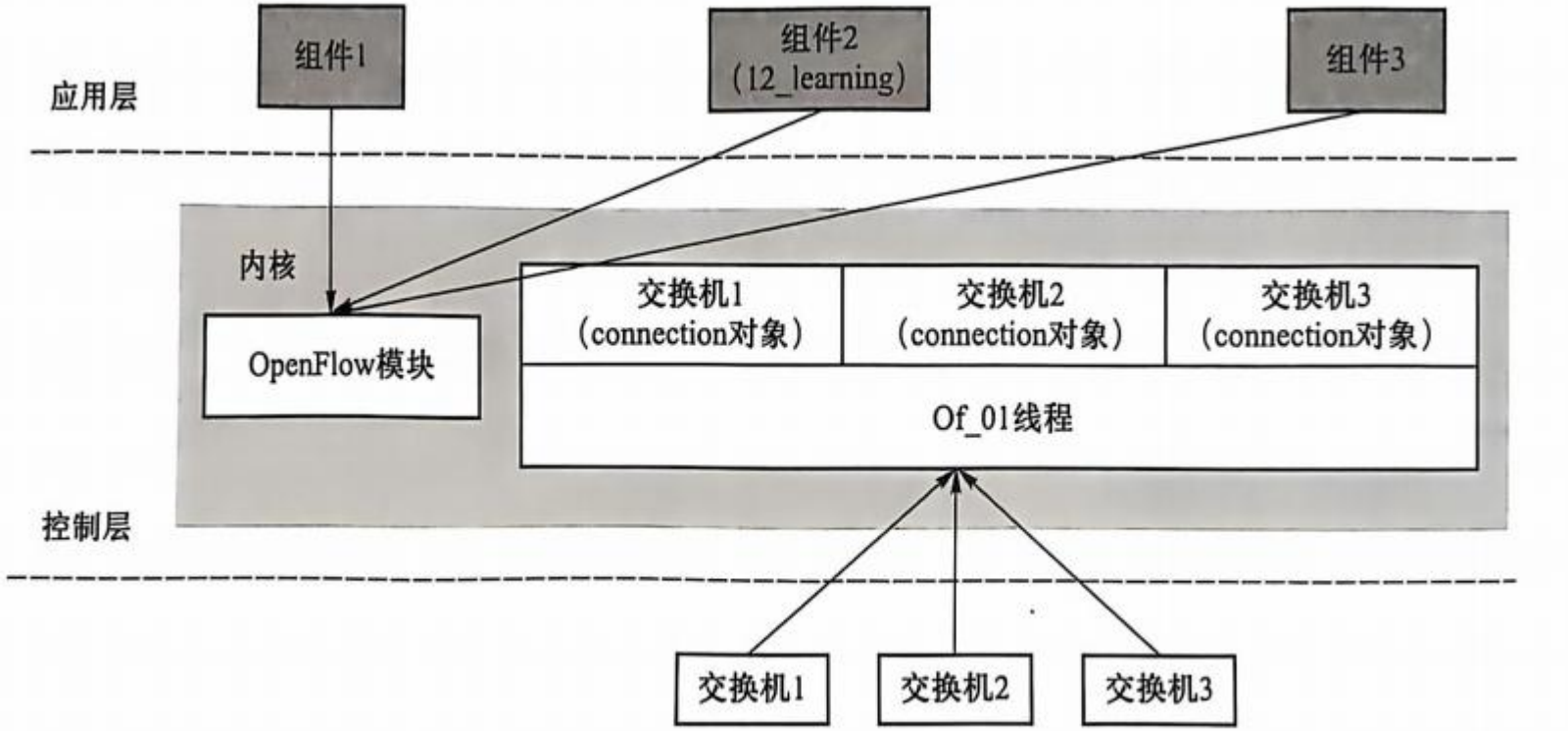


图 5.7　POX 控制器组成结构图

2. POX

在功能方面，POX的核心部分及其组件和NOX基本一致。此外，POX还提供了基于Python语言的OpenFlow API以及如Topology、Discovery等可复用的基础模块。POX也可以使用NOX的GUI和虚拟化工具。在性能方面，POX要优于NOX，尤其是在使用PyPy运行的情况下。学习SDN控制器，需要了解其代码结构以及组件的功能，表5.2对部分POX组件做了简要的介绍。

表 5.2　POX 组件及功能介绍

组件名称	功能介绍
datapath. switch	定义了软件定义的 OpenFlow 交换机相关的数据结构
forwarding. l2_learning	一个最简单的二层自学习交换机的实现
forwarding. l2_multi	基于网络拓扑信息的最短路径二层交换应用
forwarding. l3_learning	三层交换，增加了 ARP 处理逻辑
host_tracker. host_tracker	主机位置与配置追踪
info.*	信息显示的相关组件

2. POX

续表

组件名称	功能介绍
lib. graph.*	使用 Networkx 定义的图相关存储内容
lib. packet.*	定义了网络报文的解封装和处理逻辑
lib. revent	事件系统的定义
log	日志系统
messenger.*	定义了与外部进程通信的消息系统
openflow. libopenflow_01	OpenFlow v1. 0 库类的定义
openflow. spanning_tree	生成树策略
openflow. discovery	拓扑发现模块
openflow. topology	连接 OpenFlow 和 Topology
topology	拓扑模块
web. core	Web 服务模块
pox. core	POX 的核心模块

2. POX

POX控制器的优点：

1. 开发简单，Python语言的代码结构简单，初学者容易上手，且无须安装即可使用。
2. 开发效率高，由于Python语言简单高效，所以POX应用的开发效率很高，适合快速开发和功能验证的科研场景；
3. 系统兼容性强，由于Python语言支持跨平台，所以POX可以在Linux、Mac OS和Windows等不同操作系统平台上运行。

POX的主要缺点：

1. 模块组件不够丰富，功能不够全面。
2. 相比于后面要介绍的Ryu控制器，POX的代码结构不够严谨，封装性和可扩展性相对不足。
3. 由于性能上的不足，所以POX在SDN发展道路上逐渐被边缘化，在SDN控制器的市场竞争中逐渐处于下风。

开源控制器–Ryu

Ryu基于Python语言开发，代码风格流畅，模块清晰，可扩展性很强。Ryu的版本迭代非常快速，至2019年11月份已迭代至版本4.34。

Ryu控制器在系统结构上分为控制层和应用层，在功能实现上采用了基于组件的模块化风格，如图5.8所示。

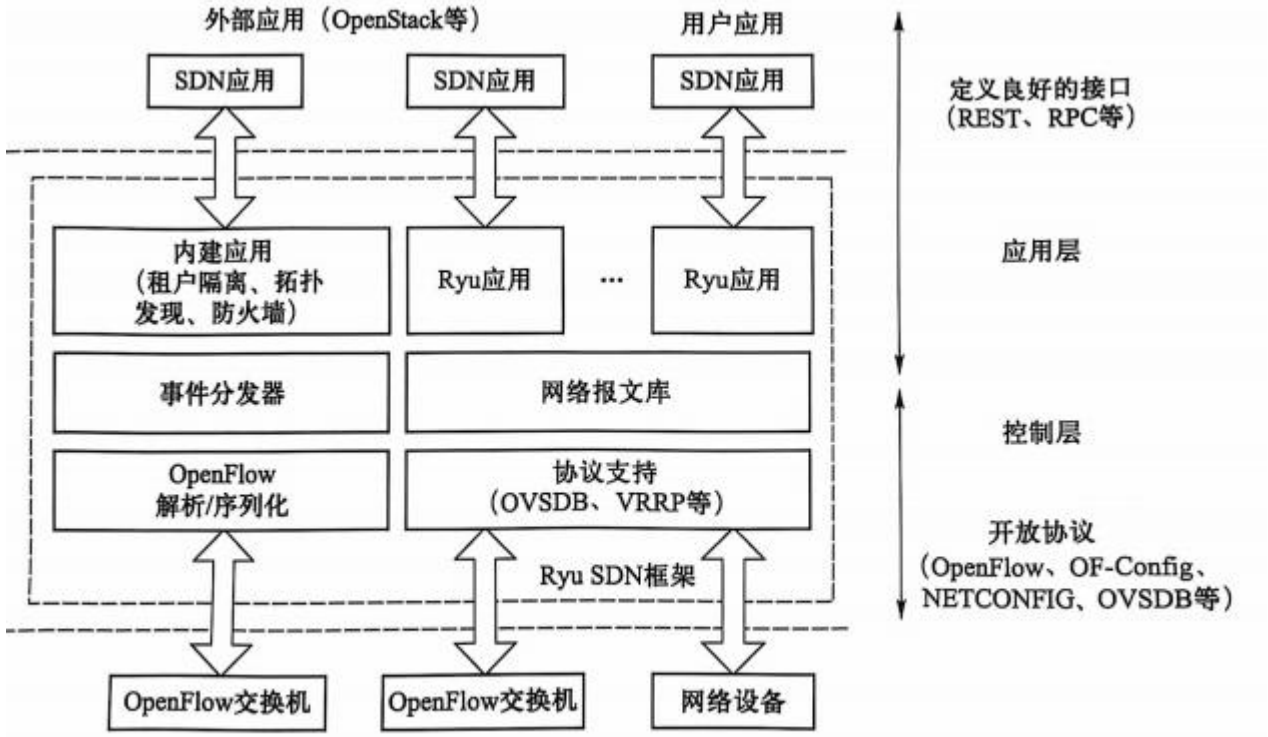


图 5.8　RYU 控制器系统结构图

开源控制器–Ryu

Ryu通过南向接口与数据平面的设备进行通信。
Ryu通过北向接口实现与外部应用的交互。Ryu是一个功能特性丰富的SDN控制器 。
表5.3描述了Ryu的主要组件及功能简述。

表 5.3　Ryu 主要组件功能介绍

组件名称	功 能 描 述
base. app_manager	Ryu 的组件（模块）调度中心
cmd.*	Ryu 的命令解析模块
controller. controller	定义了 OpenFlowController 类，用于描述控制器的行为
controller. ofp_event	定义了 OpenFlow 的事件类型，用于完成 OpenFlow 报文到事件的转换
controller. ofp_handler	OpenFlow 报文处理的基础模块，包括 Hello、Packet-in、Packet-out 等报文处理
lib.*	协议报文的定义库，包括 TCP/IP 和其他协议报文的定义
ofproto. ofproto_vx_x	定义了 OpenFlow 协议不同版本的静态参数及报文封装的格式
ofproto. ofproto_vx_x_parser	定义了 OpenFlow 报文的类及其解析函数和报文的序列化函数等内容
topology.*	定义了拓扑相关的事件和数据结构，向外提供对应的 API
app. rest	定义了基础的 REST API 接口
app. ofp_rest	定义了 OpenFlow 相关的 REST API
app. simple_switch	实现简单二层交换的应用

开源控制器-Floodlight

Floodlight 系统结构和其他控制器类似，如图5.9所示，分为控制层和应用层两个主要部分，应用层通过北向API实现与控制层的信息交互；控制层通过南向接口和数据平面通信，实现对数据平面的控制。

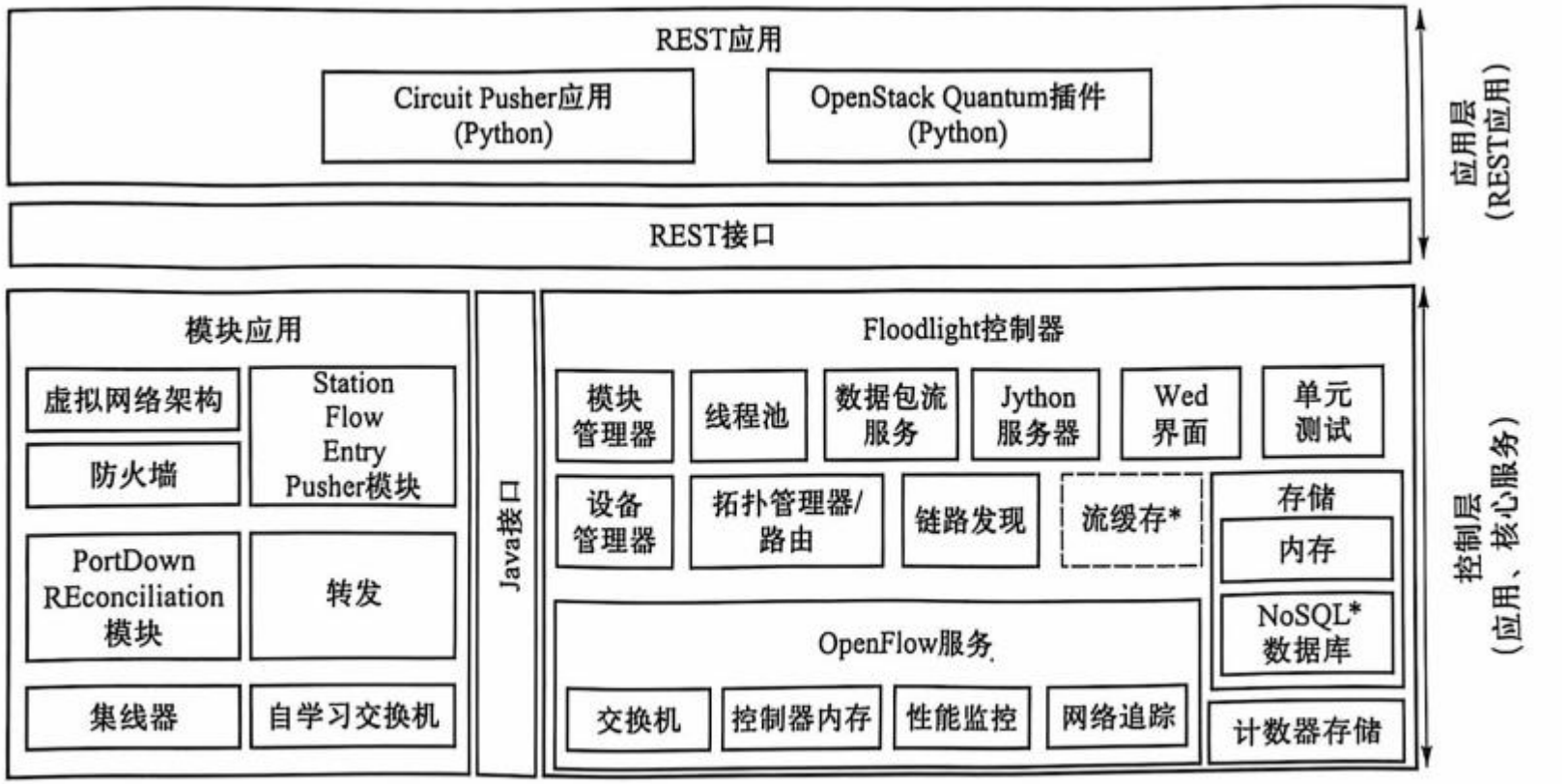


图 5.9　Floodlight 控制器系统结构图

表5.4 Floodlight 的核心模块功能介绍

表 5.4 Floodlight 的核心模块功能介绍

组件名称	功能介绍
Device Manager	通过 Packet-in 报文，可习得设备与交换机端口的连接信息
Floodlight Provider	Floodlight 的核心模块，完成交换机连接处理
Link Discovery Manager	使用 LLDP 协议获取网络链路信息
Topology Manager	拓扑模块，依赖于链路发现模块的链路信息
OFSwitch Manager	管理连接的交换机，提供如发送 OpenFlow 报文等接口

表5.4 Floodlight 的核心模块功能介绍

续表

组件名称	功能介绍
Forwarding	实现数据的转发
Learning Switch	实现了一个简单的二层交换机
ACL	简单的接入控制列表模块
Load Balancer	负载均衡模块

Floodlight支持的南向协议目前只有OpenFlow，且所能支持的OpenFlow版本比较少，这是Floodlight 特性上的劣势。

优势方面，Floodlight 开放的模块化架构，使得其极易扩展和提升，且Floodlight的依赖少，容易安装。

此外，Floodlight支持混合组网，即支持OpenFlow和非OpenFlow交换机的混合网络。而且Floodlight 性能稳定且高效，这也是它的重要优势。

1.OpenDaylight系统架构

OpenDaylight 具有高度模块化、可扩展强、易于升级、支持多协议等特性。OpenDaylight控制器的系统架构如图5.10所示。

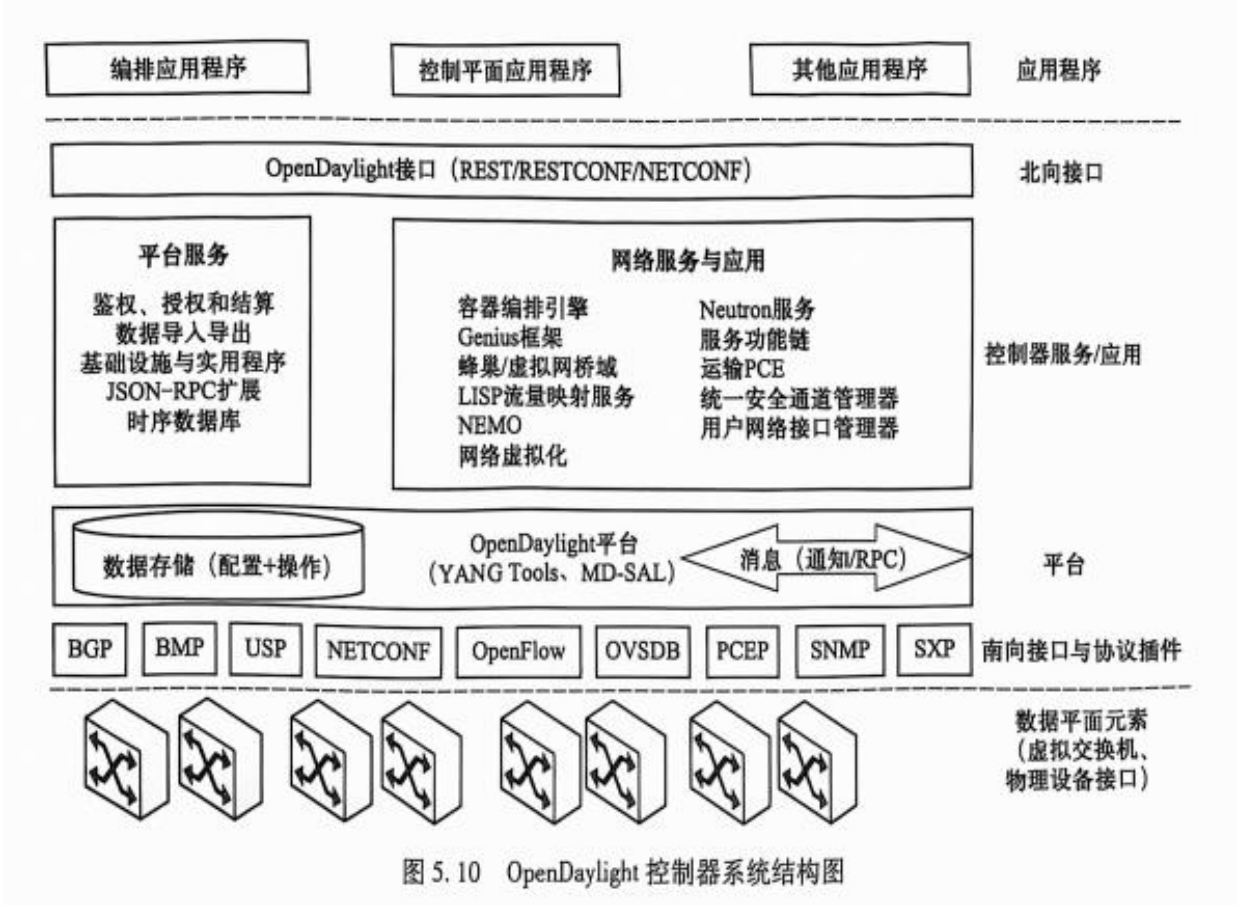


图 5.10　OpenDaylight 控制器系统结构图

1.OpenDaylight系统架构

1)南向接口与协议插件层

该层实现了对多种南向协议的支持。

2)平台层

该层是OpenDaylight平台的核心，其核心实现是模型驱动的服务抽象层(model-driven service abstraction layer. MD-SAL)，在OpenDaylight中，底层网络设备和网络应用程序都表示为对象或模型，通过YANG语言建模，YANG模型提供了有关设备或应用程序功能的一般描述，而不需要知道其具体实现细节。

3)控制器服务/应用层

该层提供平台服务(platform service)、网络服务与应用(network service andapplication)。平台服务包括鉴权、授权和结算(authentication，authorization andaccounting，AAA)、数据导入导出(data import/export)、基础设施与实用程序(infrastructure and utility)等。

4）北向接口层

该层提供OpenDaylight开放接口，包括REST API、RESTCONF、NETCONF等多种接口形式。

5)网络应用层

该层是基于OpenDaylight北向接口层的接口所开发出的应用集合。

2.OpenDaylight组件

OpenDaylight采于Java语言实现，以Maven来构建模块项目代码。

为了实现OpenDaylight良好的扩展性，OpenDaylight 基于OSGi框架运行，所有的模块均作为OSGi框架的bundle运行，OSGi是一个Java框架，其中定义了应用程序即bundle的生命周期模式和服务注册等规范。OSGi的优点是支持模块动态加载、卸载、启动和停止等行为，尤其适用于需要热插拔的大型模块化项目。

OpenDaylight作为一个SDN网络操作系统平台，基于OSGi框架开发可以实现灵活的模块加载和卸载等操作，而无须在对模块进行操作时重启整个控制器。OpenDaylight 使用Karaf容器来运行项目Karaf 基于OSGi运行环境提供了一个轻量级的OSGi容器。基于OpenDaylight控制器开发模块时，还需要使用YANG语言来建模，然后使用YANG Tools生成对应的Java API，并与其他Maven构建的插件代码共同完成服务实现。

2.OpenDaylight组件

OpenDaylight提供了数量众多、特性丰富的组件模块，也是其他控制器无相提并论的。究其原因，一方面得益于它开放的架构，使得模块插件的开发加系统化、规范化，所以开发新模块相对简单、高效；另一方面，OpenDaylig众多的支持团队和开发者的努力铸造了特性丰富的OpenDaylight。表5.5给出OpenDaylight部分组件的简介。

表 5.5 OpenDaylight 部分组件功能介绍

组件名称	功能介绍
OpenFlowPlugin	用于支持 OpenFlow 规范的实现。目前已经支持了版本 1.0 和 1.3.2
Netconf-connector	负责连接远程 NETCONF 设备的组件
Yang UI	提供一种应用于所有 OpenDaylight 控制器 YANG 模型的用户接口
ALTO	应用层流量优化模块
AAA Services	3A 授权认证模块，鉴权、授权和结算管理
BGP	BGP 协议模块，可以手动或通过 REST API 配置
PCEP	PCEP 协议模块，可以手动或通过 REST API 配置
L2Switch	二层交换模块，实现了 ARP 的处理，有环拓扑的风暴解除，主机追踪等
LACP	链路聚合控制协议模块
SDNi	实现了获取网络信息的接口，并完成多控制器之间的信息共享
VTN	虚拟租户网模块，向同一个控制器网络下的多租户提供虚拟网络
Cluster-deployer	提供集群能力，拥有多个节点提供容错，离散的点对点成员机制，避免单点故障

开源控制器-ONOS

2014年12月5日，ONOSv1.0版本Avocet的源码正式开源，标志着一个基于 Java语言开发、采用OSGI框架、支持分布式部署且性能卓越的控制器平台面世。相比OpenDaylight，最初版本的ONOS没有采用YANG，底层模块直接使用 Floodlight的核心模块，分布式系统实现上也采用了开源架构。由于ONOS采用了许多成熟的代码模块和框架，所以ONOS更新换代很快，易于扩展，也降低了对学习者的要求。由于ONOS优秀的性能、更加清晰的代码架构及它背后强大的支撑团队，ONOS自诞生以来就成为业界的热点，成为SDN控制器市场的有力竞争者。

开源控制器-ONOS

ONOS在系统结构上采用了SDN控制器普遍采用的分层架构，自下而上分为协议层、适配层、南向接口层、分布式核心层、北向接口层和应用层，如图5.11所示。

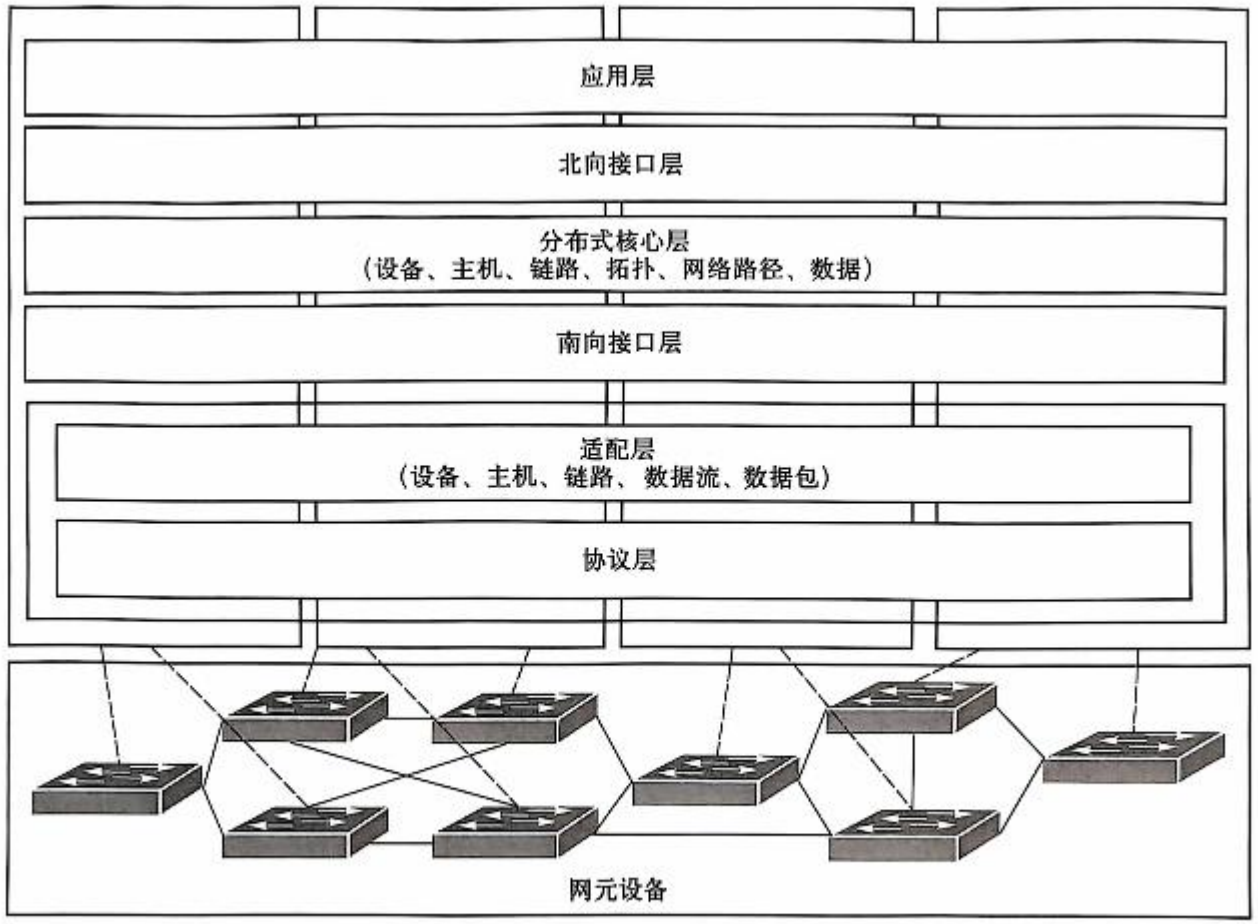


图 5.11　ONOS 系统结构图

在代码运行层面，ONOS和OpenDavlight一样，都是基于OSGi 框架来运行的。ONOS的模块由一个或多个bundle组成，每个bundle都是由 Maven构建的工程。表5.6给出了ONOS若干核心模块的简介。模块可分为生产者与消费者两种角色，生产者是产生数据的模块，消费者是处理数据的模块，核心管理模块manager负责接收生产者模块产生的事件并向消费者模块转发。

表 5.6　ONOS 部分功能模块

组 件名 称	功 能 简 介
app. sdnip	sdnip 应用，用于 SDN 域和传统网络域的通信
app. segmentrouting	SegmentRouting 应用，用于实现分段路由功能
app. proxyarp	ARP 代理模块
core. net	ONOS 核心模块，包含主机、流表、设备、拓扑等网络资源管理相关的组件
core. store	存储相关，包括提供数据序列化、持久化等功能
manager	核心管理模块，用于提供部分管理服务，包括负责请求路由，接收生产者模块产生的事件并转发到消费者模块，为其他模块提供服务接口的注册
providers	ONOS 架构中最低的一层，核心模块事件的产生者，包括南向协议 OpenFlow 等内容
openflow	OpenFlow 协议相关，包含控制器描述、握手、报文的封装和解析等内容

Part
03

商业控制器

1.APIC

APIC是应用策略基础设施控制器的英文缩写，也是这款控制器的商业名称。它是一款面向数据中心的SDN控制器，服务于思科的以应用为中心的基础设施(ACI)架构。它使用标准的REST API作为北向接口，而其南向接口主要使用OpFlex协议，用于管理思科Nexus 9000系列设备，为数据中心提供一个集安全集中管理、合规以及可扩展性于一体的全自动化一站式解决方案。

APIC控制器其核心控制功能采用模块化结构，包括系统管理器、租户管理器、网络结构管理器、虚拟机管理器和14~7层服务管理器等五大模块，提供了丰富的网络控制与管理特性。

商业控制器-思科APIC与DNA-C

(1)系统管理器：该模块负责控制器及服务器的管理，实现思科ACI交换矩阵、策略实施和健康状态监控自动化管理，能够在任何位置支持任何应用，并统一物理环境和虚拟环境的操作。

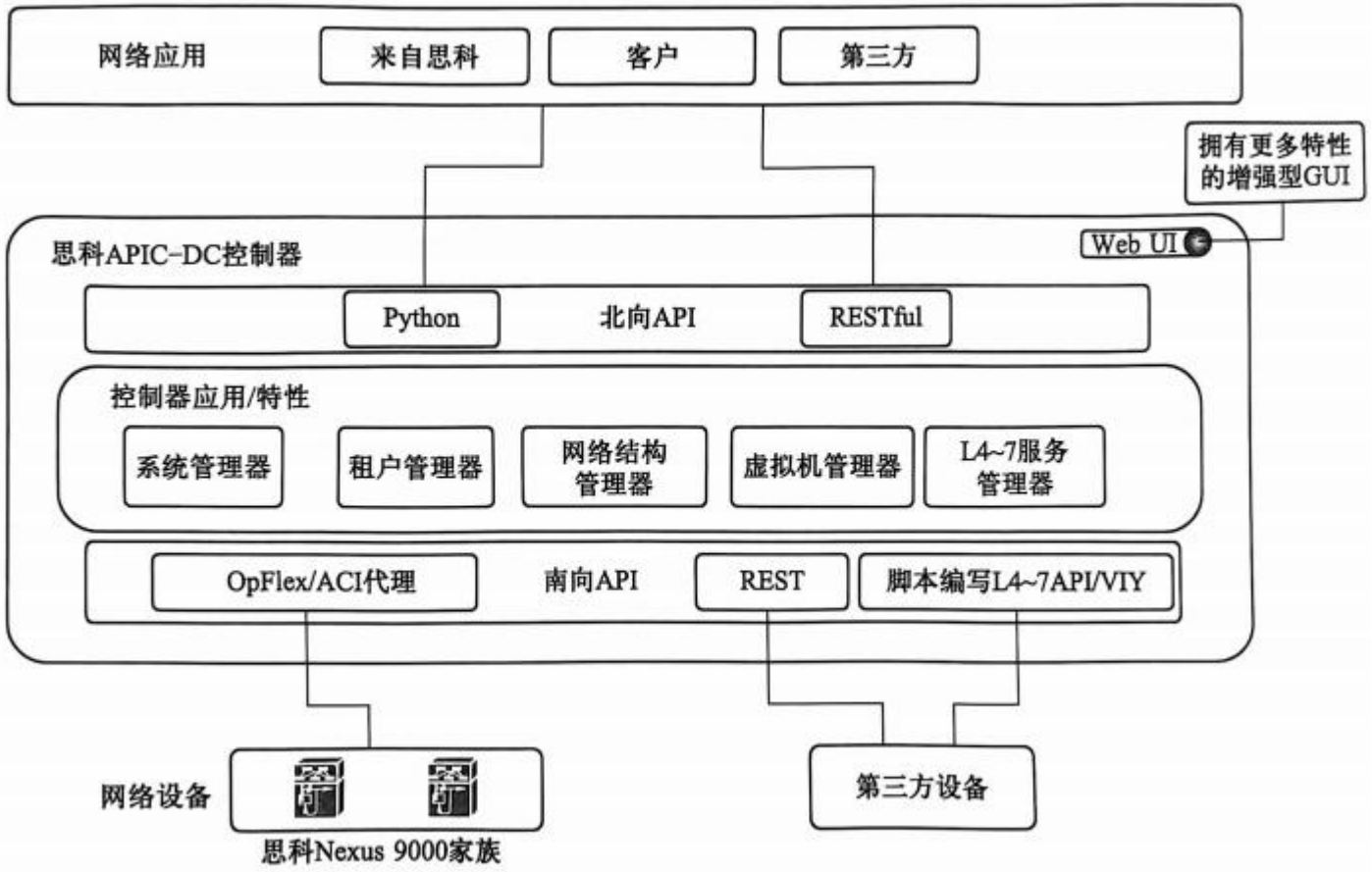


图 5.12　APIC 架构

(2)租户管理器：该模块提供用户所需的策略管理能力。

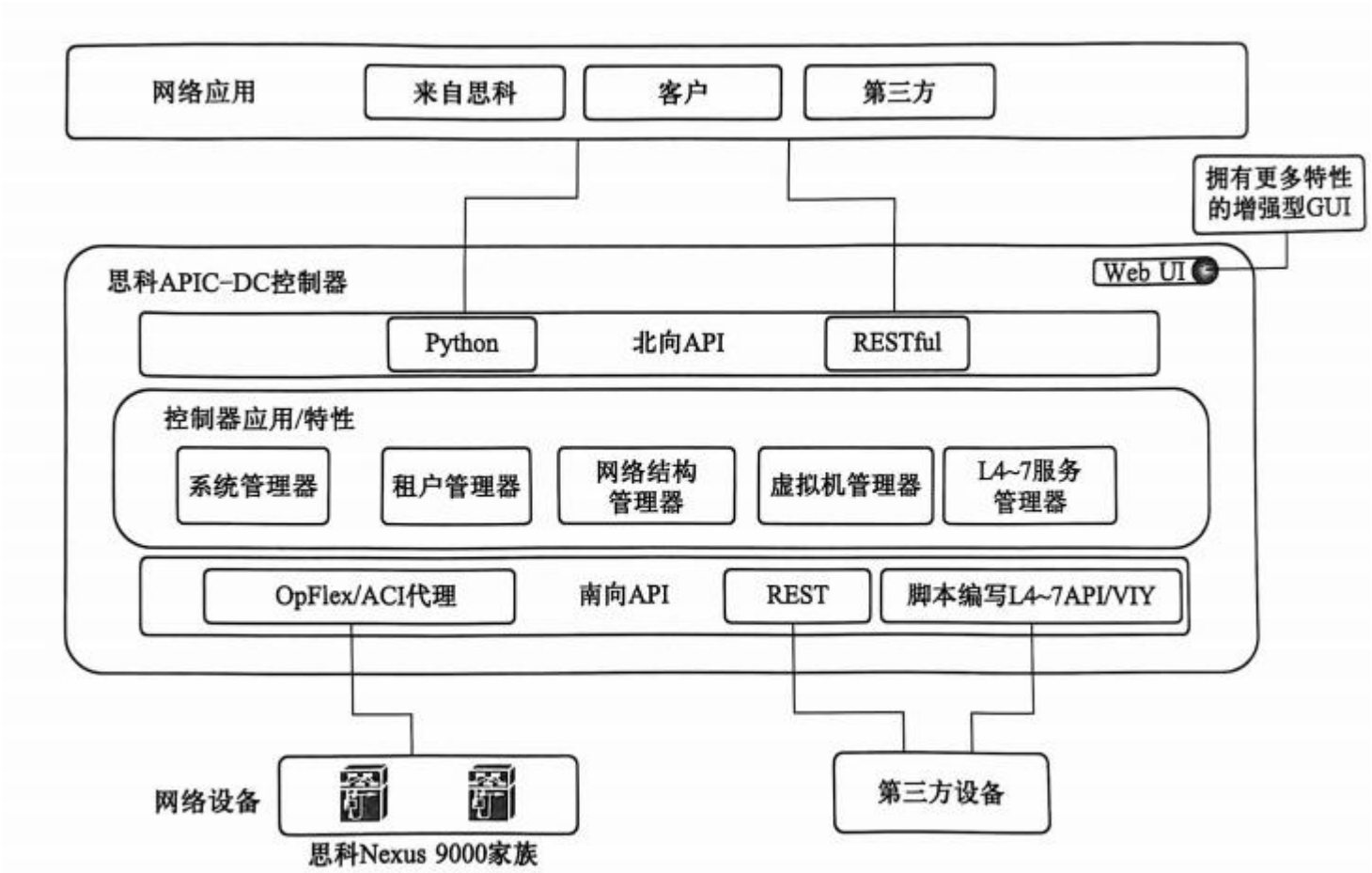


图 5.12　APIC 架构

(3)网络结构管理器：该模块主要提供对Nexus 9000的组网管理能力。

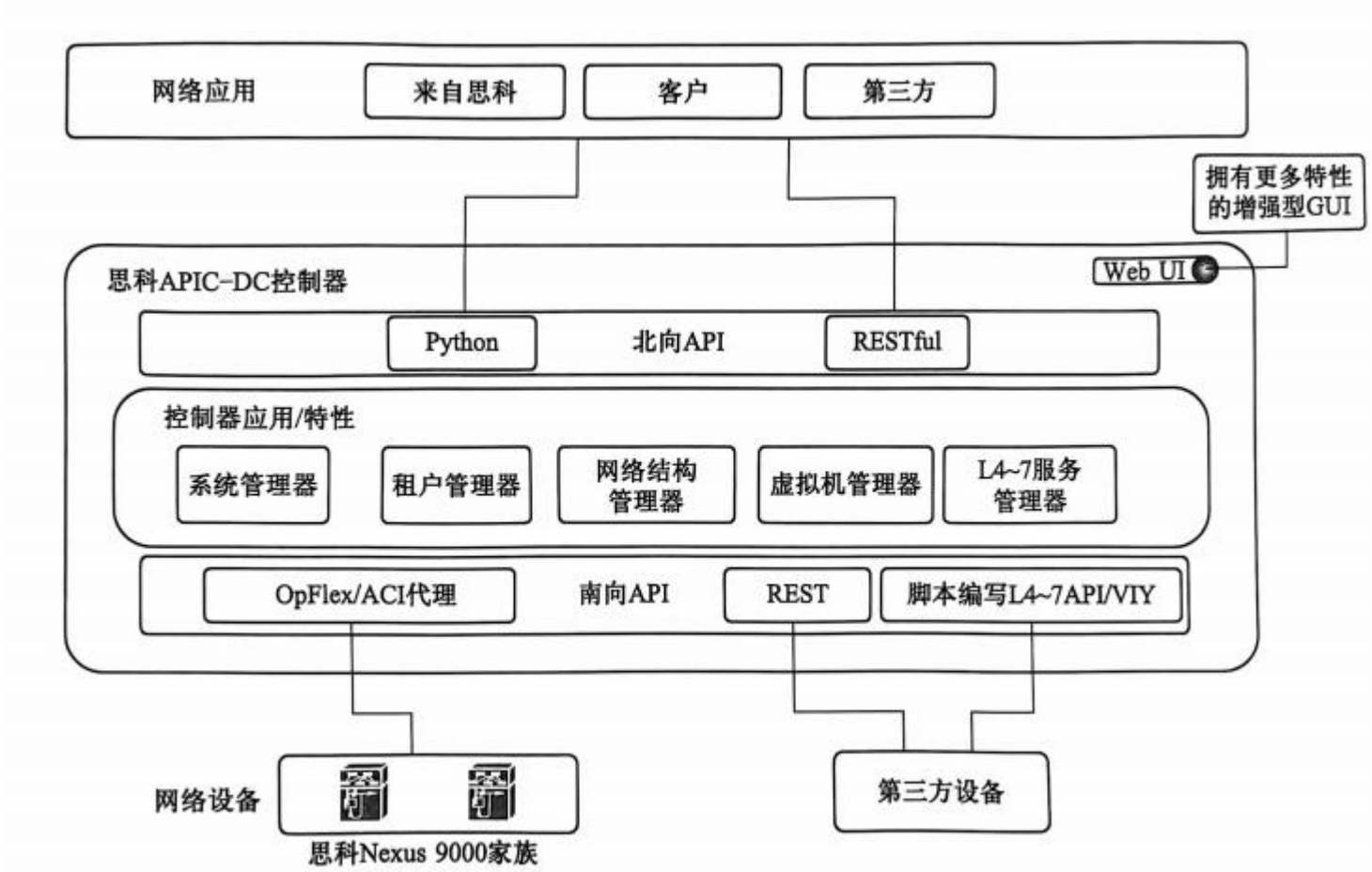


图 5.12　APIC 架构

商业控制器-思科APIC与DNA-C

(4)虚拟机管理器：该模块用于提供与私有云的对接。

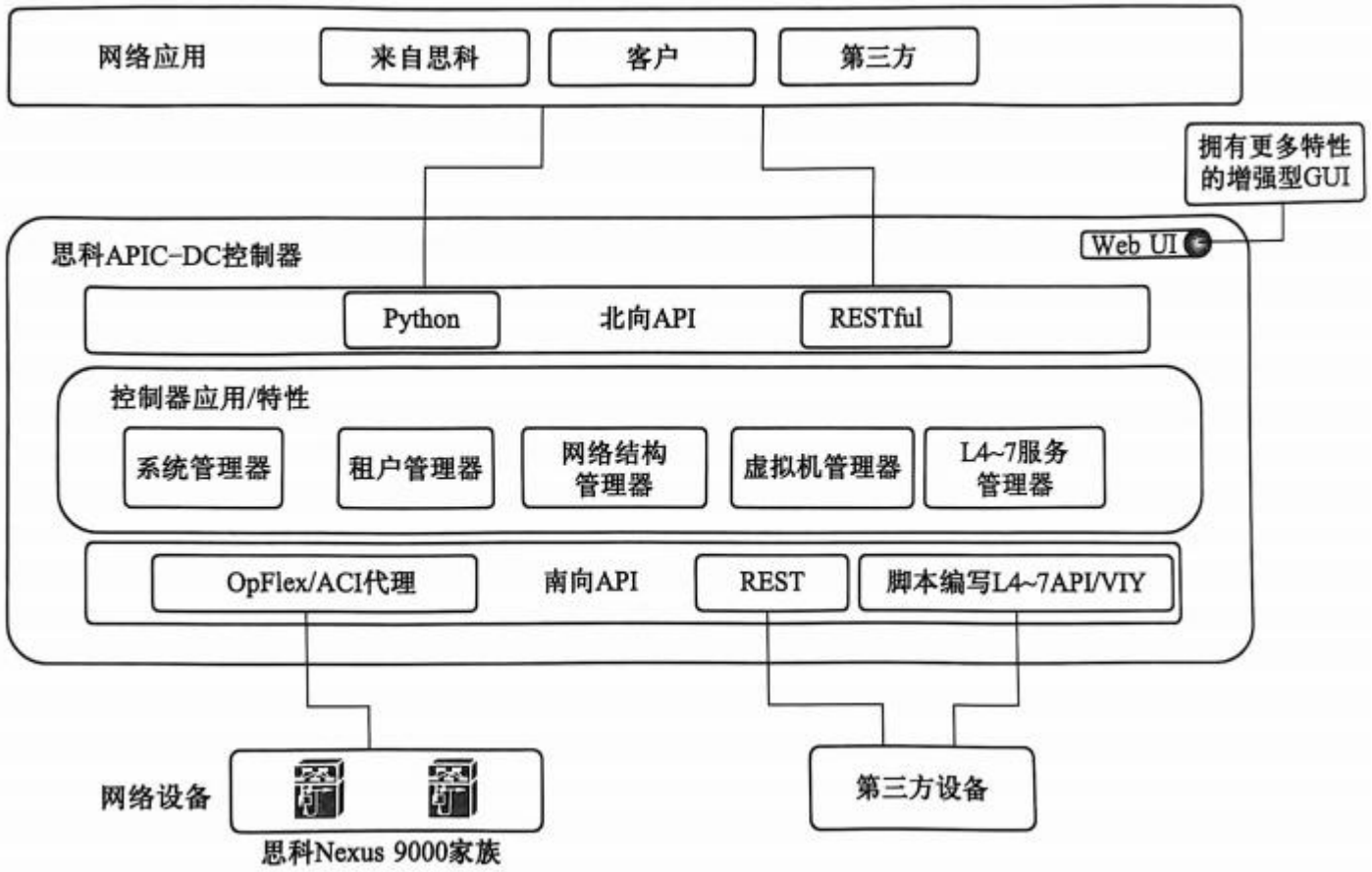


图 5.12　APIC 架构

(5)I4~7服务管理器：该模块是思科ACI体系中以应用为中心的典型体现。

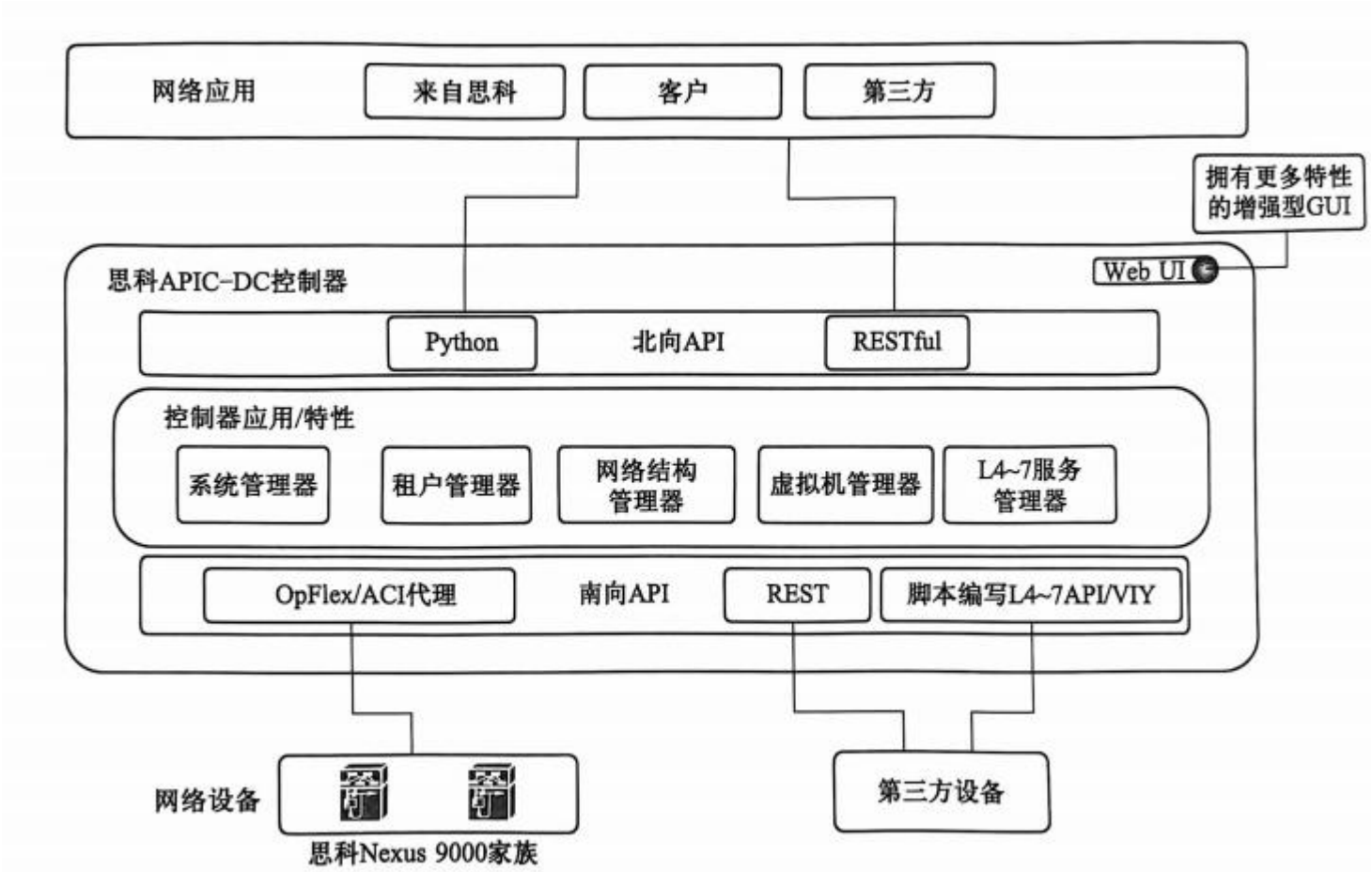


图 5.12　APIC 架构

商业控制器-思科APIC与DNA-C

APIC使用户能够更关注业务与应用本身。在北向接口方面，APIC提供的REST API接口，让网络管理人员能基于自身的意图，更自主地对网络功能进行定义；同时APIC也支持掌握Python编程技能的网络管理人员对其内部的管理信息对象进行操作，以进一步体现SDN的价值。APIC允许用户根据自身的业务在转发层面上，(Contrad)求来分类，将不同应用识别为不同的端点组(EPG)，并利用合同这一概念来替代复杂的传统ACL，实现对端点组的访问控制。端点组是VXLAN报文结构中的安全组识别位，具有端点组对应特性并且能指示报文也称为iVx.端点组策略是在源端还是在目的端上执行，这种新型的VX、LAN。

APIC的南向接口支持两大类协议。一是通过使用OpFlex协议。二是支持使用REST、第4~7层API与VTY脚本编程方式，对特定的生态合作伙伴设备进行练一管理。

商业控制器-思科APIC与DNA-C

DNA-C是思科推出的“数字化网络架构中心(digital network architecturecenter)”的简称，这是一款面向园区网以及多分支企业应用的SDN控制器。

DNA-C之前的名称为APIC-EM，即APIC企业模块(enterprise module)，于2015年发布，目的是将一整套经过数据中心网络验证、基于策略的SDN部署方法与思路在园区网落地。2017年，增加了软件定义接入(SDA)功能，并且通过网络数据平台模块提供了业务保障功能（基于大数据的智能运维），并改名为DNA-C。发展到今天，DNA-C不仅能够支持软件定义的园区网，还能为多分支的企业提供一整套跨广域网的SD-WAN解决方案。同时，它能够和基于APIC的数据中心网络无缝集成，为企业提供整合的全网络自动化及可视化解决方案。

商业控制器-思科APIC与DNA-C

图5.13描述了DNA-C控制器的系统架构，包含了大量的核心功能模块。

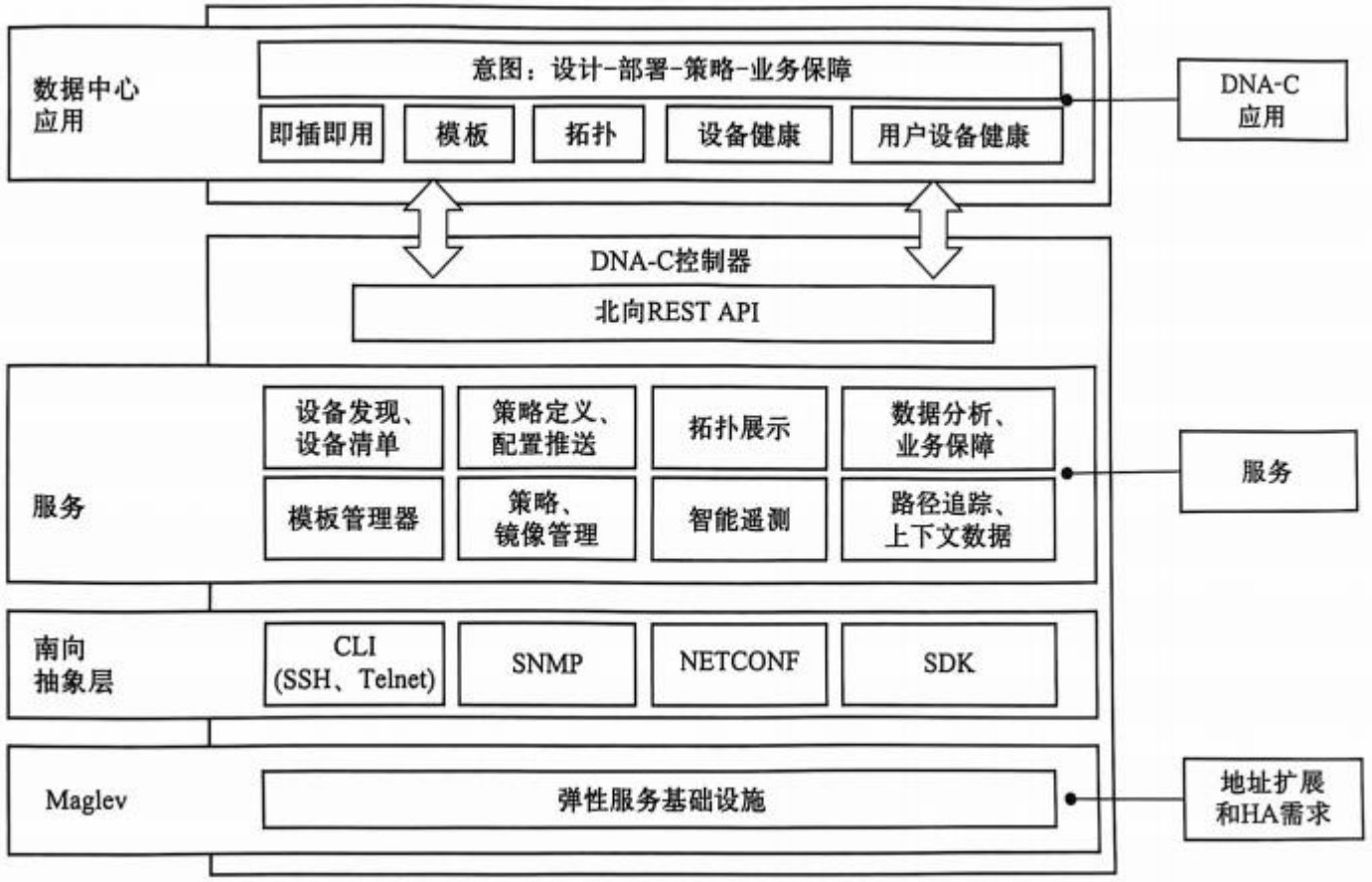


图 5.13　DNA-C 控制器系统架构

(1)设备发现服务：该服务为网络管理人员提供基于IP的设备搜索及基于CDP/LLDP的设备连接扫描能力。

(2)设备清单服务：在发现设备之后，网络管理人员即可通过设备清单列表对所有设备进行统一的管理维护。

(3)规划设计服务：为网络管理人员提供了层次化网络设计、地址规划、定义有线和无线接入等功能。

(4)策略定义服务：以DNA-C为控制器的DNA是策略型的SDN架构，因此策略定义是DNA-C的重点。该服务除了定义转发策略外，还包括组（安全）访问、服务质量、流量复制等在内的多种策略定义。另外，DNA-C可提供基于意图的策略定义。

(5)配置推送服务：网络管理人员基于意图定义的策略，最终通过配置推送服务将所需的配置命令下发到网络设备，这些配置命令既包括Underlay 网络中的全局参数设定

(6)模板管理服务：为网络管理人员定义配置脚本提供模板。(7)镜像管理服务：网络设备的软件版本管理功能，可集中管理设备的升级、打补丁等。

(8)拓扑展示服务：展示网络全貌，便于网络管理人员掌握当前网络的体状态。

VMware公司提出了软件定义数据中心(software defined data center,SDDC)解决方案，NSX-T是该解决方案中的软件定义基础架构，能够以软件定义的方式实现可跨数据中心、云环境和应用框架进行延展的网络连接。

NSX-T再现了全套网络服务，包括交换、路由、防火墙、负载均衡、QoS等。这些服务可以以编程的方式完成用户自定义组装，形成唯一且隔离的虚拟网络环境。

商业控制器-VMware的NSX-T控制器

整个网络被分为三个部分：管理平面、控制平面以及数据平面，如图5.14所示。管理平面提供系统的单个API入口点，永久保留用户配置，处理用户查询，以及执行各节点上的操作任务。NSX管理器提供了云服务管理。

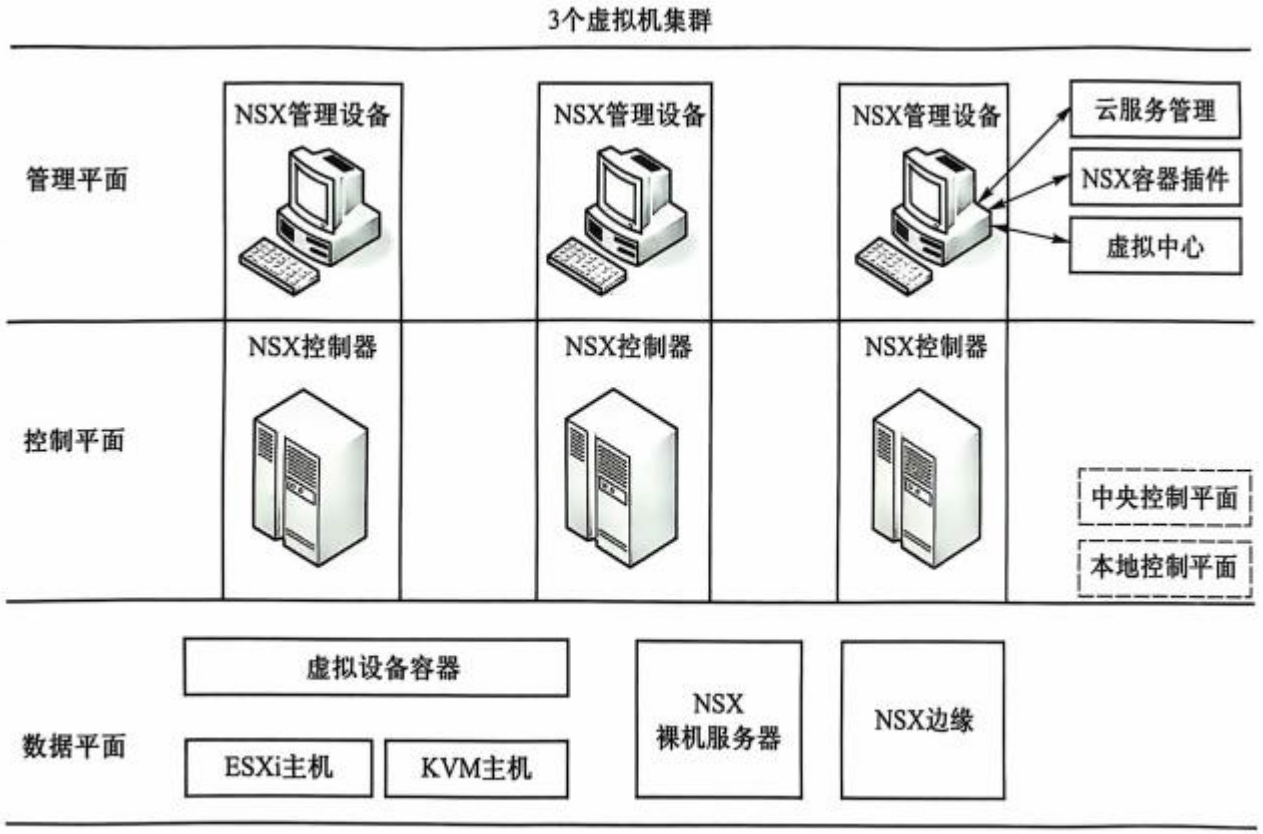


图 5.14　NSX-T 体系结构图

商业控制器-VMware的NSX-T控制器

NSX控制器(NSX controller)部署为一组高可用性的虚拟设备，它们负责在整个NSX-T架构中以编程方式部署虚拟网络。NSX-T CCP在逻辑上与所有数据平面流量隔离，这意味着，控制平面中的任何故障都不会影响现有的数据平面操作。流量不通过控制器传输，控制器负责为其他NSX控制器组件提供配置，如逻辑交换机、逻辑路由器和边缘节点配置等。

NSX控制器包含三个重要的表，分别是MAC地址表、ARP表和VTEP(VXLAN tunnel endpoint，VXLAN隧道端点)表。这些表对每个表的虚拟主机和宿主信息进行整理，并在NSX域中进行复制，这样做的最大好处就是有效降低系统的复杂度；另一方面，这些表可以起到抑制ARP的作用，可减少整个环境中的ARP请求。

NSX控制器包含五个角色，分别为API提供程序、持久性服务器、逻辑管理器、开关管理器、目录服务器。各部分作用如表5.7所示。

表 5.7　NSX 控制器角色及作用

NSX Controller 角色	承 担 作 用
API 提供程序	维护由 NSX Manager 消费的 Web 服务 API
持久性服务器	确保跨节点保存数据，确保数据不可丢失
逻辑管理器	处理策略和网络拓扑的计算

续表

NSX Controller 角色	承 担 作 用
开关管理器	将管理程序并将相关配置推送到主机
目录服务器	集中在 VXLAN 和分布式逻辑路由目录的信息

Part
04

基于控制器的编程

1. POX 的安装与启动

安装 POX 通常有两种方式。一是先安装 Mininet。Mininet 是一个 SDN 虚拟仿真环境，在该环境下可以搭建 SDN 网络，采用 Mininet 的安装选项可以指定选择安装 POX。二是采用 POX 单独安装的方式。POX 的代码可以在 Github 上下载，安装完成后，进入 POX 目录启动控制器：

```
cd pox
./pox.py openflow.of_01 forwarding.l2_learning
```

pox.py 是程序的入口，参数 openflow.of_01 是负责与 OpenFlow v1.0 的交换机通信的组件，参数 forwarding.l2_learning 是二层自学习交换机组件。控制器启动后，默认开启 6633 端口监听。

POX的代码目录结构如图5.15所示。

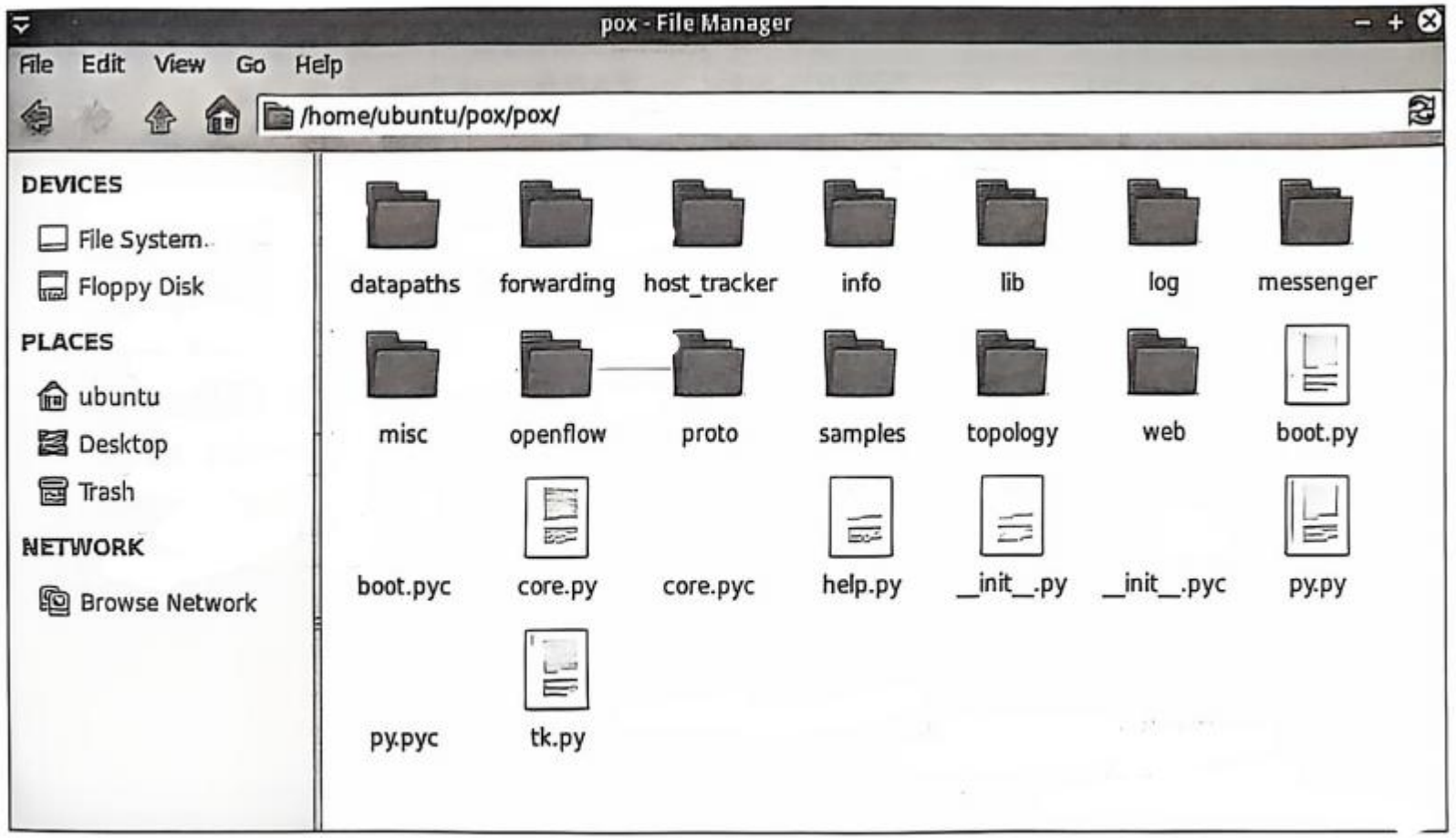


图 5.15　POX 控制器源代码目录结构

2.POX的代码结构

POX是基于组件的控制器，每一个以“.py”作为后缀的文件可以看作是一个组件，每个文件夹可以视为由多个同类或相关组件构成的一个组件包。POX提供了一系列实现核心功能与应用的组件。其中，组件包datapaths提供数据交换功能，可将从接收端口收到的数据包在流表中进行匹配，并执行匹配到的动作；forwarding提供数据转发功能；host_tracker用于追踪网络主机；openflow提供OpenFlow 协议栈的封装代码，可实现OpenFlow消息的封装与解析；messenger负责完成POX与外部进程的交互工作；web用于存放Web服务资源及配置文件；log用于存放日志信息；topology负责管理网络拓扑信息；proto提供网络服务应用，包括支持DHCP、ARP、DNS与ICMP等服务的组件；lib用于存放其他库文件。另外，源代码库中的core.py是POX的内核组件，boot.py是系统启动组件。

3.POX的组件运行机制

在POX控制器中，组件是一个能够独立运行的应用程序。以组件包forwarding为例，包含 Hub、 L2_learning、L3_learning等组件。若在交换机上运行Hub组件，将使OpenFlow交换机的行为如同集线器一样；运行L2_learning组件将使OpenFlow交换机的行为如同一个具有转发地址表自学习功能的二层交换机。除了这些控制器已有的组件外，还可以根据需求，基于POX的源码开发自己的组件。在学习开发组件之前，首先了解POX控制器组件的运行机理。组件在POX控制器上的运行流程如图5.16所示。

组件在POX控制器上的运行流程如图5.16所示。

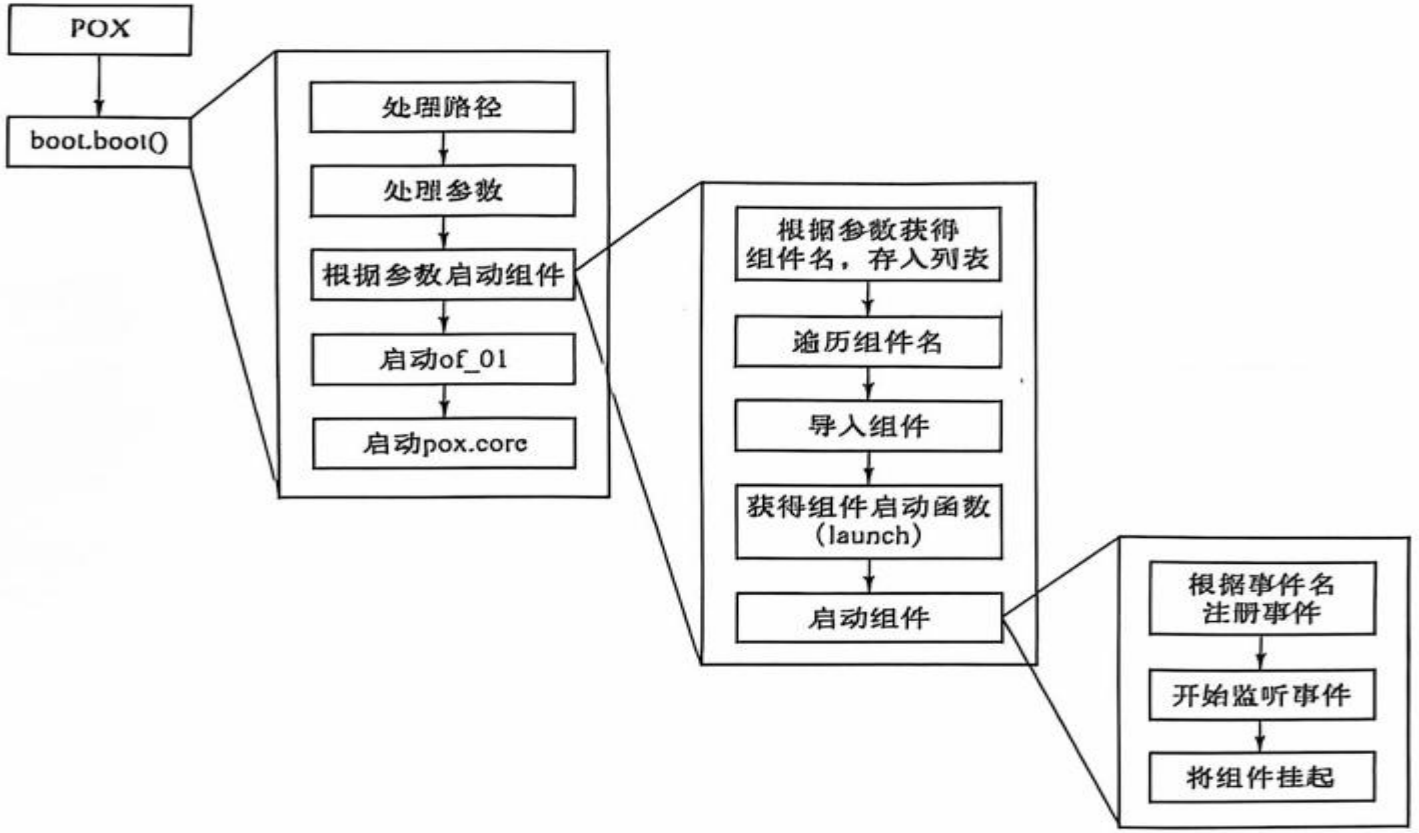


图 5.16　POX 控制器组件启动流程

在图5.16中，控制器的程序入口是pox.py文件，该文件只有3行代码实现对系统启动组件boot.py中的boot()函数的调用。pox.py的具体代码如下：

```
from pox. boot import boot
if__name__ = = '__main__':
boot( )        #启动组件
```

函数 boot( )包含了以下功能，首先获取处理路径，然后进行参数预处理，并根据参数列表启动相应的组件，接着启动 of_01 线程，最后启动内核(pox. core)。函数 boot( )的具体代码如下：

```
def boot (argv = None):
    sys. path. insert(0, os. path. abspath(os. path. join(base, 'pox')))   #获取路径
    sys. path. insert(0, os. path. abspath(os. path. join(base, 'ext')))
    try:
      argv = sys. argv[1:]                                                #预处理参数
      pre = [ ]
    while len(argv):
        if argv[0]. startswith("-"):
            pre. append(argv. pop(0))
    argv = pre + "py --disable". split( ) + argv
    if_do_launch(argv):                    #根据参数启动组件
```

```
        _post_startup()                    #启动 of-01 线程,监听事件
        core. goUp()                       #启动内核
    else:
        #return
        quiet = True
        raise RuntimeError()
```

在上述代码中，通过函数 do_launch(argv)启动组件。具体启动流程包括：根据参数获得组件名，存入列表；然后遍历组件名，导入组件；获得组件启动函数（launch），启动组件。函数 do_launch(argv)的具体代码如下。

基于控制器的编程-基于POX编程

```
def _do_launch (argv):
    #导入组件
    modules = _do_imports(n.split(':')[0] for n in component_order)
    if modules is False:
      return False
    inst = {}
     #遍历组件列表
    for name in component_order:
        cname = name
        inst[name] = inst.get(name, -1) + 1
        params = components[name][inst[name]]
        name = name.split(":", 1)
        launch = name[1]
        if len(name) == 2 else "launch"
        name = name[0]
    name,module,members = modules[name]
```

基于控制器的编程-基于POX编程

```
#预处理组件,是否有启动函数
if launch in members:
        f = members[launch]
    if getattr(f, '_pox_eval_args', False):
        import ast
        for k,v in params.items():
            if isinstance(v, str):
                try:
                    params[k] = ast.literal_eval(v)
        if multi == False and len(components[cname]) != 1:
            print(name, "does not accept multiple instances")
        return False
    try:

        if f(**params) is False:        #启动组件
            # Abort startup
            return False
```

在上述组件的启动过程中，需要调用启动函数，在内核中进行组件注册。以实现三层学习的 L3-learning 组件为例，它的启动函数代码如下。

```
Sdef launch (fakeways="", arp_for_unknowns=None, wide=False):
    #处理参数
    fakeways = fakeways.replace(",", " ").split()
    fakeways = [IPAddr(x) for x in fakeways]
    if arp_for_unknowns is None:
        arp_for_unknowns = len(fakeways) > 0
  else:
        arp_for_unknowns = str_to_bool(arp_for_unknowns)
  #注册组件,调用 core.py 的 core.registerNew 函数
  core.registerNew(l3_switch, fakeways, arp_for_unknowns, wide)
```

在该启动函数中，通过调用内核（core.py）的 registerNew 函数对组件进行注册。

POX 组件在启动后，将一直处于监听事件状态，并对监听到的事件，通过事件处理函数进行相应的处理。事件与事件处理函数的对应是通过名称匹配的机制来实现的。事件处理函数需要按照“_handle_（前缀）+xxxEvent”（事件名）的方式进行命名。例如，若要监听两种类型的事件 FooEvent 和 BarEvent，则事件的处理函数需要按_handle_FooEvent 和_handle_BarEvent 的方式分别进行命名。

4.POX组件开发示例

下面以基于 POX 控制器源码，实现一个具有二层自学习的交换机组件为例，学习 POX 组件的开发。

1）引用所有依赖的源文件包

```
from pox. core import core
import pox. openflow. libopenflow_01 as of
from pox. lib. util import dpid_to_str, str_to_dpid
from pox. lib. util import str_to_bool
import time
```

2）创建日志对象

```
log = core. getLogger( )
_flood_delay = 0
```

3)创建 LearningSwitch 类

LearmingSwitch 类主要完成二层交换机功能，该类维护一个字典 macToPort,用于保存MAC地址和端口之间的映射关系。处理逻辑在_handle_PacketIn方法中，该方法用来处理OpenFlow中的Packet-in报文。该方法中，又定义了两个特殊的动作，即泛洪和丢包。交换机将数据包上报给控制器之后，交换机组件的数据包处理流程如图5.17所示。

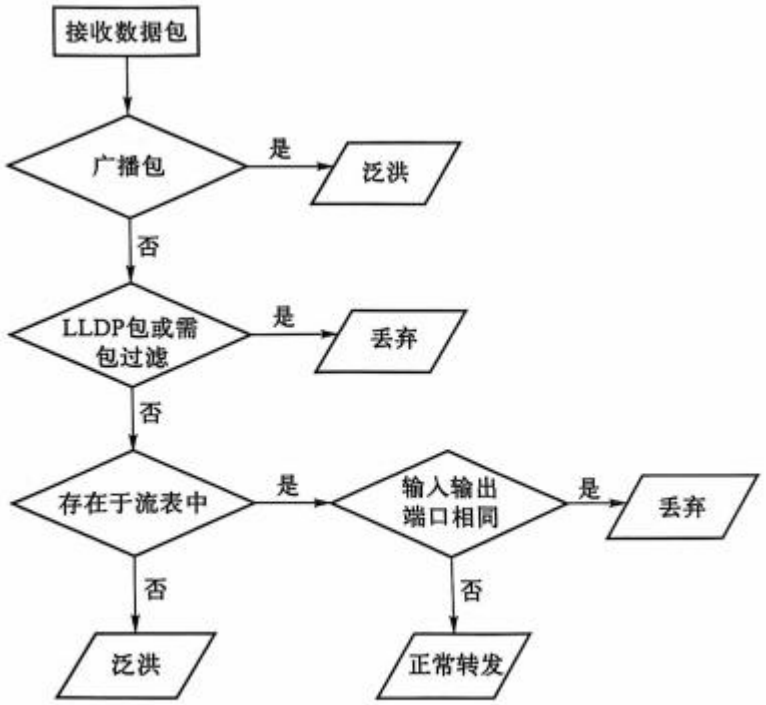


图 5.17　二层自学习交换机组件的数据包处理流程

```
class LearningSwitch (object):
  def__init__ (self, connection, transparent):
    self. connection = connection
    self. transparent = transparent
    self. macToPort = {}              #流表:MAC 与端口的映射关系字典
    connection. addListeners(self)  #监听 connection
    self. hold_down_expired =_flood_delay == 0

  def_handle_PacketIn (self, event):
    packet = event. parsed
    def flood (message = None):
```

```
#数据包为广播包或目的地址不存在于流表中,调用 flood()方法
        msg = of.ofp_packet_out()
        if time.time()-self.connection.connect_time >=_flood_delay:
#若已建立连接一定时间
            if self.hold_down_expired is False:
                self.hold_down_expired = True
                log.info("%s: Flood hold-down expired -- flooding",
                    dpid_to_str(event.dpid))
```

```
            if message is not None: log.debug(message)
            msg.actions.append(of.ofp_action_output(port = of.OFPP_FLOOD))
#将输出端口设置为泛洪

        else:
            pass                                    #不对数据包进行修改
        msg.data = event.ofp
        msg.in_port = event.port
        self.connection.send(msg)
```

```
def drop (duration = None): #丢包处理
  if duration is not None:
#若需要设置空闲时间或生存时间,设置流的参数并做泛洪处理
    if not isinstance(duration, tuple):
      duration = (duration,duration)
    msg = of.ofp_flow_mod()
    msg.match = of.ofp_match.from_packet(packet)
    msg.idle_timeout = duration[0]
    msg.hard_timeout = duration[1]
    msg.buffer_id = event.ofp.buffer_id
    self.connection.send(msg)
  elif event.ofp.buffer_id is not None:
    msg = of.ofp_packet_out()
    msg.buffer_id = event.ofp.buffer_id
    msg.in_port = event.port
    self.connection.send(msg)
```

```
self. macToPort[ packet. src] = event. port#使用源地址和端口更新地址/端口表
```

```
if not self. transparent: # 2
   if packet. type == packet. LLDP_TYPE or packet. dst. isBridgeFiltered( ):
#若为 LLDP 数据包或需要被过滤数据包则丢弃
      drop( ) # 2a
      return
```

基于控制器的编程-基于POX编程

```
if packet. dst. is_multicast:      #若为组播数据包则泛洪
  flood()
else:
  if packet. dst not in self. macToPort:  #数据包目的地址不存在于流表中则泛洪
    flood("Port for %s unknown -- flooding" % (packet. dst,))
  else:
    port = self. macToPort[packet. dst]
    if port == event. port:    #若输入端口与输出端口相同则丢弃
      log. warning("Same port for packet from %s -> %s on %s. %s.    Drop."
          % (packet. src, packet. dst, dpid_to_str(event. dpid), port))
      drop(10)
      return
    log. debug("installing flow for %s. %i -> %s. %i" %
          (packet. src, event. port, packet. dst, port))
    msg = of. ofp_flow_mod()      #更新流表时间戳
    msg. match = of. ofp_match. from_packet(packet, event. port)
    msg. idle_timeout = 10
    msg. hard_timeout = 30
    msg. actions. append(of. ofp_action_output(port = port))
    msg. data = event. ofp
    self. connection. send(msg)
```

4）创建 L2_learning 类

类 L2_learning 的主要作用是监听 OpenFlow 事件，当监听到有交换机向控制器发送连接请求时，触发_handle_ConnectionUp 方法，初始化一个 LearningSwitch 类，并执行真正的数据处理逻辑。

```
class L2_learning (object):
  def__init__ (self, transparent, ignore = None):
    core. openflow. addListeners(self)
    self. transparent = transparent
    self. ignore = set(ignore) if ignore else ()
#当监听到有交换机向控制器发送连接请求时触发
  def_handle_ConnectionUp (self, event):
```

```
if event. dpid in self. ignore:
    log. debug("Ignoring connection %s" % (event. connection,))
    return
log. debug("Connection %s" % (event. connection,))
LearningSwitch(event. connection, self. transparent)
```

5）创建启动二层交换机函数

函数 launch 是程序的人口，该函数实现组件 L2_learning 在内核中的注册。

```
def launch (transparent=False, hold_down=_flood_delay, ignore = None):
  try:
    global_flood_delay
    _flood_delay = int(str(hold_down), 10)
    assert_flood_delay >= 0
  except:
    raise RuntimeError("Expected hold-down to be a number")

  if ignore:
    ignore = ignore.replace(',', ' ').split()
    ignore = set(str_to_dpid(dpid) for dpid in ignore)
core.registerNew(l2_learning, str_to_bool(transparent), ignore)
```

1.Ryu的安装与启动

Ryu 的安装方式有多种，下面介绍通过 pip 安装和通过源码安装两种方式。

1）通过 pip 安装

检查终端上是否存在 pip，终端为 Ubuntu 时使用命令：

```
sudo pip3 --version
```

终端为 Windows 时使用命令：

```
pip -V
```

如已安装好 pip，只需要在终端上输入安装 Ryu 命令：

```
pip install ryu
```

1.Ryu的安装与启动

2）通过源码安装

（1）安装前的准备。首先，需要安装 git、pip 等常用软件，并将其更新到最新版本。如果所需的软件已是最新版本，可以跳过此步骤。

安装 git、pip 命令如下：

```
sudo apt-get update
sudo apt-get install git
sudo apt-get install python-pip
sudo pip install --upgrade pip six
```

其次，需要安装相关依赖，包括 python-eventlet、python-routes、python-webob 和 python-paramiko 等依赖包，并确保依赖包的版本满足要求。可通过 pip install 命令安装 tools/pip-requires 里面的依赖包，也可以使用命令逐条安装。前者的具体命令格式如下：

```
sudo apt-get install python-eventlet python-routes python-webob python-paramiko
```

1.Ryu的安装与启动

Ryu 安装完成后，可以通过 ryu-manager 启动应用（组件）。启动时需要读取输入的预启动应用及其依赖的应用模块，并将这些应用模块放在启动队列里，然后按顺序启动。如果启动时没有指定任何应用，将会默认启动 ofp_handler，它是处理 OpenFlow 的模块，主要负责 OpenFlow 事件的处理。当启动 Ryu 时，后台也同时启动了 AppManager 实例，该实例负责应用的注册、事件的注册和监听等内容，是 Ryu 的信息管理中心模块。启动 Ryu 的一般命令格式如下：

```
ryu-manager [app-lists] [arguments]
```

其中，app-lists 是启动的模块列表，arguments 是可选参数，其形式可为

- --verbose，打印详细信息；
- --version，打印程序版本号；
- --observe-links，自动下发 LLDP，用于拓扑发现。

例如，当启动 Ryu 的时候，要求同步启动 Ryu 内建的简单交换机组件 simple_switch和提供 REST API 的组件 ofctl_rest，同时，又希望 Ryu 控制器能够自动下发 LLDP，以进行拓扑发现，则具体的命令格式如下：

```
ryu-manager simple_switch. py ofctl_rest. py-observe-links
```

2.Ryu的代码结构

Ryu的代码目录结构如图5.18所示。

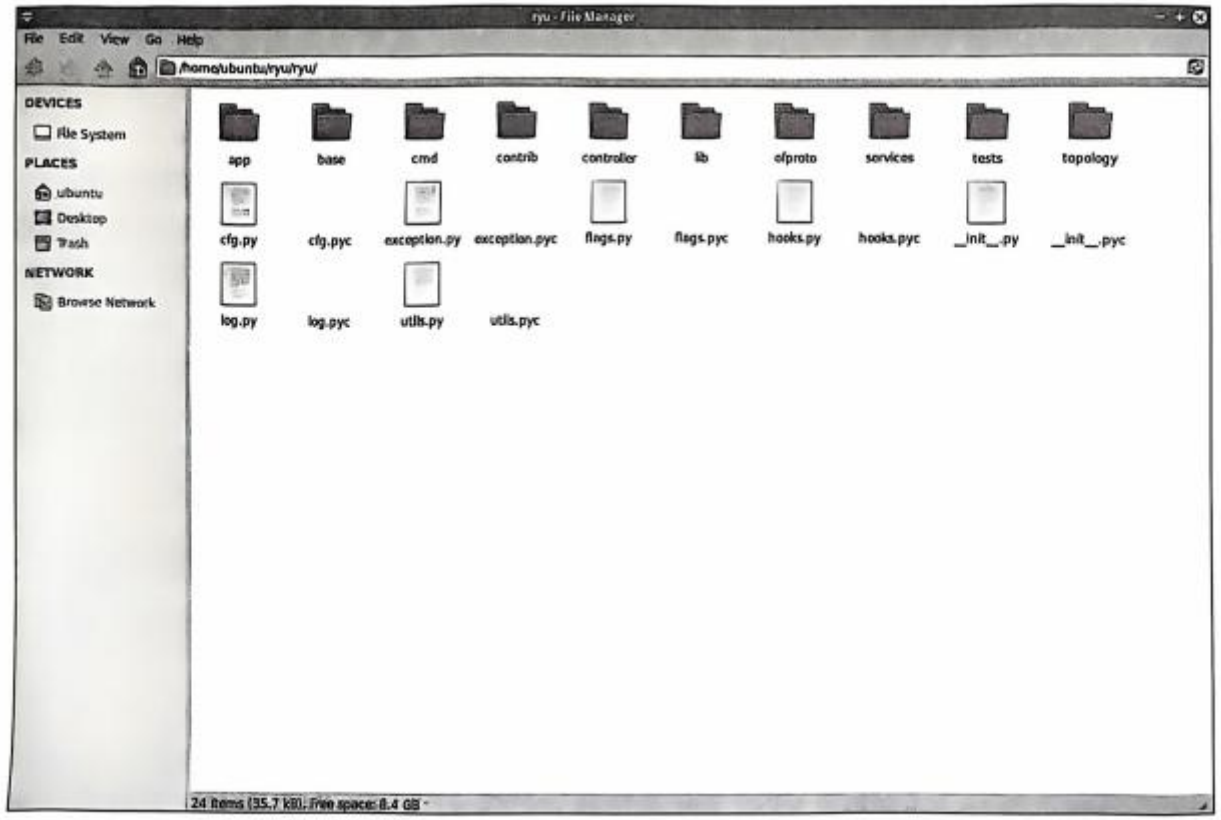


图 5.18　Ryu 控制器源代码目录结构

2.Ryu的代码结构

其中，目录app用于存放Ryu的内建应用以及用户开发的应用代码，内建应用包括simple_switch、rest_router、rest_firewall、rest_qos等组件；base用于存放应用程序管理中心的有关代码，其中包含 Ryu 组件的基类RyuAPP的定义，RyuAPP定义了RyuAPP的基本属性，如name、threads、events等成员，还定义了AppManager基类，用于管理APP；cmd存放了Ryu框架入口程序，它会调用Ryu核心组件的初始化程序；contrib存放了第三方贡献的代码；controller存放了事件系统的代码，用于实现控制器与交换机之间的关联与事件处理；ofproto包含负责OpenFlow 协议解析的代码；lib用于存放网络报文库的代码，包括常见的ICMP、DHCP、IGMP等协议报文；services包含了一些基于Ryu框架开发的协议，如VRRP、BGP、OVSDB等；tests存放了测试代码，包括单元测试以及整合测试的代码；topology下是有关拓扑的组件，如switches.py定义了交换机的数据结构，event.py定义了OpenFlow中的交换事件，dumper.py定义了网络拓扑发现事件，最后通过pi.py向上提供了一套调用topology目录中定义函数的接口。

23.Ryu的组件运行机制

与POX控制器类似，Ryu也是基于组件的框架。所有Ryu组件的运行由管理中心ryu-manager进行管理，图5.19给出Ryu组件的运行框架与简单流程。当通过ryu-manager启动一个应用时，会自动生成并运行一个对应的事件循环线程(event loop thread)。该线程循环运行，捕捉来自事件队列(event queue)的触发事件，当事件队列不为空时，事件循环线程依次从队列中取出事件，并且调用相应的事件处理器(event handler)或事件处理函数进行处理。事件队列由事件循环线程维护，其中的事件有两个来源，一是来自数据平面（交换机）产生的各种处理与交换请求；二是事件处理器对事件进行处理之后，可能会产生新的事件，例如对Packe-in事件的处理，往往会触发相应的Packet-out事件。

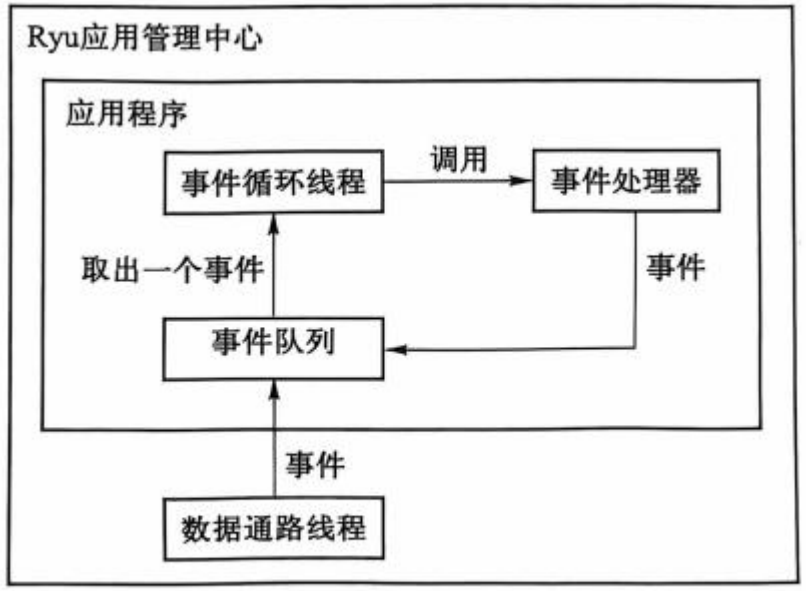


图 5. 19　Ryu 控制器的组件运行框架

3.Ryu的组件运行机制

图5.19不仅明确了Ryu控制器的组件运行机理，同时也提供了Ryu组件开发模型，即基于事件处理的编程模型。对于不同的事件或功能实现，组件开发的关键在于事件处理器。需要根据具体的需求，分析并根据事件处理或功能实现的逻辑，尽可能利用Ryu所提供的源代码资源，进行相应组件的编程开发。

但是，即使有了不同事件或功能对应的事件处理器或处理函数，对于不同类型的事件，事件循环线程如何实现事件处理器以及处理函数的准确匹配或调用是非常重要的。这个问题属于Ryu的事件管理范畴，并借助通过Python语言定义的修饰器实现。对应的修饰器定义如下：

ryu. controller. handler. set_ev_cls(ev_cls, dispatchers=None)

其中，参数ev_cls用于指定事件类型所调用的事件处理函数，也就是说，当该指定类型的事件触发后，相应的事件处理函数被调用。参数dispatchers 有HANDSHAKE_DISPATCHER、CONFIG_DAPATCHER、MAIN_DISPATCHER 和DEAD_DISPATCHER四个取值，分别对应交换机与控制器通信过程中的四个不同的阶段或状态：交换Hello消息，在控制器与交换机建立安全连接后，双方发送 Hello消息进行协议版本的协商；接收Features 消息，控制器发送Features-re-quest消息获取交换机的参数，交换机回复Features-reply消息将交换机的详细参数告知控制器，控制器发送Set-config消息，向交换机下发配置参数；一般状态，控制器与交换机之间进入Packet-in、Packet-out交互；连接中断状态，当通信完成或发生错误时，中断连接。

下面是定义 ev_cls 的一个实例：

```
@ set_ev_cls( ofp_event. EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler( self, ev) :
    ......
```

4.Ryu组件开发实例

1)二层自学习交换机组件开发

该示例的开发目标是实现一个二层自学习交换机组件，可提供收到数据包后向控制器发送Packet-in消息、学习MAC地址表、根据Packet-out报文指示交换机将数据包泛洪等基本功能。

下面结合二层自学习交换机组件实现代码的分析，帮助读者梳理组件开发的要点。

(1)对组件所要引用的所有依赖包进行声明。其中，app_manager是Ryu应用的管理中心，用于加载Ryu应用程序，接收从APP发送过来的信息，同时也完成消息的路由。ofp_event 实现从 OpenFlow 报文生成事件，CONFIG_DIS-PATCHER、MAIN_DISPATCHER用于描述事件发生时的交换机状态。set_ev_cls是修饰器，packet、ethernet、ether_types是网络报文库中的类，用于报文解析。具体代码如下：

```
from ryu. base import app_manager
from ryu. controller import ofp_event
from ryu. controller. handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu. controller. handler import set_ev_cls
from ryu. ofproto import ofproto_v1_3
from ryu. lib. packet import packet
from ryu. lib. packet import ethernet
from ryu. lib. packet import ether_types
```

4.Ryu组件开发实例

(2)定义或创建一个交换机类。定义或创建该类的目的是为了对所要实现的二层自学习交换机进行功能抽象与封装。该类继承自RyuApp父类。ByuApp是Ryu应用的抽象功能描述类，用于完成Ryu应用所需要完成的基本操作，从而使开发者能专注于业务逻辑的实现。该类的代码实现分为以下几步。

第一步：创建SimpleSwitch13类。指定Openflow的版本为1.3，SimpleSwitch13类的mac_to_port用于保存MAC地址和交换机端口的对应关系。具体代码如下：

```
class SimpleSwitch13(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
    def__init__(self, *args, **kwargs):
        super(SimpleSwitch13, self).__init__(*args, **kwargs)
        self.mac_to_port = {}      #用于保存目的 MAC 地址和端口号的映射
```

4.Ryu组件开发实例

第二步：定义修饰器，处理ofp_event.EventOFPSwtichFeatures类型事件。当数据通路与控制器建立连接之后，通过Features消息回复数据通路的特性。switch_features_handler方法通过@set_ev_cls装饰器注册 EventOFPSwtichFeatures事件。当EventOFPSwtichFeatures 事件在 CONFIG_DISPATCHER阶段生成时，就会被分发到switch_features_handler方法，从而执行处理方法，完成事件处理。具体代码如下：

```
@ set_ev_cls( ofp_event. EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler( self, ev) :
    datapath = ev. msg. datapath   #datapath 存储交换机基本信息
    ofproto = datapath. ofproto
    parser = datapath. ofproto_parser
    match = parser. OFPMatch( )
    actions = [ parser. OFPActionOutput( ofproto. OFPP_CONTROLLER,
        ofproto. OFPCML_NO_BUFFER) ]
#没有匹配任何其他流表时,发送请求给控制器
#下发一条 table-miss 流表项至 datapath
#用于指导交换机将匹配流表项失败的数据包以 Packet-in 的方式发送至控制器
    self. add_flow( datapath, 0, match, actions)
```

4.Ryu组件开发实例

```
#定义添加流表项函数
def add_flow(self, datapath, priority, match, actions, buffer_id=None):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]
    if buffer_id:
        mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                priority=priority, match=match,
                                instructions=inst)
    else:
        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)
    datapath.send_msg(mod)
```

4.Ryu组件开发实例

第三步：定义修饰器，处理ofp_event. EventOFPPacketIn 事件。packet_in_handler方法注册EventOFPPacketIn 事件，当 EventOFPPacketIn 事件在MAIN_DISPATCHER 状态下产生时，就会被分发给_packet_in_handler方法去处理。首先对数据包进行必要的解析，然后以交换机dpid的输入端口号为键值，将源MAC地址记录到mac_to_port 字典中：mac_to_port[dpid][sre]= in_port。接下来查看其目的MAC地址是否在mac_to_port字典数据项的值中，若目的MAC地址不在表中，则将输出端口设置为ofproto.OFPP_FLOOD，并通过Packet-out报文指示交换机将数据包泛洪；若查询成功，则将输出端口设置为查询所得端口；最后调用add_flow方法将流表项下发到datapath，从而完成二层交换应用。具体代码如下：

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    #如果传输出错,丢弃
    if ev.msg.msg_len < ev.msg.total_len:
        self.logger.debug("packet truncated: only %s of %s bytes",
                          ev.msg.msg_len, ev.msg.total_len)
    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
```

4.Ryu组件开发实例

```
pkt = packet.Packet(msg.data)
eth = pkt.get_protocols(ethernet.ethernet)[0]
#如果收到 LLDP 包,丢弃
if eth.ethertype == ether_types.ETH_TYPE_LLDP:
    # ignore lldp packet
    return
dst = eth.dst
src = eth.src

#获取 datapath.id,初始化 Mac 地址表
dpid = datapath.id
self.mac_to_port.setdefault(dpid, {})
```

```
self. logger. info("packet in %s %s %s %s", dpid, src, dst, in_port)
#学习 MAC 地址
self. mac_to_port[dpid][src] = in_port
#判断目的 MAC 地址是否已经学习到
if dst in self. mac_to_port[dpid]:
    out_port = self. mac_to_port[dpid][dst]
else:
    out_port = ofproto. OFPP_FLOOD#目的 MAC 地址未学习到#
actions = [parser. OFPActionOutput(out_port)]
#编写一个 flow mod 信息
if out_port ! = ofproto. OFPP_FLOOD:
    match = parser. OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
    if msg. buffer_id ! = ofproto. OFP_NO_BUFFER:
        self. add_flow(datapath, 1, match, actions, msg. buffer_id)
        return
    else:
        self. add_flow(datapath, 1, match, actions)
data = None
if msg. buffer_id == ofproto. OFP_NO_BUFFER:
    data = msg. data
out = parser. OFPPacketOut(datapath=datapath, buffer_id=msg. buffer_id,
                           in_port=in_port, actions=actions, data=data)
#发送 Packet-out 信息
datapath. send_msg(out)
```

上述的简单二层交换应用来自官方源代码，代码位于ryu/app/simple_switch_13.py。如果需要重新实现一个Ryu应用，在完成Python编写后需要保存到APP目录下。由于新加了一个应用，Ryu编译文件中并没有，所以还需对Ryu重新编安装。重新安装之后，通过ryu-manager命令启动Ryu控制器并执行新写的应即可。具体命令如下：

```
cd ryu
sudo python setup. py install
sudo ryu-manager ryu/app/test. py  （以 test. py 为例）
```

组件开发完成后，可通过Mininet连接Ryu控制器，并搭建测试拓扑完成对test.py组件的测试验证。

2）流量监控组件开发

(1)需求描述。

在如图5.20所示的网络拓扑中，要求在控制器c0上实现一个简单的网络流量监控组件，能够定期向交换机s1下发请求，获取交换机的流表项状态信息、端口状态信息；同时，当交换机状态发生变化时，控制器能做相应处理，从而使管理人员可以在网络发生异常时，快速找到原因并排除故障。

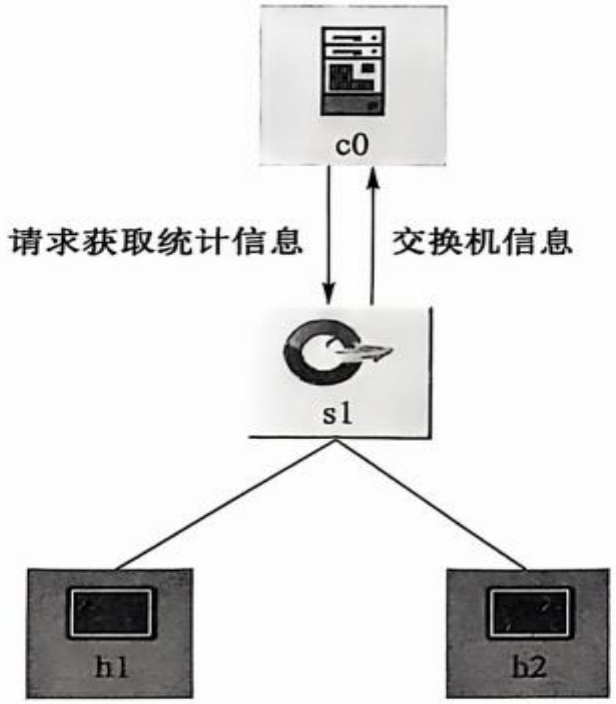


图 5.20　流量监控组件验证实验拓扑

(2)实现分析。

无论是开发什么组件，Ryu代码的整体实现框架都是基本一致的，因此，在熟悉Ryu的代码库、理解Ryu基于组件的运行机理之外，还需要理解组件的实现逻辑，最重要的是分析该组件运行中涉及哪些事件(OpenFlow消息)，根据这些事件，就可以编写事件处理器（函数）。本例中涉及最主要的三个事件包括：交换机流表状态请求回应事件(EventOFPFlowStatsReply)、交换机端口状态请求19A 回应事件(EventOFPPortStatsReply)、交换机状态变化事件(EventOFPState-Change），因此，需要编写相应的事件处理器为_flow_stats_reply_handler、_port_stats_reply_handler和_state_change_handler。同时，为了实现对交换机流量的实时监控，在实现上需要在控制器上设置一个专门的监控线程monitor，以便定时向交换机（线程）发出状态请求（Request_stats）。此外，交换机与控制器间需要定期发送Hello、Echo包等实现状态的建立与维系。组件实现原理如图5.21所示。

组件实现原理如图5.21所示。

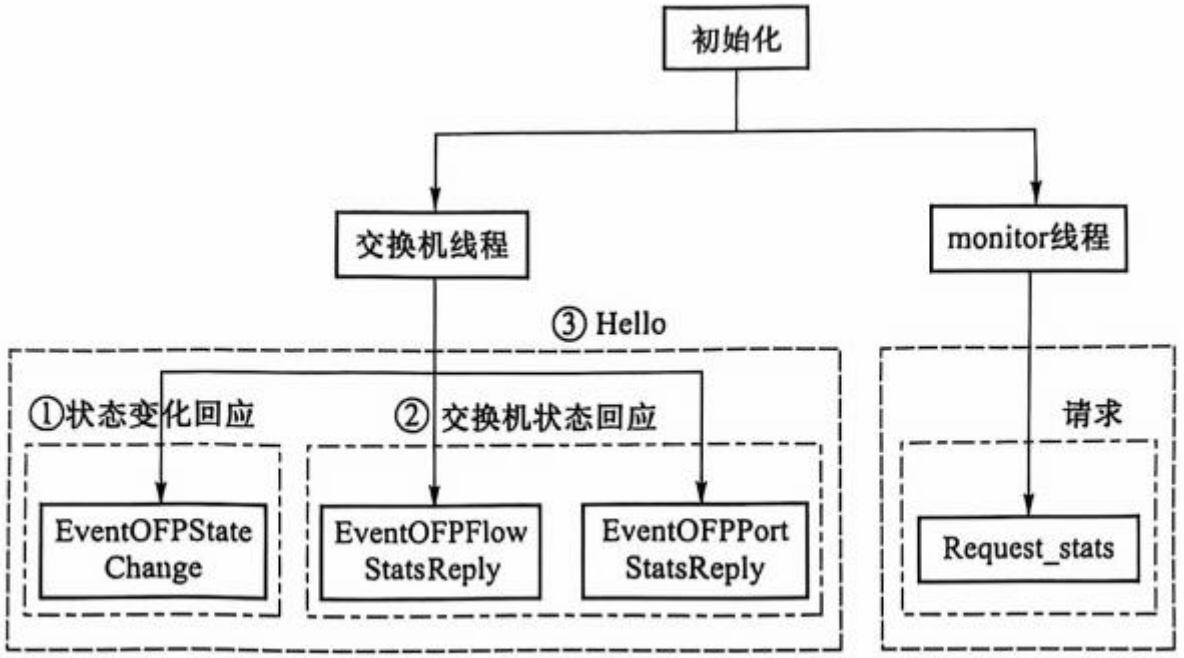


图 5.21　流量监控组件的实现原理

1.对组件所要引用的所有依赖文件包进行声明。attrgetter模块作用于attrgetter处理对象，提高代码运行效率。simple_switch_13是简单交换机组件。hub继承于Event、StreamServer和WSGIServer等类，还有一些重要的重要函数如spawn()。具体代码如下：

```
from operator import attrgetter
from ryu. app import simple_switch_13
from ryu. controller import ofp_event
from ryu. controller. handler import MAIN_DISPATCHER, DEAD_DISPATCHER
from ryu. controller. handler import set_ev_cls
from ryu. lib import hub
```

定义并初始化SimpleMonitor13类，该类继承自SimpleSwitch13。在该类中初始化了datapaths列表，用于存储交换机id，并启动一个新线程hub.spawn()，初始化self_monitor。具体代码如下：

```
class SimpleMonitor13(simple_switch_13.SimpleSwitch13):
    def__init__(self, *args, **kwargs):
        super(SimpleMonitor13, self).__init__(*args, **kwargs)
        self.datapaths = {}                    #增添 datapaths 列表,存储交换机 id
        self.monitor_thread = hub.spawn(self_monitor)    #函数启动一个新线程
```

2.编写事件处理器_state_change_handler，实现对ofp_event.EventOFPState-Change事件的处理。通过 ev.state取值MAIN_DISPATCHER或DEAD_DISPATCHER判断交换机的状态。当ev.state 取值为MAIN_DISPATCHER 且 datapath.id not inself.datapaths时，表示有新加入的交换机；当ev.state取值为DEAD_DISPATCHER且datapath.id in self.datapaths时，表示有掉线的交换机，即需要注册解除的交换机。对于有交换机状态变化的情况，需要对datapaths列表进行更新。具体代码如下：

```
@set_ev_cls(ofp_event.EventOFPStateChange,
                [MAIN_DISPATCHER, DEAD_DISPATCHER])
def_state_change_handler(self, ev):
    datapath = ev.datapath
```

```
#判断机器状态,是否新加入的交换机(注册、被监控)
if ev.state == MAIN_DISPATCHER:
    if datapath.id not in self.datapaths:
        self.logger.debug('register datapath: %016x', datapath.id)
        self.datapaths[datapath.id] = datapath
#是否是掉线的交换机(注册解除)
elif ev.state == DEAD_DISPATCHER:
    if datapath.id in self.datapaths:
        self.logger.debug('unregister datapath: %016x', datapath.id)
        del self.datapaths[datapath.id]
```

3.编写monitor函数。用hub.sleep(10)设置控制器每隔10s向datapaths列表内所有的交换机下发状态信息请求。用_request_stats()方法下发请求，请求内容分别为交换机的流表项状态信息(OFPFlowStatsRequest)与端口状态信息(OFPPortStatsRequest)，具体代码如下：

```
def_monitor(self):
        while True:
            for dp in self.datapaths.values():
                self._request_stats(dp)
            hub.sleep(10)      #每隔10s发一次请求
    #request_stats()方法请求交换机状态
    def_request_stats(self, datapath):
        self.logger.debug('send stats request: %016x', datapath.id)
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        #请求流表项状态
        req = parser.OFPFlowStatsRequest(datapath)
        datapath.send_msg(req)
        #请求端口状态信息
        req = parser.OFPPortStatsRequest(datapath, 0, ofproto.OFPP_ANY)
        datapath.send_msg(req)
```

4.编写事件处理器_flow_stats_reply_handle，实现对 ofp_event.EventOFPFlow-StatsReply事件的处理。该事件监听交换机回复的流表项状态信息，并对信息进行打印输出。具体代码如下：

```
@set_ev_cls(ofp_event.EventOFPFlowStatsReply, MAIN_DISPATCHER)
def_flow_stats_reply_handler(self, ev):
    body = ev.msg.body
    self.logger.info('datapath          '
                     'in-port  eth-dst          '
                     'out-port packets  bytes')
    self.logger.info('----------------'
                     '-------- ----------------'
                     '-------- -------- --------')
    #打印返回 Flow 状态信息
    for stat in sorted([flow for flow in body if flow.priority == 1],
                       key=lambda flow:(flow.match['in_port'],
                                        flow.match['eth_dst'])):
        self.logger.info('%016x %8x %17s %8x %8d %8d',
                         ev.msg.datapath.id,
                         stat.match['in_port'], stat.match['eth_dst'],
                         stat.instructions[0].actions[0].port,
                         stat.packet_count, stat.byte_count)
```

5.编写事件处理器_port_stats_reply_handler，实现对 ofp_event.EventOFPPort-StatsReply事件的处理。该事件监听交换机回复的端口状态信息，并对信息进行打印输出。具体代码如下：

```
@set_ev_cls(ofp_event.EventOFPPortStatsReply, MAIN_DISPATCHER)
    def_port_stats_reply_handler(self, ev):
        body = ev.msg.body

        self.logger.info('datapath             port     '
                         'rx-pkts   rx-bytes rx-error'
                         'tx-pkts   tx-bytes tx-error')
        self.logger.info('---------------- -------- '
                         '-------- -------- -------- '
                         '-------- -------- --------')
    #打印返回 Port 状态信息
    for stat in sorted(body, key=attrgetter('port_no')):
        self.logger.info('%016x %8x %8d %8d %8d %8d %8d %8d',
                         ev.msg.datapath.id, stat.port_no,
                         stat.rx_packets, stat.rx_bytes, stat.rx_errors,
                         stat.tx_packets, stat.tx_bytes, stat.tx_errors)
```

以上是整个组件的代码实现，运行组件可以通过在命令行输入：ryu-manager ryu. app. simple_monitor_13，组件成功运行后显示以下信息，交换机流表项状态如图 5. 22 所示，端口状态如图 5. 23 所示。

datapath	in-port	eth-dst	out-port	packets	bytes
0000000000000001	1	f2:7d:ed:00:e7:13	2	4	280
0000000000000001	2	46:5d:4a:27:46:17	1	3	238

图 5. 22　交换机流表项状态图

datapath	port	rx-pkts	rx-bytes	rx-error	tx-pkts	tx-bytes	tx-error
0000000000000001	1	12	928	0	19	1486	0
0000000000000001	2	12	928	0	19	1486	0
0000000000000001	fffffffe	0	0	0	0	0	0

图 5. 23　交换机端口状态图

明德新民　　止于至善

明德新民 | 止于至善

第六章 SDN北向接口

学院：软件学院　　任课教师：申凌峰

河南大學

目录

CONTENTS

- 01 北向接口概述
- 02 REST API
- 03 RESTCONF协议

Part
01

北向接口概述

北向接口功能与类型

SDN北向接口的功能主要体现在两方面，一是为上层业务应用提供网络资源与基础功能抽象，二是为上层业务应用和网络管控提供可编程的开发接口。借助北向接口所提供的这两方面功能，可通过网络抽象和虚拟化实现基本网络功能，如路径计算、环路检测、安全等；可以开发出满足各种业务需求的应用软件，如防火墙、负载均衡器、流量工程等网络应用软件；可以实现网络资源与业务的编排；对于网络管理，可实现基于全网拓扑感知、状态与流量监控的网络管理与控制，能够敏捷响应高层应用与业务的网络资源统一调度以及向其他云计算编排系统（如OpenStack Quantum、VMware vCloud Director等）提供网络管理的功能。

北向接口功能与类型

北向接口作为SDN中最重要的编程接口，丰富了SDN的应用。但由于上层业务应用的多样性，北向接口的设计需要充分考虑各种需求之间的差异，同时还需要满足合理性和开放性要求，从而给北向接口的标准化带来了很大挑战。目前，尚未形成业界公认的北向接口标准，不同的SDN控制器采用了不同的北向接口规范。

尽管SDN北向接口未形成统一的标准，但从北向接口的功能立足点来看，主要有两种类型的接口：功能型北向接口(functional NBI)和基于意图的北向接口(intent-based NBI)。

北向接口功能与类型

1. 功能型北向接口

此类接口基于自下而上的网络视角和方法，重点在网络资源抽象及控制能力的开放，关注的是“网络能提供什么”，可将底层网络的类型与特性抽象为不同类型的网络资源或逻辑功能，如拓扑、L2VPN、L3VPN、隧道等。

功能型北向接口主要面向具体的网络功能，与具体的网络实现技术方案相关。它主要对网络功能模型与网络管理模型进行抽象，网络功能模型主要在控制器上实现，侧重于业务下发与流量统计等功能；网络管理模型主要在运行支撑系统(operational support system，OSS)上实现，侧重于告警、性能监测和日志统计等功能。

2. 基于意图的北向接口

此类接口基于自上而下的网络视角和方法，重点在满足上层应用与业务实现逻辑和服务质量的需求，关注的是“应用需要什么”，通过对用户或业务需求的意图分析与洞察，定义与抽象其所需要的连接服务、资源需求、访问控制、流处理和策略逻辑等内容。基于意图的北向接口提出了一种与网络实现技术无关的北向接口，这是一种新的设计思想，它自顶向下地从需求视角对网络对象与能力进行抽象，并通过声明式的表达体现使用者的意图，也就是关注应用想做什么，而不关注如何去做。

北向接口功能与类型

基于意图的北向接口有利于实现以应用为中心、意图驱动的智能网络，目前已经引起产业界的广泛关注，并成为北向接口发展的重要方向。基于意图的业务模型描述方式是用户在某一上下文环境中基于意图的表述，意图可以由对象和操作，或者对象和结果组成，如图6.1所示。

(1)对象(object)：用于描述网络业务模型的主体，主要包括面向用户的节点(customer facing node, CFN)、连接(connection)和业务流(service flow)。

(2)操作(operation)：用于描述用户期望的行为，可以用“在某个条件下，做某个动作，同时遵守某种约束(constraint)”的模式来表述。例如，在满足带宽、延时等网络条件下，在匹配哪些节点不可互通等限制下，标记优先级，实现转发操作。

(3)结果(result)：用于描述用户希望达到的状态，包括期待结果和避免结果两类，可以用“期望达到某一状态”或者“避免达到某一状态”的子句来表述。例如，预期保证带宽利用率大于80%避免端到端延时大于100ms等。

表6.1以关于企业多分支站点及其总部站点之间的访问需求为例，给出基于意图的接口描述示例。

北向接口功能与类型

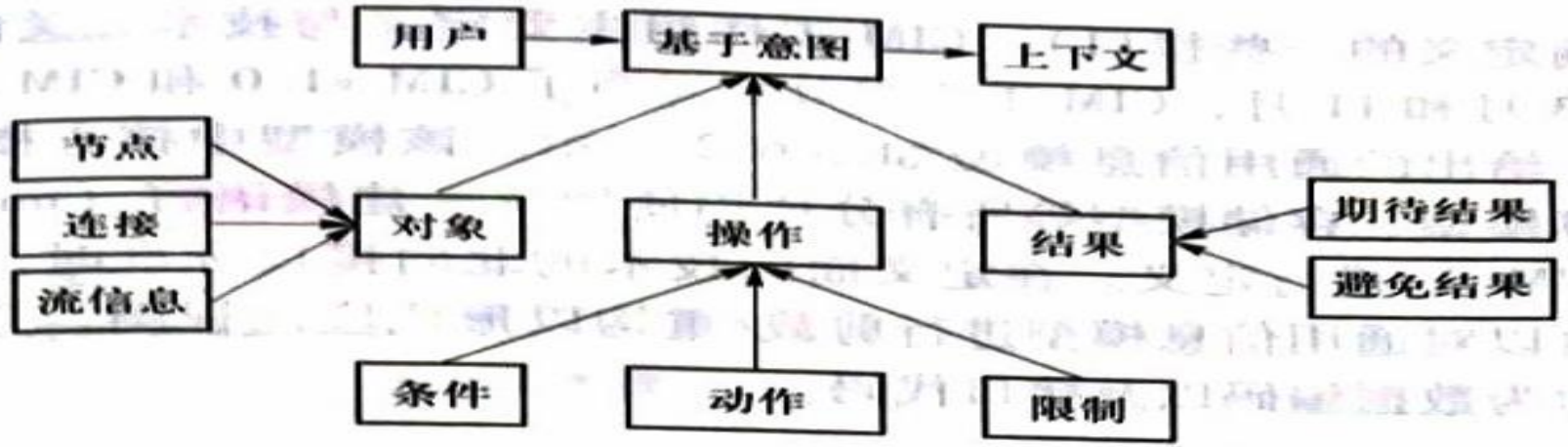


图 6.1　基于意图的描述方式

表 6.1　基于意图的接口描述示例

意　图	对　象	操作/结果
各分支站点可以访问总部站点的网络数据库，但不可以访问总部站点的用户数据库	各分支、总部网络数据库和总部用户数据库	操作： 可以访问； 不可以访问
各分支站点间的带宽利用率保持大于 80%	各站点间的带宽资源	结果： 期待带宽利用率大于 80%

北向接口的标准化

1. ONF 的相关工作

1)CIM 工作组

CIM(common information model，通用信息模型)工作组致力于定义一个对于各种网络技术公共的信息模型。信息模型是对物理资源的抽象表示，它包含了一组对象、对象之间的关系、对象属性和对象可以执行的操作。北向接口根据信息模型来定义接口上交互的数据内容和操作。虽然不同的网络技术具有不同的特点，但是这些技术所共有的一些信息模型基本一致。采用不同的信息模型会增加北向接口的复杂度，降低接口互操作性和可扩展性。

CIM 继承了ITU-T、TMF等标准组织对于电信网络的建模方法。通用信息模型包含与具体网络技术无关的核心信息模型（例如拓扑、转发等对象）、与特定网络技术相关的信息模型（比如OTN、IP等）以及应用相关模型（比如NBI工作组当前定义的一些接口)。CIM工作组主要定义与技术无关的核心信息模型.2015年3月和11月，CIM工作组分别发布了CIM v1.0和CIM v1.1版本。

北向接口的标准化

CIM给出的通用信息模型如图6.2所示。该模型中核心模型、转发技术模型、计算模型、存储模型等所有分片均使用统一建模语言(unified modeling language，UML)进行定义。在定义面向技术的北向接口标准时，针对不同的应用场景，可以对通用信息模型进行剪裁/重构以形成应用视图，并可进一步将应用视图映射为数据编码以及接口代码。

北向接口的标准化

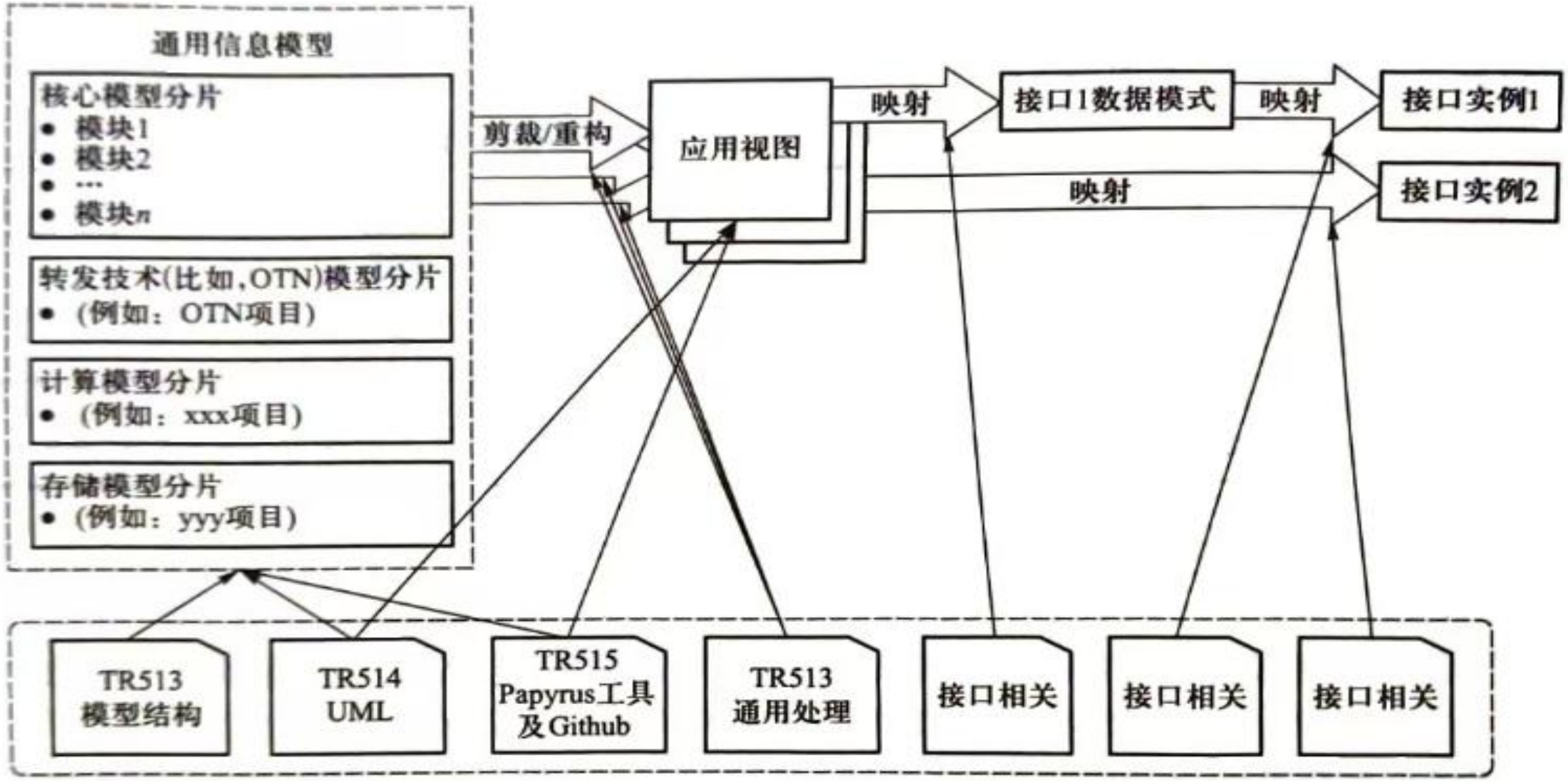


图 6.2　通用信息模型

2)NBI工作组

NBI（北向接口）工作组是ONF最早定义北向接口的工作组，主要研究功能型北向接口，重点是网络资源抽象及控制能力的开放，而在CIM的工作得到了ONF认可后，转而尝试采用CIM方法论集成部分信息模型。在NBI中引入了“维度”的概念，如图6.3所示。横向维度也称为宽度，表示接口适用的范围以及接口面向的功能；纵向维度也称为高度，用于描述抽象的等级，包括底层抽象和上层抽象。图6.3中北向接口按抽象等级进行分类，不同抽象层(abstraction level)及接口范围(interface scope)需要不同的应用程序接口。底层抽象接口主要面向网络资源及功能，包括OpenFlow接口、编程语言、网络设备抽象层、网络分片、网络功能（如拓扑、寻址、交换/路由、负载分担、防火墙)等；上层抽象接口主要面向应用，如应用于虚拟网络管理或QoS等的特定应用程序接口。

北向接口的标准化

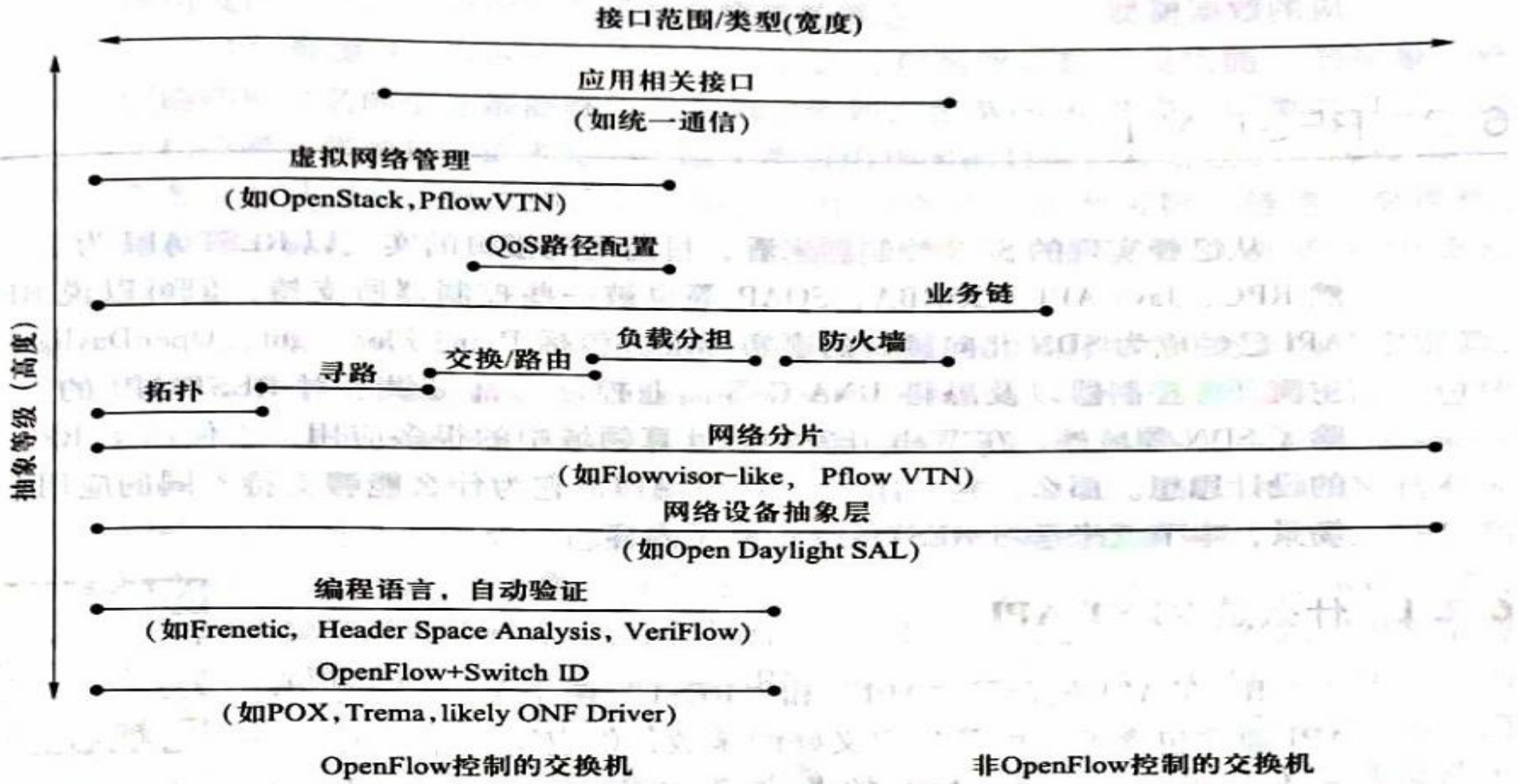


图 6.3　北向抽象接口维度

北向接口的标准化

2. IRTF 的工作

因特网研究任务部(Internet Research Task Force, IRTF)专门成立SDNRG研究组，主要研究SDN模型的分离、SDN模型的可扩展性和适用性等，同时为SDN 领域中出现的一些有趣和关键的问题提供了讨论平台。SNDRG 提出了SDN的层次化架构，在管理和控制平面上给出网络服务抽象层(network service ab- straction layer, NSAL)，也就是北向接口，为应用、服务、控制和管理提供接入服务，其他还包括应用平面、控制层分为控制抽象层和管理抽象层、设备和资源管理层。

3. IETF 的工作

因特网工程任务部(IETF)并没有独立的北向接口工作组，但是也成立了SFC工作组。该工作组关注现有协议和架构的重用，立足于以演进的方式实现集中控制和网络的可编程，目标是确立各网络功能服务整合的体系结构以及对外的接口，定义如何将多个业务功能通过业务链连接起来形成不同的业务功能集合，满足不同场景的需求和部署场景，SFC的对外控制接口是SDN北向接口的重要组成部分。目前，SFC工作组已经发布了SFC架构RFC，并在努力制定相应的数据模型。

Part 02

REST API

什么是REST API

REST API包含了“API”和“REST”两个关键词。其中，API通常包含了一组预先定义好的函数，供应用程序或编程开发人员访问调用。REST的英文全称是representational state transfer，直译成中文是“表述化的状态转移”，该词最早出现在2000年 Roy Thomas Fielding 的博士论文中。REST本质上是一组 Web架构的约束条件和原则，满足 REST约束条件和原则的设计规范或者架构风格就称之为RESTful，遵循RESTful 设计的API就是REST API。RESTful 并不是专门为SDN提出的，而是针对Web 应用中 HTTP 使用中的问题提出的。长期以来，在Web应用中，HTTP协议的使用很不规范、非常随意，甚至很混乱。例如，要删除一个ID号为10的学生信息，URL有多种写法，至少有下面三种比较普遍的写法：

/student/delete/id=10

/student/id=10&method=delete

/deleteStudentById?id=10

由此可见，URL的设计缺乏规范性，同时还存在一些其他的问题，比如HTTP 的动词使用不当、HTTP 的返回状态码使用不规范等。RESTful 正是要解决Web应用中的HTTP的诸如此类问题。如果用RESTful设计，上面的例子可以统一写成以下形式：

DELETE /students/10

这种以“一个HTTP动作+一个URI”的书写形式，显得非常规范和简洁。

什么是REST API

为实现HTTP使用的规范与约束，Fielding博士在其博士论文中提出了“REST”架构风格，在该架构中引入了几个重要的概念。

(1)资源(resource)：资源是对信息(包括所有数据或功能)的抽象。任何能够被命名的信息都能够作为资源，例如，在Web开发中，资源可以是一份文档或者一张图片，或者是一个超文本引用指向的目标；在普通的博客应用中，资源可能是用户、博文或者评论等；而在SDN中，资源可能是链路、交换机、流表等。资源对于REST是一个非常重要的概念，因为REST是面向资源的设计；

(2)资源标识符(resource identifier)：用以标识组件之间交互的特定资源，在 Web 中的实例就是 URI (uniform resource identifier，统一资源标识符)，包括URL (uniform resource locator，统一资源定位符)和URN(uniform resourcename，统一资源名称)两种形式。这里要说明的是，URI、URL和 URN 是有区别的。URI是一个标准，它提供了一个简单、可扩展的资源标识方式；URL和URN 都是URI 的一个子集（或者说一种实现方式），URL不仅标识了Web资源，还指定了操作或者获取方式，同时指出了主要访问机制和网络位置；URN用特定命名空间的名字标识资源，使用URN 可以在不知道其网络位置及访问方式的情况下标识资源。对于REST API来说，一个资源一般对应一个唯一的URI，简单来讲就是，REST通过 URI来标识资源。因此、URI 的设计合理性和规范性十分重要。

(3)表述(representation)：用于捕获一个资源当前或预期的状态、在组件之间传递该表述，表述由数据、描述数据的元数据以及描述元数据的元数据组成（通常用来验证消息的完整性）。一个特定的表述可能表示的是被请求资源当前的状态或预期的状态。以Web为例，典型的表述包括 HTML 文档、JPEG 图片等。

(4)元数据(meta data)：用于描述数据的数据，分为表述元数据和资源元数据。表述元数据用于描述表述的数据，如媒体类型、最后修改时间等；资源元数据用于描述资源的数据，如源链接、alternates标签、vary标签等。

REST给出的重要约束有五个，分别为客户-服务器(client/server)、无状态(stateless)、缓存(cache)、统一接口(uniform interface)、分层系统(layeredsystem)。此外，还有一个可选约束：按需代码(code-on-demand)。

什么是REST API

下面结合Web的例子，具体解释这些约束。

(1)客户-服务器：要求客户端用户接口和服务器端数据存储之间实现分离与解耦，从而允许客户端和服务器端分组优化，彼此之间不受影响，既提高了客户端的便捷性，也增强了服务器的可伸缩性。

(2)无状态：要求来自客户端的每一个请求必须包含服务器处理该请求所需要的所有信息。由于每个请求都被单独考虑，提高了可见性和可靠性。同时，由于服务器是无状态的，能够很方便地实现水平扩展，提高了可扩展性。

(3)缓存：要求一个请求响应中的数据标记为是否可缓存，如果可以，客户端可以重用相同请求的响应数据；这样做的好处是减少了客户端和服务器端的交互次数，提高了效率。

(4)统一接口：强调组件之间的交互或通信应有一个统一的接口。以Wek客户和服务器之间通信为例，要求其方法必须是统一的，使用标准的HTTP动作、如GET、POST、PUT、DELETE等。

(5)分层系统：通过将架构分为若干等级的分层，限制组件的行为。例如允许服务器和客户端之间的中间层，例如反向代理、API网关等组件，代替服务器对客户端的请求进行回应，而这对于客户端来说都是透明的，它只关心接收到的HTTP响应，而并不关心与它通信的组件是谁。分层系统提高了系统的可扩展性，但也增加了系统的复杂性。

1. HTTP动词

在REST API中，比较常用的是以下6个HTTP动词，如表6.2所示。

表 6.2 HTTP 动词及功能描述

HTTP 动词	描　述
HEAD	获取某个资源的头部信息
GET	获取资源
POST	创建资源
PATCH	部分更新资源
PUT	完整更新资源
DELETE	删除资源

2. 资源的命名规范

资源的原型主要有文档(Document)、集合(Collection)、仓库(Store)和控制器(Controller)几种。

- 文档：单一资源的表现形式，可以理解为一个对象，或者数据库中的一条记录
- 集合：资源的一个容器或目录，可以向集合里面添加资源或文档。
- 仓库：由客户端管理的一个资源库，客户端可以向仓库中新增资源或者删除资源，也可以从仓库中获取资源。
- 控制器：可以执行一个方法，支持参数输入，结果返回。

在RESTful设计中，统一资源标识符需要遵循一定的命名规范。不同资源的命名规范如下：

- 文档类型，用名词单数命名；
- 集合和仓库类型，用名词复数命名；
- 控制器类型，用包含动词的词语命名。

3.URI的设计规范

URI的格式规范主要包括下面几点。

(1)关于分隔符“/”的使用。分隔符“/”一般用来对资源层级进行划分，在REST API中，“/”不应该出现在URI的末尾。

(2)URI中尽量使用连字符“-”代替下画线“_”。连字符“-”一般用来分割URI 中出现的字符串，以提高URI的可读性。例如 http://api.example.org/mark-masse/this-is-my-first-post。

(3) URI中统一使用小写字母。

(4)URI中不要包含文件(或脚本)的扩展名。例如，不要出现，php或者.json之类的后缀名。

(5)增删改查(CRUD)的操作不能出现在 URI中。以对 ID 为 123 的 user进行相关操作为例，GET/deleteUser?id=123、DELETE/deleteUser/123、POST/users/123/delete等都在URI中出现了删除操作，这些写法都是错误的；而GET/users/123、POST/users/123、DELETE /users/123 则是规范的，简洁且可读性好。

(6)URI 可以使用query字段，作为查询的参数补充，以表示一个唯一的资源。query可以作为过滤条件使用，例如，请求GET/users?role=admin表示通过设置role=admin获取角色为管理员的所有用户。

(7)可以设置过滤参数作为资源列表分页标识使用。例如，请求GET/users?pageSize=25&pageStartIndex=50，通过设置 pageSize 和 pageStartIndex，获取指定页的用户。

4. HTTP的响应状态码

在RESTful设计中，需要正确使用HTTP的响应状态码。REST API相关的响应状态码以三位长度的十进制表示，包括以下四类：

- 2xx：以2开头的表示操作成功；
- 3xx：以3开头的表示重定向，资源位置发生了变化；
- 4xx：以4开头的表示客户端错误；
- 5xx：以5开头的表示服务器错误。

5. 元数据的设计

RESTul设计中包括元数据的设计。可以在HTTP头部设置元数据字段HTTP头部的常用元数据字段如下。

(1) Content-Type：表示body的数据格式，例如，Content-Type设置为appkcation/json，表示类型是application，数据格式是json。

(2) Content-Length：表示body数据体的大小。

(3) Last-Modified：表示资源最后被修改的时间戳。

开源控制器的REST API

1. Floodlight REST API

   Floodlight 控制器的北向接口提供了丰富的 REST API，除了模块管理器等控制器内部的功能模块外，Floodlight中的应用模块，如虚拟网络过滤器(virtual network filter)模块、静态流表项推送(static flow entry pusher)模块防火墙模块、学习交换机(learning switch)模块等都提供了REST API，可以供上层应用调用。

2. Ryu REST API

Ryu 提供了三类REST API作为控制器的北向接口。第一类是而向运营商的北向接口，可以通过Ryu控制器中的组件配置管理SDN网络。第二类是面向OpenStack 平台的REST API，主要为OpenStack平台的Ouantum组件提供服务，使得网络资源、计算资源和存储资源被统一交付给用户，实现全资源的池化和按需调度。第三类是面向用户的自定义 REST APL Ryu 使用 WSGI Web 服务器来创建REST API，使得用户可以使用REST或RPC定义自己的应用。

Ryu 中使用REST API的方式与Floodlight相似，其REST API定义在所属模块的文件内。因此，如果希望在Ryu中使用REST API，需要提前使用ryu-manager加载一些与之相关的REST模块。这些模块在ryu/ryu/app目录下，主要包括ofetl_rest.py、rest_topology. py、rest_firewall. py、rest_gos.py、rest_router.py等类型的REST模块。具体有哪些REST API可以使用，可以查看源代码中位于开头的注释，这些注释表明了该模块中可以使用的REST API。

开源控制器的REST API

REST API在被广泛应用的同时，也存在着一些缺点。例如，容易受限于控制器所能提供的接口能力，在一些复杂业务操作场合下执行效率低等。为此，有一些控制器对外提供更加丰富的北向接口形式，如Java接口等，供不同需求的网络应用开发人员使用。以OpenDaylight 为例，其在北向接口API的设计中，开放能力涵盖网络服务功能、网络编排功能、服务管理等，还提供了开放服务网关协议(OSGi)框架和双向的REST API两种接口形式。其中开放服务网关协议框架主要用于与控制器处于同一地址空间的Java应用开发（如应用开发可作为控制器提供服务的一部分），而REST API则主要用于开发远程的、基于Web的应用，并为之提供了完备的接口描述、参数、响应设置、状态编码等信息。利用这些北向接口，可充分利用控制器调用网络的能力，同时通过应用中的算法驱动控制器对全网资源进行编排。

开源 SDN 控制器的出现，以及SDN 网络中北向接口的出现和成熟，为上层这无疑给传统应用开发带来巨大的便利性，从而用户可以灵活调配网络资源，的设备厂商带来了巨大压力。为了应对SDN 带来的竞争，一些传统网络厂商也推出了具有编程能力的相应设备，以供上层业务与应用的直接调用，这成为北向接口的另一种形式，例如思科推出的DNA-C API等。

思科DNA-C API

1. DNA-C的开放接口

DNA-C的各种接口服务与DNA的开放平台策略，如图6.4所示。

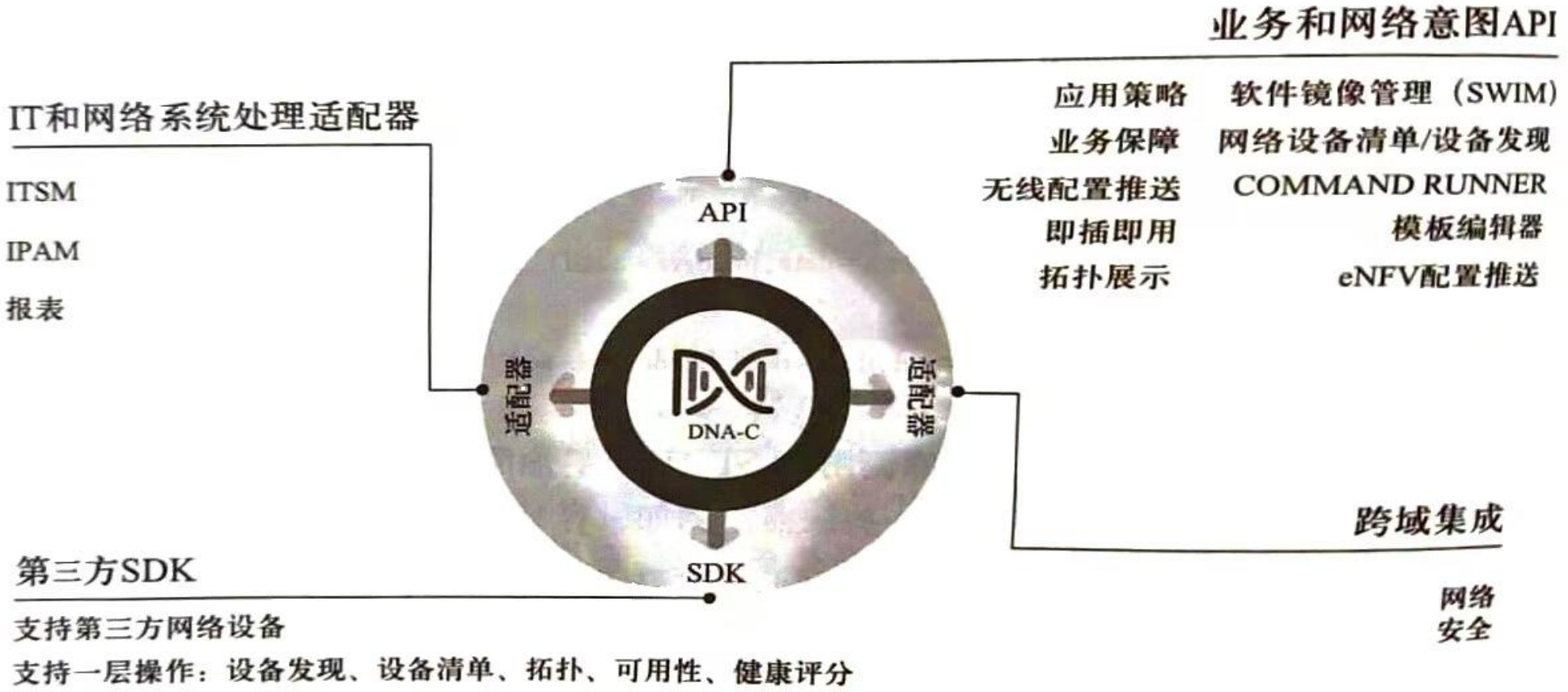


图 6.4　DNA-C 的开放接口

思科DNA-C API

DNA-C中有专门的平台模块管理四种类型的接口。

1） 北向接口

北向接口的平台模块中使用了基于意图的API概念，包括应用策略、业务保障、无线配置推送、即插即用、拓扑展示、软件镜像管理、网络设备清单/设备发现、COMMAND RUNNER（用于运行命令行命令）、模板编辑器、eNFV配置推送等，这类开放性的REST API为开发人员提供了标准接口，开发人员可以对DNA-C的自动化及业务保障工作流进行统一访问。

2） 南向接口

南向接口的平台模块中使用了多厂商SDK工具的概念。这个工具允许思科的合作伙伴利用 SDK 去支持和管理非思科的第三方设备。常用的通信协议包括NETCONF、SNMP、Telnet/SSH等。

3） 西向接口

思科有意将东西向接口分成西向接口与东向接口，以实现不同的功能。其中有两个功能很重要。一是集成API功能，这个功能能够与IT生态系统中的ITSM 体系、IPAM 体系以及报表体系等紧密集成，ITSM和IPAM 体系分别表示IT服务管理和IP地址管理。二是事件及通告功能，这个功能通过推送事件触发的通告来使第三方应用去检测并响应DNA-C中发现的运维问题。

4）东向接口

东向接口的平台模块中提供了跨域集成的功能。这类集成的目的在于将思科的各类企业级SDN域连通，使得企业可以在园区网、广域网、数据中心网络等不同域之间执行统一的策略部署，并且通过思科的安全软件Stealthwatch进行统一的风险防范。

2. DNA-C 的北向接口

DNA-C 的北向接口也被称为基于意图的 API (intent-based API，IAPI)，表示其能够对上层应用与业务的意图进行理解，并提供基于策略的抽象。IAPI也是基于 REST API的，使用HTTPS的操作动作，例如GET、POST、PUT、DELETE等，并利用JSON的语法去管理网络。

如图6.5所示，IAPI的具体功能可分为四类：了解网络(Know Your Network)、站点管理(Site Management)、运维工具(Operational Tools)、认证(Authentication)。

1）了解网络

这类API是用于发现站点、拓扑、网络、设备、客户等的细节，并对网络设备进行添加、删除，还能实现设备数据的导出，可细分为：

(1)站点API：获取站点结构，了解网络健康度；

(2)网络API：获取网络设备数据，了解设备健康度；

(3)设备API：获取设备详细信息，例如设备MAC地址、UUID、主机名、功能特性、接口、设备配置、证书状态、模块、VLAN绑定等，还可对设备进行增删改查；

(4)客户API：获取有线及无线客户的详细信息，包括各种健康度数据等。

思科DNA-C API

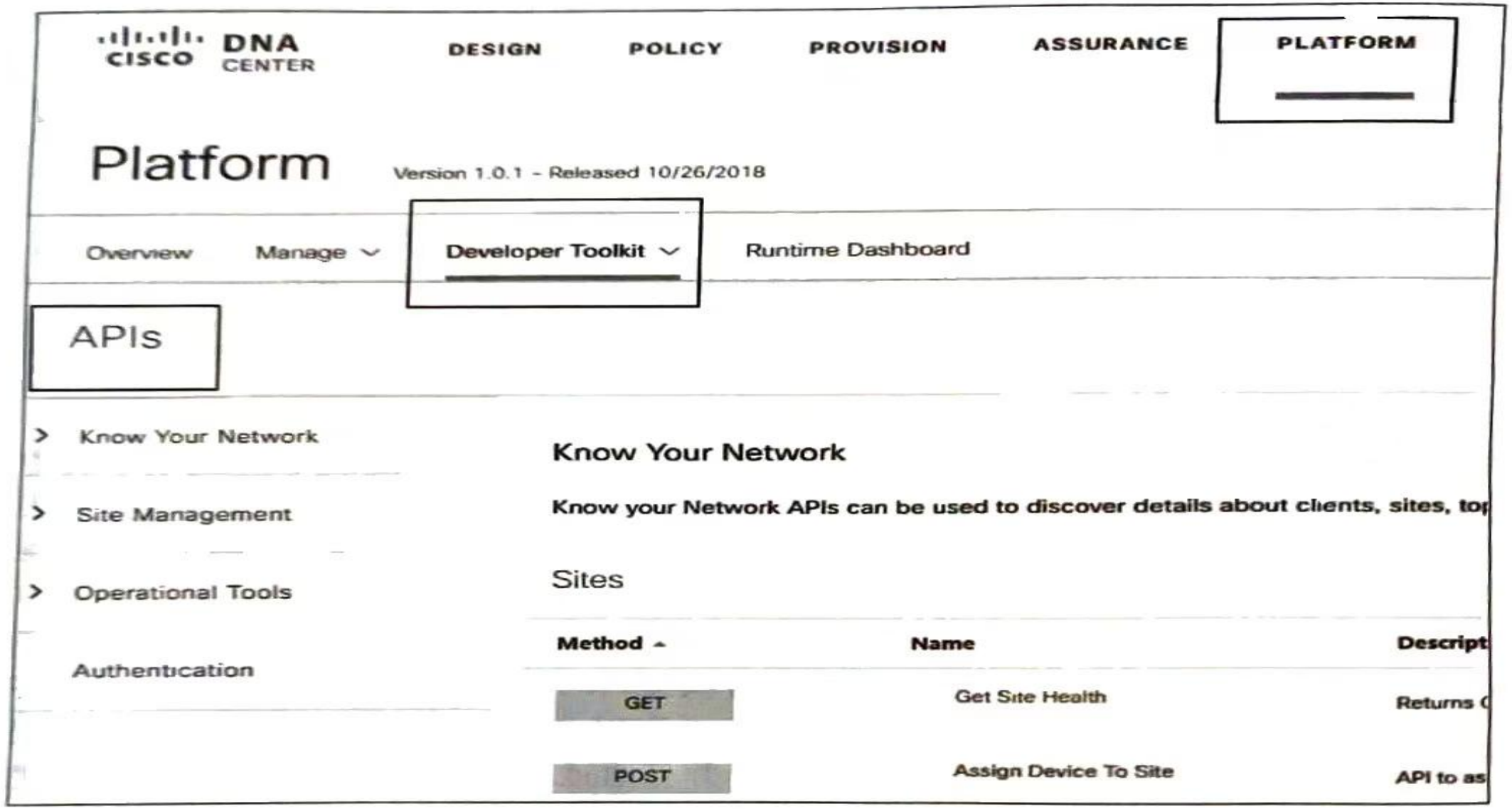


图 6.5　DNA-C 的界面截图

2）站点管理

这类 API 用于提供企业网络的零接触部署功能，并能对网络内的软件镜像进行分发和激活管理，可细分为：

(1)站点配置文件API：用于提供网络功能虚拟化(NFV)等相关的功能；

(2)SWIM API: SWIM 是Software Image Management的英文缩写，该API用于提供软件的镜像管理；

(3) PnP API: PnP是即插即用(plug and play)的英文缩写，该API用于提供零接触的部署管理。

3)运维工具

这类API是用于管理DNA-C中的运维任务，可细分为：

(1)命令行运行API：可用于执行针对网络设备的只读命令，以获取设备实时信息；

(2) 网络发现API：可用于执行基于特定用户凭证的网络设备发现;

(3)配置模板编程API：可用于执行DNA-C 设计及配置推送环节中的模板加载，并对模板本身做增删改查以及版本化操作；

(4)路径追踪API：可用于执行基于两个端点之间的路径追踪，以确定数据路径及迅速定位故障、排除故障；

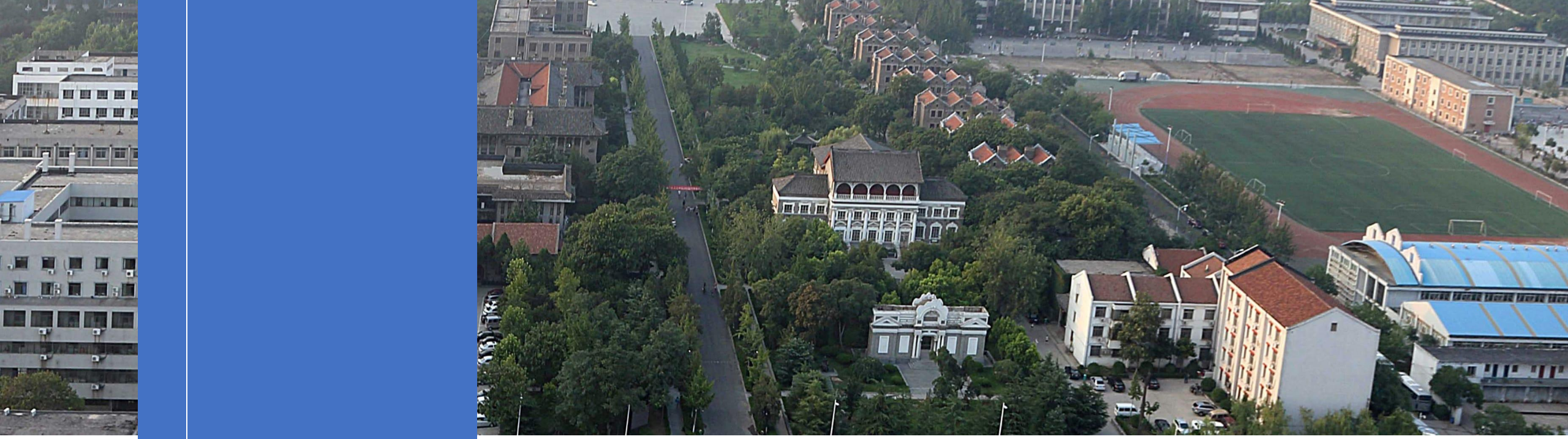
(5)文件API：可用于获取文件，包括设备软件镜像及数字证书等；

(6)任务API：可用于获取任务进度信息。

4)认证

这类API用于生成安全令牌，以区分不同访问行为的权限，并保障后续API调用过程的安全性。

应该指出，不同厂商的REST API会有不同的表达方式，同一厂商不同软件版本的REST API表达方式也可能会有差异，使用时应以官方参考文档为准。下面以POSTMAN 作为调试工具对DNA-C的REST API进行演示，POSTMAN是一个用于API开发的协作平台，关于它的使用方法超出了本书范围，有兴趣的读者可访问POSTMAN官网了解详情。



Part 03

RESTCONF 协议

RESTCONF 概述

从协议RESTCONF的名称可以看出，它包含了两个关键字“REST”和“Config”。顾名思义，这是一个基于REST模式、用于网络配置与管理的协议，目的在于为Web应用提供获取配置数据、状态数据、通知事件的标准机制。因此，该协议以HTTP作为传输协议，使用YANG语言来定义数据结构，并使用NETCONF定义数据存储。其中，数据存储可以通过文件系统、数据库、闪存或者它们互相结合来实现。在NETCONF协议中，定义了配置数据存储和一系列的创建(create)、获取(retrieve)、更新(update)和删除(delete)操作(简称CRUD)，通过这些操作，可用于访问数据。而YANG则是一种数据建模语言，它定义了数据存储内容、操作数据、协议操作、通知事件的语法和语义等。RESTCONF 是基于YANG来定义的标准数据模型，以创建针对NETCONF数据存储的CRUD操作。

RESTCONF协议的出现，并非是为了取代NETCONF协议，而是提供一种额外的简易接口，遵循RESTful原则并且与面向资源的设备抽象兼容。与NETCONF相比，RESTCONF提供的功能较简单，有一些功能它是不提供的，例如不提供对数据库的加锁操作，不支持数据库备份。 因此，当需要提供复杂功能时，仍然需要使用NETCONF。

资源模型

RESTCONF 协议的操作对象是基于层次结构的资源。首先它的每个资源代表设备中的一个可管理组件，由概念数据集合和允许操作该数据的方法集组成。每个资源有它自己的媒体类型标识符，用HTTP响应头中的“Content-Type”字段表示。

资源可以包含子节点（资源），也称嵌套资源或域，一个资源可以包含0个或多个嵌套资源。子资源的类型和允许操作它们的方法是与数据模型相关的即使一个资源有父资源，子资源的创建和删除操作也可以独立于它的父资源。顶级资源为API。

RESTCONF 协议定义了一些与应用相关的媒体类型，用于标识每个可用的资源类型。表6.3总结了资源和媒体类型的对应关系。

表 6.3　资源与媒体类型

资　　源	媒 体 类 型
API 资源	application/vnd. yang. api
数据存储资源	application/vnd. yang. datastore
数据资源	application/vnd. yang. data
事件资源	application/vnd. yang. event
操作资源	application/vnd. yang. operation
补丁资源	application/vnd. yang. patch

(1)API 资源(API resource)：包含 RESTCONF特性的状态和接入点。它是顶层资源，并且拥有application/vnd.yang.api+XML 或者application/vnd.yang.api+JSON介质类型。它可以通过“/.well-known/restconf”来获取。API资源拥有以下子资源。

①/.well-known/restconf/datastore：这个强制资源代表了运行中的配置数据存储和任何可用的非配置数据，它可能被直接获取或编辑，不能被客户端直接创建或删除。

②/.well-known/restconf/modules：这种强制性资源的标识符可支持YANG数据模型模块。

③/.well-known/restconf/operations：这个可选资源提供服务器端支持的数据模型的特定协议操作。如果服务器端没有关于数据模型的特定操作，服务器端将返回一个默认的资源。任何数据模型的操作都由YANG模型定义，并在服务器端返回这个资源可用的子节点。

(2)数据存储资源(datastore resource)：数据存储资源代表数据资源概念树的根。数据存储资源可以使用GET方法来查询，以获取配置数据资源或无配置数据资源存储中的数据资源。通过depth查询参数，可以控制子树的深度检索操作。

(3)数据资源(data resource)：一个数据资源代表一个YANG数据节点，它是一个数据存储资源的子孙节点。YANG容器和列表数据节点类型也被认为是数据资源。与数据存储资源相同，数据资源也同样可以通过GET获取，也可以通过depth参数控制查询的深度。

(4)事件资源(event resource)：媒体类型为application/vnd.yang.event +XML或application/vnd.yang.event+JSON 的一种资源。它描述了一种由通知消息递交的概念体系或数据模型特定的事件。

(5)操作资源(operation resource)：为 application/vnd.yang.operation+XML或application/vnd.yang.operation+JSON的一种资源。

(6)补丁资源(patch resource)：为application/vnd.yang.data+XML 或application/vnd.yang.data+JSON的“PATCH”操作。

消息

RESTCONF 协议以HTTP 作为传输协议，因此采用HTTP报文实体表示消息。一个HTTP消息对应一个单独的协议方法或操作，大部分消息可以在一个单独的资源上执行一个单独的任务，比如获取资源或编辑资源。但有关“PATCH”方法的消息例外，如果采用“YANG Patch”格式，可以利用一个消息编辑多个数据仓库。

图6.15给出了RESTCONF消息的一般格式，具体说明如下。

(1)方法(method)：消息的必需部分，表示消息所涉及的资源操作方法。如GET、PUT等HTTP方法名。

(2)入口(entry)：消息的必需部分，表示为每一台HTTP服务器上配置的RESTCONF入口、例如/.well-known/restconf。

(3)资源(resource)：消息的可选部分，表示将要被访问的资源，，以路径表达式的形式表示。如果这个域为空，表明目标资源就是API本身，媒体类型为application/vnd.yang.api。

(4)查询(query)：消息的可选部分，表示RESTCONF消息携带的参数以键值对的形式出现。

(5)片段(fragment)：该域目前尚未用于RESTCONF协议。

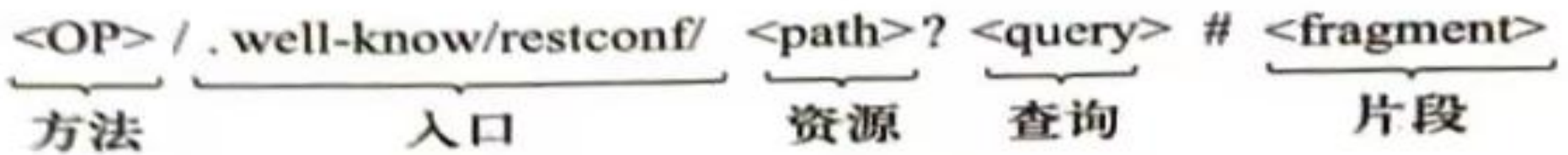


图 6.15　RESTCONF 消息格式

消息

上述RESTCONF消息报文作为HTTP报文实体，封装在HTTP报文中进行传输。为此，一些典型的HTTP 报文消息头可用于 RESTCONF 消息，包括请求和应答两种类型。表6.4给出了常用的请求消息头及描述，表6.5给出了常用的应答消息头及描述。

每个消息代表着一些资源访问，一个HTTP状态码表示每一个请求的返回状态。如果返回码的状态是4XX或者是5XX，则错误信息也会在消息响应中一并返回。

表 6.4　HTTP 请求消息头及描述

名　　称	描　　述
Accept	可接收的相应内容类型（例如，text/html）
Content-Type	请求消息体的类型（例如，application/json）
Host	服务主机地址
If-Match	如果实体与 ETag 匹配才生效
If-Modified-Since	在规定时间后的行为被修改才生效
If-Unmodified-Since	在规定时间后的行为不被修改才生效

表 6.5　HTTP 应答消息头及描述

名　　称	描　　述
Allow	定义了在返回 405 错误时的合法行为（例如，GET）
Content-Type	响应消息体的类型（例如，application/json）
Date	发送消息的时间，包含日期和时分秒
ETag	一个标示符号用于指定资源的特定版本
Last-Modified	某一资源的最后修改时间，包含日期和时分秒
Location	用于指定一个新创建的资源

1. GET

GET方法由客户端发出，用于从资源那里获取数据和元数据。除了操作资源之外的所有资源类型都支持该方法。请求必须包含一个至少含有一个端点组的请求URL。请求参数如表6.6所示。

表 6.6 GET 方法请求 URL 参数

名　　称	描　　述
Config	请求的是配置的还是非配置的数据
Depth	控制获取请求的深度
Format	对请求的响应是 JSON 还是 XML
Select	指定目标资源内部的一个嵌套资源

需要指出的是，服务器端在返回数据资源时，不会返回那些用户不具备读取权限的数据资源。如果用户未被授权而读取目标资源的任何部分，就会向客户端返回一个包含“403 Forbidden”状态行的错误响应信息。如果用户被授权读取目标资源的某部分而不是全部，则未被授权的内容就会从响应体之中被删除，只返回授权部分的内容。

2. POST

POST方法用于创建数据资源或者调用操作资源，可由客户端以各种不同的原因发出。服务器端根据目标资源的媒体类型来决定如何处理请求。请求必须包含一个请求URI，它包含了一个目标资源，标识的资源类型如表6.7所示。

表 6.7 POST 方法请求的目标资源类型

类　型	描　述
Datastore	创建一个顶层配置的数据资源
Data	创建一个配置的数据子资源
Operation	调用协议操作

如果目标资源的类型是一个数据存储资源或数据资源，于创建一个资源。表6.8的查询参数是用来支持POST方法来处理数据存储和数据资源的。

表 6.8　POST 方法处理数据存储和数据资源的查询参数

名　　称	描　　述
Insert	指定在何处插入资源
Point	指定插入资源的点

如果POST方法成功，状态行返回“204 No Content”且不会有消息体响应。如果用户未经授权而创建目标资源，状态行会向客户端返回错误信息“403 For.bidden”。

如果目标资源类型是一个操作资源，POST方法将调用该操作，若POST方法成功则状态行返回“200OK”，否则状态行返回“204 No Content”。同样，如果用户未被授权对目标进行操作，状态行会返回“403 Forbidden”。

3. PUT

PUT方法用于替换目标资源。如果PUT方法成功，会在状态行返回信息“200OK”，它不会返回消息体。如果用户未被授权而试图替换目标资源，则会返回一个包含“403 Forbidden”的回应状态行给客户端。

4. DELETE

DELETE方法用于删除目标资源。如果DELETE方法成功，就会在状态行返回信息“200OK”，且没有任何消息体。如果用户未被授权，则在状态行中会得到回应“403 Forbidden”。

5. PATCH

PATCH 用于对资源进行部分的更新。如果PATCH方法成功，状态行会返回“200 OK”，但不返回任何消息体。如果用户未被授权，则在状态行会得到回应“403 Forbidden”。

将RESTCONF协议与NETCONF协议的基本操作做一比较，表6.9给出了两个协议中相应操作的对应关系。

表 6.9 RESTCONF NETCONF 协议中操作的对应关系

RESTCONF	NETCONF
OPTIONS	none
HEAD	none
RESTCONF	NETCONF
GET	<get-config>，<get>
POST	<edit-config>(operation = " create")
PUT	<edit-config>(operation = " replace")
PATCH	<edit-config>(operation = " merge")
DELETE	<edit-config>(operation = " delete")

明德新民 | 止于至善